

AWS for AI/ML Engineers

A practitioner's reference for Senior Lead engineers (Career_upskill — Topic 04, Capital One lens)

Compiled for Vatsal Raicha

2026

Contents

How to read this ebook	22
Capital One on AWS — Dossier for Sr Lead AI/ML Engineer	23
Strategic posture in one paragraph	23
1. Strategic posture & migration history	23
2. AWS service-by-service map	24
Compute & orchestration	24
Data services	25
Streaming & event-driven	25
ML/AI	26
Networking, IaC, governance	26
3. Open-source signals (read the code, learn the idioms)	26
4. AI/ML platform shape	28
Products users interact with	28
Internal teams	28
MLOps spine (inferred from public talks + JDs)	28
Public ML focus areas (per C1 Tech blog)	28
University partnerships	28
5. Regulatory & compliance posture	29
The 2019 breach lessons	29
6. 2024–2026 GenAI direction	30
GenAI stack signals	30
Databolt expanded to unstructured data (March 2026)	30
7. Sr Lead AI/ML Engineer — hiring bar	30
What three current postings require	30
Compensation bands (McLean, VA, 2026)	31
What to study (priority order)	31
Cert priorities (rank by signal-to-effort for this specific role)	31
Interview talking points (memorize)	31
8. Open questions to ask in interviews	32
9. Reading list (priority order)	32
Must-read before any interview	32
Strongly recommended	33
Background (if time)	33
Cross-references in this topic	33
Module 1 — AWS Account, Organizations & Landing-Zone Model	34
1. The account is the boundary	34

2. AWS Organizations	35
3. SCPs and RCPs	35
Service Control Policies (SCPs)	35
Resource Control Policies (RCPs)	36
4. Control Tower vs Landing Zone Accelerator	36
5. IAM Identity Center	37
6. 2024–2026 changes worth knowing	37
7. Pitfalls and anti-patterns	38
8. Capital One lens	38
9. Sanity check	38
10. Cross-references	39
Primary sources	39
Module 2 — IAM Deep	40
1. The six-layer policy evaluation chain	40
2. Users, roles, and what to use when	40
3. ABAC vs RBAC at scale	41
4. STS endpoints, IRSA, Pod Identity, IAM Roles Anywhere	41
5. The 2019 Capital One breach in IAM detail	42
6. IAM Access Analyzer	43
7. Permission Boundaries (the delegated-admin enabler)	43
8. 2024–2026 changes worth knowing	44
9. Pitfalls and anti-patterns	44
10. Capital One lens	44
11. Sanity check	45
12. Cross-references	45
Primary sources	45
Module 3 — Regions, Availability Zones, Edge	46
1. Partitions — the highest boundary	46
2. Regions and AZs	47
3. Edge, Local Zones, Outposts, Wavelength	47
4. Regional service quirks (the ML-specific ones)	47
5. VPC endpoints and PrivateLink (preview)	48
6. 2024–2026 changes worth knowing	48
7. Pitfalls and anti-patterns	48
8. Capital One lens	48
9. Sanity check	49
10. Cross-references	49
Primary sources	49
Module 4 — Billing, Pricing Models & Tagging Discipline	50
1. Pricing models for compute	50
2. The cost-management toolkit	51
Cost Explorer	51
AWS Budgets	51
AWS Cost Anomaly Detection	51
AWS Cost Categories	51
Cost and Usage Report (CUR / CUR 2.0)	51
3. Consolidated billing	52

4. Tagging discipline (as policy)	52
5. ML-specific cost gotchas	53
6. 2024–2026 changes worth knowing	53
7. Pitfalls and anti-patterns	53
8. Capital One lens	54
9. Sanity check	54
10. Cross-references	54
Primary sources	55
Module 5 — Networking primer for ML engineers (assume zero)	56
1. IP addresses and CIDR	56
2. RFC 1918 private address space	57
3. Subnets, route tables, and gateways	57
4. DNS basics	57
5. TCP vs UDP, ports, ephemeral ports	58
6. Firewalls: stateful vs stateless	58
7. Load balancers: L4 vs L7	59
8. Public vs private IPs, Elastic IPs	59
9. Common pitfalls (right now, before you touch AWS)	59
10. Sanity check	59
11. Cross-references	60
Primary sources	60
Module 6 — VPC Deep	61
1. VPC anatomy	61
2. Subnet classes (by routing intent)	61
3. IPv4/IPv6 dual stack	62
4. NAT Gateway vs NAT Instance	62
5. VPC endpoints — the regulated-cloud essential	62
Gateway endpoints	62
Interface endpoints (PrivateLink)	63
Endpoint policies	63
6. IPAM and shared VPCs via RAM	63
7. Pitfalls and anti-patterns	64
8. Capital One lens	64
9. Sanity check	64
10. Cross-references	64
Primary sources	65
Module 7 — Security Groups, NACLs, Flow Logs	66
1. Security Groups	66
2. NACLs (Network ACLs)	66
3. Evaluation order	67
4. The 7-step “SageMaker notebook can’t reach S3” troubleshooting checklist	67
5. VPC Flow Logs	67
6. Reachability Analyzer and Network Access Analyzer	68
7. Pitfalls and anti-patterns	68
8. Capital One lens	68
9. Sanity check	69
10. Cross-references	69

Primary sources	69
Module 8 — Multi-VPC and Hybrid Connectivity	70
1. VPC Peering	70
2. Transit Gateway (TGW)	70
3. AWS PrivateLink (for your own services)	71
4. Direct Connect (DX)	71
5. Site-to-Site VPN and Client VPN	72
6. Cloud WAN	72
7. TGW Network Manager	72
8. Asymmetric routing — the silent killer	72
9. Pricing pitfalls	72
10. Capital One lens	72
11. Sanity check	73
12. Cross-references	73
Primary sources	73
Module 9 — DNS, Load Balancing, and Edge	74
1. Route 53	74
7 routing policies	74
Health checks	75
Route 53 Resolver	75
2. Application Load Balancer (ALB) — L7	75
3. Network Load Balancer (NLB) — L4	75
4. Gateway Load Balancer (GWLB)	75
5. CloudFront	75
6. AWS Global Accelerator (GA)	76
7. AWS WAF	76
8. AWS Shield	76
9. Pitfalls and anti-patterns	77
10. Capital One lens	77
11. Sanity check	77
12. Cross-references	77
Primary sources	78
Module 10 — S3 Deep	79
1. Storage classes (price/latency tradeoff)	79
2. Lifecycle, versioning, replication	80
3. S3 Tables (GA Dec 2024)	80
4. Object Lambda & Access Points	80
5. Encryption & compliance	81
6. Performance & limits	81
7. Pitfalls and anti-patterns	82
8. Capital One lens	82
9. Sanity check	82
10. Cross-references	82
Primary sources	82
Module 11 — EFS, FSx, EBS for ML	83
1. EBS volume types	83

2. EFS (NFSv4)	84
3. FSx flavors	84
FSx for Lustre	84
FSx for OpenZFS	84
FSx for Windows File Server	84
FSx for NetApp ONTAP	84
4. Picking storage for ML	84
5. Pitfalls	85
6. Capital One lens	85
7. Sanity check	85
8. Cross-references	85
Primary sources	86
Module 12 — Lake Formation, Glue Catalog, Iceberg, DataZone	87
1. Glue Data Catalog (the metastore)	87
2. Lake Formation	87
Three grant modes	87
Granularity	87
Cross-account sharing	88
The grant-order pitfall	88
3. Iceberg on AWS (2024–2026)	88
4. AWS DataZone	88
5. DataZone vs Unity Catalog	89
6. Capital One lens	89
7. Pitfalls	89
8. Sanity check	90
9. Cross-references	90
Primary sources	90
Module 13 — Glue Fundamentals	91
1. The core components	91
2. DynamicFrame vs DataFrame	91
3. Schema evolution	92
4. Example pipeline (raw to curated)	92
5. 2024–2026 changes	93
6. Pitfalls	93
7. Capital One lens	93
8. Sanity check	93
9. Cross-references	93
Primary sources	93
Module 14 — Glue Spark Deep	94
1. Worker types (2026)	94
2. Autoscaling	94
3. Glue 3.0 vs 4.0 vs 5.0	94
4. Glue Streaming	95
5. Pushdown predicates	95
6. Partitioning rules	95
7. Performance tuning checklist	95
8. Flex execution	96

9. Pitfalls	96
10. Capital One lens	96
11. Sanity check	96
12. Cross-references	96
Primary sources	97
Module 15 — ETL Orchestration Choices	98
1. The options	98
2. Step Functions deep	99
3. MWAA versions and sizing	99
4. EventBridge Pipes	99
5. SageMaker Pipelines	99
6. Glue Workflows	100
7. The Capital One MLOps orchestration spine	100
8. Pitfalls	100
9. Capital One lens (talking point)	100
10. Sanity check	100
11. Cross-references	101
Primary sources	101
Module 16 — Relational Basics — RDS Family	102
1. Supported engines (as of 2026-05)	102
2. High availability — Multi-AZ Instance vs Cluster	102
3. RDS Proxy	103
4. Blue/Green Deployments	103
5. Other operational essentials	103
6. 2024-2026 changes	103
7. Pitfalls	104
8. Capital One lens	104
9. Pricing nuance	104
10. Sanity check	104
11. Cross-references	104
Primary sources	104
Module 17 — Aurora Deep	106
1. The architecture innovation	106
2. Aurora Postgres vs Aurora MySQL	106
3. Aurora Serverless v2	106
4. Aurora Limitless Database (GA Dec 2024)	107
5. Aurora DSQL (GA May 2025) — the headline	107
6. Aurora Global Database	107
7. Babelfish	108
8. Aurora I/O-Optimized	108
9. Aurora ML integration	108
10. Pitfalls	108
11. Capital One lens	108
12. Sanity check	108
13. Cross-references	109
Primary sources	109

Module 18 — DynamoDB Deep	110
1. Single-table design	110
2. Keys and indexes	110
3. Streams and triggers	110
4. TTL	111
5. Capacity modes	111
6. DAX (DynamoDB Accelerator)	111
7. Transactions	111
8. Global Tables	112
9. Point-in-Time Recovery (PITR)	112
10. Import/Export with S3	112
11. Hot partitions and adaptive capacity	112
12. Expression syntax	112
13. Consistent vs eventually consistent reads	112
14. DynamoDB Local	113
15. Pitfalls	113
16. Capital One lens	113
17. Sanity check	113
18. Cross-references	113
Primary sources	113
Module 19 — DocumentDB Deep	114
1. Architecture	114
2. MongoDB compatibility	114
3. Sharding via Elastic Clusters	114
4. Indexes	115
5. Vector search	115
6. Aggregation pipeline	115
7. Change streams	115
8. IAM authentication	115
9. Migration patterns	115
10. DocumentDB vs MongoDB Atlas on AWS	115
11. 2024-2026 changes	116
12. Pitfalls	116
13. Capital One lens	116
14. Sanity check	116
15. Cross-references	116
Primary sources	117
Module 20 — Amazon Keyspaces (for Apache Cassandra)	118
1. The pitch	118
2. Capacity modes	118
3. Multi-region replication	118
4. Partition design	118
5. CQL surface — what’s supported vs gaps	119
6. Other constraints	119
7. Keyspaces vs DynamoDB	119
8. Use cases	119
9. Pitfalls	119
10. Capital One lens	119

11. Sanity check	120
12. Cross-references	120
Primary sources	120
Module 21 — ElastiCache & MemoryDB	121
1. The Valkey story (2024)	121
2. ElastiCache — the cache tier	121
3. MemoryDB — durable Redis-compatible primary DB	121
4. ElastiCache vs MemoryDB	122
5. Cache patterns (ElastiCache)	122
6. Hot key mitigation	122
7. Pricing snapshot	122
8. 2024-2026 changes	123
9. Pitfalls	123
10. Capital One lens	123
11. Sanity check	123
12. Cross-references	123
Primary sources	123
Module 22 — Redshift Deep	125
1. Node types	125
2. Redshift Serverless (GA 2022)	125
3. Redshift Spectrum	125
4. Data Sharing	126
5. Zero-ETL integrations (the 2024 story)	126
6. Materialized views	126
7. Concurrency Scaling	126
8. Workload Management (WLM)	126
9. Streaming Ingestion	126
10. Redshift ML	126
11. MERGE	127
12. AQUA (folded)	127
13. 2024-2026 changes	127
14. Pitfalls	127
15. Capital One lens	127
16. Sanity check	128
17. Cross-references	128
Primary sources	128
Module 23 — Snowflake on AWS	129
1. The three-layer architecture	129
2. Virtual Warehouse sizes	129
3. Snowpipe and Snowpipe Streaming	130
4. Snowpark	130
5. Time Travel and zero-copy cloning	130
6. Streams + Tasks	130
7. Iceberg tables	130
8. Horizon governance	130
9. Cortex (Snowflake’s AI/ML surface)	130
10. Why Capital One picked Snowflake	131

11. Snowflake vs Redshift (decision points)	131
12. 2024-2026 changes	131
13. Pitfalls	131
14. Capital One lens	132
15. Sanity check	132
16. Cross-references	132
Primary sources	132
Module 24 — Athena & Query Federation	133
1. Athena v3 engine	133
2. Table formats supported	133
3. Federated query	133
4. Workgroups	134
5. Performance tuning	134
6. Partition projection	134
7. Athena ACID via Iceberg	134
8. CTAS (CREATE TABLE AS SELECT)	135
9. Athena for Apache Spark	135
10. 2024-2026 changes	135
11. Pitfalls	135
12. Capital One lens	135
13. Sanity check	135
14. Cross-references	135
Primary sources	136
Module 25 — Neptune — Graph Database	137
1. The dual nature	137
2. Capacity tiers	137
3. Neptune Analytics (the 2024 story)	137
4. Neptune ML	138
5. GraphRAG	138
6. Bulk loading	138
7. Cluster topology	138
8. Capital One lens — fraud and graph	138
9. Pitfalls	139
10. Sanity check	139
11. Cross-references	139
Primary sources	139
Module 26 — OpenSearch — Search & Vector	140
1. Service vs Serverless	140
2. The 2021 Elasticsearch fork	140
3. k-NN engine choices	140
4. Hybrid search (BM25 + vector)	141
5. Ingestion pipelines	141
6. Security plugin	141
7. Neural search	141
8. OpenSearch as RAG backend	141
9. OpenSearch Service vs Elastic Cloud on AWS	141
10. 2024-2026 changes	142

11. Pitfalls	142
12. Capital One lens	142
13. Sanity check	142
14. Cross-references	142
Primary sources	143
Module 27 — Vector Capabilities Decision Framework	144
1. The options	144
2. The decision tree	145
3. Bedrock Knowledge Bases native backends	145
4. Cost shapes	145
5. Multi-tenant patterns	146
6. The “two-tier” pattern (Capital One likely)	146
7. When to pick which (cheat sheet)	146
8. Pitfalls	146
9. Capital One lens	146
10. Sanity check	147
11. Cross-references	147
Primary sources	147
Module 28 — Amazon MSK Deep	148
1. MSK Provisioned vs Serverless	148
2. KRaft vs ZooKeeper	148
3. Authentication	148
4. MSK Connect	149
5. Glue Schema Registry	149
6. MSK Replicator	149
7. Producer/Consumer patterns	149
8. MSK vs Self-managed Kafka vs Confluent Cloud on AWS	150
9. 2024–2026 changes	150
10. Pitfalls	150
11. Capital One lens	151
12. Sanity check	151
13. Cross-references	151
Primary sources	151
Module 29 — Kinesis Family + Flink	152
1. Kinesis Data Streams (KDS)	152
2. Amazon Data Firehose (formerly Kinesis Firehose)	152
3. Managed Service for Apache Flink (MSF, formerly Kinesis Data Analytics for Apache Flink)	153
4. EventBridge	153
5. EventBridge Pipes	153
6. Decision framework: Kinesis vs MSK vs EventBridge vs SQS/SNS	153
7. Pricing comparison (rough)	154
8. Capital One serverless streaming SDK pattern	154
9. Pitfalls	154
10. Sanity check	155
11. Cross-references	155
Primary sources	155

Module 30 — EC2 & Accelerators	156
1. The taxonomy	156
2. NVIDIA GPU instances	156
3. AWS-designed accelerators	157
4. EC2 Capacity Blocks for ML	157
5. UltraClusters and EFA	157
6. Nitro System	157
7. AMIs (Amazon Machine Images)	157
8. Auto Scaling Groups (ASG)	158
9. Spot economics for ML	158
10. Placement Groups	158
11. Pricing examples (us-east-1, May 2026)	158
12. Capital One lens	158
13. Sanity check	159
14. Cross-references	159
Primary sources	159
Module 31 — ECS + Fargate + AWS Batch	160
1. ECS	160
2. Fargate	160
3. ECS vs EKS	160
4. AWS Batch	161
5. ECS Anywhere	161
6. Pricing comparison (~)	161
7. Capital One lens	161
8. Pitfalls	162
9. Sanity check	162
10. Cross-references	162
Primary sources	162
Module 32 — AWS Lambda Deep	163
1. Execution model	163
2. SnapStart (the cold-start killer)	163
3. Container images	163
4. Layers and Extensions	164
5. Concurrency	164
6. Destinations	164
7. Event sources	164
8. Lambda + VPC	164
9. NAT GW cost trap	165
10. Observability	165
11. Capital One serverless-first patterns	165
12. Pricing	165
13. Pitfalls	165
14. Sanity check	166
15. Cross-references	166
Primary sources	166
Module 33 — EKS Foundations	167
1. Cluster anatomy	167

2. Data plane options	167
Managed Node Groups (MNG)	167
Karpenter (the modern choice)	167
Fargate Profiles	168
3. EKS Auto Mode (GA Dec 2024)	168
4. VPC CNI (pod networking)	168
5. IAM Roles for Service Accounts (IRSA) vs Pod Identity	168
IRSA (GA 2019-09-03)	168
EKS Pod Identity (GA 2023-11-26)	168
6. GPU support	169
7. Upgrades	169
8. EKS Anywhere & EKS Distro	169
9. Pricing	169
10. 2024-2026 changes	169
11. Pitfalls	169
12. Capital One lens	170
13. Sanity check	170
14. Cross-references	170
Primary sources	170

Module 34 — SageMaker Platform Map 171

1. The three eras of SageMaker	171
2. Studio Domains	171
3. User profiles and Spaces	171
4. The 2024 story: SageMaker Unified Studio	172
5. Code Editor	172
6. SageMaker Catalog	172
7. RStudio in SageMaker	172
8. Custom Studio images	172
9. Evolution timeline	172
10. 2024-2026 changes	173
11. Pitfalls	173
12. Capital One lens	173
13. Sanity check	173
14. Cross-references	173
Primary sources	173

Module 35 — SageMaker Data 174

1. Data Wrangler	174
2. Feature Store	174
3. Processing Jobs	174
4. Ground Truth & Ground Truth Plus	175
5. Clarify	175
6. Model Cards (the SR 11-7 anchor)	175
7. A2I (Augmented AI)	175
8. 2024-2026 changes	175
9. Pitfalls	176
10. Capital One lens	176
11. Sanity check	176
12. Cross-references	176

Primary sources	176
Module 36 — SageMaker Training	177
1. Training Jobs — the core	177
2. Distributed training libraries	177
SMDDP (SageMaker Distributed Data Parallel)	177
SageMaker Model Parallel (SMP)	177
Native PyTorch FSDP / DeepSpeed	177
3. Spot training (SageMaker Managed Spot Training)	178
4. Warm pools	178
5. Trainium training	178
6. Heterogeneous Clusters	178
7. Training Compiler (deprecated)	178
8. MLflow integration	178
9. Entrypoint patterns	179
10. 2024-2026 changes	179
11. Pitfalls	179
12. Capital One lens	179
13. Sanity check	180
14. Cross-references	180
Primary sources	180
Module 37 — SageMaker Inference	181
1. The four inference modes	181
2. Real-time endpoints	181
3. Serverless inference	182
4. Async inference	182
5. Batch Transform	182
6. Multi-Model Endpoints (MME)	182
7. Multi-Container Endpoints	182
8. Inference Components (the 2024 cost story)	183
9. Inference Recommender	183
10. Shadow testing	183
11. Auto-scaling	183
12. Latency tuning	183
13. SageMaker LMI	183
14. 2024-2026 changes	184
15. Pitfalls	184
16. Capital One lens	184
17. Sanity check	184
18. Cross-references	184
Primary sources	184
Module 38 — SageMaker MLOps	186
1. SageMaker Pipelines	186
Python DSL	186
@step decorator (2024)	186
2. Step types	187
3. Projects (templated CI/CD)	187
4. Model Registry	187

5. Model Monitor	187
6. Lineage Tracking	188
7. Edge Manager (deprecated)	188
8. The MLOps lifecycle (full picture)	188
9. 2024-2026 changes	188
10. Pitfalls	189
11. Capital One lens	189
12. Sanity check	189
13. Cross-references	189
Primary sources	189
Module 39 — SageMaker JumpStart, Canvas, Autopilot	190
1. JumpStart	190
2. JumpStart vs Bedrock	190
3. Canvas	190
4. Autopilot (programmatic AutoML)	191
5. Decision matrix	191
6. Bedrock integration	191
7. Pitfalls	191
8. Capital One lens	192
9. Sanity check	192
10. Cross-references	192
Primary sources	192
Module 40 — SageMaker HyperPod	193
1. The problem HyperPod solves	193
2. Two flavors	193
Slurm-based HyperPod (original GA 2023)	193
EKS-based HyperPod (GA 2024)	193
3. Resilience features	193
4. HyperPod Recipes	194
5. Task Governance (2024)	194
6. FSx Lustre storage	194
7. Instance group definitions	194
8. Picking Slurm vs EKS-based HyperPod	194
9. 2024-2026 changes	194
10. Pitfalls	195
11. Capital One lens	195
12. Sanity check	195
13. Cross-references	195
Primary sources	195
Module 41 — SageMaker Networking & Security	196
1. VPC mode	196
2. No-internet egress mode	196
3. KMS encryption	196
4. Private SageMaker API endpoints	197
5. IAM execution roles	197
6. Cross-account ML patterns	197
7. Custom Studio images	197

8. Network restrictions for sensitive workloads	197
9. Internet-free Studio domains	197
10. Multi-account ML topology	198
11. 2024-2026 changes	198
12. Pitfalls	198
13. Capital One lens	198
14. Sanity check	198
15. Cross-references	198
Primary sources	199
Module 42 — Amazon Bedrock	200
1. The model catalog	200
2. Knowledge Bases (managed RAG)	200
3. Agents	201
4. Guardrails	201
5. Flows	201
6. Prompt Management	201
7. Model Distillation (GA Dec 2024)	201
8. Custom Models	201
9. Provisioned Throughput	202
10. Application Inference Profiles (2024)	202
11. Bedrock Data Automation (2024)	202
12. 2024-2026 changes	202
13. Pitfalls	202
14. Pricing examples (May 2026, us-east-1)	202
15. Capital One lens	203
16. Sanity check	203
17. Cross-references	203
Primary sources	203
Module 43 — Amazon Q Family	204
1. Q Developer	204
2. Q Business	204
3. Q Apps	204
4. Q in QuickSight	205
5. Q in Connect	205
6. Governance	205
7. 2024-2026 changes	205
8. Pitfalls	205
9. Capital One lens	205
10. Sanity check	206
11. Cross-references	206
Primary sources	206
Module 44 — Self-Hosted FM Training & Serving	207
1. When to self-host vs Bedrock	207
2. NVIDIA NIM (Inference Microservices)	207
3. NVIDIA Triton Inference Server	208
4. EC2 Capacity Blocks for ML	208
5. NVIDIA AI Enterprise on AWS	208

6. Nemo Guardrails	208
7. Self-hosted vector DBs (preview)	208
8. SageMaker HyperPod (the AWS-managed alternative)	208
9. EKS + KServe pattern (the Capital One way)	208
10. Cost math: self-hosted vs Bedrock	209
11. 2024-2026 changes	209
12. Pitfalls	209
13. Capital One lens	209
14. Sanity check	209
15. Cross-references	210
Primary sources	210
Module 45 — Databricks on AWS Architecture & Control Plane	211
1. The control plane / data plane split (recap from Topic 02)	211
2. AWS deltas from Azure	211
3. Control plane regions (AWS)	211
4. Classic vs Serverless compute	212
5. The MWS Account API	212
6. Deployment architecture (typical Capital One pattern)	212
7. 2024-2026 changes	213
8. Pitfalls	213
9. Capital One lens	213
10. Sanity check	213
11. Cross-references	213
Primary sources	213
Module 46 — Unity Catalog on AWS	214
1. Unity Catalog on AWS — what changes	214
2. Storage credentials	214
3. External locations	214
4. Self-assuming role mandate (Jan 2025)	215
5. Lake Formation interop	215
Pattern A: UC primary, no LF	215
Pattern B: LF primary, UC reads via Glue Catalog federation	215
6. Three-level namespace	215
7. Governance differences from Azure	215
8. Credential vending	216
9. Pitfalls	216
10. Capital One lens	216
11. Sanity check	216
12. Cross-references	216
Primary sources	217
Module 47 — Databricks Networking on AWS	218
1. AWS PrivateLink for Databricks	218
2. Secure Cluster Connectivity (SCC)	218
3. BYO VPC (Customer-managed VPC)	218
4. NAT Gateway requirements	219
5. Classic vs serverless data plane network paths	219
Classic	219

Serverless	219
6. Network Connectivity Configuration (NCC)	219
7. Cross-account considerations	219
8. Pitfalls	220
9. Capital One lens	220
10. Sanity check	220
11. Cross-references	220
Primary sources	220
Module 48 — EMR vs Databricks Decision	221
1. The four contenders	221
2. When EMR wins	221
3. When Databricks wins	222
4. Photon on AWS economics	222
5. Mosaic AI Vector Search on AWS	222
6. Cost-per-DBU on AWS	222
7. Decision tree	223
8. Pitfalls	223
9. Capital One lens	223
10. Sanity check	223
11. Cross-references	224
Primary sources	224
Module 49 — Databricks-AWS Admin & Ops	225
1. MWS Account API endpoints	225
2. Terraform two-provider pattern	225
3. Workspace deployment patterns	226
4. Billing and DBU economics	226
5. Audit log delivery	226
6. KMS key separation	226
7. Audit logs vs CloudTrail	226
8. Workspace deployment in CI	227
9. Operational runbooks	227
10. Pitfalls	227
11. Capital One lens	227
12. Sanity check	228
13. Cross-references	228
Primary sources	228
Module 50 — AWS Security Primitives	229
1. KMS (Key Management Service)	229
2. CloudHSM	229
3. Secrets Manager vs Parameter Store	230
4. Macie	230
5. GuardDuty	230
6. Inspector v2	230
7. Security Hub	230
8. AWS Config	231
9. IAM Access Analyzer	231
10. CloudTrail and CloudTrail Lake	231

11. 2024-2026 changes	231
12. Pitfalls	231
13. Capital One lens	232
14. Sanity check	232
15. Cross-references	232
Primary sources	232
Module 51 — Governance & Policy-as-Code	233
1. The governance landscape	233
2. SCPs and RCPs (recap)	233
3. Cloud Custodian — the Capital One project	233
c7n architecture	234
Common patterns	234
Dry-run discipline	234
Mode: deployment patterns	234
Why Capital One built this	234
4. cfn-guard	235
5. cdk-nag	235
6. CloudFormation Hooks	235
7. AWS Audit Manager	235
8. The four pillars (when to use what)	235
9. Pitfalls	236
10. Capital One lens	236
11. Sanity check	236
12. Cross-references	236
Primary sources	237
Module 52 — Financial-Services Compliance Lens	238
1. The regulatory stack	238
2. PCI-DSS v4 on AWS	238
3. SOC reports	239
4. FFIEC Cloud Computing Statement	239
5. OCC supervision	239
6. SR 11-7 — Model Risk Management	239
Mapping to AWS services	239
7. GLBA Safeguards Rule (FTC 2023 update)	240
8. Databolt-style tokenization	240
9. The 2019 Capital One breach postmortem	240
Each lesson → AWS feature	240
10. AWS Artifact	241
11. Capital One lens	241
12. Sanity check	241
13. Cross-references	241
Primary sources	241
Module 53 — The Capital One MLOps Spine	242
1. The full diagram	242
2. The named platform team — IFX	244
3. The rubicon-ml choice	244
4. Fraud-graph ML	244

5. The hybrid serving decision (SageMaker vs KServe)	245
6. Talking points for interviews	245
7. Cross-references	245
8. Sanity check	245
Primary sources	245
Module 54 — EKS for ML Serving — KServe Deep	247
1. The KServe story	247
2. InferenceService CRD	247
3. Runtimes	248
4. ModelMesh — multi-model serving	248
5. Deployment modes	248
6. KEDA — event-driven autoscaling	248
7. Karpenter for GPU	249
8. Istio service mesh	249
9. Gateway API (newer)	249
10. IRSA / Pod Identity	249
11. NVIDIA GPU Operator	250
12. Observability	250
13. Why Capital One picks KServe over SageMaker endpoints	250
14. KServe + Triton + NeMo Guardrails (the C1 Servicing Tool stack)	250
15. 2024-2026 changes	250
16. Pitfalls	251
17. Sanity check	251
18. Cross-references	251
Primary sources	251
Module 55 — CI/CD for ML on AWS	252
1. The CI/CD chains	252
GitHub Actions + OIDC pattern (the modern default)	252
2. CodeCommit deprecation note	253
3. SageMaker Pipelines vs Step Functions for ML CI/CD	253
4. Model registry promotion strategies	253
Manual approval	253
Automated approval gates	253
Progressive rollout	254
5. Packaging models	254
6. IaC patterns — CDK + Terraform	254
cdk-nag + cfn-guard in CI	254
7. Container security scanning	254
8. Secrets and config management	254
9. Automated rollback	255
10. 2024-2026 changes	255
11. Pitfalls	255
12. Capital One lens	255
13. Sanity check	255
14. Cross-references	255
Primary sources	256
Module 56 — Observability & Cost for AI Workloads	257

1. CloudWatch — the AWS-native backbone	257
2. X-Ray	257
3. AWS Distro for OpenTelemetry (ADOT)	257
4. AMP + AMG (Managed Prometheus + Managed Grafana)	258
5. Datadog and Splunk forwarding patterns	258
6. AWS Cost Anomaly Detection	258
7. Cost Explorer + Budgets + Budget Actions	258
8. CUR 2.0 + FOCUS 1.0	258
9. Application Signals	258
10. The SageMaker cost gotchas hit list	259
11. GenAI cost discipline	259
12. EKS and Lambda cost shapes	259
13. Capital One pattern — cost-optimizing GenAI	260
14. Slingshot insight (the Capital One product)	260
15. Pitfalls	260
16. Sanity check	260
17. Cross-references	261
Primary sources	261
Module 57 — AWS Cert Paths for Sr Lead AI/ML	262
1. Recommended sequence	262
2. Domain-to-module mapping	262
SAA-C03 (Solutions Architect Associate)	262
MLA-C01 (ML Engineer Associate)	263
DEA-C01 (Data Engineer Associate)	263
SCS-C03 (Security Specialty)	263
SAP-C02 (Solutions Architect Professional)	264
3. Study plan (46-week ladder)	264
4. Hands-on lab plan per cert	264
SAA-C03	264
MLA-C01	264
DEA-C01	264
SCS-C03	264
SAP-C02	265
5. Hands-on lab budget	265
6. Mock-exam strategy	265
7. Exam-day strategy	265
8. Recertification	265
9. Pitfalls	266
10. The Capital One signal	266
11. Sanity check	266
12. Cross-references	266
Primary sources	266
Appendix A — FACTS.md	267
Topic 04 — FACTS.md	268
Capital One on AWS	268
Capital One open-source AWS tooling	269
Capital One 2019 breach (compliance context)	269

AWS Certification — active (as of 2026-05-14)	269
AWS Certification — retired	270
Exam domain weightings (top three for this role)	270
Hands-on lab budget guidance	271
(Module-specific FACTS to be added as modules are written)	271
Appendix — Sources & References	272
1. Capital One — primary sources	272
Capital One Tech blog and corporate sites	272
Capital One open-source projects (read the code to learn their idioms)	272
Capital One job postings (signal what the role expects)	273
Conference presentations referenced	273
2. AWS official documentation (archived as PDFs/HTML)	273
AWS Well-Architected Framework	273
AWS Reference Architectures	273
Networking	274
Storage & Lake	274
Databases	274
Streaming	275
Compute	275
SageMaker (the biggest section)	275
Generative AI	276
Databricks on AWS	276
Security & Compliance	277
Observability & Cost	277
3. AWS Certifications (official exam guides)	278
4. Regulatory and standards documents	278
5. Open-source projects referenced	279
6. Internal research reports (in this repository)	279
7. Books, papers, and additional references	280
8. About archived documents	280
9. Caveats	281

How to read this ebook

This is the consolidated reading copy of **Topic 04 — AWS for AI/ML Engineers**, from the **Career_upskill** project. The source markdown files live at `topics/04_aws_for_ai_ml/` and remain the canonical version; this ebook is a derived artifact.

Audience: a Senior/Lead AI/ML engineer preparing for the AWS-native, regulated-finance environment at Capital One. Networking is taught from first principles. Database depth is maximized (12 dedicated modules). SageMaker gets 8 modules. Databricks-on-AWS gets a full 5-module Part since the user already has Azure Databricks (Topic 02) fluency.

Ordering is the natural numeric sequence — Modules 1 through 57, organized into 14 Parts (A–N).

What’s included: - All 57 conceptual modules - The Capital One dossier (interview/role-prep) - The FACTS.md appendix (citable facts, dates, costs) - A full Sources & References appendix

What’s NOT included: - Code artifacts (Terraform, KServe YAML, Custodian policies, Python pipelines — they run, they don’t read) - Quizzes in their original collapsible form - The Table of Contents file (this ebook’s auto-generated TOC supersedes it)

Diagrams are Mermaid where present. The EPUB build pre-renders them to PNG via the `mmdc` CLI. In the PDF they appear as code blocks unless `mmdc` is installed.

Cross-references in the source modules use relative paths like `[Module 42](42_bedrock.md)`. In this single-file ebook they appear as in-document links.

Capital One on AWS — Dossier for Sr Lead AI/ML Engineer

Audience: You — preparing for the Sr Lead AI/ML Engineer role at Capital One. **Purpose:** A single document you can read before an interview, recruiter call, or architecture discussion to ground yourself in Capital One’s AWS reality. **Source:** Curated from [../../../../research_inputs/04_aws_for_ai_ml/00_capital_one_aws_usage.md](#) and continually refined as more research lands. **Last updated:** 2026-05-14

Strategic posture in one paragraph

Capital One closed its last on-premises data center in **November 2020**, completing a five-year migration begun in 2015 and becoming the **first US bank to publicly run 100% on public cloud**. They are AWS-only for production — no Azure, no GCP, no hybrid story. Their commercial software arm (Capital One Software) sells **Slingshot** (Snowflake cost governance) and **Databolt** (vaultless tokenization for Redshift / Aurora / RDS / now unstructured GenAI data) — both betting on the AWS + Snowflake + Databricks stack. Their internal culture is “**serverless-first**” (Lambda + Step Functions + Fargate + DynamoDB), and they self-host AI/ML model serving on **EKS with KServe** rather than relying solely on SageMaker endpoints.

1. Strategic posture & migration history

Year	Event
2015	All-in-on-AWS strategy announced
2015	Hygieia released as their first open-source DevOps tooling (OSCON 2015)
2017	Eno (AI assistant) launched
2019	March/April breach: misconfigured WAF SSRF + permissive IAM → ~106M customer records exfiltrated from S3

Year	Event
2020 (Aug)	OCC \$80M fine; consent order outlines failures
2020 (Nov)	Last data center decommissioned; 41 tons of copper recycled
2021	Capital One Software arm launches; Slingshot enters market
2024	Eno upgraded to LLM-backed; re:Invent 2024 dominated by C1 sessions on cost-optimizing GenAI, IaC governance, FM deployment, serverless 10-year retrospective
2025	NVIDIA GTC EXS74442 — Generative AI Agent Servicing Tool; Eno adds Spanish
2026 (Mar)	Databolt expanded to unstructured data for GenAI

Implication for you: Capital One has lived the cloud-native journey at scale longer than almost any other regulated-finance org. They expect that you’ve internalized the discipline that comes from a 5-year migration and a public breach — not just feature familiarity.

2. AWS service-by-service map

Each row cites the primary public source. Where multiple talks mention a service, the most authoritative is named.

Compute & orchestration

Service	Capital One usage	Source
Lambda	Centerpiece of serverless-first strategy; re:Invent 2024 “10 years of pioneering serverless” with Catherine McGarvey	re:Invent 2024
Step Functions	Powers check-processing pipeline (80% latency reduction); orchestrates ML pipelines	re:Invent + tech blog
API Gateway + SAM	Public API surface for microservices	AWS case study
CodeDeploy	Gradual 2%/min traffic shifting on Lambda	re:Invent 2024

Service	Capital One usage	Source
Fargate, ECS	Containers without managing servers; pairs with Lambda	re:Invent 2024
EKS	Self-managed Kubernetes — explicit in MLE postings; backs KServe model serving	Workday R239877
Step Functions	ML orchestration alongside SageMaker Pipelines	C1 Sr Lead MLE JD

Data services

Service	Capital One usage	Source
S3	Primary data lake; target of 2019 breach (now wrapped in Cloud Custodian + KMS + access analyzer + Macie)	universal
DynamoDB	Operational KV, low-latency transactional store	AWS case study
Aurora, RDS	Relational, including Postgres + Oracle migration targets	AWS case study
DocumentDB	Document workloads	AWS case study
Redshift	Analytic warehouse	tech blog
Snowflake on AWS	Primary analytics data warehouse per HBR sponsor piece (2021); Slingshot targets it	Capital One Software docs
Databricks on AWS	ML/data engineering workloads; commercial Slingshot supports it	Sr Platform Eng JD

Streaming & event-driven

Service	Capital One usage	Source
Kinesis	Primary serverless event bus; internal SDK abstracts Kinesis + Lambda source/processing/sink	capitalone.com/tech/cloud/serverless-streaming
MSK / Kafka	Referenced in patterns; Kinesis is the named production service publicly	tech blog

ML/AI

Service	Capital One usage	Source
SageMaker	Named in re:Invent 2024 (“Reduce FM deployment costs and latency with SageMaker”, Grant Gillary); Sr Lead MLE JDs list it explicitly	re:Invent 2024 + JDs
EMR	Big-data feature engineering	Sr Lead MLE JD
Bedrock	re:Invent 2024 “Control the cost of your generative AI services” (Brent Segner)	re:Invent 2024
Amazon Q Business	Same talk	re:Invent 2024
QuickSight + Amazon Q	BI	re:Invent 2024
EKS + KServe	Self-hosted model serving — the differentiator	Workday R239877

Networking, IaC, governance

Service / Tool	Capital One usage	Source
Route 53	Multi-region active-active	AWS case study
CloudFormation + AWS CDK	Primary IaC for AWS resources	re:Invent 2024 (Ishu Gupta)
cdk-nag	CDK guardrails	same
cfn-guard	CloudFormation policy-as-code	same
CloudFormation hooks	Deployment-time policy enforcement	same
HashiCorp Terraform	“VPC bones” — networking is Terraform; app stacks are CDK	inference from JDs

3. Open-source signals (read the code, learn the idioms)

Project	Repo	What it reveals about Capital One
Cloud Custodian	github.com/cloud-custodian	YAML policy-as-code for AWS governance. CNCF Incubating (Apache 2.0). C1 enforces guardrails continuously at scale, integrated with Security Hub. <i>The single most important repo to read before interviewing.</i>
rubicon-ml	github.com/capitalone/rubicon-ml	Model experiment tracking with git integration. Designed for auditability and reproducibility — a direct response to SR 11-7 model risk management.
DataProfiler	github.com/capitalone/DataProfiler	Schema + statistics + entity extraction for sensitive-data scanning. ~1.6k stars. Used to identify PII before it lands in analytic systems.
datacompy	github.com/capitalone/datacompy	Dataframe diff across Pandas / Polars / Spark / Snowpark. Pipeline validation pattern.
locopy	github.com/capitalone/locopy	Redshift + Snowflake load/unload helper. Direct signal both warehouses are in production.
Hygieia	github.com/Hygieia	DevOps dashboard, OSCON 2015. Their earliest OSS release.
edgetest	github.com/capitalone/edgetest	Dependency-update testing — supply-chain hygiene.
Stratum-Observability	github.com/capitalone	Observability instrumentation library.

Action: before any interview, spend 90 minutes reading the Cloud Custodian docs and skimming three example policy YAMLs. You will impress interviewers by referencing the project by name and identifying which AWS APIs Custodian wraps.

4. AI/ML platform shape

Products users interact with

- **Eno** — AI assistant, launched 2017 (SMS / Alexa). LLM upgrade in 2024; Spanish in 2025. Credited with ~50% call-center volume reduction.
- **Generative AI Agent Servicing Tool** — internal tool that helps human call-center agents retrieve account info in real time. Presented at **NVIDIA GTC 2025** (session EXS74442) and re:Invent 2024.

Internal teams

- **Intelligent Foundations and Experiences (IFX)** — the named internal ML platform team per the Lead MLE job posting. This is who you'll work with or near.

MLOps spine (inferred from public talks + JDs)

1. **Feature engineering:** EMR + Glue + Spark
2. **Experimentation & training:** SageMaker (Training Jobs, Studio) + Databricks (distributed PyTorch / TensorFlow)
3. **Lineage & auditability:** rubicon-ml
4. **Orchestration:** Step Functions + Airflow (Kubeflow appears in JDs)
5. **Model serving:** **EKS** + **KServe** (their differentiator) + SageMaker endpoints (selectively)
6. **Governance:** Cloud Custodian + cfn-guard + cdk-nag + CloudFormation hooks
7. **Observability:** CloudWatch + likely Datadog (industry norm at scale)
8. **Tokenization for PII:** **Databolt** (their own product) — sits between data lake and analytical / ML systems

Public ML focus areas (per C1 Tech blog)

- **Graph ML for fraud** (Neptune + Neptune ML + custom transformers)
- **NLP for assistants** (Eno + Servicing Tool)
- **Explainable AI** (SR 11-7 alignment)
- **Anomaly detection** (fraud + ops)
- **Privacy-preserving ML**
- **ML at scale** (the platform problems IFX solves)

University partnerships

- Global graph transformers
- Dynamic customer embeddings

5. Regulatory & compliance posture

Capital One operates under **PCI-DSS, SOC 2, FFIEC, GLBA, OCC** oversight, plus the structural memory of the **2019 breach**. This shapes every public statement they make about AWS:

Regulation/Standard	What it constrains	How C1 implements on AWS
PCI-DSS	Cardholder data protection	Network segmentation via VPC + SGs; tokenization (Databolt); KMS for at-rest; TLS for in-transit
SOC 2	Operational controls	CloudTrail + CloudTrail Lake; Cloud Custodian; Audit Manager
FFIEC	Federal financial institution exam framework	Multi-region active-active (Route 53); documented recovery objectives
OCC	Bank supervision (the 2019 breach was their enforcement action)	Cloud Custodian for continuous compliance; IAM Access Analyzer; SCP-enforced guardrails
GLBA	Customer financial info privacy	Macie for sensitive-data discovery; tokenization
SR 11-7	Model risk management (Fed/OCC guidance)	rubicon-ml for lineage; Model Cards; Clarify for bias; documented validation processes

The 2019 breach lessons

The breach was a **misconfigured WAF SSRF** combined with **over-permissive IAM** that allowed an attacker to retrieve EC2 metadata credentials and exfiltrate ~106M customer records from S3. Everything in their public security posture is downstream of this:

- **Cloud Custodian** — *continuous* policy enforcement, not point-in-time audits
- **IMDSv2 enforcement** — the SSRF vector is dead with IMDSv2 token-required mode
- **IAM Access Analyzer** — surface unintended public/cross-account access
- **Macie** — alert if PII ends up in unexpected buckets
- **cfn-guard / cdk-nag / CloudFormation hooks** — catch the misconfiguration *before* deploy
- **Databolt** — even if exfiltrated, tokenized data is useless without the vault

Interview signal: if asked about the breach, do not name-drop it as “the Capital One incident.” Frame it as: “*the industry learned that SSRF + over-permissive instance profiles is the classic exfiltration chain; IMDSv2 + least-privilege roles + Access Analyzer + tokenization are the layered response.*”

6. 2024–2026 GenAI direction

The most useful single artifact is the re:Invent 2024 round-up page on [capitalone.com/tech: https://www.capitalone.com/tech/cloud/aws-reinvent-2024/](https://www.capitalone.com/tech/cloud/aws-reinvent-2024/)

Session theme	Speaker	What it tells you
Control the cost of your generative AI services	Brent Segner	Bedrock + Q Business cost governance is a <i>first-class</i> discipline
Reduce FM deployment costs and latency with SageMaker	Grant Gillary	SageMaker is their managed-inference path
10 years of pioneering serverless	Catherine McGarvey	Lambda + Fargate is core to their culture
Governance and security with IaC	Ishu Gupta	CDK + cdk-nag + cfn-guard + CloudFormation hooks
Advancing state-of-the-art science and AI in financial services	James Montgomery	They invest in research, not just productionizing
AI-driven value	Prem Natarajan + NVIDIA	NVIDIA partnership is real (likely G5/P5/Capacity Blocks)

GenAI stack signals

- **Bedrock** (managed FMs)
- **Amazon Q Business** (enterprise assistant)
- **EC2 GPU + NVIDIA AI Enterprise** (self-hosted FM workloads)
- **EKS + KServe** (self-hosted model serving)
- **Nemo Guardrails** (named in Sr Distinguished MLE preferred quals)
- **Cost governance** specifically called out as a discipline

Databolt expanded to unstructured data (March 2026)

This is the most recent public move — they’re productizing “**safe data for GenAI**”: keeping PII out of vector indexes, prompt logs, and fine-tuning corpora. If GenAI work is part of your scope, expect Databolt to come up.

7. Sr Lead AI/ML Engineer — hiring bar

What three current postings require

Posting	Tech surface called out
Sr Lead MLE — GenAI/Python/AWS	Lambda, Step Functions, EMR, SageMaker, S3; HuggingFace; vector DBs; Nemo Guardrails; LLM optimization; AWS Ultraclusters
Lead MLE — MLOps/KServe	KServe, Kubernetes (self-managed clusters!), PyTorch, TensorFlow on AWS; Kubeflow; Airflow; Docker; CI/CD
Sr Distinguished MLE — Personalization	Distributed computing 6–10+ years; explainable + responsible AI; production retraining; low-latency event-driven systems

Compensation bands (McLean, VA, 2026)

- **Sr Lead MLE:** \$229k – \$262k
- **Sr Distinguished MLE:** \$315k – \$359k

What to study (priority order)

1. **SageMaker** — every component (Training, Inference, Pipelines, Feature Store, Studio). Modules 34-41 in this topic.
2. **EKS + KServe** — Kubernetes for model serving, including IRSA, Karpenter, GPU operator. Modules 33 + 54.
3. **Lambda + Step Functions** — serverless-first patterns. Modules 32, 15.
4. **Networking** — VPC, PrivateLink, NAT GW patterns. Modules 5-9. *You said this is your gap; close it.*
5. **Cloud Custodian** — read policies, write one. Module 51.
6. **Bedrock + cost governance** — Modules 42, 56.
7. **rubicon-ml + Model Cards + Clarify** — model governance for SR 11-7. Module 35, 38.
8. **Databricks on AWS** — UC + PrivateLink + instance profiles. Modules 45-49.

Cert priorities (rank by signal-to-effort for this specific role)

1. **SAA-C03** — baseline credibility, recruiter-scannable
2. **MLA-C01** — strongest current AI/ML signal
3. **SCS-C03** — regulated-finance differentiator
4. **DEA-C01** — data platform fluency
5. **SAP-C02** — promotion lever toward Architect/EM
6. **AIP-C01** — capstone once stable

Interview talking points (memorize)

- “I know Capital One has been 100% on AWS since November 2020 — first US bank — and that the migration shaped a serverless-first culture I want to be part of.”

- “I’ve read through Cloud Custodian and understand why policy-as-code is the backbone of compliant infrastructure at scale.”
 - “I see the EKS + KServe path in your MLE postings — coming from Azure Databricks, I’m interested in how you split workloads between managed SageMaker endpoints and self-hosted KServe.”
 - “I’ve followed Databolt’s evolution from structured tokenization to unstructured-data support in March 2026 — the GenAI safety story matters.”
 - “Eno’s 2024 LLM upgrade and the Servicing Tool at NVIDIA GTC 2025 show production GenAI at scale in finance — that’s the work I want to do.”
-

8. Open questions to ask in interviews

Things public sources don’t fully reveal. Asking *one* of these well demonstrates depth:

1. **“How does the team decide between SageMaker endpoints and KServe-on-EKS for a given model?”** — they have both; the criteria are the interesting part.
 2. **“How is Unity Catalog (Databricks on AWS) reconciled with Lake Formation for the same S3 data?”** — there’s a known interop tension; how they solve it reveals their data-governance maturity.
 3. **“What’s the multi-region DR posture for SageMaker training and inference workloads?”** — public talks emphasize active-active for app tier, but ML platform DR is rarely discussed publicly.
 4. **“How does rubicon-ml integrate with Model Registry and Model Cards in practice?”** — they’re related but not identical; the integration is custom.
 5. **“What’s the breakdown of Bedrock vs Anthropic-direct vs self-hosted FM usage?”** — they use all three; the proportions reveal their tradeoffs.
 6. **“How is Databolt deployed at the data-plane layer — sidecar, library, gateway?”** — implementation detail not public.
 7. **“What’s Cloud Custodian’s coverage of EKS + Kubernetes resources today, given the legacy is AWS-resource policies?”** — Custodian’s reach into K8s is evolving.
 8. **“How is the IFX platform team structured — federated MLEs embedded in product teams, or centralized platform with self-service?”** — org design tells you what your day-to-day will look like.
-

9. Reading list (priority order)

Must-read before any interview

1. [Capital One AWS case study](#) — the executive narrative
2. [Capital One Tech — re:Invent 2024 roundup](#) — 2024 lineup
3. [Capital One Tech — AI page](#) — Eno + Servicing Tool + ML focus

4. [Capital One Tech — serverless streaming](#) — internal Kinesis SDK pattern
5. [Cloud Custodian docs](#) — at minimum, the AWS provider page and 5 example policies
6. [rubicon-ml README](#) — what it tracks, how it integrates with git

Strongly recommended

7. [Databolt for AWS product page](#) — tokenization story
8. [Capital One open-source org](#) — skim repo names
9. AWS Well-Architected Framework — Machine Learning Lens (PDF) — what AWS officially expects from an ML architect
10. AWS Well-Architected Framework — Security Pillar (PDF) — the foundation for regulated workloads
11. OCC consent order against Capital One (Aug 2020) — what the regulator actually wrote

Background (if time)

12. Re:Invent 2024 session videos — Brent Segner (GenAI cost), Grant Gillary (FM on SageMaker), Catherine McGarvey (serverless), Ishu Gupta (IaC governance)
13. NVIDIA GTC 2025 session EXS74442 — Generative AI Agent Servicing Tool
14. HBR 2021 piece (sponsored) on Capital One + Snowflake — strategic data architecture narrative

Cross-references in this topic

- Cloud Custodian deep-read: **Module 51** (Governance & policy-as-code)
- KServe + EKS production setup: **Module 33** (EKS foundations) + **Module 54** (KServe deep)
- Databolt-style tokenization patterns: **Module 52** (Financial-services compliance lens)
- rubicon-ml integration: **Module 38** (SageMaker MLOps) + **Module 53** (C1 MLOps spine)
- Snowflake on AWS strategic choice: **Module 23**
- Bedrock cost governance: **Module 42** + **Module 56**
- 2019 breach lessons (IMDSv2, Access Analyzer, etc.): **Module 2** (IAM deep) + **Module 50** (Security primitives) + **Module 52**
- Cert sequence: **Module 57**

Module 1 — AWS Account, Organizations & Landing-Zone Model

What this is: the multi-account topology that 99% of regulated-finance AWS estates use, why it exists, and how Capital One operates ~600+ accounts under it.

Why it matters: an AWS account is the security, billing, and quota boundary. “One big account with namespaces” does not survive regulatory audit. Architects are expected to fluently describe Organizations, OUs, Control Tower vs Landing Zone Accelerator, and the SCP/RCP model. This shows up in the SAA-C03 exam (security domain), SAP-C02 (heavily), and every Capital One architecture review.

Cert mapping: SAA-C03 (Security 30%), SAP-C02 (Org & multi-account heavily), SCS-C03 (governance).

1. The account is the boundary

A single AWS account is a hard security boundary. Inside one account, everything shares IAM, default VPCs, service quotas, and one root credential. You **cannot** subdivide an account into independent control planes — IAM policies, SCPs, and tagging are *not* substitutes for account separation.

The “many small accounts under one Organization” model has been the AWS best practice since at least 2018, codified in [Organizations_Multi_Account_Strategy.pdf](#). The pattern:

- **One management account** (formerly called the “master account”) — holds Organizations, billing, payer, IAM Identity Center, GuardDuty/SecurityHub/Config delegated admins. *Never* run workloads here.
- **One Log Archive account** — locked-down S3 bucket destination for org-wide CloudTrail and Config.
- **One Audit account** — read-only access for SecurityHub, GuardDuty, Access Analyzer findings aggregation.
- **N workload accounts** — one per environment-stage × business unit (or finer).

Capital One operates **600+ accounts** under this model. The exact count is not public, but their re:Invent talks confirm a hundreds-of-accounts estate organized by line of business and environment

tier.

2. AWS Organizations

AWS Organizations (GA 2017-02-28) provides the tree:

```
Root
├── Security OU
│   ├── Audit account
│   └── Log Archive account
├── Infrastructure OU
│   ├── Network account (Transit Gateway hub)
│   └── Shared Services account
├── Sandbox OU
├── Workloads/Non-Prod OU
│   ├── card-ml-dev
│   ├── card-ml-stage
│   └── ...
├── Workloads/Prod OU
│   ├── card-ml-prod
│   └── ...
├── Exceptions OU      ← regulated/exempted accounts
├── Suspended OU       ← lifecycle holding pen
└── PolicyStaging OU   ← rehearse SCP/RCP changes here
```

Limits: max **1,000 OUs** per org, OU nesting depth **5**, soft limit on accounts (raisable to thousands), max **5 SCPs per entity**, max policy size **5,120 bytes** (whitespace-stripped).

Pattern rule: organize by **policy zone**, not team. Teams move; policy zones don't. "Card-ML Team OU" is wrong; "Workloads/Prod OU" is right.

3. SCPs and RCPs

These are the two policy types that bound the *org*, not individual identities.

Service Control Policies (SCPs)

SCPs are **guardrails** that set the *maximum* permissions an IAM principal in a member account can have. **SCPs do not grant** — they are filters on what identity policies can grant. The classic SCP set for a regulated-finance estate:

- Deny region usage outside `us-east-1`, `us-east-2`, `us-west-2` (with narrow Bedrock exceptions when a model is only available elsewhere).
- Deny `ec2:RunInstances` unless `aws:RequestTag/CostCenter` is present.
- Deny `ec2:RunInstances` unless `MetadataHttpTokens=required` (mandates IMDSv2 — the post-2019-breach control).

- Deny disabling CloudTrail, GuardDuty, Config, Security Hub.
- Deny `s3:PutBucketPolicy` actions that grant `Principal: *` (block public S3).
- Deny use of the `root user` except via a specific MFA condition.

Resource Control Policies (RCPs)

RCPs (GA **2024-11-13**) are the dual of SCPs. SCPs constrain **principals**; RCPs constrain **resources**. Before RCPs, you could not centrally enforce “no S3 bucket in this org may be public” — SCPs only bound your own principals, not anonymous or cross-account callers hitting your resources.

RCPs initially cover **S3, SQS, KMS, Secrets Manager, STS**, with more services planned. They attach to `root/OU/account` like SCPs. A typical RCP:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyExternalAccessToS3",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": "*",
    "Condition": {
      "StringNotEqualsIfExists": {
        "aws:PrincipalOrgID": "o-xxxxxxxxxx"
      },
      "BoolIfExists": {
        "aws:PrincipalIsAWSService": "false"
      }
    }
  }]
}
```

This single RCP would have prevented the 2019 Capital One breach exfil step — the attacker’s principal was outside the org.

4. Control Tower vs Landing Zone Accelerator

Two AWS-provided multi-account orchestrators. They are not interchangeable.

	AWS Control Tower	Landing Zone Accelerator (LZA)
GA	2019-06-24	2022-03-22
Form	Managed AWS service	Solutions Library reference deployment (CDK)
Config	Console-driven	YAML declarative

	AWS Control Tower	Landing Zone Accelerator (LZA)
Audit profiles	Generic	PCI-DSS, HIPAA, NIST 800-53, CMMC, CIS, PBMM, CCCS Medium
Partitions	aws only	aws, aws-us-gov, aws-cn
Ceiling	~300 accounts smoothly	Thousands
Customization	Account Factory Customization (AFC), AFT	YAML, full CDK extension
Who picks it	SMB, mid-market, simple orgs	Banks, federal, healthcare

Capital One predates Control Tower and runs a custom account-vending pipeline closer in spirit to LZA. New regulated-finance shops in 2026 building from scratch should look at **LZA + AFT (Account Factory for Terraform)** rather than Control Tower.

5. IAM Identity Center

Renamed from “AWS SSO” on **2022-07-26**. The workforce-identity layer:

- SAML/OIDC federation to Okta, Azure AD, Ping, Google Workspace.
- **Permission sets** = templated IAM roles, materialized in each member account as `AWSReservedSSO_<Name>_<hash>` roles.
- **Account assignments** — which users/groups get which permission set in which accounts/OUs.
- **Session policies** — set at AssumeRole time, scope down further.
- **Delegated administration** — run Identity Center from a non-management account (almost always required).
- **Trusted identity propagation** (GA 2023-11; expanded 2024-25) — lets QuickSight, Redshift, EMR, S3 Access Grants, Athena consume the *user’s* identity end-to-end. Big for fine-grained data access in banking.

The rule: **zero IAM Users for humans**. Identity Center for all human access. IAM Users only for break-glass and the small set of third-party integrations that cannot federate.

6. 2024–2026 changes worth knowing

- **RCPs GA** (2024-11-13) — resource-perimeter enforcement.
- **Declarative policies** (re:Invent 2024) — enforce account-wide defaults (IMDSv2, EBS encryption, default-VPC blocking) that survive user toggling.
- **AI services opt-out policies** (2024) — control Bedrock/Q data collection at the org level.
- **Centralized root access management** (GA 2024-11) — remove root credentials from member accounts entirely; assume root from management when needed.
- **Control Tower Account Factory Customization (AFC)** GA; **AFT (Account Factory for Terraform)** widely adopted.

- **Bedrock cross-region inference profiles** (2024-08) — service-specific multi-region access pattern relevant for region-locked models.

7. Pitfalls and anti-patterns

- Running workloads in the **management account**.
- **OU = team** instead of **OU = policy zone**.
- Treating SCPs as grants. They are filters.
- Skipping the **delegated admin** pattern for Identity Center, GuardDuty, Config, Security Hub — leaves the management account doing operational work.
- **Single CloudTrail in the management account** with no org-trail — tamper-evident logging gone if mgmt-account access is compromised.
- IAM Users for humans (despite SSO existing).
- Buying RIs across the org and discovering RI sharing pooled the discount somewhere unexpected.

8. Capital One lens

Capital One’s public reference architecture and re:Invent talks describe:

- **600+ AWS accounts** with custom landing zone predating Control Tower.
- **Per-LOB OUs nested under environment-tier OUs** — Card-ML-Prod, Auto-ML-NonProd, etc.
- **Account-vending automation** (similar in spirit to AFT) — new accounts spawn with org-baseline policies, IAM Identity Center permission sets, mandatory tags, and a default VPC topology.
- **Mandatory SCPs**: deny non-US regions, deny root, deny IMDSv1, deny non-Capital-One-approved AMIs.
- **Hub-and-spoke Transit Gateway** in a dedicated Networking account.
- **RCPs adoption** — natural fit to harden the S3 estate against the historical SSRF exposure pattern.

Interview talking point: *“I’d expect Capital One to run delegated admin from the Security OU for GuardDuty + Config + Access Analyzer + Security Hub, with org-trail in CloudTrail to a locked Log Archive account, and RCPs on S3/KMS to enforce `aws:PrincipalOrgID` boundaries — those are the post-2019 baseline.”*

9. Sanity check

1. What’s the difference between an SCP and an RCP, and why couldn’t SCPs alone prevent the 2019 Capital One breach exfiltration?
2. When would you pick Landing Zone Accelerator over Control Tower?
3. Why is “OU = team” wrong?
4. What is delegated administration and why is it almost always required?
5. What does centralized root access management (2024) actually do?

10. Cross-references

- **Module 2** — IAM deep, IRSA, the 2019 breach lessons in IAM detail
- **Module 6, 8** — VPC + multi-VPC + hub-and-spoke TGW in a Networking account
- **Module 51** — Cloud Custodian (Capital One's own org-governance OSS) layered on top of SCPs/RCPs
- **Module 57** — exam-domain mapping for SAA-C03 and SAP-C02

Primary sources

- [Organizations_Multi_Account_Strategy.pdf](#) — official multi-account whitepaper
- [AWS_SRA.pdf](#) — AWS Security Reference Architecture
- [Control_Tower_UserGuide.pdf](#)
- [Landing_Zone_Accelerator.html](#)
- [RCP_announcement.html](#) — 2024-11-13 RCP launch
- Research report: [02_aws_foundations.md](#)

Module 2 — IAM Deep

What this is: the IAM model, the six-layer policy evaluation chain, the IMDSv2 / Access Analyzer / IRSA / Pod Identity story, and the 2019 Capital One breach in IAM detail.

Why it matters: every Capital One architecture review starts with IAM. The 2019 breach was an IAM failure. Every cert in the recommended ladder (SAA, MLA, DEA, SCS, SAP) tests IAM heavily. As a Sr Lead at Capital One, you will be expected to design role boundaries that survive a regulator audit and to spot over-permissive policies in code review.

Cert mapping: SAA-C03 (Security 30%), MLA-C01 (Domain 4 ML Security), DEA-C01 (Data Security 18%), SCS-C03 (heavily), SAP-C02.

1. The six-layer policy evaluation chain

When a principal makes a request to an AWS API, IAM evaluates a chain in roughly this order:

1. **SCP** — Org-level, bounds member-account principals.
2. **RCP** — Org-level, bounds resource access (GA 2024-11-13).
3. **Resource policy** — Attached to the resource (S3 bucket policy, KMS key policy, IAM role trust policy).
4. **Identity policy** — Attached to the principal (managed, customer-managed, or inline).
5. **Permissions boundary** — Caps the principal's *effective* permissions.
6. **Session policy** — Set at AssumeRole time, scopes further down.

The rules: - The request succeeds **only if every applicable layer allows it**. - An **explicit deny** anywhere wins. Always. - **Implicit deny** is the default — if no layer explicitly allows, it's denied. - **Permissions boundaries do not apply to resource policies** — a bucket policy can still grant access wider than the boundary intends.

2. Users, roles, and what to use when

Construct	Lifetime	Use case
IAM User	Long-lived (access key)	Break-glass + third-party integrations that can't federate. Near zero count in a regulated org.
IAM Role (assumed via STS)	Short-lived session (1h default, max 12h)	Everything else — humans (via Identity Center), workloads (via IRSA/Pod Identity/IAM Roles Anywhere), cross-account, federation
Managed policy (AWS-managed)	Versioned by AWS	Quick start; never use <code>*:FullAccess</code> in production
Managed policy (customer-managed)	You version	The default for reusable identity policies
Inline policy	Anonymous, attached directly	Avoid — no reuse, hard to audit

3. ABAC vs RBAC at scale

RBAC = role per persona $\times$ per environment. At 600 accounts $\times$ 20 personas $\times$ 4 environments $\times$ 30 services, RBAC explodes combinatorially. Capital One-scale orgs cannot operate it.

ABAC tags principals and resources (e.g., `cost-center=card-ml`, `data-classification=restricted`, `env=prod`) and uses condition keys like `aws:PrincipalTag/...` and `aws:ResourceTag/...`:

```
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "aws:ResourceTag/CostCenter": "${aws:PrincipalTag/CostCenter}"
    }
  }
}
```

This single policy lets a Card-ML principal read only Card-ML resources. The number of policies stays $O(1)$ as the org scales. Capital One heavily favors ABAC for workload access; RBAC remains for human personas via Identity Center permission sets.

4. STS endpoints, IRSA, Pod Identity, IAM Roles Anywhere

STS endpoints — regional (`sts.us-east-1.amazonaws.com`) is preferred. The global endpoint is a SPoF and slower in many regions. Make global STS tokens valid in all regions only if you explicitly opt in.

IRSA (IAM Roles for Service Accounts) — GA **2019-09-03** for EKS. Maps a K8s ServiceAccount to an IAM role via OIDC federation. The pod gets short-lived credentials via projected service-account token. Replaced kiam/kube2iam.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: card-ml-trainer
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/CardMLTrainerRole
```

The role’s trust policy must allow `sts:AssumeRoleWithWebIdentity` for the cluster’s OIDC provider, conditioned on the namespace+SA pair.

EKS Pod Identity — GA **2023-11-26**. Simpler alternative using an EKS-managed agent instead of OIDC + cluster trust policy. Easier cross-account, fewer trust-policy edits at scale. New EKS clusters should default to Pod Identity in 2026 unless you have specific OIDC requirements.

IAM Roles Anywhere — GA **2022-07-13**. On-prem servers or non-AWS workloads obtain temporary AWS creds via X.509 certs signed by a CA you register as a Trust Anchor. The replacement for “store an IAM access key in a bare-metal datacenter.”

5. The 2019 Capital One breach in IAM detail

This is the canonical IAM teaching case study. Disclosed **2019-07-29**; \$80M OCC civil penalty **2020-08-06**; \$190M class-action settlement Dec 2021.

Chain of failure:

1. A **WAF / reverse-proxy EC2 instance** had a role with permissions to `s3:ListBucket` and `s3:GetObject` against customer-data buckets — way too broad.
2. The attacker exploited an **SSRF vulnerability** in the WAF to make the instance call `http://169.254.169.254/latest/meta-data/iam/security-credentials/...` — the **IMDSv1** endpoint.
3. **IMDSv1** returned the role’s temporary credentials with no token/header challenge.
4. Attacker exfiltrated ~100 GB from S3 across ~106 million customer records.

Defenses now considered table stakes (every one of these is something a Capital One architecture review will check):

- **IMDSv2 required** — token-based, hop-limit=1. Enforce via SCP `Deny ec2:RunInstances` unless `MetadataHttpTokens=required`. Default-on for new instance types from **2024-03**. Declarative policies (2024) make this account-level default.
- **VPC endpoint policies** on S3 — restrict bucket access to org-owned principals only via `aws:PrincipalOrgID`.
- **S3 Block Public Access** at account *and* org level (now via RCP).
- **Least-privilege role**: scope to a single bucket prefix, not `s3:*`.

- **GuardDuty** detects credential exfil patterns (`UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration, Recon:IAMUser/MaliciousIPCaller.Custom`).
- **No EC2 instance role with broad S3 access** — instead, use signed S3 presigned URLs from a control plane.

Interview framing: name the chain (SSRF → IMDSv1 → role creds → broad S3 grant) not the company. The industry learned this; everyone uses these defenses now.

6. IAM Access Analyzer

GA **2019-12-02**. Uses **Zelkova** (automated reasoning / SMT) to *prove* whether a resource policy permits external access.

2023–24 additions worth knowing: - **Unused access findings** (GA 2023-11-26) — flags unused IAM users, roles, role permissions, access keys. - **Custom policy checks** (GA 2024) — CI gate: “does this PR grant new permissions?” Reject if so. - **Internal access analyzers** (GA 2024) — scan within the org boundary, not just external. - **Policy generation from CloudTrail** — auto-generate scoped policies based on what a role actually used.

This is the engine that lets a CI pipeline say “this PR introduces over-broad permissions” without manual review.

7. Permission Boundaries (the delegated-admin enabler)

A **permissions boundary** is an advanced feature that caps the *effective* permissions of a user or role regardless of identity policy. Heavily used for **delegated administration**: developers can create roles, but only within the boundary you defined.

Critical pattern in CI/CD where pipelines mint application roles:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "iam:CreateRole",
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "iam:PermissionsBoundary":
          "arn:aws:iam::123456789012:policy/CardMLAppBoundary"
      }
    }
  }]
}
```

Without this condition, your pipeline can create god-mode roles. With it, every role the pipeline mints is hard-bounded by `CardMLAppBoundary`.

Note: boundaries apply to the principal, not to resource policies. A bucket policy can still grant access wider than the boundary suggests.

8. 2024–2026 changes worth knowing

- **EKS Pod Identity** preferred over IRSA for new clusters (2024+).
- **Access Analyzer unused-access** and **policy generation from CloudTrail** widely adopted; integrated with Identity Center to flag dormant permission sets.
- **Centralized root management** via Organizations (2024-11) lets you delete root creds on member accounts.
- **IAM Identity Center → trusted identity propagation** for S3 Access Grants, Redshift, Athena (2024) — finally true end-user identity propagation, not role-pooling.
- **Verified Permissions** (Cedar policy language) — maturing as the answer for **app-layer authz**, separate from IAM. AWS’s pitch for in-app authorization.

9. Pitfalls and anti-patterns

- `"Action": "*", "Resource": "*"` in identity policies — the #1 finding in any audit.
- **IMDSv1 enabled** — must die.
- **Wildcard trust policies** (`"Principal": "*"`) without a condition.
- **Cross-account roles** without `aws:PrincipalOrgID` or `sts:ExternalId`.
- Long-lived access keys in CI/CD instead of OIDC federation (GitHub Actions OIDC → `AssumeRoleWithWebIdentity` is the modern pattern).
- Forgetting **permission boundaries don’t bind resource policies** — a bucket policy can still grant wider access.
- Letting `AWSReservedSSO_*` roles accumulate from defunct permission sets.

10. Capital One lens

Post-breach, Capital One is publicly on record (re:Inforce 2021/2022 talks, “*Building security with developer velocity*”) about:

- **Org-wide IMDSv2 enforcement** via SCP.
- **Mandatory permission boundaries** on all dev-created roles.
- **Centralized Access Analyzer** with policy-as-code gates in CI.
- **Cedar / Verified Permissions** for in-app authz.
- **Zero IAM users for humans** — Identity Center for all human access.
- **Internal “paved road” pipeline** that mints IRSA / Pod Identity roles automatically with prefix-scoped S3 grants.
- **Cloud Custodian** policies (their OSS) detect drift from these baselines and auto-remediate or alert.

Talking point: *“My first IAM check on any AWS architecture is the metadata hop limit on EC2 — if MetadataHttpTokens isn’t ‘required’ the design is incomplete.”*

11. Sanity check

1. What is the six-layer IAM evaluation chain, and which type of “deny” always wins?
2. Walk through the 2019 breach attack chain in IAM terms. Name three controls that would have prevented it.
3. When should you pick EKS Pod Identity over IRSA?
4. What does Zelkova do, and what’s the practical value of “custom policy checks” in 2024?
5. Why is `aws:PrincipalOrgID` a more useful condition than `aws:SourceAccount` for blocking cross-org access?

12. Cross-references

- **Module 1** — Organizations, SCPs, RCPs (the org-level container for IAM)
- **Module 33** — EKS, where IRSA/Pod Identity live
- **Module 50, 52** — KMS key policies, the breach response, compliance
- **Module 51** — Cloud Custodian for IAM drift detection
- **Module 55** — GitHub Actions OIDC federation pattern

Primary sources

- [IAM_Best_Practices.html](#)
- [AWS_SRA.pdf](#) — Security Reference Architecture
- [SCS-C03_Exam_Guide.pdf](#)
- Research report: [02_aws_foundations.md](#)
- [CAPITAL_ONE.md](#) — 2019 breach context

Module 3 — Regions, Availability Zones, Edge

What this is: AWS's global infrastructure — partitions, regions, AZs, Local Zones, Outposts, Wavelength, and edge — with the regional quirks that bite ML workloads.

Why it matters: SageMaker, Bedrock, Trainium, and HyperPod availability varies sharply by region. You will design multi-region active-active for tier-0 systems, deal with Bedrock model-access requests per region/account/model, and explain the difference between an AZ letter and an AZ ID to people who think they're the same.

1. Partitions — the highest boundary

AWS is divided into **partitions** — entirely separate trust roots and ARN namespaces:

Partition	ARN prefix	What it is
aws	arn:aws:...	Commercial, 34 regions as of 2026-05
aws-us-gov	arn:aws-us-gov:...	GovCloud (West, East). FedRAMP High, ITAR.
aws-cn	arn:aws-cn:...	China (Beijing operated by Sinnnet, Ningxia by NWCD). Separate console, separate accounts.
aws-iso, aws-iso-b, aws-isof	(restricted)	Air-gapped for U.S. intelligence community

You **cannot** federate identity, peer VPCs, or replicate S3 across partitions. An account in `aws` is a different universe from an account in `aws-us-gov`.

2. Regions and AZs

A **Region** is a geographic area with **3–6 Availability Zones**.

An **AZ** is one or more discrete data centers with independent power, cooling, and networking, interconnected via sub-millisecond fiber to the others in the region.

AZs are randomized per account. `us-east-1a` for your account may be a different physical AZ than `us-east-1a` for someone else's. Use **AZ IDs** (`use1-az1`, `use1-az2`, etc.) when correlating across accounts (e.g., when peering or sharing subnets via RAM).

```
aws ec2 describe-availability-zones --query "AvailabilityZones[][ZoneName,ZoneId]"
# Returns the mapping for your account
```

3. Edge, Local Zones, Outposts, Wavelength

- **Edge locations** — 600+ PoPs for CloudFront, Route 53, Global Accelerator, WAF.
- **Local Zones** — single-AZ extensions of a parent region in metro areas (LA, Boston, Chicago, Phoenix, Las Vegas, Atlanta, etc.). Sub-10 ms latency to specific cities. Limited service catalog: EC2, EBS, VPC, ELB, RDS, ECS/EKS data plane. Use for low-latency to a metro that's not a region.
- **AWS Outposts** — AWS-managed hardware shipped to your data center. Two flavors: Outposts rack (42U) and Outposts servers (1U/2U). Same APIs as the region; control plane stays in AWS, data plane is on-prem. Used by banks for trading-floor latency and data-residency that even GovCloud can't satisfy.
- **AWS Wavelength** — EC2/EKS embedded in carrier 5G networks (Verizon, KDDI, Vodafone). Niche for ML inference at carrier edge (AR/VR, V2X).

4. Regional service quirks (the ML-specific ones)

This is where engineers actually get burned:

- **Bedrock model availability** varies sharply by region. Anthropic Claude Opus is in `us-east-1`, `us-west-2`, `eu-central-1`, etc. — but not all models everywhere. **Cross-region inference profiles** (GA 2024-08) bridge the gap by routing requests to whichever region currently has capacity for that model.
- **SageMaker HyperPod, Trainium / Trainium2, P5/P5e**, and **Capacity Blocks** are only in select regions. Plan training-cluster location early.
- **IAM is global** but its data plane has regional replicas. STS, ACM, Route 53 have regional/global modes — be deliberate.
- **CloudFront, WAF Global, Shield** are edge-attached; their config lives in `us-east-1` even when serving globally.
- **us-east-1** has the broadest service set *and* the worst blast radius — the December 2021 outage took down half the internet. Banks treat `us-east-1` as a hot region but actively design for `us-east-2` and `us-west-2` failover.

- **Opt-in regions** (Hong Kong, Bahrain, Cape Town, Milan, Jakarta, UAE, Hyderabad, Zurich, Spain, Tel Aviv, Melbourne, Calgary, Malaysia, Thailand, Mexico, Taipei) must be explicitly enabled per account. In a regulated org these are SCP-controlled.

5. VPC endpoints and PrivateLink (preview)

(Full treatment in Module 8.)

- **Gateway VPC endpoints** — free; only S3 and DynamoDB. Routed via prefix list in the route table.
- **Interface VPC endpoints** — ENI-based. ~\$0.01/hr per endpoint per AZ + \$0.01/GB. The currency of “S3/KMS/Bedrock/SageMaker traffic must stay on AWS backbone.”
- **Endpoint policies** + `aws:PrincipalOrgID` — the post-2019 baseline: lock down endpoint usage to org-owned principals only.

6. 2024–2026 changes worth knowing

- **New regions GA:** Malaysia (`ap-southeast-5`) 2024-08, Mexico (`mx-central-1`) 2025-01, Thailand (`ap-southeast-7`) 2025-01, Taipei (`ap-east-2`) GA 2025.
- **Bedrock cross-region inference profiles** GA 2024-08.
- **Cross-region PrivateLink** GA 2024-06 — interface endpoints can target services in another region without VPC peering.
- **Outposts servers** broader instance type support.

7. Pitfalls and anti-patterns

- Treating **AZ letter as identity** across accounts. Use AZ IDs.
- Assuming a service is in every region. **Always check the regional services table.**
- Forgetting Bedrock model access is **per-region, per-account, per-model**. Capital One’s account-vending pipeline has to handle these requests programmatically.
- Running production in **only us-east-1**.
- Forgetting that CloudFront/WAF Global config is “global” but anchored in **us-east-1**.
- Pinning data residency to a partition you can’t actually use (a Brazilian bank can’t suddenly use `aws-cn`).

8. Capital One lens

Capital One operates primarily in **us-east-1**, **us-east-2**, **us-west-2**. Multi-AZ across 3 AZs for every prod workload, **active-active across regions** for tier-0 systems.

Heavy PrivateLink usage — S3, KMS, Bedrock, SageMaker runtime endpoints all flow over PrivateLink with endpoint policies pinning `aws:PrincipalOrgID`.

SCPs deny `ec2:RunInstances` outside the allowed three regions, with narrow Bedrock-only exceptions

when a model is region-locked. They likely manage Bedrock model-access requests via automation since they're in production with many model variants.

9. Sanity check

1. Why is `us-east-1a` not the same as another account's `us-east-1a`?
2. Name three ML services whose regional availability you'd check before designing a new training pipeline.
3. What's the difference between Local Zones and Outposts?
4. Why is `us-east-1` simultaneously the most-used and most-feared region?
5. What does Bedrock cross-region inference profiles solve?

10. Cross-references

- **Module 4** — billing, where partition + region differences manifest in invoices
- **Module 8** — VPC peering, Transit Gateway, PrivateLink (incl. cross-region)
- **Module 30** — EC2 instance types per region (P5e/Trainium availability)
- **Module 36, 40** — SageMaker training, HyperPod regional considerations
- **Module 42** — Bedrock model regional availability

Primary sources

- AWS Global Infrastructure page (live)
- Research report: [02_aws_foundations.md](#)

Module 4 — Billing, Pricing Models & Tagging Discipline

What this is: the pricing models that actually matter (Savings Plans, Spot, On-Demand), Cost Explorer + Budgets + Cost Anomaly Detection, CUR 2.0, and tagging discipline as an enforceable policy.

Why it matters: Capital One's annual AWS spend is estimated at \$1B+. At that scale, FinOps is a dedicated function and SageMaker cost gotchas (idle endpoints, NAT GW, Capacity Blocks commitments) routinely run into seven figures. As a Sr Lead, you will design pipelines under Compute Savings Plan commitments, defend Spot-vs-OD choices, and explain why an untagged GPU instance costs the team \$30k/month.

Cert mapping: SAA-C03 (Cost-Optimized Architectures 20%), SAP-C02 (cost governance heavily), DEA-C01 (operations 22%).

1. Pricing models for compute

The choices matter most for ML — training and inference are where the bill lives.

Model	Commit	Max discount	Flexibility
On-Demand	none	0%	full
Reserved Instance (Standard)	1y / 3y, no/partial/all upfront	~72% (3y all-upfront)	locked to instance family/region/tenancy
Reserved Instance (Convertible)	1y / 3y	~54%	exchangeable to any family
Compute Savings Plan	1y / 3y hourly \$-commit	~66%	EC2 + Fargate + Lambda; any region, family, OS, tenancy
EC2 Instance Savings Plan	1y / 3y hourly \$-commit	~72%	locked to instance family + region; OS/size flexible

Model	Commit	Max discount	Flexibility
SageMaker Savings Plan	1y / 3y \$-commit	~64%	SageMaker training, inference, processing, notebook
Spot	none	~90%	interruptible (2-min warning)

The rule: Compute Savings Plans dominate in practice. Same max-discount tier as Convertible RIs but with Lambda/Fargate coverage and no exchange ceremony. Use EC2 Instance SPs only when you have rigid steady-state demand on one family.

Spot for ML training is real money — large-scale checkpointed training (SageMaker Managed Spot Training, EKS Karpenster with spot pools) routinely saves 70-90%. Spot interruptions on p4d/p5 GPU instances are not theoretical (10-20% in busy regions), so you need checkpointing every 10-30 minutes. SageMaker Managed Spot Training handles this for you.

2. The cost-management toolkit

Cost Explorer

UI + API (`ce:GetCostAndUsage`). 12 months default, up to **38 months** retention. **Hourly granularity** for the last 14 days. The first place to look when a bill spikes.

AWS Budgets

Alert on actual or forecasted spend. **Budget Actions** — auto-stop EC2/RDS, auto-apply an SCP, auto-disable an IAM policy. Underused feature: a \$100 dev-account budget action that suspends instances when crossed is a fantastic guardrail.

AWS Cost Anomaly Detection

GA **2020-12**. ML-based detection on cost/usage segmented by service, account, tag, or cost category. The cheapest insurance for catching a runaway SageMaker endpoint left on overnight. Set it up day one in every account.

AWS Cost Categories

Virtual dimensions that group accounts/tags/services into a logical rollup (e.g., “Card-ML-Platform”). Compose into Cost Explorer for chargeback that doesn’t depend on perfect tag discipline.

Cost and Usage Report (CUR / CUR 2.0)

Parquet to S3, hourly granularity, every column. **CUR 2.0** (GA **2024-06**) is the new schema with split cost allocation for shared services (e.g., EKS containers, ECR) and **FOCUS 1.0**-compliant exports

(FinOps Open Cost & Usage Specification). Capital One almost certainly pipes CUR 2.0 to Snowflake or Redshift for per-LOB chargeback dashboards.

3. Consolidated billing

Organizations automatically aggregates invoices to the management (payer) account. Three knobs:

- **Volume discounts** on S3, data transfer, and CloudFront — tier across the entire org.
- **RI sharing** — RIs and Savings Plans pool across accounts by default. Can be configured per-account (RI sharing on/off). Useful for: keeping one team’s discount from being absorbed by another.
- **Credits** — applied org-wide unless pinned to specific accounts.

4. Tagging discipline (as policy)

Tagging is not optional. Untagged resources are the largest cost-allocation hole — typically **15-30%** “unallocated” without enforcement.

Mandatory tag taxonomy for a regulated finance org:

Tag	Values
CostCenter	numeric code
BusinessUnit	card, auto, retail, ...
Project	free-form
Environment	prod / nonprod / sandbox
Owner	email
DataClassification	public / internal / confidential / restricted
Compliance	pci / sox / glba / none
AutoShutdown	true / false
BackupPolicy	tier-0 / tier-1 / tier-2 / none

Enforce via:

- **Tag Policies** (Organizations feature) — define required keys and allowed values.
- **SCPs** — Deny `ec2:RunInstances` if `RequestTag/CostCenter` is missing.
- **AWS Config rules** — `required-tags`, with auto-remediation via SSM Automation.
- **Resource Groups Tagging API** — for retro-tagging existing resources.
- **Cloud Custodian** (Capital One’s OSS) — `mark-for-op` policies that warn at T-7 days and stop at T-0.

Tags must be **activated** in the Billing console before they appear in Cost Explorer / CUR. There’s a 24-hour lag.

5. ML-specific cost gotchas

The line items that blow up bills for ML teams:

Line item	Why it's expensive	Fix
Idle SageMaker Studio domain	Always-on storage + IDE compute	Lifecycle hooks to shut down idle apps
Idle SageMaker real-time endpoint	\$/hr per instance even at 0 RPS	Auto-scaling to 0 (serverless inference) or scheduled shutdown
Idle SageMaker Notebook Instance	\$/hr forever	Auto-shutdown lifecycle config (well-known script)
NAT Gateway	\$0.045/GB + \$0.045/hr per AZ	VPC endpoints for AWS services (S3 + DynamoDB gateway endpoints are free)
EC2 Capacity Blocks	Reserved GPU windows — pay even if unused	Cancellation policy + accurate forecasting
Cross-AZ data transfer	\$0.01-\$0.02/GB	Place training datasets in the same AZ as training instances
S3 Standard for cold training data	1.5-3x the cost of IA or Glacier	Lifecycle policies; archive raw inputs once features are computed
PrivateLink endpoint hours	\$/hr × every region × every service × every AZ	Consolidate, share endpoints across accounts via RAM

6. 2024–2026 changes worth knowing

- **CUR 2.0 GA 2024-06** with split cost allocation for shared services (EKS containers, ECR pull through).
- **FOCUS 1.0** published **2024-06-13**; AWS FOCUS export GA same window.
- **Savings Plans for SageMaker** expanded coverage 2024.
- **Bedrock cost allocation by inference profile / tenant** improvements 2024-25 (Application Inference Profiles).
- **Compute Optimizer for SageMaker** (newer recommendations).
- **AWS re:Post billing console redesign** 2024.

7. Pitfalls and anti-patterns

- **Untagged resources** — 15-30% unallocated bill at most orgs without enforcement.
- **Tag drift** between IaC and console mutations.
- Buying **RIs without first analyzing SP fit** — you almost always want Compute SPs now.

- Forgetting that **Savings Plans** apply to compute usage across the org payer, so a high-discount account can “absorb” another account’s bill — desirable for pooling, confusing for charge-back.
- **Idle SageMaker endpoints / Studio domains** — the silent killer.
- **NAT GW egress for S3** — should be a VPC gateway endpoint (free).
- Letting **Bedrock provisioned throughput** sit when on-demand would be cheaper at current volume.

8. Capital One lens

Capital One discloses ~\$1B+ annual AWS spend (analyst estimates, post-data-center-exit 2020). At that scale they almost certainly run:

- A dedicated **FinOps team**.
- **Org-wide Compute Savings Plan** portfolio managed at the payer.
- **Tag policies enforced via SCPs and preventive Config rules**.
- **CUR 2.0 piped to Snowflake/Redshift** for per-LOB chargeback dashboards (their Slingshot product is the commercial version of this insight applied to Snowflake).
- **Budget actions auto-stopping non-prod after-hours**.
- **Cost Anomaly Detection scoped per cost category**.
- **SageMaker Savings Plans** for the training fleet; **Spot + Karpenter** for batch model training jobs.
- The re:Invent 2024 talk “Control the cost of your generative AI services” (Brent Segner) suggests **Bedrock cost governance** via Application Inference Profiles, per-app tags, and provisioned-throughput break-even analysis.

Interview talking point: *“At Capital One scale, the FinOps lever I’d reach for first is Compute Savings Plans pooled at the payer, then SageMaker Savings Plans for the training fleet, then Cost Anomaly Detection scoped per LOB cost category, and Cloud Custodian to enforce auto-shutdown on tagged non-prod resources.”*

9. Sanity check

1. When would you pick EC2 Instance Savings Plans over Compute Savings Plans?
2. Why is Spot a credible option for SageMaker training but not for a low-latency real-time endpoint?
3. What does CUR 2.0 add over CUR 1.0?
4. Name three ML-specific cost gotchas and the fix for each.
5. Tag policies vs SCPs for tag enforcement — which is preventive and which is detective?

10. Cross-references

- **Module 30** — EC2 instance pricing detail
- **Module 32** — Lambda pricing model
- **Module 37** — SageMaker inference endpoint pricing (the big cost gotcha)

- **Module 42** — Bedrock per-token vs Provisioned Throughput break-even
- **Module 51** — Cloud Custodian policies for tag enforcement, idle cleanup
- **Module 56** — Observability + cost (the operational follow-through)

Primary sources

- [Savings_Plans_UserGuide.pdf](#)
- [Tagging_Best_Practices.pdf](#)
- Research report: [02_aws_foundations.md](#)

Module 5 — Networking primer for ML engineers (assume zero)

What this is: networking from first principles, no AWS. CIDR, subnets, routes, NAT, DNS, TCP/UDP, ports, firewalls, load balancers. The substrate everything else in Part B sits on.

Why it matters: every “my SageMaker notebook can’t reach S3”, “the endpoint returns 504 only from prod”, and “why is our NAT bill \$14k/month?” is a networking ticket in disguise. At Capital One — where every workload runs inside a regulated VPC with no Internet Gateway egress — networking *is* the system, not an optional layer.

Pre-req: none. This module exists for ML engineers who have never had to think about this.

1. IP addresses and CIDR

An **IPv4 address** is 32 bits, written as four 8-bit octets: **10.20.30.40**. Each octet is 0-255.

A **CIDR block** (Classless Inter-Domain Routing) is **address/prefix-length**, where the prefix is the number of leading bits fixed for the network. The remaining bits are host bits.

CIDR	Host bits	Total addresses	AWS usable
/16	16	65,536	65,531
/20	12	4,096	4,091
/24	8	256	251
/28	4	16	11 (smallest VPC subnet AWS allows)

Rule of thumb: every step smaller in prefix length doubles the size. /23 is 2× /24.

AWS reserves 5 addresses per subnet (a frequent gotcha): - **.0** — network address - **.1** — VPC router - **.2** — DNS (Amazon-provided) - **.3** — future use (reserved) - **.255** — broadcast

So a /28 subnet has $16 - 5 = 11$ **usable IPs**. This bites you when launching SageMaker training clusters: an 8-node distributed job + Studio app ENI + a couple of VPC endpoint ENIs and you're out.

2. RFC 1918 private address space

Never routable on the public internet — these are the building blocks of every private network:

Range	Size	Common usage
10.0.0.0/8	16.7 M	Enterprise default; large orgs subdivide
172.16.0.0/12	1 M	Docker default; avoid colliding
192.168.0.0/16	65 K	Home networks; small offices

RFC 6598 100.64.0.0/10 is “carrier-grade NAT” space — AWS uses it for some service-managed ENIs (e.g., EKS pod IPs in some CNI modes).

3. Subnets, route tables, and gateways

A **subnet** is a slice of a VPC's CIDR, bound to exactly one Availability Zone.

A **route table** is a list of **destination CIDR** → **target** rules consulted **longest-prefix-match first**. Every subnet has exactly one route table (the main one if not explicitly associated).

Gateways:

- **Internet Gateway (IGW)** — a horizontally scaled VPC component that lets resources with public IPs reach the internet *and* receive return traffic. A subnet is “public” iff its route table has `0.0.0.0/0 → igw-....`
- **NAT Gateway** — AWS-managed Network Address Translation. Instances in **private subnets** send egress to the NAT GW, which rewrites the source IP to its own (public) Elastic IP. Return packets come back to the NAT and are de-translated. Stateful, one-way (no inbound from internet).
- **NAT Instance** — legacy: an EC2 instance you manage doing the same thing. Useful only for tiny scale or special routing tricks.
- **Egress-Only Internet Gateway (EIGW)** — IPv6 equivalent of NAT GW. Stateful, one-way, free.

4. DNS basics

DNS turns names into IPs. Record types you must know:

Record	What it does
A	Name → IPv4. <code>api.example.com → 52.1.2.3</code>

Record	What it does
AAAA	Name → IPv6
CNAME	Name → another name. Cannot exist at zone apex
MX	Mail exchanger
TXT	Arbitrary strings (SPF, DKIM, domain verification)
NS	Delegates a zone to authoritative name servers
SOA	Start-of-authority metadata
PTR	Reverse DNS (IP → name); rare in cloud-native apps

Route 53 invented **ALIAS** records to fix the “CNAME at zone apex” problem for AWS-managed targets (ELB, CloudFront, S3 website).

TTL (Time-To-Live) controls caching duration. Short TTLs (60s) for failover; long TTLs (24h) for stable infra.

5. TCP vs UDP, ports, ephemeral ports

	TCP	UDP
Connection model	Connection-oriented	Connectionless
Reliability	Reliable, ordered	Unreliable, no ordering
Overhead	Three-way handshake (SYN/SYN-ACK/ACK)	Minimal
Use cases	HTTP, gRPC, databases, SSH	DNS, QUIC (HTTP/3), real-time media

A **port** is a 16-bit number (0-65535). Well-known: 22 SSH, 80 HTTP, 443 HTTPS, 53 DNS, 5432 PostgreSQL, 3306 MySQL, 6443 Kubernetes API.

Ephemeral ports — when a client connects out, the kernel picks a random source port from a range (Linux 32768-60999, Windows 49152-65535). **Return traffic must be allowed back on that ephemeral port.** This is *the* most common NACL-mistake source.

6. Firewalls: stateful vs stateless

Stateful firewalls remember connections. If you allow outbound, the response is automatically allowed. AWS Security Groups, modern host firewalls.

Stateless firewalls evaluate every packet independently. You must explicitly allow both directions, including ephemeral return ports. AWS NACLs.

This single distinction explains why “I opened port 443 outbound” doesn’t work on a NACL — you also need to allow inbound on the ephemeral port range.

7. Load balancers: L4 vs L7

The **OSI layer** they operate on:

	L4 (transport)	L7 (application)
Sees	IP + port	HTTP headers, paths, cookies, gRPC frames
Speed	Fast, simple	Slightly higher latency
Client IP preservation	Easy	Requires X-Forwarded-For or PROXY protocol
Examples	AWS NLB	AWS ALB
Capabilities	TCP/UDP passthrough	Path/host/header routing, JWT auth, WAF

8. Public vs private IPs, Elastic IPs

A **public IP** is reachable from the internet (subject to firewalls). A **private IP** is RFC 1918 — only routable within your VPC or peered networks.

An **Elastic IP (EIP)** is a static public IPv4 you own and can reassign. AWS now charges **\$0.005/hr** (~\$3.60/mo) for every public IPv4 (in use *or* idle), since Feb 2024 — a meaningful line item at fleet scale.

9. Common pitfalls (right now, before you touch AWS)

- **Overlapping CIDRs** between VPCs make peering and TGW attachments impossible. Centralize allocations with IPAM.
- **/28 subnet for a training cluster**: 11 IPs is not enough for an 8-node distributed job once you count ENIs, Studio apps, and VPC endpoints.
- **Forgetting the AWS-reserved 5 addresses** when sizing subnets.
- **Confusing “no internet access” with “no AWS service access”** — S3 needs *routing*, which a VPC endpoint provides without an IGW.
- **Confusing CNAME and ALIAS** at zone apex.
- **NACL ephemeral-port mistake** — opening 443 outbound but forgetting to allow 32768-60999 inbound on the return path.

10. Sanity check

1. How many usable IPs does a /24 AWS subnet have, and why?
2. Which RFC 1918 range is the Docker default, and why does that matter?
3. What’s the difference between an IGW and a NAT GW?
4. Stateful vs stateless firewall — which one needs you to allow ephemeral return ports explicitly?
5. Why can’t you have a CNAME at zone apex, and what does AWS use as a workaround?

11. Cross-references

- **Module 6** — VPC, where these primitives become AWS objects
- **Module 7** — SG and NACL in AWS detail
- **Module 8** — multi-VPC connectivity (peering, TGW, PrivateLink, DX, VPN)
- **Module 9** — DNS, load balancing, edge — Route 53, ALB/NLB/GWLB, CloudFront, WAF, Shield

Primary sources

- [VPC_Connectivity_Options.pdf](#)
- Research report: [03_networking.md](#)
- RFC 1918, RFC 6598 (Internet Engineering Task Force)

Module 6 — VPC Deep

What this is: the AWS Virtual Private Cloud in production detail — subnets by routing intent, IGW vs NAT GW, VPC endpoints (Gateway + Interface), IPAM, shared VPCs via RAM.

Why it matters: the VPC is the boundary every workload sits in. Capital One workload VPCs are **isolated** (no IGW) by default — getting AWS service access to that workload means VPC endpoints and PrivateLink. The PrivateLink endpoint bill alone is a non-trivial line item at scale.

Cert mapping: SAA-C03 (Resilient + Secure 56%), SAP-C02 (heavily), SCS-C03 (network controls), MLA-C01 (Domain 4).

1. VPC anatomy

A **VPC** is a logically isolated virtual network in **one AWS region**. It has:

- One or more **CIDR blocks** (primary + up to 4 secondary IPv4; plus an Amazon-provided /56 IPv6 if enabled).
- **Subnets** — one per AZ; classification (public/private/isolated) is by route table, **not by AWS itself**.
- **Route tables, Internet Gateway, NAT Gateway(s), VPC endpoints, DHCP options set, Network ACLs, Security Groups.**
- **Elastic Network Interfaces (ENIs)** — virtual NICs attached to EC2, Lambda-in-VPC, SageMaker, RDS, ECS task, etc.

Default VPC: every AWS account gets one per region with a **172.31.0.0/16** block, public subnets in each AZ, and an IGW. **Capital One disables/deletes default VPCs via SCP** — production workloads never sit in them.

2. Subnet classes (by routing intent)

There are three patterns. None of them are AWS objects — they're emergent from the route table:

Class	Default route	Hosts
Public	0.0.0.0/0 → IGW	NLBs, ALBs, NAT GWs, bastions
Private (with egress)	0.0.0.0/0 → NAT GW	App servers, SageMaker training, EKS nodes that pull container images from the internet
Isolated	no default route	RDS, DynamoDB-via-endpoint, anything regulated. Capital One's default for PCI/PII workloads.

3. IPv4/IPv6 dual stack

A VPC can have an Amazon-provided /56 IPv6 block. Every IPv6 address is **globally routable** — there is no NAT for IPv6 in AWS.

To make a subnet “private” for IPv6, use an **Egress-Only Internet Gateway (EIGW)** — stateful, one-way like a NAT but **free**. The IPv6 future-proof equivalent of a NAT GW.

4. NAT Gateway vs NAT Instance

NAT Gateway (managed): - Scales to 45 Gbps and 1M packets/sec **per AZ**. - Deploy **one per AZ** to avoid cross-AZ data charges. - \$0.045/hr (~\$32/mo per GW) **plus \$0.045/GB processed**. - **The data-processing charge is what blows up bills** — 1 TB/month of egress costs \$45 just in processing on top of the \$32 hourly.

NAT Instance (EC2 you manage): - Cheaper at tiny scale. - No managed failover, capped by instance NIC bandwidth. - Use only if you need a special routing trick (source-NAT to a fixed allowlisted IP for a partner) or at very small scale.

5. VPC endpoints — the regulated-cloud essential

VPC endpoints let instances reach AWS service APIs **without leaving the AWS network** (no IGW, no NAT, no internet).

Gateway endpoints

- **Free**
- Only for **S3 and DynamoDB**.
- Implemented as a prefix-list route (e.g., `pl-63a5400a` for S3) in the route table. Traffic uses the existing ENI. **No DNS magic** — you target the regional S3 hostname normally.

Interface endpoints (PrivateLink)

- **ENI inserted into your subnets**, given a private IP, hooked to the service via PrivateLink.
- Works for ~150+ AWS services: SageMaker API/runtime, Bedrock, ECR API/dkr, STS, KMS, Secrets Manager, CloudWatch Logs, SSM, etc.
- **\$0.01/hr per endpoint per AZ + \$0.01/GB processed.**
- **Bill math:** 3 AZs × 30 endpoints × \$0.01/hr × 730h **\$657/mo before any traffic.**

DNS behavior: Interface endpoints support “Private DNS” — when enabled, the public DNS name (`sagemaker.us-east-1.amazonaws.com`) resolves to private endpoint IPs inside your VPC. This is what makes SDK code work unmodified.

Endpoint policies

Endpoint policies let you scope what’s accessible via the endpoint. The post-2019 baseline:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "aws:PrincipalOrgID": "o-xxxxxxxxxx"
      }
    }
  }]
}
```

This single statement enforces “only my org’s principals can use this endpoint to reach S3.” The Capital One 2019 breach exfiltrator’s principal was outside the org — this would have blocked it.

6. IPAM and shared VPCs via RAM

IPAM (IP Address Manager) — a service for planning, allocating, and auditing CIDRs across accounts and regions. Hierarchical pools (top-level → region → environment → account). Free tier; advanced tier (\$0.0001/IP/hr \$0.072/IP/mo) adds compliance tracking and IPv6 BYOIP.

RAM (Resource Access Manager) — share a VPC’s subnets with other accounts in your Organization. The owner manages networking; participants launch resources.

Capital One pattern: one **network account** owns shared VPCs, dozens of workload accounts launch SageMaker/EKS into them. This centralizes network design while keeping workloads in their own accounts.

7. Pitfalls and anti-patterns

- **Forgetting the per-AZ Interface endpoint charge** → bill shock.
- **Single-AZ NAT GW with multi-AZ workloads** → cross-AZ data-transfer charges (\$0.01/GB each way) silently eat the savings.
- **Secondary CIDR overlap** with a peered VPC or on-prem network → invisible blackhole.
- **Private DNS enabled** on an Interface endpoint **and** a private hosted zone with the same name → resolution conflict.
- **Using NAT GW when a VPC Gateway endpoint to S3 would do** — S3 traffic over NAT costs \$0.045/GB; over the gateway endpoint, free.
- **Not having an isolated subnet tier** at all — every workload ends up “private with NAT egress” and the bill reflects it.

8. Capital One lens

- **Hub-and-spoke**: a centralized “network” account owns transit-VPCs and TGW attachments.
- All workload VPCs are **isolated** (no IGW). Egress (if any) goes via a **centralized egress VPC** with proxies, AWS Network Firewall, and per-domain allow-listing.
- Every AWS service call goes through **PrivateLink Interface endpoints**, often centralized via private hosted zones in Route 53 and shared across accounts.
- **IPAM enforces non-overlapping /16s per business unit** — critical at 600+ accounts.
- The “no NAT GW in PCI zones” rule means SageMaker training jobs in PCI workloads need pre-staged container images in ECR with an Interface endpoint for `ecr.dkr` and `ecr.api`.

9. Sanity check

1. What makes a subnet “public” vs “private” vs “isolated”?
2. When would you use a Gateway endpoint vs an Interface endpoint?
3. Why is the data-processing charge on NAT GW often larger than the hourly charge?
4. Walk through the math: 3 AZs, 30 Interface endpoints, no traffic. What’s the monthly cost?
5. What’s the role of `aws:PrincipalOrgID` in an endpoint policy, and how does it relate to the 2019 breach?

10. Cross-references

- **Module 5** — primer, prerequisite for this module
- **Module 7** — SGs, NACLs, Flow Logs
- **Module 8** — Transit Gateway, PrivateLink for inter-VPC/hybrid
- **Module 41** — SageMaker VPC mode + no-internet egress
- **Module 47** — Databricks on AWS BYO VPC

Primary sources

- [VPC_Connectivity_Options.pdf](#)
- Research report: [03_networking.md](#)

Module 7 — Security Groups, NACLs, Flow Logs

What this is: Security Groups (stateful, allow-only, attached to ENIs), NACLs (stateless, allow + deny, attached to subnets), and VPC Flow Logs — the L3/L4 access controls and the audit trail. Plus a 7-step “SageMaker can’t reach S3” troubleshooting checklist worth memorizing.

1. Security Groups

- Attach to **ENIs** (so to EC2, RDS, Lambda-in-VPC, SageMaker, ALB, etc.).
- **Stateful:** return traffic is automatically allowed.
- **Allow-only:** there is **no deny rule**. Absence of allow = block.
- Rules: protocol + port range + source/destination (CIDR, another SG, or prefix list).
- Default outbound: allow all. Default inbound: deny all.
- **Can reference another SG as source** — powerful. “Allow port 5432 from **sg-app**” means anything wearing the app SG can hit Postgres, independent of IPs.
- Quota: 60 inbound + 60 outbound rules per SG; 5 SGs per ENI by default (raisable to 16).

The **SG-as-source pattern** is the cleanest tagging-by-membership control. You don’t have to know CIDRs; you tag application servers with **sg-app** and database access is automatically scoped to anything wearing that SG.

2. NACLs (Network ACLs)

- Attach to **subnets** (not ENIs).
- **Stateless:** must allow both directions, **including ephemeral return ports**.
- Have **both Allow and Deny rules**, evaluated in **rule-number order**, first match wins.
- Default NACL: allow all both ways. Custom NACLs default to deny all.

Use **case** in regulated finance: a coarse extra layer. Common patterns: - Block known-bad IP ranges at the subnet level. - Enforce “this isolated subnet can never talk to the internet.” - Add a defense-in-depth layer where SGs are the primary control.

NACLs are blunt; you should not use them for fine-grained access control. That's SGs' job.

3. Evaluation order

For a packet entering an ENI from outside the subnet:

1. **NACL inbound** on subnet.
2. **SG inbound** on ENI.
3. (instance processes the request, generates response)
4. **SG outbound** on ENI (stateful — return is often pre-allowed by the inbound rule).
5. **NACL outbound** on subnet — **must allow ephemeral source port back**.

The NACL outbound step is where engineers get burned. “I allowed 443 inbound but the response is dropped” → you forgot to allow ephemeral ports (32768-60999 on Linux) on the way out.

4. The 7-step “SageMaker notebook can't reach S3” troubleshooting checklist

This is the single most-tested troubleshooting scenario in AWS exam prep, and it's something you will use weekly in production. Memorize the order:

1. **Is the notebook in VPC-only mode?** If so, there's no internet — must use a Gateway endpoint for S3.
2. **Does the notebook's subnet route table have `pl-s3 → vpce-...`?** Without this, S3 traffic has nowhere to go.
3. **Does the endpoint policy allow the bucket and action?** Endpoint policies override; “Allow * Resource *” is fine, but if it's locked down, S3 calls fail with AccessDenied.
4. **Does the bucket policy allow the role's principal from this VPC endpoint?** Use `aws:SourceVpce` condition to allow access from your endpoint specifically.
5. **SG outbound rule allowing HTTPS (443) to the S3 prefix list?**
6. **NACL on the subnet allowing 443 out and ephemeral 1024-65535 in?**
7. **KMS** — if bucket is SSE-KMS, you need a **KMS Interface endpoint** *and* a key policy that allows the role to decrypt.

When in doubt, run **Reachability Analyzer** between the notebook ENI and an S3-related resource (e.g., a test ENI), or check **CloudTrail** for AccessDenied with reason codes.

5. VPC Flow Logs

Per-ENI / per-subnet / per-VPC capture of accepted/rejected/all flows.

Destinations: - CloudWatch Logs (expensive at scale). - S3 (cost-sane choice for high-volume flow logs). - Kinesis Data Firehose (when you want to fan to SIEM in real time).

Formats: default v2 (14 fields); custom format up to v7 — adds: - `vpc-id`, `subnet-id`, `instance-id` - `tcp-flags` (SYN/ACK/FIN/RST) - `pkt-srcaddr` (for NAT'd flows — the *real* source IP) - `flow-direction`

(ingress/egress) - **traffic-path** (through IGW, TGW, peering, VPN, ...)

Captures: src/dst IP+port, protocol, packets, bytes, start/end timestamps, action (ACCEPT/REJECT), log status.

Does NOT capture: - Packet payload (use Traffic Mirroring for that). - DNS lookups (use Route 53 Resolver query logs). - Traffic to/from **169.254.169.254** (IMDS) and **169.254.169.123** (NTP) link-local addresses. - Traffic between endpoints of certain managed services.

Pricing: data-ingestion charges on the destination. CloudWatch Logs ingestion is the killer at scale. **S3 + Athena is the cost-sane pattern** for finance.

6. Reachability Analyzer and Network Access Analyzer

Reachability Analyzer: - “Can ENI A reach ENI B on port 443?” - Static analysis of SGs, NACLs, route tables, peering, TGW. - Pay-per-analysis (~\$0.10). - Great for spot debugging.

Network Access Analyzer (newer, more powerful): - Declarative scopes: “no resource in the PCI VPC should be reachable from the internet.” - Continuous compliance checking against invariants. - Premium feature, but cheap compared to a breach. - Heavily used in regulated industries to encode PCI/HIPAA reachability invariants.

7. Pitfalls and anti-patterns

- **NACL ephemeral-port mistake** — opening 443 outbound on a NACL but forgetting to allow ephemeral inbound on the return path.
- **SGs as the only control with 0.0.0.0/0 source** on common ports — fine for an ALB facing the internet, terrible for anything internal.
- **Flow Logs to CloudWatch Logs at scale** — ingestion bill surprise. Use S3.
- **Using NACLs for fine-grained control** — they’re not the right tool. Use SGs.
- **No flow logs at all** — your audit trail is incomplete; regulators will ask why.

8. Capital One lens

- **SGs are the primary control**; NACLs are coarse guard rails (“this subnet can never egress”).
- **VPC Flow Logs to S3** with v5+ custom format, queried via Athena + QuickSight. Retained per regulatory schedule.
- **Network Access Analyzer scopes** encode regulatory invariants (“PCI DSS scoped resources cannot reach non-PCI scope” / “isolated subnets have no path to the internet”).
- **Cloud Custodian policies** (their OSS) check SG rules nightly: e.g., flag any SG with **0.0.0.0/0** on ports other than 80/443 inbound to an ALB.

9. Sanity check

1. SG vs NACL — which is stateful, which is stateless, and which one always evaluates rules in numeric order?
2. Walk through the 7-step SageMaker → S3 checklist. Where does KMS come in?
3. What does Flow Logs **not** capture, and what should you use instead?
4. Why is CloudWatch Logs the wrong destination for VPC Flow Logs at scale?
5. What's the difference between Reachability Analyzer and Network Access Analyzer?

10. Cross-references

- **Module 6** — VPC, subnets, endpoints (prerequisite)
- **Module 8** — multi-VPC connectivity
- **Module 41** — SageMaker VPC mode + no-internet egress (where the 7-step checklist lives in production)
- **Module 50** — Security primitives (where Flow Logs feed into Security Hub findings)

Primary sources

- [VPC_Connectivity_Options.pdf](#)
- Research report: [03_networking.md](#)

Module 8 — Multi-VPC and Hybrid Connectivity

What this is: VPC Peering, Transit Gateway, PrivateLink (for your own services), Direct Connect, VPN, Cloud WAN. The connectivity layer that links VPCs to each other and to on-prem.

Why it matters: Capital One operates 600+ accounts under a hub-and-spoke TGW topology with DX into multiple colos. Every architecture review asks how a new workload integrates — you need to fluently distinguish “TGW route” from “PrivateLink service” from “VPC peering.”

1. VPC Peering

- 1:1 connection between two VPCs.
- Region-local or inter-region.
- **Non-transitive** — A B and B C does *not* give A C.
- Cheap (only data-transfer charges); CIDRs must not overlap.
- Doesn't scale past ~10 VPCs — mesh becomes $O(n^2)$ and the route-table maintenance becomes painful.

Use case: two VPCs that need to talk to each other and that's it. Beyond two, switch to Transit Gateway.

2. Transit Gateway (TGW)

The hub-and-spoke replacement for peering meshes.

- Attaches **VPCs, VPNs, Direct Connect Gateways, peer TGWs (cross-region), Connect attachments** (SD-WAN/GRE).
- **Route tables** on the TGW segment traffic. Pattern: one RT per environment — “prod can talk to prod-shared-services, not to dev.”
- Quotas: ~5,000 attachments per TGW; 10,000 routes per RT.

- **Pricing:** \$0.05/hr per attachment (~\$36/mo each) + \$0.02/GB processed. A 100-VPC TGW: ~\$3,600/mo before traffic.

TGW route table pattern (typical):

prod-rt:

```
attached: prod-card-ml, prod-card-app, prod-shared-services
prod-card-ml ↔ prod-shared-services  ✓
prod-card-ml ↔ prod-card-app         ✓
prod-card-ml ↔ dev-*                 ✗ (no route)
```

3. AWS PrivateLink (for your own services)

Built on the same plumbing as Interface endpoints, but for **your own services**:

1. You publish a service backed by an **NLB** or **GWLB**.
2. Consumers create an **Interface endpoint** into your service from their own VPCs.
3. One-directional (consumer → provider only).
4. **No CIDR overlap concerns** since the consumer sees only an ENI IP in their own VPC.

Capital One pattern: every shared internal service (model registry, feature store, internal LLM gateway, central tokenization service like Databolt) is exposed via PrivateLink rather than via TGW reachability. **Least-privilege at the L4 level** — each consumer is an explicit endpoint resource, easy to audit, easy to revoke.

This is the architect-grade insight: **TGW gives broad reachability between networks; PrivateLink gives narrow access to one specific service.** Use TGW for tiers that share resources broadly (e.g., shared monitoring); use PrivateLink for everything else.

4. Direct Connect (DX)

A **physical fiber circuit** from your DC/colo into an AWS DX location.

Type	Description
Dedicated connection	Full 1/10/100 Gbps port; you own it
Hosted connection	Sub-port slice from an APN partner (50 Mbps to 25 Gbps)

Virtual Interfaces (VIFs): - **Private VIF** — one VPC (via VGW) or many (via DX Gateway → TGW). - **Public VIF** — AWS public service endpoints (S3, etc.) without using the internet. - **Transit VIF** — DX Gateway → Transit Gateway.

Pricing: port-hour + data-transfer-out (cheaper than internet egress, much cheaper at scale).

Always pair with VPN backup. A DX circuit is a single physical path and can fail.

5. Site-to-Site VPN and Client VPN

Site-to-Site VPN: - IPsec tunnels (always two for redundancy) between an AWS VGW/TGW and a customer gateway (your firewall). - Up to ~1.25 Gbps per tunnel. - Often used as DX backup or for low-traffic remote sites.

Client VPN: - Managed OpenVPN-based service for human users to reach VPCs. - Mutual TLS + optionally federated auth. - \$0.10/hr per associated subnet + \$0.05/hr per connected client. - Used for engineers' workstation access to private VPCs.

6. Cloud WAN

AWS's "global network as a service": you define **network segments** (e.g., prod, dev, shared) in **policy JSON**; Cloud WAN provisions the underlying TGWs, peerings, and DX gateways across regions.

The replacement for the hand-rolled "global TGW mesh." Use when you have 4+ regions or want declarative segmentation across them.

7. TGW Network Manager

A monitoring/observability console for TGWs, Cloud WAN, DX, VPN. Topology view, CloudWatch metrics, events.

8. Asymmetric routing — the silent killer

A common failure mode in inspection architectures (Network Firewall, third-party appliances):

1. Packet enters via TGW route table A → reaches firewall → traffic returns.
2. Return packet hits a different TGW route table B → routed via a different path.
3. Stateful firewall in the path drops the asymmetric return.

Fix: TGW **appliance mode** for stateful inspection appliances. Forces symmetric routing through the same appliance.

9. Pricing pitfalls

- **TGW data charges count both directions** in many cross-account patterns.
- **VPC Peering is "free" within an AZ** but cross-AZ traffic still incurs \$0.01/GB.
- **Adding a secondary CIDR** that overlaps with a peered VPC or on-prem → invisible blackhole.
- **Direct Connect without a VPN backup** → an outage waiting to happen.
- **Interface endpoint cost** scales with AZ count × endpoint count × hours.

10. Capital One lens

- **Hub-and-spoke TGW per region**, peered across regions, ideally evolving to **Cloud WAN**.

- **DX from multiple Capital One colos to AWS**, redundant per region, with S2S VPN backup.
- **Inter-account service access via PrivateLink** rather than open TGW routes — every service consumption is auditable as an endpoint resource.
- **Egress to internet** (where unavoidable, e.g., partner APIs) goes via a centralized egress VPC with **AWS Network Firewall + Squid proxy + domain allowlisting**.
- **Network Access Analyzer scopes** encode regulatory invariants (“PCI workloads cannot reach non-PCI workloads”).

11. Sanity check

1. VPC Peering is non-transitive. What does that mean operationally, and when does it bite?
2. When would you pick PrivateLink for one of your own services over a TGW route?
3. What’s the difference between a Private VIF and a Transit VIF on Direct Connect?
4. What is “TGW appliance mode” and what problem does it solve?
5. When does Cloud WAN start to make sense vs hand-rolled multi-region TGW?

12. Cross-references

- **Module 6** — VPC + Interface endpoints (PrivateLink for AWS services)
- **Module 7** — SGs/NACLs for traffic that crosses the connectivity layer
- **Module 47** — Databricks on AWS PrivateLink / Secure Cluster Connectivity
- **Module 50, 51** — Network Firewall as part of the security primitives

Primary sources

- [Hybrid_Connectivity.pdf](#)
- [Building_Scalable_Secure_Multi_VPC_Network_Infrastructure.pdf](#)
- [AWS_Transit_Gateway_Best_Practices.html](#)
- Research report: [03_networking.md](#)

Module 9 — DNS, Load Balancing, and Edge

What this is: Route 53 (hosted zones + 7 routing policies + Resolver), ALB/NLB/GWLB, CloudFront, Global Accelerator, WAF, Shield. The DNS + L7/L4 + CDN layer.

Why it matters: every public-facing AI/ML surface (LLM API gateway, model inference endpoint, RAG-backed chatbot) sits behind some combination of Route 53 + ALB + WAF + Shield + CloudFront. Designing for DDoS resilience and global low-latency inference is part of the Sr Lead role.

1. Route 53

Public hosted zones — authoritative DNS for an internet domain.

Private hosted zones — DNS visible only inside associated VPCs. Critical for **PrivateLink override patterns** — you create a private hosted zone for `sagemaker.us-east-1.amazonaws.com` and override the AWS public DNS with your endpoint IPs.

7 routing policies

Policy	When to use
Simple	One record set, no logic
Weighted	Traffic split by weights (canary deploys, A/B)
Latency-based	Route to the lowest-latency AWS region for the resolver
Failover	Primary/secondary with health-check-driven cutover
Geolocation	By user country/continent (sovereignty)
Geoproximity	By physical distance, with a “bias” knob (Traffic Flow only)
Multi-value answer	Up to 8 healthy records returned, basic DNS-level load distribution

Health checks

HTTP/HTTPS/TCP, can **chain** (calculated health checks combining sub-checks) and integrate with CloudWatch alarms.

Route 53 Resolver

The DNS server every VPC gets at `VPC-base+2`. **Resolver endpoints** (inbound/outbound) let on-prem resolvers query VPC names and vice versa — essential for hybrid PrivateLink architectures.

Resolver query logs to S3/CWL/Firehose — fills the DNS blind spot in Flow Logs.

2. Application Load Balancer (ALB) — L7

- HTTP/HTTPS/gRPC/HTTP-2/WebSocket.
- Path-based, host-based, header-based, query-string-based routing.
- Targets: EC2, IP, Lambda, containers (via target groups).
- Native integration with **WAF**, **Cognito** (auth offload), **OIDC**.
- Sticky sessions via cookies.
- **Slow start mode** for new targets — gentle ramp.
- Pricing: hourly + **LCU** (Load Balancer Capacity Units, mix of new connections, active connections, processed bytes, rule evaluations).

3. Network Load Balancer (NLB) — L4

- TCP/UDP/TLS passthrough or termination.
- **Static IPs per AZ** (and Elastic IP support). Critical when partners need to allowlist your IPs.
- Ultra-low latency (~100 μ s), tens of millions of concurrent flows.
- Preserves client source IP by default (instance mode).
- **The L4 layer underneath PrivateLink services.**

4. Gateway Load Balancer (GWLB)

- L3, uses **GENEVE encapsulation** (UDP port 6081).
- Inserts third-party network/security appliances (Palo Alto, Fortinet, Check Point, Aviatrix, Suricata) transparently into the traffic path.
- The “service-chain” primitive: route VPC egress $\rightarrow$ GWLB endpoint $\rightarrow$ appliance fleet $\rightarrow$ out.

5. CloudFront

- Global CDN, 600+ POPs.
- Origins: S3, ALB, EC2, MediaStore, custom HTTP.
- **Lambda@Edge** and **CloudFront Functions** for compute at the edge.

- **Origin Access Control (OAC)** locks S3 origins so the bucket is only reachable via CloudFront — table stakes for serving static SageMaker docs or generated assets.
- TLS termination, **HTTP/3**, Brotli, signed URLs/cookies.

AI/ML angles: - Serve **Bedrock-generated assets** with edge caching. - Host **LLM-app static frontends** (React/Next.js) close to users. - Cache **embeddings or model artifacts** close to inference clients.

6. AWS Global Accelerator (GA)

Anycast static IPs in front of regional ALB/NLB/EC2 endpoints. Traffic enters the AWS backbone at the nearest edge POP and rides AWS’s network the rest of the way — typically **30-60% faster** than public-internet routing for cross-region traffic.

Useful for a globally distributed inference API (e.g., a latency-critical fraud-decision model serving customer requests worldwide).

7. AWS WAF

Rule engine in front of ALB, CloudFront, API Gateway, AppSync, App Runner.

Rule type	Use case
Managed rule groups	AWS, Marketplace (OWASP Top 10, account-takeover, bot control)
Custom rules	String match, regex, geo, IP set, size, SQLi/XSS
Rate-based rules	Count requests per 5-min sliding window per IP (or per header/cookie/JA3); essential against scrapers hitting your LLM endpoint
Bot Control	Paid managed rule group, behavioral bot detection
CAPTCHA / Challenge	Interactive challenges as an action

Pricing: \$5 / web ACL / mo + \$1 / rule / mo + \$0.60 per million requests + managed-rule subscriptions.

Connection to the 2019 Capital One breach: the attack started via an SSRF in a misconfigured WAF (ModSecurity, third-party at the time). Today’s AWS WAF doesn’t have that specific vulnerability, but the lesson is “WAF rules are a security boundary; treat them like code.”

8. AWS Shield

- **Standard** — free, auto-enabled, defends against common L3/L4 DDoS.
- **Advanced** — \$3,000/mo per organization (12-month commit) + data-transfer protection. Adds:
 - **DRT (DDoS Response Team)** 24/7

- **Cost protection** for scaling during attacks
- **L7 mitigation** when paired with WAF
- **Attack analytics**
- Shield Advanced metrics in CloudWatch

For regulated finance, Shield Advanced is table-stakes on any public-facing surface.

9. Pitfalls and anti-patterns

- **ALB sticky sessions** break stateless container scaling assumptions.
- **NLB source-IP preservation** requires `client_ip = on` in `nginx` / `proxy_protocol_v2` for some apps.
- **CloudFront and S3** default bucket policy collisions — always use OAC, never bucket-level “public” toggles.
- **WAF rate-based rules with too-broad keys** (IP only) miss authenticated abuse; use composite keys (IP + session token or IP + JA3 fingerprint).
- **Route 53 ALIAS vs CNAME confusion at zone apex** — use ALIAS for AWS-managed targets.
- **No CloudFront in front of S3 static assets** — CORS pain and no edge cache.

10. Capital One lens

- **ALBs internal-only inside VPC**, fronted by API Gateway (private) or by an on-prem F5/edge stack via DX.
- All public-facing surfaces behind **CloudFront + WAF + Shield Advanced**.
- **Route 53 private hosted zones** (resolver rules + endpoints) used to centralize PrivateLink DNS overrides across the org.
- **Global Accelerator** considered for multi-region active-active inference APIs (e.g., latency-critical fraud-decision models).
- Public-facing surfaces have **AWS Shield Advanced** + DRT engagement runbook.

11. Sanity check

1. Name the 7 Route 53 routing policies and one production use case for each.
2. ALB vs NLB — which preserves client IP by default, and which one underpins PrivateLink?
3. What is GENEVE encapsulation and what does GWLB use it for?
4. What does OAC do, and why is it critical for S3+CloudFront patterns?
5. When does Global Accelerator pay off vs just using CloudFront?

12. Cross-references

- **Module 8** — multi-VPC connectivity (PrivateLink is built on NLB)
- **Module 33** — EKS Ingress (often ALB Ingress Controller or ALBv2)

- **Module 42, 44** — Bedrock + self-hosted LLM endpoints (CDN + WAF patterns)
- **Module 50** — security primitives (WAF + Shield + Network Firewall combo)

Primary sources

- [Route53_Routing_Policies.html](#)
- [ELB_Types_Comparison.html](#)
- Research report: [03_networking.md](#)

Module 10 — S3 Deep

What this is: S3 storage classes, lifecycle, replication, S3 Tables (Iceberg-native), Express One Zone, Object Lambda, Access Points, encryption, performance limits, Mountpoint. The data-lake substrate.

Why it matters: every Capital One workload reads or writes S3. The 2019 breach exfiltrated from S3. S3 cost optimization can be 7-figure annual savings. SageMaker training, EMR, Glue, Athena, Lake Formation all sit on S3.

Cert mapping: SAA-C03, DEA-C01 (Data Store 26%), SCS-C03 (S3 protections).

1. Storage classes (price/latency tradeoff)

Class	Latency	Storage cost (relative)	When
Standard	ms first-byte	1×	Default — hot training data, model artifacts
Standard-IA	ms	0.55×	30-day min; per-GB retrieval fee. Older training inputs
One Zone-IA	ms	0.44×	Single-AZ — only for reproducible derived features
Intelligent-Tiering	ms (Freq/IA), ms-min (Archive)	varies	Unknown access patterns. Small monitoring fee per 1000 objects (<128KB never tiered)
Glacier Instant Retrieval	ms	0.2×	Quarterly access; 90-day min
Glacier Flexible Retrieval	1–5 min Expedited / 3–5 hr Std / 5–12 hr Bulk	0.07×	Backups, occasional access

Class	Latency	Storage cost (relative)	When
Glacier Deep Archive	12+ hr	0.04×	7-year regulatory retention (SOX, FFIEC)

11 nines durability across all classes; 99.99% availability on Standard.

2. Lifecycle, versioning, replication

- **Lifecycle policies** transition or expire objects on age, tags, size filters. **AbortIncompleteMulti-partUpload** is the single most-omitted rule and silently bleeds money.
- **Versioning** is bucket-level. Once enabled, only “suspended” — never disabled. Required for **Object Lock** and MFA-Delete.
- **Replication:**
 - **SRR** (same region) for log aggregation, dev/prod fanout.
 - **CRR** (cross-region) for DR.
 - **Multi-Region Access Points (MRAP)** route to lowest-latency replica via Global Accelerator.
 - **S3 Replication Time Control (RTC)** — 99.99% replication within 15 min with billing SLA.

3. S3 Tables (GA Dec 2024)

A new bucket type purpose-built for tabular Iceberg data:

- **Automatic compaction** and snapshot expiration.
- **Unreferenced-file cleanup.**
- Table-level RBAC.
- Native Athena/EMR/Redshift integration; **Glue Iceberg REST catalog** integration mid-2025.
- **Up to 10×** higher **TPS** on Iceberg writes vs self-managed Iceberg on Standard S3.
- **3×** faster **queries** vs self-managed Iceberg.

Federation pattern for Capital One: raw zone on S3 Standard, curated Iceberg tables in S3 Tables for SLA-bound consumers.

4. Object Lambda & Access Points

- **Access Points** — named hostnames with their own bucket policy. Scoped per VPC or per workload. Capital One uses one per microservice.
- **Object Lambda Access Points** — invoke a Lambda on **GetObject** to transform data inline: **PII redaction**, **Databolt token swap** for downstream consumers without re-materializing the dataset.
- **Multi-Region Access Points** — single global endpoint over replicated buckets.

5. Encryption & compliance

	What it is	Use case
SSE-S3	AES-256, S3-managed	Default since Jan 2023; no extra cost
SSE-KMS	Customer-managed key	PCI-DSS — required to demonstrate key custody
SSE-C	Customer-supplied key per request	HSM-backed key custody outside AWS
DSSE-KMS	Dual-layer KMS	CNSA / FIPS-mandated (GovCloud)

Bucket Key — caches a data key per bucket, cuts KMS API calls by **up to 99%**. Always enable for KMS-encrypted buckets at scale.

Object Lock — WORM: - **Governance mode** — IAM with privilege can override. - **Compliance mode** — no override, not even root. Required for FINRA 17a-4(f) immutable broker-dealer records.

Block Public Access (BPA) — account-level kill switch. **ON by SCP for every Capital One account.**

S3 Access Grants (2023) — IAM Identity Center-based grants that issue temporary credentials scoped to prefix. Replaces sprawling bucket policies for user-level access.

6. Performance & limits

3,500 PUT/POST/DELETE per second per prefix. 5,500 GET/HEAD per second per prefix.

The prefix is whatever character sequence precedes the partition you key on, **not the leading directory**. The partition workarounds:

- **Hash-prefix object keys:** <8-char hash>/<rest of path> distributes load across virtual prefixes.
- **Partitioned prefixes:** dt=2026-05-15/hour=03/ naturally distributes hot writes.
- **Iceberg sidesteps** this naturally — metadata writes distribute across many manifests.

S3 Express One Zone (Nov 2023) — single-AZ “directory bucket” class: - **Single-digit-millisecond** latency. - ~50% lower request cost than S3 Standard. - **7-10× faster** on small-object workloads. - Trade-off: single AZ. Replicate critical training shards into S3 Standard.

S3 Mountpoint (GA Aug 2023) — POSIX-ish FUSE mount; high-throughput read, append-only write, no rename. SageMaker training jobs use this when pipe mode isn’t fast enough.

S3 Batch Operations — operate across billions of objects via inventory manifest (re-encrypt, restore, invoke Lambda).

7. Pitfalls and anti-patterns

- **Lifecycle to Glacier on tiny objects** costs more than the storage savings (per-object overhead).
- **Forgetting `s3:RestoreObject`** in IAM blocks Athena queries against Glacier partitions.
- **Replication doesn't replicate pre-existing objects** — run S3 Batch Replication explicitly (since 2022).
- **KMS key policy must allow the role, not the bucket** — KMS denies don't show in S3 logs, only in CloudTrail.
- **Missing `AbortIncompleteMultipartUpload`** lifecycle rule — multipart upload fragments accumulate forever.
- **Bucket Key disabled** at scale → high KMS API costs.

8. Capital One lens

- S3 is **the** lake substrate. Bucket naming convention: `co-<lob>-<env>-<purpose>-<region>` (e.g., `co-card-prd-features-use1`).
- Every bucket: **BPA on, default SSE-KMS w/ Bucket Key, versioning on**, lifecycle for expiring multipart and aging non-current versions.
- **Databolt tokenization runs upstream of S3** — raw PAN never lands; tokens only.
- Iceberg curated tables migrating to **S3 Tables** buckets through 2026.

9. Sanity check

1. What are the per-prefix request rate limits, and how do you work around them?
2. When would you use S3 Express One Zone vs Standard?
3. What does Object Lock Compliance mode enforce that Governance mode doesn't?
4. How does Bucket Key reduce KMS cost?
5. Why does S3 Tables matter for Iceberg performance?

10. Cross-references

- **Module 12** — Lake Formation FGAC layered on S3 catalog tables
- **Module 14** — Glue Spark reading/writing S3 Iceberg
- **Module 22, 24** — Redshift Spectrum, Athena reading S3
- **Module 41** — SageMaker training reading S3 via Mountpoint / pipe / DRA
- **Module 52** — Databolt tokenization upstream of S3

Primary sources

- [S3_Best_Practices.pdf](#)
- Research report: [04_storage_data_lake.md](#)

Module 11 — EFS, FSx, EBS for ML

What this is: the block (EBS) and file (EFS, FSx) storage options for ML workloads, with the decision matrix for “what storage do I need for this training job?”

1. EBS volume types

Type	Max IOPS	Max throughput	Notes
gp3	16,000	1,000 MB/s	Default. Decouples IOPS/throughput from size; 20% cheaper than gp2.
gp2	16,000	250 MB/s	Legacy; IOPS scale with size. Avoid for new builds.
io2 Block Express	256,000	4,000 MB/s	Sub-ms latency, 99.999% durability, multi-attach. SQL Server / kdb+.
st1	500	500 MB/s	Throughput-optimized HDD; sequential (Kafka logs, big-data temp).
sc1	250	250 MB/s	Cold HDD; cheap colocated archive.

- **Snapshots** are incremental, stored in S3. **Fast Snapshot Restore (FSR)** pre-warms blocks; charged per AZ-snapshot pair.
- **EBS Multi-Attach** (io1/io2) — same volume to up to 16 Nitro instances in one AZ. Filesystem must be cluster-aware (GFS2, OCFS2). Mostly Oracle RAC, not ML.
- **Encryption** — default-on at account level; KMS CMK.

2. EFS (NFSv4)

Two performance modes: - **General Purpose** (default) — low-latency. - **Max I/O** (legacy) — higher latency but more parallel IOPS.

Three throughput modes: - **Bursting** — credit-based; exhausts silently at sustained load. - **Provisioned** — pay for committed throughput. - **Elastic** (the modern default) — pays per usage, no provisioning.

EFS Intelligent-Tiering — lifecycle to IA / Archive after N days idle; up to 92% cheaper for cold data. First read after tiering pays a per-GB retrieval fee.

Use cases: - Shared notebooks - SageMaker Studio home directories - Airflow DAG sync across workers
- Not for training I/O at >10 GB/s

3. FSx flavors

FSx for Lustre

POSIX parallel filesystem; up to **terabits/s throughput**, sub-ms latency.

- **Scratch** — single-AZ, no replication, cheapest, ideal for ephemeral training runs.
- **Persistent** — replicated within AZ; 50/100/200/500/1000 MB/s/TiB throughput tiers.
- **S3-linked filesystems (DRA — Data Repository Association)** — lazy-loads from S3 on first read; writes back via `lustre_release` or DRA export. **Critical for SageMaker / EKS training** where dataset lives in S3 but training expects POSIX.
- **2024:** metadata IOPS provisioning — decouples metadata IOPS for many-small-file ML workloads (image / token shards).

FSx for OpenZFS

Single-AZ or multi-AZ; snapshots, zero-copy clones. Multi-AZ added 2023. Shared dev environments, low-volume analytics.

FSx for Windows File Server

SMB; AD-integrated. Legacy app shares, not ML.

FSx for NetApp ONTAP

Multi-protocol (NFS + SMB + iSCSI), FlexClone snapshots, SnapMirror to on-prem NetApp; FabricPool tiering to S3. Capital One uses ONTAP as landing zone for on-prem feeds bridging to cloud.

4. Picking storage for ML

Workload	Pick
Training I/O > 1 GB/s, > 1 TB dataset Many small files (images, audio, JSONL)	FSx Lustre Persistent + S3 DRA FSx Lustre with provisioned metadata IOPS or S3 Express One Zone via Mountpoint
Shared notebook + small dataset (< 100 GB) Single-node training, dataset fits	EFS Elastic Throughput gp3 EBS
Distributed training checkpoint store	S3 Express One Zone (2024+ SageMaker recommendation)

5. Pitfalls

- **FSx Lustre data hydration:** lazy-load means **first epoch is slow** unless you `hsm_restore` (preload). Always preload for benchmarking.
- **EFS Bursting** throughput exhausts burst credits silently — switch to Elastic for production.
- **gp2 burst-balance starvation** on small (<100 GB) volumes — gp3 with provisioned IOPS solves it.
- **io2 BX is region-limited** — check before architecting.
- **Snapshots without lifecycle** pile up — use Data Lifecycle Manager (DLM).

6. Capital One lens

- gp3 EBS as compute root default.
- EFS Elastic Throughput for Studio domains.
- FSx Lustre Persistent for large training runs (linked to S3 DRA for dataset access).
- ONTAP for on-prem-to-cloud feed bridges (treating it as the gateway, not the lake).

7. Sanity check

1. When does FSx Lustre Persistent + S3 DRA beat just reading from S3 with Mountpoint?
2. What's the difference between EFS Bursting and Elastic throughput modes, and when does Bursting silently break?
3. What's the gp3 vs gp2 difference and why are new builds always gp3?
4. When would you pick FSx ONTAP over OpenZFS?
5. What's the recommended checkpoint store for SageMaker distributed training in 2024+?

8. Cross-references

- **Module 10** — S3 (DRA endpoint, Mountpoint)
- **Module 36** — SageMaker training (FSx Lustre integration patterns)
- **Module 40** — HyperPod (FSx Lustre as the cluster storage tier)

Primary sources

- [EBS_UserGuide.pdf](#)
- [EFS_UserGuide.pdf](#)
- [FSx_Lustre_UserGuide.html](#)
- Research report: [04_storage_data_lake.md](#)

Module 12 — Lake Formation, Glue Catalog, Iceberg, DataZone

What this is: the AWS data-governance stack — Glue Data Catalog as metastore, Lake Formation as FGAC enforcer, Iceberg as table format, DataZone as federated governance. Compared to Databricks Unity Catalog.

1. Glue Data Catalog (the metastore)

The canonical Hive-Metastore-compatible metastore for AWS analytics. Athena, EMR, Glue ETL, Redshift Spectrum, and EMR Serverless all read it.

- Supports **Iceberg, Hudi, Delta Lake** table formats. Iceberg has the deepest integration; Glue 4.0+ writes Iceberg natively.
- **Federation:** Glue Catalog can federate to Hive Metastore, Snowflake, and (via Iceberg REST) other catalogs.
- **Glue Iceberg REST Catalog** (GA 2024) makes the Glue Catalog accessible to any Iceberg-compliant engine — Trino, Spark on K8s, Databricks.

2. Lake Formation

Centralized fine-grained access control (FGAC) on Glue Catalog tables.

Three grant modes

1. **Named-resource grants** — explicit `GRANT SELECT ON db.table TO principal`.
2. **LF-Tag-Based Access Control (LF-TBAC)** — tags on databases/tables/columns; policies grant by tag expression. Scales to thousands of tables.
3. **Hybrid mode** (2023+) — coexistence with IAM-only access for tables not yet onboarded.

Granularity

- Database
- Table

- Column
- Row
- Cell

Row/cell-level via **data filters** (predicate + column projection).

Cross-account sharing

Lake Formation grants to an AWS account or org via **RAM** (Resource Access Manager). The receiving account creates a **resource link** to consume.

The grant-order pitfall

If a table is registered with Lake Formation **but the IAM principal also has direct S3 read on the underlying bucket**, the user bypasses FGAC by reading raw S3.

Fix: remove direct S3 IAM to LF-managed prefixes; rely on LF temporary credentials vending (`lakeformation:GetTemporaryGlueTableCredentials`).

3. Iceberg on AWS (2024–2026)

- Apache Iceberg 1.5+ supported by Glue, Athena v3, EMR 6.15+, Redshift, SageMaker Processing.
- **S3 Tables** is the managed-Iceberg substrate (Module 10).
- **Schema evolution, partition evolution, time travel** (snapshot-id / as-of-timestamp), **branching/tagging** are first-class.
- **Compaction:** automatic in S3 Tables; in raw-S3 Iceberg you run Glue table optimization (`ALTER TABLE ... COMPACT`) — without it, small-file proliferation kills query perf within weeks.

4. AWS DataZone

GA Sep 2023; 2024–2025 additions: business glossary, ML Model Catalog, GenAI subscription workflows.

Layers on top of Lake Formation + Glue Catalog:

- **Domains** — LOB boundaries.
- **Projects** — cross-functional groups.
- **Data assets** — tables, dashboards, models.
- **Subscriptions** — audited access requests with publisher approval.

Approval workflows route to a *publisher*; once approved, DataZone calls Lake Formation to grant access — **LF remains the enforcement plane**.

SageMaker Unified Studio (re:Invent 2024 → GA 2025) embeds DataZone as the governance backbone for the unified data + AI workspace.

5. DataZone vs Unity Catalog

	DataZone (AWS-native)	Unity Catalog (Databricks)
Where it lives	AWS; sits above Lake Formation	Inside Databricks control plane
Enforcement	Delegates to LF	Cluster-level (Photon, SQL Warehouse)
Scope	Tables, dashboards, notebooks, models	Tables, ML features, registered models
Open source?	No	Yes — Unity Catalog OSS (2024) with Iceberg REST endpoints

Interop: Iceberg REST is the bridge. Both Unity (since 2024) and Glue (since 2024) expose Iceberg REST endpoints, so Spark on Databricks can read a Glue-cataloged Iceberg table with LF credentials vending, and Athena can read a Unity-cataloged Iceberg table.

6. Capital One lens

Strategic stack pattern:

S3 raw → Iceberg curated (→ S3 Tables) → Glue Catalog metastore →
Lake Formation FGAC with LF-Tags → DataZone for discovery & subscription →
Databricks Unity Catalog for ML/feature pipelines via Iceberg REST

- PCI-DSS posture: every table tagged `data_classification = PCI | NPI | Confidential | Public`; LF-TBAC denies non-PCI roles.
- KMS keys per LOB; CMK rotation 90 days.
- Databolt tokenization gates landing — Lake Formation enforces post-tokenization views (raw-PAN columns physically absent).

7. Pitfalls

- **Hybrid mode confusion** — users see both LF-managed and IAM-only tables and don't know which enforces. Fix: split databases, name them `_lf_` vs `_iam_`.
- **Forgetting `lakeformation:GetDataAccess`** — Athena returns “Insufficient Lake Formation permissions” even when grants look correct.
- **Glue Catalog version drift** — Glue 3.0 vs 4.0 vs 5.0 Iceberg behavior differs; pin EMR / Glue versions in PR templates.
- **DataZone subscription latency** — grants are asynchronous (seconds to minutes), unsuitable for break-glass workflows.
- **Iceberg compaction skipped** in self-managed buckets → query times degrade within weeks.

8. Sanity check

1. What are the three Lake Formation grant modes, and when does Hybrid bite?
2. Walk through the cross-account sharing flow — what does RAM do, and what does the receiving account create?
3. Why does Iceberg + S3 Tables solve the compaction problem?
4. DataZone vs Unity Catalog — what's the Iceberg REST interop story?
5. How does LF-TBAC scale better than named-resource grants in a 600-account estate?

9. Cross-references

- **Module 10** — S3 (the substrate, S3 Tables)
- **Module 14** — Glue Spark writes/reads via the catalog
- **Module 22, 24** — Redshift Spectrum + Athena consume catalog
- **Module 46** — Unity Catalog on AWS (the Databricks side of the interop)
- **Module 52** — Databolt-style tokenization upstream of LF

Primary sources

- [Lake_Formation_DG.pdf](#)
- [Glue_DeveloperGuide.pdf](#)
- [DataZone_UserGuide.pdf](#)
- Research report: [04_storage_data_lake.md](#)

Module 13 — Glue Fundamentals

What this is: AWS Glue — crawlers, Data Catalog, jobs (Spark + Python shell + Ray), Glue Studio, DataBrew, bookmarks, triggers, workflows.

1. The core components

Glue Data Catalog — see Module 12. Metastore consumed by everything.

Crawlers — automated schema-discovery jobs that scan a data source (S3 path, RDS, Aurora, etc.) and create/update Glue Catalog tables. Run on schedule or on-demand. Common pattern: hourly crawler on a partitioned S3 path.

Glue Jobs — three runtimes: - **Spark** (Apache Spark on Glue’s managed cluster). - **Python shell** — single-node Python for small ETL. - **Ray** — distributed Python via Ray (newer, niche).

Glue Studio — visual editor that compiles to PySpark or Scala. Useful for discovery; engineers tend to graduate to handwritten scripts.

DataBrew — visual data prep with 300+ transformations, recipes that compile to processing jobs. Targeted at data analysts; the “I want to clean a dataset without writing code” surface.

Bookmarks — Glue’s incremental-ETL primitive. Tracks processed files / partitions / rows so re-runs skip already-processed data. **The single best feature of Glue for cost discipline.**

Triggers — schedule (cron) or event-based (job state changes).

Workflows — DAG of jobs + crawlers + triggers. Simpler than Step Functions but limited; usually outgrown.

2. DynamicFrame vs DataFrame

Glue introduces a `DynamicFrame` on top of Spark `DataFrame`:

- **DynamicFrame** — schema is *per record* (handles inconsistent schemas in semi-structured data), supports `ResolveChoice` for type ambiguity, integrates with Glue Catalog + bookmarks.
- **DataFrame** — standard Spark `DataFrame`; faster for known schemas.

Practical rule: start with DynamicFrame for source ingest (raw data has schema drift); convert to DataFrame for transformations once schema is stable; convert back to DynamicFrame for write-out via Glue Catalog if you need bookmarks.

3. Schema evolution

- Crawlers detect added columns and partition values.
- Glue 4.0+ writes Iceberg natively, which gives proper schema evolution semantics (rename, drop, reorder).
- For non-Iceberg outputs, schema evolution is by Glue Catalog version, with manual reconciliation.

4. Example pipeline (raw to curated)

```
# Glue 5.0 Spark job
from awsglue.context import GlueContext
from pyspark.context import SparkContext
from awsglue.job import Job

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# Read raw with bookmarks
raw = glueContext.create_dynamic_frame.from_catalog(
    database="raw", table_name="card_transactions",
    transformation_ctx="raw_card_transactions" # enables bookmarks
)

# Standardize
df = raw.toDF().filter("amount > 0").withColumn("event_date", to_date("ts"))

# Write Iceberg
df.writeTo("curated.card_transactions") \
    .using("iceberg") \
    .partitionedBy(days("event_date")) \
    .createOrReplace()

job.commit()
```

5. 2024–2026 changes

- **Glue 5.0 GA** — Spark 3.5, Python 3.11.
- **Glue Iceberg REST Catalog** (GA 2024).
- **Glue Data Quality** built-in (DQDL — Data Quality Definition Language).
- **Amazon Q in Glue** — natural-language-to-Glue-job generation.

6. Pitfalls

- **Crawler discovers a new partition** but the job hasn't seen it → bookmark misses it.
- **DynamicFrame everywhere** — slow vs DataFrame.
- **Glue Studio code drift** — engineers edit the generated script, then Studio overwrites it on next save.
- **Workflows for complex orchestration** — outgrow it; switch to Step Functions.

7. Capital One lens

Glue is a core piece of the MLOps spine — feature engineering jobs that feed SageMaker training. Step Functions + Glue + EMR + SageMaker is the pattern (Sr Lead MLE JD).

8. Sanity check

1. When would you pick a DynamicFrame over a DataFrame?
2. What does a Glue bookmark track, and why does it matter for cost?
3. Why might a crawler discover a partition that a job then misses?
4. Glue Workflows vs Step Functions — when do you outgrow Workflows?
5. What does Glue 5.0 change vs Glue 4.0?

9. Cross-references

- **Module 12** — Glue Catalog + Lake Formation
- **Module 14** — Glue Spark performance deep
- **Module 15** — orchestration alternatives (Step Functions, MWAA)
- **Module 22** — Redshift consuming Glue-cataloged data via Spectrum

Primary sources

- [Glue_DeveloperGuide.pdf](#)
- [Glue_Best_Practices.pdf](#)
- Research report: [05_glue_etl.md](#)

Module 14 — Glue Spark Deep

What this is: Glue Spark architecture, worker types, autoscaling, Glue 3.0 vs 4.0 vs 5.0, performance tuning. The depth a Sr Lead needs to optimize a slow Glue job.

1. Worker types (2026)

Worker	vCPU	Memory	Use case
G.025X	2	4 GB	Streaming (small)
G.1X	4	16 GB	Default ETL
G.2X	8	32 GB	Memory-hungry ETL
G.4X	16	64 GB	Large joins, wide DataFrames
G.8X	32	128 GB	Very large memory-pressure workloads
Z.2X	8	64 GB	Ray (distributed Python)

A **DPU** (Data Processing Unit) = 4 vCPU + 16 GB a G.1X worker. Pricing is per-DPU-second.

2. Autoscaling

Glue autoscales Spark executors based on Spark task scheduler signals. The autoscaling is *additive only by default* — once scaled up, Glue tends to hold workers. Enable **--enable-auto-scaling true** and pair with **--enable-job-insights true** for visibility.

3. Glue 3.0 vs 4.0 vs 5.0

	Spark	Python	Highlights
Glue 3.0	3.1	3.7	Photon-equivalent Glue runtime

	Spark	Python	Highlights
Glue 4.0	3.3	3.10	Native Iceberg/Hudi/Delta, Adaptive Query Execution defaults, ~30% faster than 3.0
Glue 5.0	3.5	3.11	Iceberg REST catalog, improved auto-tune, Arrow-based shuffle

Pin Glue version in PR templates. Behavior differences in Iceberg writes, AQE behavior, and Catalog interactions across versions are real.

4. Glue Streaming

Spark Structured Streaming on Glue with windowing, watermarking, exactly-once. Sources: Kinesis, MSK, Kafka. Sinks: S3 (Iceberg), Redshift, OpenSearch.

Worker type **G.025X** is purpose-built for streaming (low memory, fan-out).

5. Pushdown predicates

`pushdownPredicate` in `create_dynamic_frame.from_catalog` — Glue translates the filter into a partition predicate that filters Glue Catalog partitions *before* Spark sees them. **Critical for large partitioned tables** — without it, Glue scans every partition.

```
glueContext.create_dynamic_frame.from_catalog(
    database="raw", table_name="txns",
    push_down_predicate="dt >= '2026-05-01' AND dt < '2026-05-15'"
)
```

6. Partitioning rules

- Partition keys should be **low-cardinality** (year/month/day, region) and **frequently filtered on**.
- Avoid partitioning by high-cardinality fields (`user_id`) — too many tiny files.
- Aim for **~256 MB - 1 GB** per output file.

7. Performance tuning checklist

1. **Enable Job Insights** (`--enable-job-insights true`) for Spark UI access.
2. **Push down predicates** via `push_down_predicate`.
3. **Watch the Spark UI** for skew (long tail tasks).

4. **Increase parallelism** with `spark.sql.shuffle.partitions` (default 200; bump to 800-2000 for big jobs).
5. **Coalesce output** before write to avoid small files: `df.coalesce(N)` with $N = \text{total_size} / 512\text{MB}$.
6. **Use DataFrame over DynamicFrame** for known-schema transformations.
7. **Flex execution** (Glue 4.0+) — runs on spare capacity at 35% lower cost, with longer latency. Good for non-urgent batches.

8. Flex execution

--execution-class FLEX — Glue tries to run on idle capacity; takes longer to start but cheaper. Standard runs immediately at full price.

9. Pitfalls

- **Forgetting push_down_predicate** on partitioned tables → full scan.
- **Tiny output files** → query slowness downstream.
- **Skewed joins** → one task takes 90% of the job's time.
- **DynamicFrame for transformations** that don't need it → 2-3x slowdown.
- **G.1X workers for memory-pressured jobs** → executor OOMs.
- **Bookmark not configured** on incremental jobs → re-process everything.

10. Capital One lens

Feature engineering jobs run as Glue Spark with **Flex execution for non-urgent batches**, full price for SLA-bound jobs. Pinned Glue version per PR. Iceberg writes via Glue 4.0/5.0 with the Iceberg REST catalog.

11. Sanity check

1. What's a DPU and how does it map to worker types?
2. How does `push_down_predicate` change query performance?
3. When does Flex execution pay off vs standard?
4. How do you tune `spark.sql.shuffle.partitions` for a 10 TB join?
5. Why is partitioning by `user_id` usually a mistake?

12. Cross-references

- **Module 13** — Glue fundamentals (prerequisite)
- **Module 12** — Glue Catalog + Iceberg
- **Module 22** — Redshift Spectrum consumes Glue-managed Iceberg
- **Module 35** — SageMaker Processing Jobs as an alternative for ML preprocessing

Primary sources

- [Glue_DeveloperGuide.pdf](#)
- Research report: [05_glue_etl.md](#)

Module 15 — ETL Orchestration Choices

What this is: the decision framework for AWS orchestration — Step Functions vs MWAA vs EventBridge Pipes vs SageMaker Pipelines vs Glue Workflows. Plus a walk-through of the **Capital One MLOps orchestration spine**.

1. The options

Tool	Strengths	Weaknesses
Step Functions	Native AWS service integrations (200+), JSON/JSONata definition, error handling, parallel & map states, two flavors (Standard, Express)	State payload size limits, JSON DSL learning curve
MWAA (Managed Airflow)	Python DAGs, huge ecosystem of operators, familiar to data engineers	Environment management overhead, version upgrades painful, \$\$\$ at scale
EventBridge Pipes	Source→filter→enrich→target serverless pattern, no Lambda glue needed	Limited per-pipe transformations
EventBridge Rules + Schedules	Cron + event matching, scheduler GA 2022	Not a full DAG engine
SageMaker Pipelines	ML-native (steps for training/processing/registry), integrated with SM Studio, supports @step decorator	SageMaker-only; less flexible for non-ML steps
Glue Workflows	Native to Glue jobs/crawlers/triggers	Limited; outgrown quickly

2. Step Functions deep

Standard vs Express:

	Standard	Express
Max duration	1 year	5 min
Pricing model	Per state transition (\$0.025 / 1k)	Per request + execution time
Use case	Long-running ML pipelines	High-volume event handling
Execution history	Visible	CloudWatch Logs only

Variables and JSONata (2024) — Step Functions added inline variables and JSONata expressions, dramatically reducing the boilerplate. Now you can do `$states.input.txnId` and not need a Lambda just to extract a field.

3. MWAA versions and sizing

- MWAA 2.7.x (current stable as of 2026-05).
- Environment classes: `mw1.small` (\$0.49/hr), `mw1.medium` (\$0.78/hr), `mw1.large` (\$1.55/hr), `mw1.xlarge` (\$3.10/hr) — plus workers + scheduler + DB. A medium env with 5 workers runs ~\$1,500/mo before DAG run cost.
- Upgrade pain: in-place upgrades are non-trivial; many shops spin a new env and migrate DAGs.

4. EventBridge Pipes

A serverless source→filter→enrich→target pattern:

```
[Source: SQS]    → [Filter: only orders > $100]
                  → [Enrich: Lambda fetches customer profile]
                  → [Target: SageMaker async endpoint for fraud scoring]
```

No Lambda glue needed for the filter; no SQS-to-Lambda-to-something boilerplate. Patterns once expressed as “Lambda fetches from SQS, applies filter, transforms, calls API” collapse to a 30-line Pipe definition.

5. SageMaker Pipelines

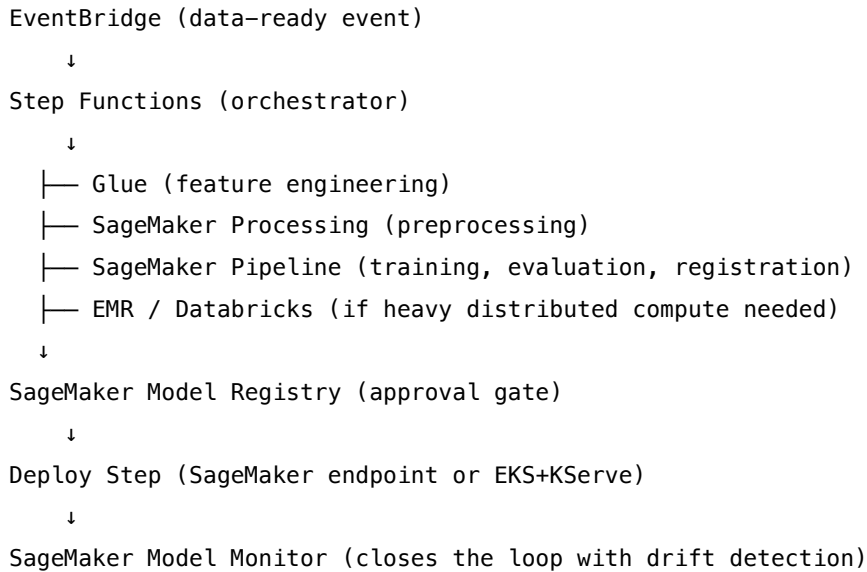
ML-native DAG. Steps: - `ProcessingStep`, `TrainingStep`, `TuningStep` - `ModelStep`, `RegisterModelStep` - `ConditionStep`, `ClarifyCheckStep`, `QualityCheckStep` - `LambdaStep`, `CallbackStep` (escape hatches) - **2024: @step decorator** — Python function-as-step pattern. - **MLflow integration:** SageMaker hosts MLflow tracking server.

6. Glue Workflows

Limited. Use for DAGs that are 100% Glue. Anything with Lambda, ECS, SageMaker — graduate to Step Functions.

7. The Capital One MLOps orchestration spine

Based on their re:Invent 2024 talks and Sr Lead MLE job posting:



8. Pitfalls

- **Step Functions state payload size limits** — 256 KB. Pass S3 references, not blobs.
- **MWAA env upgrades** — often easier to rebuild than upgrade in place.
- **EventBridge Pipes payload size** — 256 KB before/after enrichment.
- **SageMaker Pipelines step max size** — practical limit on the number of steps and the size of step inputs/outputs.
- **Glue Workflows complexity** — outgrow quickly; bite the Step Functions migration early.

9. Capital One lens (talking point)

In an architecture review: *“For new ML pipelines I’d default to Step Functions for orchestration with SageMaker Pipelines for the training sub-DAG, EventBridge Pipes for source-side fan-in, and MWAA only for legacy DAGs the team already has.”*

10. Sanity check

1. Step Functions Standard vs Express — when does Express pay off?
2. What was the 2024 Step Functions feature that reduced the need for “Lambda just to extract a field”?

3. EventBridge Pipes — what's the source→filter→enrich→target shape useful for?
4. When does Glue Workflows stop scaling, and what do you graduate to?
5. Walk through the Capital One MLOps orchestration spine end-to-end.

11. Cross-references

- **Module 13, 14** — Glue (the data layer Step Functions orchestrates)
- **Module 28, 29** — Kinesis / MSK / EventBridge (the streaming sources)
- **Module 38** — SageMaker MLOps (Pipelines deep)
- **Module 53** — Capital One MLOps spine (the integrated view)

Primary sources

- [Step_Functions_DeveloperGuide.pdf](#)
- [SageMaker_Pipelines_DG.pdf](#)
- Research report: [05_glue_etl.md](#)

Module 16 — Relational Basics — RDS Family

What this is: Amazon RDS managed relational databases (Postgres, MySQL, MariaDB, Oracle, SQL Server, Db2), with the production essentials — Multi-AZ, read replicas, RDS Proxy, Blue/Green deployments, IAM auth, encryption.

1. Supported engines (as of 2026-05)

- **PostgreSQL** — default for greenfield in regulated enterprises. Versions through 17.x. **pgvector** ships in 0.7+ for embedding-similarity search in OLTP.
- **MySQL** — 5.7 (extended support), 8.0, 8.4 LTS.
- **MariaDB** — drop-in MySQL alternative; less common in regulated estates.
- **Oracle** — BYOL or License Included; CDB architecture required as of 21c.
- **SQL Server** — Express, Web, Standard, Enterprise editions; Always On not exposed (use Multi-AZ).
- **Db2** — added Dec 2023 (LUW, BYOL). Capital One historically ran Db2 on z/OS; managed RDS-Db2 is the off-ramp.

2. High availability — Multi-AZ Instance vs Cluster

Feature	Multi-AZ Instance	Multi-AZ Cluster
Standby count	1 (sync)	2 (semi-sync)
Standbys readable?	No	Yes (read-after-write caveats)
Failover time	60-120s	<35s typical
Engines supported	All	Postgres, MySQL only
Storage	Single EBS per node	Per-node local storage

Read replicas are async-replicated copies for read scale-out. Intra-region uses engine-native replication; cross-region adds encryption-key replication and ~seconds-to-minutes lag. Promotion breaks the replication relationship.

3. RDS Proxy

Fully managed connection pool. Mandatory for **Lambda** → **RDS** at any scale: Lambda's per-invocation connection burst kills Postgres `max_connections`.

- Supports IAM auth pass-through.
- Secrets Manager rotation — no app restart on credential rotation.
- Adds ~5ms latency.
- Costs ~\$0.015 / vCPU-hour of the underlying instance.

4. Blue/Green Deployments

Clone the production cluster, apply schema / engine changes on the green stack, then switch over with replication ensuring data parity. **Switchover typically ~1 minute** of write downtime.

2024 milestones: GA for SQL Server, cross-region promotion for Aurora.

Use for: major-version upgrades, schema changes, instance class moves.

5. Other operational essentials

- **Parameter groups** (engine config: `work_mem`, `max_connections`) vs **option groups** (engine features: Oracle OEM, SQL Server Audit). DB-level vs cluster-level scope matters for Aurora.
- **Automated backups** — 1-35 day retention, PITR to a second. Manual snapshots persist until deleted; encrypted snapshots can be copied cross-region/cross-account.
- **IAM authentication** — 15-min token replaces password. Cap: 200 new connections/sec per instance; use RDS Proxy to amortize.
- **Encryption at-rest** — via KMS (customer or AWS-managed CMK). Cannot toggle on a live instance — must snapshot, copy with encryption, restore.
- **TLS** — `rds-ca-rsa2048-g1` is the current bundle as of 2026. The 2024 CA rotation forced fleet-wide client updates — plan for the next rotation.
- **Performance Insights** — 7 days free retention, paid up to 2 years. **AAS (Average Active Sessions)** is the headline metric — anything above vCPU count means saturation.

6. 2024-2026 changes

- **RDS for Db2 GA** Dec 2023.
- **Postgres 17** support early 2025.
- **MySQL 8.4 LTS** late 2024.
- **Extended Support pricing** kicked in for MySQL 5.7 and Postgres 11 at \$0.10/vCPU-hr after community EOL — budget this for legacy.
- **Blue/Green for SQL Server** GA 2024.
- **pgvector 0.7** with HNSW filtered search.
- **RDS storage autoscaling now supports gp3 from gp2 in-place** (no rebuild).

7. Pitfalls

- **Storage autoscaling traps** — max storage threshold is a hard ceiling, not advisory. Once hit, writes fail. Set with 50% headroom and alarm at 70% used.
- **gp2 burst exhaustion** — under 1TB you live on burst credits. Move to gp3 for predictable performance below the 1TB threshold.
- **Free storage = log files + temp** — long-running Postgres transactions bloat WAL and `pg_temp`. Alert on `FreeStorageSpace` not just CPU.
- **Connection storm during failover** — Multi-AZ failover invalidates DNS; clients with cached connections must reconnect. RDS Proxy hides this from app pods.

8. Capital One lens

Capital One is Postgres-heavy for transactional services (per public re:Invent talks), with Aurora for the largest workloads. RDS-on-Postgres is the default for medium services where Aurora's premium isn't justified. Db2 RDS is a likely candidate for mainframe-modernization tracks. **RDS Proxy is mandatory for any Lambda fronting Postgres.**

9. Pricing nuance

- A `db.r6g.large` Multi-AZ instance is roughly 2x single-AZ price (you pay for the standby).
- gp3 storage is ~20% cheaper than gp2 and decouples IOPS from size.
- Reserved Instances yield 30-60% discount over 1y/3y terms; aligns poorly with cloud-native autoscaling but well with steady-state OLTP.

10. Sanity check

1. Multi-AZ Instance vs Cluster — when does Cluster pay off?
2. When is RDS Proxy mandatory?
3. What does Blue/Green deployment buy you, and what's the typical downtime?
4. Why does IAM auth need RDS Proxy at any scale?
5. What's the AAS metric in Performance Insights?

11. Cross-references

- **Module 17** — Aurora (RDS's higher-performance sibling)
- **Module 19** — DocumentDB (Aurora-style architecture, NoSQL)
- **Module 22, 24** — Redshift/Athena query federation reading from RDS
- **Module 32** — Lambda + RDS Proxy pattern

Primary sources

- [RDS_Best_Practices.pdf](#)

- Research report: [06a_relational_dynamodb.md](#)

Module 17 — Aurora Deep

What this is: Aurora architecture (6-way replicated storage), Aurora Postgres vs MySQL, Aurora Serverless v2, **Aurora Limitless** (sharded, GA Dec 2024), and **Aurora DSQL** (distributed multi-region active-active Postgres, GA May 2025 — the 2025 headline service). Plus Global Database, Babelfish, I/O-Optimized.

1. The architecture innovation

Aurora's key idea: **separate compute from storage**.

- Compute nodes (writer + readers) are stateless DB engines.
- Storage is a distributed, log-structured fleet replicating each **10 GB segment 6 ways across 3 AZs** (2 copies per AZ).
- Write quorum: **4 of 6**. Read quorum: **3 of 6**. Aurora can lose an entire AZ plus one more copy and stay available for both reads and writes.

Replication is via **redo-log shipping to storage**, not buffer-page replication. Readers tail the same log; replica lag is typically **10-20 ms** vs RDS's seconds.

2. Aurora Postgres vs Aurora MySQL

	Aurora PG	Aurora MySQL
Compatibility	PG 11–16	MySQL 5.7, 8.0
Babelfish	Yes (SQL Server wire compat)	No
ML integration	Yes (Bedrock, SageMaker, Comprehend)	Yes
Limitless	Yes (GA Dec 2024)	Preview only
Parallel Query	Limited	Broader

3. Aurora Serverless v2

v2 fixes v1's biggest sin: cold starts. v2 maintains a warmer pool of pre-provisioned ACUs and scales in **0.5 ACU increments within seconds**, not minutes.

- v1 is deprecated as of late 2024 — migrate.
- v2 supports Global Database, read replicas, Data API.
- Pricing: 1 ACU 2GB RAM + proportional CPU at ~\$0.12/ACU-hour.
- **Crossover vs provisioned r6g.large** (~\$0.29/hr): ~50% sustained utilization. Below that, serverless v2 wins.

4. Aurora Limitless Database (GA Dec 2024)

Horizontally-sharded Postgres-compatible database.

- Define **sharded tables** (partitioned by a column like `customer_id`).
- Define **reference tables** (replicated to every shard).
- A **routing layer** (DB shard group endpoint) parses SQL and routes to the right shard.
- **Cross-shard transactions** use distributed 2PC.
- Claimed throughput at re:Invent 2024: **2M+ writes/sec**.

Use when you've outgrown single-writer Aurora (typically 200K+ writes/sec sustained) and don't need DSQL's multi-region active-active.

5. Aurora DSQL (GA May 2025) — the headline

Aurora DSQL (Distributed SQL) is a serverless, **multi-region active-active Postgres-compatible** database. AWS's answer to Spanner and CockroachDB.

Key properties: - **Multi-region active-active strongly consistent writes** with single-digit-ms commit latency in-region. - Writes use a distributed time-ordered commit protocol on **Time Sync Service** (microsecond-precise hardware clocks). - **Postgres-compatible** wire protocol and SQL surface — subset at GA (no foreign keys, no triggers, no sequences with strict ordering). - **No instance management** — fully serverless. - **Optimistic concurrency control** — transactions can abort with serialization errors; app must retry. - Pricing: **per DPU** (Distributed Processing Unit), per-byte-stored, per-region.

Use cases: - Globally distributed financial transactions. - Multi-region SaaS where every region needs read+write. - Regulatory data sovereignty with availability.

Capital One implication: DSQL maps directly onto multi-region active-active aspirations for **card auth and fraud-decision systems**. This is strategically the most important new database service AWS has shipped since DynamoDB Global Tables. Watch closely.

6. Aurora Global Database

Storage-level replication to **up to 5 secondary regions** with <1s lag typical. Secondary is read-only until promoted.

- Managed planned failover: 2 min RPO.
- Unplanned failover: RPO seconds, RTO ~1 min.

Different from DSQL: GD is asynchronous primary/secondary; DSQL is sync active-active.

7. Babelfish

Postgres extension that speaks the **Microsoft SQL Server TDS wire protocol** and T-SQL dialect.

Migrate SQL Server apps to Aurora PG **without rewriting client code**. Compatibility is partial — not all T-SQL constructs supported. Use the **Babelfish Compass** tool to assess.

8. Aurora I/O-Optimized

Pricing tier where you pay ~30% more on instance hours and storage, but **I/O is free**.

Standard Aurora bills per I/O at \$0.20 per million requests.

Breakeven: workloads where I/O > ~25% of total Aurora bill. **High-write, high-scan OLTP almost always wins on I/O-Optimized; sleepy report DBs stay on Standard.**

9. Aurora ML integration

`aws_ml.invoke_endpoint` calls SageMaker; `aws_bedrock.invoke_model` calls Bedrock; `aws_ml.detect_sentiment` calls Comprehend — all from SQL.

Useful for inline embedding generation and feature serving — **but careful:** a hot SELECT against Bedrock will rack up cost fast. Cache results.

10. Pitfalls

- **Reader endpoint stickiness** — Aurora's reader endpoint round-robins on DNS resolution, but JDBC pools cache one IP. Use **AWS JDBC Wrapper Driver** for proper reader load balancing.
- **Storage cost surprises** — Aurora bills storage on high-water-mark; deletes don't shrink. Use `VACUUM FULL` (carefully) or dump/restore.
- **Failover read-after-write** — after failover, old writer becomes a reader briefly; stale reads possible. Application must handle.

11. Capital One lens

- **Aurora Postgres** for transactional services at scale (likely the platform's default for new builds).
- **Aurora Serverless v2** for non-prod environments and bursty workloads.
- **Aurora DSQL** is the architecturally interesting service for multi-region card auth and fraud decisioning — expect Capital One to be a public reference customer at some re:Invent in 2025-26.
- **I/O-Optimized** for write-heavy fraud / fraud-feature tables.

12. Sanity check

1. How does Aurora's 6-way replicated storage tolerate failure?
2. When does Aurora Serverless v2 win economically over provisioned?

3. Limitless vs DSQL — what's the difference in consistency model?
4. What does Babelfish solve, and what's its compatibility tool?
5. When is I/O-Optimized worth the 30% instance premium?

13. Cross-references

- **Module 16** — RDS family (Aurora is built on the RDS control plane)
- **Module 27** — pgvector in Aurora for RAG
- **Module 18** — DynamoDB Global Tables (Aurora DSQL alternative for K-V)
- **Module 41** — SageMaker integration with Aurora ML extensions

Primary sources

- [Aurora_User_Guide.pdf](#)
- [Aurora_DSQL_Whitepaper.pdf](#)
- Research report: [06a_relational_dynamodb.md](#)

Module 18 — DynamoDB Deep

What this is: the canonical AWS NoSQL key-value/document store — single-table design, partition keys + GSI/LSI, Streams, TTL, on-demand vs provisioned, DAX, transactions, Global Tables, PITR, hot-partition pathologies.

1. Single-table design

DynamoDB rewards denormalization. The canonical pattern: **one table holds all entity types**, distinguished by generic partition (PK) and sort keys (SK) with overloaded values:

PK	SK	... attributes
USER#123	PROFILE	email="...", name="..."
USER#123	ORDER#2026-05	amount=100, status="shipped"
USER#123	SESSION#abc	last_seen=...

Access patterns are encoded into key prefixes; queries are `PK = X AND SK BEGINS_WITH Y`. **Trade-off:** schema lives in app code, tooling weaker, but reads are $O(1)$ with no joins.

2. Keys and indexes

	Description
Partition key alone (HASH)	Point lookups only
PK + SK (HASH + RANGE)	Range scans within partition
GSI (Global Secondary Index)	Different PK/SK; async-replicated; separate throughput. Up to 20 per table (raised 2023 from 5)
LSI (Local Secondary Index)	Same PK, different SK; sync; must be defined at table creation ; 10 GB per-partition cap inherits

3. Streams and triggers

DynamoDB Streams emit ordered change records per partition for **24 hours**. Common sinks:

- **Lambda triggers** — fire on each batch; the classic CDC pattern (item updated → invalidate cache, send notification, fan to OpenSearch).
- **Kinesis Data Streams** integration (since 2020) — 1-year retention, multiple consumers, replay capability. Preferred for analytics fanout.

4. TTL

Per-item epoch attribute. Items are deleted **within 48 hours** of expiry — **not at the second** (common bug). TTL deletes emit Stream events with `userIdentity: dynamodb`. **Free**.

5. Capacity modes

- **Provisioned** — you set RCU/WCU; auto-scaling supported. Reserved Capacity 1y/3y for ~50-75% discount.
- **On-demand** — pay-per-request, scales instantly up to **2× the previous peak** (peak limit raised significantly in 2024 to 1M RCU / 1M WCU per table on request).

Switching mode allowed once per 24h.

Crossover: on-demand wins below ~14-18% sustained utilization of equivalent provisioned. Spiky traffic → on-demand; flat 24/7 traffic → provisioned + reserved.

6. DAX (DynamoDB Accelerator)

Write-through cache for **microsecond reads** (vs single-digit-ms direct).

- VPC-bound cluster.
- Item cache (`GetItem`) and query cache (`Query/Scan`).
- **Limitations:** no consistent-reads through DAX item cache; cluster failover ~30s.
- Pricing: per node-hour like ElastiCache (~\$0.04/hr for `dax.t3.small` up to \$4+/hr for `dax.r5.16xlarge`).

7. Transactions

`TransactWriteItems` / `TransactGetItems`.

- Up to **100 items** (raised 2022 from 25).
- **4 MB total** payload.
- ACID, all-or-nothing.
- **Twice the cost** of normal writes/reads.

Use for multi-item invariants (debit + credit ledger pair).

8. Global Tables

Multi-region active-active replication with **last-writer-wins** conflict resolution based on stream timestamps.

- Eventual consistency cross-region (~1s typical).
- Replication is free per-row; you pay RCU/WCU in each region.
- **Global Tables v2 (2019)** is the only supported version; v1 deprecated mid-2024.

9. Point-in-Time Recovery (PITR)

35-day continuous backup, second-granular restore to any new table. **Doubles the storage bill while enabled** — budget for it.

10. Import/Export with S3

- **Import from S3** (2022) — bulk-create a new table from S3 (DynamoDB JSON, ION, CSV). Free except for table writes — but writes are billed as $\text{WCU} \times \text{items}$.
- **Export to S3** — point-in-time snapshot to S3, ION or JSON. Doesn't consume RCU. Use for analytics via Athena / Glue.
- **2024:** incremental export to S3 — only changed items since last export. Turns DynamoDB $\rightarrow$ S3 $\rightarrow$ Athena into a low-cost CDC pipeline.

11. Hot partitions and adaptive capacity

Each partition caps at **3000 RCU / 1000 WCU**.

A skewed key (e.g., `customerId` for a viral user) saturates one partition while the table is nominally fine.

Adaptive capacity (always-on since 2019) rebalances throughput across partitions but **cannot break the per-partition cap**.

Fix: write sharding — append a random suffix `customerId#0..9` to spread writes, then query 10 partitions and merge.

12. Expression syntax

Three expression types in queries: - **Key Condition Expression** — what defines the range (`PK = :v AND SK BEGINS_WITH :p`). - **Filter Expression** — post-filter applied after read (still consumes RCU on filtered-out items). - **Projection Expression** — which attributes to return.

13. Consistent vs eventually consistent reads

- **Eventually consistent** (default) — half the RCU cost; possible to read stale data within ~1 sec.
- **Strongly consistent** — full RCU cost; latest write returned.

14. DynamoDB Local

A downloadable Java app that mimics the API for local development. Doesn't perfectly replicate Streams or some auto-scaling behaviors, but good enough for unit tests.

15. Pitfalls

- **Hot partition** — single key dominating. Use write sharding.
- **GSI throttling** — separate WCU; if you under-provisioned the GSI, writes propagate slowly. **Provisioned mode: pay attention to GSI capacity separately.**
- **Filter Expression illusion** — filtering doesn't reduce RCU; it just hides items after the read.
- **Scan instead of Query** — scans every item. Almost never the right answer; design access patterns to be Queries.

16. Capital One lens

DynamoDB is the natural fit for Capital One's serverless-first culture: - Lambda + DynamoDB is the canonical pattern. - Per-LOB tables with consistent naming and tagging. - On-demand mode for new services; switch to provisioned + reserved as traffic stabilizes. - DynamoDB Streams → Lambda → fraud-detection model is a common pattern.

17. Sanity check

1. What is single-table design and what's the trade-off?
2. GSI vs LSI — when does each make sense, and what's the LSI gotcha?
3. What's the per-partition throughput cap, and what's the workaround for hot keys?
4. Why does on-demand vs provisioned crossover happen around 14-18%?
5. What does PITR cost, and what does it cover?

18. Cross-references

- **Module 21** — ElastiCache + MemoryDB (alternative low-latency stores)
- **Module 22** — DynamoDB → Redshift zero-ETL
- **Module 32** — Lambda + DynamoDB Streams (the canonical CDC pattern)
- **Module 56** — DynamoDB cost discipline

Primary sources

- [DynamoDB_Best_Practices.pdf](#)
- Research report: [06a_relational_dynamodb.md](#)

Module 19 — DocumentDB Deep

What this is: Amazon DocumentDB — MongoDB-compatible document database with Aurora-style architecture, Elastic Clusters for sharding, vector search for RAG, change streams.

1. Architecture

DocumentDB borrows Aurora’s separation of compute and storage. Each cluster has:

- **Primary** (writer)
- **Replicas** (readers, up to 15)
- **6-way replicated storage across 3 AZs**, like Aurora.

You pay per-instance, like RDS. **Instance-based pricing** is the biggest cost differentiator from MongoDB Atlas’s cluster-tier pricing.

2. MongoDB compatibility

- Compatible with **MongoDB 5.0 API**. Older docs say 4.0; this was upgraded to 5.0 in 2023.
- Some MongoDB API gaps: change streams structure differs slightly; certain operators not implemented; no `$lookup` across collections (added in 4.0+); aggregation pipeline supported.
- Use the official `mongo` shell or Compass; clients work as long as they target compatible API version.

3. Sharding via Elastic Clusters

DocumentDB **Elastic Clusters** (GA 2023) allow horizontal sharding for large workloads:

- Up to **32 shards** per cluster.
- Up to **64 instances per shard**.
- Multi-billion-document collections supported.
- Shard keys define partitioning.
- Capital One use case: large document workloads (e.g., per-customer JSON profiles).

4. Indexes

- **Compound** indexes — multi-field, ordered.
- **Multikey** indexes — on array fields.
- **Geospatial** — 2dsphere for lat/lon queries.
- **Text** — full-text on string fields.
- **Hashed** indexes — even-distribution sharding.
- **TTL indexes** — auto-expire docs.

5. Vector search

DocumentDB vector search (GA 2024) — vector indexes for RAG and similarity search:

- **HNSW** and **IVFFlat** index types.
- Up to **2,000 dimensions**.
- Inner product, cosine, Euclidean distance metrics.
- Capital One use case: **in-VPC RAG** — embeddings of compliance docs / customer FAQs stored in DocumentDB without leaving the VPC.

6. Aggregation pipeline

Standard MongoDB-style aggregation: `$match`, `$group`, `$project`, `$lookup`, `$unwind`, `$facet`, etc. Used for analytics inside DocumentDB without separate ETL.

7. Change streams

Like MongoDB. Tail the change stream from a Lambda or Glue job to react to inserts/updates/deletes. Used for CDC, cache invalidation, downstream sync.

8. IAM authentication

GA 2023. Replaces password auth with IAM SigV4 tokens. Pairs with IRSA / Pod Identity for EKS workloads.

9. Migration patterns

- **AWS DMS (Database Migration Service)** can migrate from self-managed MongoDB to DocumentDB.
- Some operators may need rewriting due to API gaps.
- Capital One pattern: lift acquired companies' MongoDB workloads onto DocumentDB via DMS.

10. DocumentDB vs MongoDB Atlas on AWS

	DocumentDB	MongoDB Atlas on AWS
Native AWS integration	Yes (IAM, KMS, VPC)	Limited (BYO IAM)
Latest MongoDB version	5.0 API	Latest (7.x+ at 2026)
Aggregation pipeline	Most	All
Atlas-specific features	No	Atlas Search, Charts, Triggers
Pricing	Instance-based	Cluster-tier
FedRAMP / regulated	Yes (AWS-native)	Limited

For regulated finance, **DocumentDB wins** on AWS-native security integration. For pure feature parity with MongoDB, Atlas wins.

11. 2024-2026 changes

- **DocumentDB vector search** GA 2024.
- **Elastic Clusters** (sharding) GA 2023, refined 2024.
- **IAM auth** GA 2023.
- **MongoDB 5.0 API parity** improvements.

12. Pitfalls

- **API gaps vs MongoDB** — some operators absent. Test before migration.
- **Index size limits** — like MongoDB, indexes count against memory.
- **Cluster failover** — primary failover ~30s, not seconds like Aurora.
- **Connection pooling** — DocumentDB has lower default max connections than self-managed MongoDB. Use a connection pool (mongoose connection pooling, MongoEngine, etc.).

13. Capital One lens

DocumentDB is the natural store for: - Member-profile JSON documents. - Compliance / regulatory document libraries (paired with vector search for RAG). - Acquired-company MongoDB migrations.

14. Sanity check

1. What's DocumentDB's architecture, and what AWS service is it modeled on?
2. What MongoDB API version does DocumentDB target as of 2026?
3. When would you use Elastic Clusters?
4. What's the vector search story, and what use case fits at Capital One?
5. DocumentDB vs Atlas — when does each win?

15. Cross-references

- **Module 27** — vector capabilities decision framework (DocumentDB vs alternatives)

- **Module 42** — Bedrock Knowledge Bases (can DocumentDB be a backend?)
- **Module 17** — Aurora (the architectural parent)

Primary sources

- [DocumentDB_Best_Practices.html](#)
- [DocumentDB_Vector_Search.html](#)
- Research report: [06b_nosql_siblings.md](#)

Module 20 — Amazon Keyspaces (for Apache Cassandra)

What this is: managed Cassandra-API-compatible service. When to pick it over DynamoDB. The “Cassandra without operating Cassandra.”

1. The pitch

Amazon Keyspaces is a managed Cassandra-API service. It exposes the **CQL (Cassandra Query Language)** wire protocol and most CQL surface, but **the underlying storage is not Apache Cassandra**. AWS implements the Cassandra protocol on a proprietary backend that looks more like DynamoDB.

This matters: behaviors that come from Cassandra’s gossip / vnodes / hinted handoff don’t apply.

2. Capacity modes

- **On-demand** — pay-per-request, no provisioning.
- **Provisioned** — RCU/WCU like DynamoDB.

Switch mode once per 24h. Same model as DynamoDB.

3. Multi-region replication

Active-active across up to 6 regions. Conflict resolution is last-writer-wins (similar to DynamoDB Global Tables).

4. Partition design

Same principles as Cassandra: - Choose partition keys that distribute writes evenly. - Clustering keys define order within partition. - Bounded partitions (under ~100MB).

5. CQL surface — what’s supported vs gaps

Supported: - Most basic CQL — SELECT, INSERT, UPDATE, DELETE, CREATE TABLE. - Indexes (limited). - Time-To-Live. - Lightweight transactions (LWT) within a single partition.

Not supported: - **Materialized views** — not implemented. - **User-defined functions (UDFs)**. - **Triggers**. - **Cross-partition LWT**. - Some CQL functions (e.g., `toTimestamp`). - **Tunable consistency** — only ONE and LOCAL_QUORUM are supported.

6. Other constraints

- **Max row size: 1 MB** (Cassandra OSS allows larger). Common trap when porting.
- **No tunable replication factor** — managed by AWS.

7. Keyspaces vs DynamoDB

	Keyspaces	DynamoDB
API	CQL (Cassandra)	DynamoDB API
Use it for	Existing Cassandra apps	New AWS-native designs
Max row	1 MB	400 KB
Cross-partition transactions	No	Yes (TransactWriteItems)
Streams	No (different mechanism)	Yes
Multi-region	Up to 6 regions	Global Tables
Vector search	No	No

Rule: if you don’t have an existing Cassandra investment, default to DynamoDB.

8. Use cases

- **Time-series data** — events with high write throughput, queried by partition + time range.
- **Existing Cassandra apps** migrating to AWS without rewriting CQL.
- **High-throughput KV** with large rows up to 1MB.

9. Pitfalls

- **Porting from open-source Cassandra** — many features absent. Test first.
- **CQL gotchas** — materialized views, UDFs, triggers won’t work; rewrite using app-layer logic.
- **Row size limit** — 1 MB cap surprises ports from Cassandra clusters with multi-MB rows.

10. Capital One lens

Probably used selectively for time-series workloads where the team has Cassandra expertise. DynamoDB is the broader default per their public patterns.

11. Sanity check

1. What's the Keyspaces vs Cassandra OSS difference under the hood?
2. What's the max row size, and how does that constrain Cassandra ports?
3. What CQL features are NOT supported?
4. When does Keyspaces win over DynamoDB?
5. What's the multi-region active-active limit?

12. Cross-references

- **Module 18** — DynamoDB (the AWS-native alternative)
- **Module 21** — ElastiCache + MemoryDB (other low-latency options)

Primary sources

- [Keyspaces_Developer_Guide.html](#)
- [Keyspaces_Best_Practices.html](#)
- Research report: [06b_nosql_siblings.md](#)

Module 21 — ElastiCache & MemoryDB

What this is: AWS's in-memory data stores — ElastiCache (Redis / Valkey / Memcached cache), MemoryDB (durable Redis-compatible primary DB), the Valkey fork story, vector search at sub-millisecond latency.

1. The Valkey story (2024)

In March 2024, Redis Inc. relicensed Redis from BSD to a **dual SSPL/RSALv2 license** — effectively non-open-source.

The Linux Foundation forked Redis as **Valkey** (March 28, 2024, fork point Redis 7.2.4), under BSD. AWS, Google Cloud, Oracle, Ericsson, and SnapChat became founding sponsors.

AWS dropped ~33% off ElastiCache Valkey pricing vs ElastiCache Redis. ElastiCache Serverless and MemoryDB now support Valkey as the modern default for new builds.

2. ElastiCache — the cache tier

Three engine options: - **Valkey** — the open-source fork. Default in 2026 for new builds. - **Redis OSS** — versions through 7.4 still supported. - **Memcached** — simpler key-value cache; no persistence, no replication, no clustering coordination. Niche.

Deployment modes: - **Cluster mode disabled** — single primary + replicas, single shard. - **Cluster mode enabled (CME)** — multi-shard cluster, 500 shards max, 16,384 hash slots. **Migration from CMD → CME is painful** — application must handle hash-slot routing.

ElastiCache Serverless (GA Nov 2023) — pay per data + per ECPU (ElastiCache Processing Unit). No node sizing. Great default for unpredictable workloads.

3. MemoryDB — durable Redis-compatible primary DB

MemoryDB is positioned as a **system-of-record**, not a cache. Same Redis API, but:

- **Multi-AZ durability** via transactional log to S3-like layer.
- **Strong consistency** within primary; eventual consistency across replicas.
- Used as **primary DB** for ultra-low-latency workloads — session state, real-time leaderboards, feature stores.

MemoryDB vector search (GA 2024) — HNSW indexes for sub-millisecond similarity search. The use case: **real-time agent/RAG context retrieval** where the user-facing latency budget is < 50ms.

4. ElastiCache vs MemoryDB

	ElastiCache	MemoryDB
Durability	Cache (data can be lost)	Durable primary DB
Replication	Async to replicas	Sync via transactional log
Use case	Cache, ephemeral state	System of record, feature store
Cost	Lower per GB	~2-3× ElastiCache
Vector search	(via Redis 7.x ANN)	Yes (HNSW, 2024)

5. Cache patterns (ElastiCache)

Pattern	Description
Cache-aside (lazy loading)	App reads cache; on miss, queries DB, writes to cache. Most common.
Write-through	App writes both to DB and cache atomically. Lower miss rate, higher write latency.
Write-behind (write-back)	App writes only to cache; async background writes DB. Risky if cache fails.
TTL-based expiration	Set TTL on every key; cache evicts on expiry. Simple staleness control.

6. Hot key mitigation

Hot key = single key getting disproportionate traffic.

Fixes: - **Read from replicas** for read-heavy hot keys. - **Client-side caching** (Redis 6+ tracking) — clients cache responses, server invalidates on change. - **Sharding by key suffix** — split one logical key across N physical keys, client picks one.

7. Pricing snapshot

- **ElastiCache Valkey** cache.r7g.large ~\$0.227/hr (33% cheaper than Redis on same hardware after AWS pricing change).

- **ElastiCache Serverless** — per ECPU + per GB-hr; ~\$0.0035 per ECPU.
- **MemoryDB Valkey db.r7g.large** ~\$0.515/hr — about 2.3× ElastiCache.

8. 2024-2026 changes

- **Valkey fork** March 2024; AWS GA on ElastiCache + MemoryDB.
- **AWS ~33% Valkey price drop** vs Redis on equivalent hardware.
- **ElastiCache Serverless** GA Nov 2023.
- **MemoryDB Vector Search** GA 2024.
- **Redis 7.4** still supported but Valkey is the recommended forward-looking choice.

9. Pitfalls

- **Cluster mode migration** (CMD → CME) is a non-trivial app rewrite.
- **MemoryDB cost overshoot** when used as a cache instead of system-of-record.
- **Memcached's lack of persistence** means cache restart = full miss.
- **Vector search on small datasets** — adds operational complexity for marginal benefit vs Aurora pgvector.

10. Capital One lens

Likely patterns: - **ElastiCache Valkey Serverless** for service-tier caching. - **MemoryDB** for the online feature store where sub-millisecond is required. - **MemoryDB Vector** for agent context retrieval (Eno, Servicing Tool) where latency budget is tight.

11. Sanity check

1. What's the Valkey story and why does it matter to AWS pricing?
2. When does MemoryDB make sense vs ElastiCache?
3. Cache-aside vs write-through vs write-behind — which is most common and why?
4. How do you mitigate a hot key in Redis?
5. What's the MemoryDB vector search use case at Capital One scale?

12. Cross-references

- **Module 18** — DynamoDB (the durable KV alternative)
- **Module 27** — vector capabilities decision framework
- **Module 35** — SageMaker Feature Store (offline-online split; MemoryDB as the online tier)

Primary sources

- [ElastiCache_Best_Practices.html](#)

- [Valkey_Announcement.html](#)
- [MemoryDB_Vector_Search.html](#)
- Research report: [06b_nosql_siblings.md](#)

Module 22 — Redshift Deep

What this is: Amazon Redshift — RA3 nodes (managed storage), Redshift Serverless, Spectrum (query S3), Data Sharing, Zero-ETL integrations, Streaming Ingestion, Redshift ML.

1. Node types

	Description
RA3	Managed storage (decoupled from compute). Sizes: ra3.xlplus, ra3.4xlarge, ra3.16xlarge. The current default.
DC2	Legacy compute-storage coupled nodes. Deprecation announced; migrate to RA3.

2. Redshift Serverless (GA 2022)

- Pay per **RPU (Redshift Processing Unit)**-second.
- **Base capacity** (min, default 32 RPU; 2024 added 8 RPU floor for smaller workloads) and **max capacity** ceiling.
- Auto-scales between base and max.
- No cluster management.
- **Crossover vs provisioned:** Serverless wins under ~30-40% sustained utilization.

3. Redshift Spectrum

Query **S3 directly** from Redshift using external tables (Glue Data Catalog as metastore). No data movement needed.

- Pricing: \$5/TB scanned (same as Athena).
- Supports Parquet, ORC, JSON, CSV, Iceberg (since 2023).
- Use case: hybrid Redshift-cluster + S3-lake architectures.

4. Data Sharing

Cross-cluster, cross-account, cross-region data sharing **without data movement**.

- Live read-only views into another cluster’s data.
- Producer/consumer model.
- Enables a “share data across LOBs without copies” pattern critical at Capital One scale.

5. Zero-ETL integrations (the 2024 story)

Aurora → Redshift, RDS for MySQL/Postgres → Redshift, DynamoDB → Redshift, Salesforce → Redshift. **Continuously replicates** without you building pipelines.

Source	GA
Aurora MySQL → Redshift	2023
Aurora Postgres → Redshift	2024
RDS MySQL → Redshift	2024
DynamoDB → Redshift	2024
Salesforce → Redshift	2024

6. Materialized views

Pre-computed aggregations refreshed on a schedule or **auto-refresh** (Redshift detects underlying changes).

7. Concurrency Scaling

When a workload exceeds cluster capacity, Redshift transparently spins up additional clusters to absorb the overflow. **1 hour/day free** per cluster.

8. Workload Management (WLM)

- **Automatic WLM** (default, 2024+) — Redshift sizes queues based on workload.
- **Manual WLM** — you define queues with memory, concurrency, query timeouts.

9. Streaming Ingestion

Native ingestion from **Kinesis Data Streams** and **MSK**. Sub-second latency for incoming records to be queryable. Replaces “Kinesis → S3 → COPY into Redshift” pipelines.

10. Redshift ML

In-database ML via **SageMaker integration**. SQL:

```
CREATE MODEL my_model
FROM (SELECT * FROM training_data)
TARGET label_column
FUNCTION my_predict_fn
IAM_ROLE 'arn:...'
SETTINGS (S3_BUCKET '...');
```

```
SELECT my_predict_fn(features) FROM scoring_data;
```

Trains in SageMaker behind the scenes; SQL function calls the deployed model.

2024: Bedrock invocation from SQL (SELECT bedrock_invoke('claude-3-5', prompt)).

11. MERGE

Standard SQL MERGE for upserts (GA in Redshift 2023). Replaces older staging+delete+insert patterns.

12. AQUA (folded)

Advanced Query Accelerator (announced 2020) — quietly folded into RA3 nodes as a performance enhancement, not a separate purchase. Don't worry about it as a separate feature in 2026.

13. 2024-2026 changes

- **Zero-ETL** integrations multiplied (above).
- **Iceberg on Spectrum**.
- **Serverless 8 RPU floor** for small workloads.
- **MERGE** statement GA.
- **Bedrock invocation from SQL**.

14. Pitfalls

- **DC2 deprecation** — migrate to RA3.
- **Forgetting to enable Concurrency Scaling** — workloads queue.
- **Spectrum without partitioning** — scans entire data lake; \$\$\$.
- **Wrong distribution style (DISTSTYLE)** on a large table — joins shuffle entire data.
- **Materialized view staleness** — auto-refresh is best-effort, not real-time.

15. Capital One lens

Capital One runs **both** Redshift and Snowflake (Module 23). Likely pattern: - **Redshift Serverless** for service-team marts. - **Snowflake** for enterprise-shared analytics (data marketplace). - **Zero-ETL** for OLTP-to-warehouse where it fits.

16. Sanity check

1. RA3 vs DC2 — what's deprecating?
2. When does Redshift Serverless beat provisioned?
3. What's the difference between Data Sharing and Spectrum?
4. Walk through the Redshift ML pattern.
5. What's Concurrency Scaling, and what's the free tier?

17. Cross-references

- **Module 23** — Snowflake on AWS (the alternative warehouse)
- **Module 24** — Athena (the serverless lake-query alternative)
- **Module 12** — Glue Catalog + Iceberg
- **Module 28, 29** — MSK / Kinesis (streaming ingest sources)

Primary sources

- [Redshift_Best_Practices.html](#)
- [Redshift_RA3_Nodes.html](#)
- [Redshift_Zero_ETL.html](#)
- Research report: [06c_analytical_vector_graph.md](#)

Module 23 — Snowflake on AWS

What this is: Snowflake the data platform, running on AWS. Why Capital One picked Snowflake despite Redshift being native, and how the two coexist.

1. The three-layer architecture

Snowflake's signature design:

1. **Storage layer** — S3 (when running on AWS). Snowflake-managed; you don't see buckets directly.
2. **Compute layer** — **Virtual Warehouses** — independent compute clusters that read/write the same shared storage.
3. **Cloud services layer** — metadata, query planning, security, access control. Runs in Snowflake's account.

Key consequence: storage and compute scale independently. You spin up a 4XL warehouse for one heavy query, kill it after, and the data stays put.

2. Virtual Warehouse sizes

Size	Credits/hr	Use
XS	1	Dev, small ad-hoc
S	2	Small teams
M	4	Standard analytical
L	8	Heavy ETL
XL	16	Large queries
2XL	32	Very large
3XL	64	Massive ETL
4XL	128	Edge cases
5XL	256	(some regions)
6XL	512	(some regions)

Multi-cluster warehouses auto-scale horizontally (add clusters of the same size for concurrency).

3. Snowpipe and Snowpipe Streaming

- **Snowpipe** — file-based continuous ingest (S3 → Snowflake table). Latency: ~60-90 sec.
- **Snowpipe Streaming** (newer, 2023) — row-based, sub-second latency. For real-time analytics.

4. Snowpark

DataFrame API + Python/Java/Scala UDFs running inside Snowflake. Replaces “extract data → process in Spark → write back” for many use cases.

Snowpark Container Services (GA 2024) — run containerized workloads inside Snowflake (closest analog to SageMaker training in-warehouse).

5. Time Travel and zero-copy cloning

- **Time Travel** — query data as of any moment in the last 90 days (Enterprise edition+).
- **Zero-copy cloning** — instantly create a clone of a table/schema/database without copying data. Critical for dev-prod parity.

6. Streams + Tasks

- **Streams** — CDC: track inserts/updates/deletes on a table.
- **Tasks** — scheduled SQL execution (cron-like).
- Combined: continuous ELT pipelines inside Snowflake.

7. Iceberg tables

Snowflake added **read + write support for Apache Iceberg** in 2024.

- Reads: external Iceberg tables on S3 (Glue Catalog or other catalogs).
- Writes: Snowflake can write Iceberg tables that other engines can read.

This is the **bridge to Redshift, Athena, Databricks** — Iceberg becomes the lingua franca.

8. Horizon governance

Snowflake’s data governance pillar: object tagging, masking policies, row access policies, lineage, anomaly detection, classification. Roughly equivalent to Lake Formation + DataZone for the Snowflake-native side.

9. Cortex (Snowflake’s AI/ML surface)

Snowflake **Cortex** (GA 2024) — built-in functions for embeddings, LLM completions, semantic search, document AI. Operates on Snowflake-resident data without exporting.

```
SELECT cortex.complete('claude-3-5', 'Summarize:' || review_text)
FROM customer_reviews;
```

10. Why Capital One picked Snowflake

Per [the HBR 2021 sponsor piece](#) and inferred from their commercial Slingshot product:

- **Separation of compute and storage** at extreme scale (PB-class).
- **Time Travel** for compliance/audit recovery.
- **Cross-team concurrency** via multi-cluster warehouses.
- **Data marketplace** for sharing datasets across LOBs (and externally).
- **Existing investment** when the AWS migration completed in 2020 — Redshift was less mature in zero-ETL and concurrency at the time.

Slingshot is Capital One Software’s commercial Snowflake cost-governance product — they built it to manage their own large Snowflake spend, then productized it.

11. Snowflake vs Redshift (decision points)

	Snowflake	Redshift
Cost model	Credits per warehouse-second	Per RPU-second (Serverless) or instance-hour (Provisioned)
Compute isolation	Per-warehouse	Per-cluster
Cross-team concurrency	Multi-cluster warehouses	Concurrency Scaling
Time Travel	90 days (Enterprise)	35 days (PITR-like via UNDROP)
Iceberg	Native read+write	Spectrum (read) + native (write)
Pricing predictability	Less (varied query cost)	More (predictable Serverless RPU)
AWS-native integration	Limited (BYO IAM, KMS via key pair auth)	Deep

12. 2024-2026 changes

- **Cortex GA** 2024.
- **Iceberg read+write GA** 2024.
- **Snowpark Container Services GA** 2024.
- **Snowpipe Streaming** matured.

13. Pitfalls

- **Auto-suspend warehouse** misconfigured → idle credits burn.

- **Result cache illusion** — query results cached 24h; clients can think a slow query is fast.
- **Cross-region replication cost** — significant data transfer.
- **Many small warehouses** vs one large multi-cluster — usually the latter wins for throughput.

14. Capital One lens

Both **Redshift** and **Snowflake-on-AWS** in production. Snowflake for enterprise-shared analytics (the data marketplace pattern); Redshift for service-team marts and tight-loop OLTP-to-warehouse via Zero-ETL.

Interview talking point: *“I expect Capital One to run Snowflake as the enterprise-shared warehouse (the data marketplace) and Redshift for service-team marts with Zero-ETL from Aurora — the choice depends on whether cross-team concurrency or AWS-native integration is the dominant requirement.”*

15. Sanity check

1. What are the three layers of Snowflake’s architecture?
2. What’s the difference between Snowpipe and Snowpipe Streaming?
3. What does zero-copy cloning enable?
4. Why is Iceberg interoperability with Redshift/Athena strategically important?
5. Snowflake vs Redshift — name three decision factors.

16. Cross-references

- **Module 22** — Redshift (the AWS-native alternative)
- **Module 24** — Athena (serverless query on lake)
- **Module 12** — Iceberg, Glue Catalog (the interop layer)
- **Module 56** — Slingshot (Snowflake cost governance)
- **CAPITAL_ONE.md** — Snowflake strategic context

Primary sources

- [Snowflake_on_AWS.html](#)
- Research report: [06c_analytical_vector_graph.md](#)

Module 24 — Athena & Query Federation

What this is: Athena v3 (Trino-based), Iceberg/Hudi/Delta support, federated query connectors, workgroups, performance tuning, Athena for Apache Spark.

1. Athena v3 engine

Athena's query engine since 2023 is **Trino-based** (formerly Presto). Compatible SQL surface with significant performance improvements over v2.

- **Pricing:** \$5 per TB scanned (same as Spectrum).
- **No infrastructure to manage** — fully serverless.
- **Glue Data Catalog** as the metastore.

2. Table formats supported

- **Hive-style external tables** (Parquet, ORC, JSON, CSV).
- **Apache Iceberg** — full read + write, including MERGE, time travel, branching.
- **Apache Hudi** — read.
- **Delta Lake** — read.

Iceberg is the most active path — Athena v3 supports the latest Iceberg spec features.

3. Federated query

Athena can query data **outside S3** via Lambda-based connectors:

- **RDS / Aurora** (MySQL, Postgres)
- **DynamoDB**
- **Redshift**
- **DocumentDB**
- **MSK / Kafka**
- **OpenSearch**

- **CMDB sources** (Vertica, MS SQL Server, etc.)
- **Snowflake**

Use case: **join data across stores without ETL** — e.g., join an S3 transaction table with a DynamoDB customer profile.

The Lambda connector is a community/AWS-provided package; you deploy it once per source type.

4. Workgroups

Workgroups give you: - **Cost separation** — bill per workgroup tag. - **Query result location** — each workgroup writes to its own S3 prefix. - **Result reuse** — queries can reuse prior results (within TTL). - **Engine version pinning** — pin v2 or v3. - **Query limits** — max scan limits per workgroup.

5. Performance tuning

Optimization	Impact
Columnar format (Parquet/ORC vs CSV)	10-100× cheaper
Partitioning	Skip data — major reduction in scan
Partition projection	Avoid Glue catalog 1M partition limit; project partitions from name pattern
File size (256 MB - 1 GB target)	Fewer files = faster scan
Predicate pushdown	Filter early via WHERE clause on partition columns
CTAS to materialize aggregations	Pay scan cost once, reuse

6. Partition projection

Tell Athena how to project partition values from a key pattern, avoiding the need to list partitions in the Glue Catalog. For tables with 1M+ partitions, this is the only viable approach.

```
ALTER TABLE my_table SET TBLPROPERTIES (
  'projection.enabled'='true',
  'projection.date.type'='date',
  'projection.date.range'='2020-01-01,NOW',
  'projection.date.format'='yyyy-MM-dd'
);
```

7. Athena ACID via Iceberg

INSERT, UPDATE, DELETE, MERGE on Iceberg tables. Time travel via `AS OF VERSION` / `AS OF TIMESTAMP`. The path for warehouse-style operations on lake data.

8. CTAS (CREATE TABLE AS SELECT)

Materialize query results as new tables. Pattern: define an Iceberg table; CTAS to populate it with aggregations; downstream queries hit the smaller table.

9. Athena for Apache Spark

Notebook-style serverless Spark inside Athena. Useful for ad-hoc Python/Spark workloads without spinning up Glue or EMR.

10. 2024-2026 changes

- **Athena v3 (Trino)** is the current engine.
- **Iceberg MERGE + time travel** GA.
- **Federated query** continues expanding source list.
- **Athena for Spark** matured.

11. Pitfalls

- **Forgetting partitioning** — full table scans on TB-scale data are expensive.
- **CSV instead of Parquet** — 10× cost.
- **Many small files** — slower than fewer big files.
- **Partition projection on a tiny table** — overengineering.
- **Joining federated sources without filters** — pulls all data through Lambda connector.

12. Capital One lens

Athena is the natural lake-query surface for ad-hoc analytical work and exploratory data science. Federated query likely used to join S3 lake data with DynamoDB / RDS without ETL.

13. Sanity check

1. What engine does Athena v3 use?
2. When does partition projection beat Glue partition listing?
3. What does the Lambda-based federated query connector do?
4. When would you use CTAS?
5. What's the cost difference between scanning Parquet vs CSV?

14. Cross-references

- **Module 12** — Glue Catalog (the metastore)
- **Module 10** — S3 + Iceberg
- **Module 22** — Redshift Spectrum (same scan cost; cluster-attached)

- **Module 23** — Snowflake (cross-engine Iceberg interop)

Primary sources

- [Athena_Performance_Tuning.html](#)
- [Athena_Federated_Query.html](#)
- Research report: [06c_analytical_vector_graph.md](#)

Module 25 — Neptune — Graph Database

What this is: Amazon Neptune — property graph + RDF, Gremlin/SPARQL/openCypher languages, Neptune Serverless, Neptune Analytics, Neptune ML, and the GraphRAG pattern.

1. The dual nature

Neptune supports **two graph models simultaneously**:

- **Property graph** — nodes + edges with properties. Queried with **Gremlin** (Apache TinkerPop) or **openCypher** (Neo4j's language).
- **RDF (Resource Description Framework)** — subject-predicate-object triples. Queried with **SPARQL**.

Most non-academic use cases pick **property graph + openCypher** (modern, familiar to engineers from Neo4j).

2. Capacity tiers

- **Neptune Provisioned** — instance-based, like Aurora.
- **Neptune Serverless** (GA late 2022) — Neptune Capacity Units (NCU); 1 NCU = 2 GB RAM. Autoscale.

3. Neptune Analytics (the 2024 story)

Neptune Analytics (GA Nov 2023, expanded 2024) is a separate service for **analytical graph workloads**.

- **In-memory graph database** for one-off analytical workloads.
- Built-in algorithms: PageRank, connected components, shortest path, community detection.
- **Native vector search** built in — combines graph traversal with vector similarity.
- Used for ad-hoc graph analytics where Neptune Database (the transactional store) is overkill.

4. Neptune ML

GNN (Graph Neural Network) training and inference, integrated with SageMaker.

- Trains on **Deep Graph Library (DGL)** under the hood.
- Inductive node classification, link prediction, edge classification.
- Capital One use case: **fraud detection** — entities and relationships as graph, GNN learns suspicious-relationship patterns.

5. GraphRAG

The intersection of LLMs and graph databases: use graph traversal as part of retrieval for RAG.

- Vector search retrieves candidates by semantic similarity.
- Graph traversal expands context (e.g., from a customer node, hop to recent transactions, related accounts).
- LLM generates with both vector + graph context.

Bedrock Knowledge Bases added Neptune Analytics as a supported vector backend in 2024.

6. Bulk loading

S3 → Neptune via the **bulk loader** (CSV with `~id`, `~from`, `~to`, `~label` columns). Much faster than per-record inserts for initial population.

7. Cluster topology

- Primary (writer) + up to 15 replicas.
- 6-way replicated storage (Aurora-style).
- Multi-AZ failover.

8. Capital One lens — fraud and graph

Capital One’s tech blog publishes work on:

- **Graph ML for fraud** — entity-relationship graphs detect identity-fraud rings, money laundering patterns.
- **University partnerships** on global graph transformers and dynamic customer embeddings.

Likely architecture pattern: 1. Stream transactions to Neptune (writer). 2. Run periodic Neptune ML GNN training. 3. Score new transactions via real-time SageMaker inference using GNN-derived features.

Talking point: *“For fraud-graph workloads, I’d put the transactional graph in Neptune Database with Neptune ML for periodic GNN retraining, and use Neptune Analytics for ad-hoc investigations that benefit from in-memory speed.”*

9. Pitfalls

- **Choosing the wrong language** — Gremlin’s API is imperative; openCypher is declarative. Most teams pick openCypher.
- **Large traversals** — graph queries that visit millions of nodes time out; design queries with hop limits.
- **Neptune ML training time** — non-trivial; GNNs are slow vs tabular ML.
- **Bulk loader CSV format** — strict; one mistake aborts the load.

10. Sanity check

1. What are the three query languages, and which one most teams pick?
2. Neptune Database vs Neptune Analytics — when do you pick each?
3. What is Neptune ML, and what’s the underlying library?
4. What does GraphRAG add over plain vector RAG?
5. How might Capital One use Neptune for fraud detection?

11. Cross-references

- **Module 27** — vector capabilities decision framework (Neptune Analytics as one of the vector options)
- **Module 42** — Bedrock Knowledge Bases (Neptune Analytics backend)
- **Topic 01 Module 22** — GraphRAG deep dive
- **Module 53** — Capital One MLOps spine (fraud-graph patterns)

Primary sources

- [Neptune_Best_Practices.html](#)
- [Neptune_Analytics.html](#)
- [Neptune_ML.html](#)
- Research report: [06c_analytical_vector_graph.md](#)

Module 26 — OpenSearch — Search & Vector

What this is: Amazon OpenSearch Service (managed) vs OpenSearch Serverless. k-NN plugin engines (Lucene, FAISS, nmslib). HNSW vs IVF indexes. Hybrid BM25 + vector search. The post-Elasticsearch-fork story.

1. Service vs Serverless

	OpenSearch Service	OpenSearch Serverless
Form	Provisioned cluster (you size nodes)	Per-capacity (OCU) auto-scaling
Use case	Steady-state log analytics, large indices	Vector workloads, time-series, unpredictable load
Capacity unit	Instance types (r6g.large, etc.)	OCU = 6 GB RAM + 2 vCPU + 120 GB storage
Dev-mode minimum	Tiny clusters	2 OCU floor (the cheapest viable Serverless deployment)

2. The 2021 Elasticsearch fork

Elastic relicensed Elasticsearch in 2021 to a non-OSS license; AWS forked it as **OpenSearch** (Apache 2.0). Since then OpenSearch has diverged: own roadmap, different feature set in some areas (e.g., k-NN engines, alerting), strong AWS integration.

3. k-NN engine choices

OpenSearch's k-NN plugin supports three engines:

- **Lucene** — pure Java, in-process. Easier to manage, lower throughput.
- **FAISS** — Facebook's library; high throughput, more memory.
- **nmslib** — older; less commonly chosen for new builds.

Index types (for FAISS / nmslib):

- **HNSW** (Hierarchical Navigable Small World) — graph-based; high recall, ANN search.
- **IVF** (Inverted File Index) — clustering-based; better for very large datasets where memory matters.

Default choice: **HNSW on FAISS** for production.

4. Hybrid search (BM25 + vector)

OpenSearch supports combining **BM25** (lexical relevance) with **vector** (semantic) search results via **Reciprocal Rank Fusion (RRF)** or weighted score combination.

- BM25 catches exact-term matches that embedding similarity misses.
- Vector search catches semantic similarity that exact-term misses.
- Combined: substantially higher relevance for RAG and search workloads.

5. Ingestion pipelines

- **OpenSearch Ingestion** (formerly Data Prepper, managed in 2024) — managed pipelines for log, trace, metric ingestion.
- **Bulk API** for batch indexing.
- **Index templates** for schema-on-read patterns.

6. Security plugin

Fine-grained access control: - Document-level security. - Field-level security. - Tenants (isolated workspaces). - Integration with Cognito, SAML, IAM Identity Center.

7. Neural search

ML-based query rewriting: an embedding model translates a query to better matches. Reduces the “I searched for X but meant Y” problem.

8. OpenSearch as RAG backend

The big use case in 2024-2026:

- **Bedrock Knowledge Bases** uses OpenSearch Serverless as the default vector store.
- **Hybrid search** improves recall for noisy queries.
- **Native integration** with Bedrock for embedding generation.

9. OpenSearch Service vs Elastic Cloud on AWS

	OpenSearch Service	Elastic Cloud on AWS
Latest features	OpenSearch roadmap (k-NN improvements, neural search, AWS integration)	Latest Elastic features (Lens, security, ML jobs)
AWS-native auth/encryption	Deep	Limited
Cost	Generally lower at scale	Premium for Elastic features

For Capital One: OpenSearch Service is the AWS-native choice.

10. 2024-2026 changes

- **Serverless GA** for vector and time-series workloads (refined 2024).
- **Neural search** matured.
- **Bedrock KB integration** as the default vector backend.
- **OpenSearch 2.x** versions advanced.

11. Pitfalls

- **Serverless 2 OCU minimum** — ~\$700/month floor for dev environments.
- **HNSW memory footprint** — vector indexes can dominate RAM; size accordingly.
- **Index rotation** in Service — managed by Index State Management policies; complex to design.
- **Hybrid search tuning** — relevance scoring needs experimentation per workload.

12. Capital One lens

OpenSearch is likely used for: - **Log analytics** at scale. - **Hybrid BM25 + vector RAG** for policy/compliance document retrieval. - **Eno's answer retrieval** layer (combined with Bedrock for generation).

13. Sanity check

1. Service vs Serverless — when does each win?
2. Three k-NN engines — what's the default pick?
3. HNSW vs IVF — when does IVF matter?
4. Why does hybrid BM25+vector beat either alone?
5. What is the 2 OCU minimum cost implication for OpenSearch Serverless?

14. Cross-references

- **Module 27** — vector capabilities decision framework
- **Module 42** — Bedrock Knowledge Bases
- **Module 56** — log analytics as observability backend

Primary sources

- [OpenSearch_Vector_Search.html](#)
- [OpenSearch_Serverless.html](#)
- [OpenSearch_kNN.html](#)
- Research report: [06c_analytical_vector_graph.md](#)

Module 27 — Vector Capabilities

Decision Framework

What this is: the decision matrix for “which AWS service stores my vectors for RAG?”
Aurora pgvector, RDS pgvector, DocumentDB vector, MemoryDB vector, OpenSearch k-NN, Neptune Analytics. Plus Bedrock Knowledge Bases backends.

1. The options

Service	Index types	Max dims	Latency	Throughput	Bedrock KB native?
Aurora pgvector	HNSW, IVFFlat	16,000	ms	Mid	Yes
RDS Postgres pgvector	HNSW, IVFFlat	16,000	ms	Mid	Yes
DocumentDB vector	HNSW, IVFFlat	2,000	ms	Mid	Yes
MemoryDB vector	HNSW	1,000	sub-ms	High	No (use OpenSearch)
OpenSearch k-NN (FAISS HNSW)	HNSW, IVF	16,000	ms	High	Yes (default)
OpenSearch Serverless	HNSW	16,000	ms	Auto-scale	Yes
Neptune Analytics vectors	Graph-aware	16,000	ms	Mid	Yes
S3 Vectors (new 2024)	HNSW	varies	varies	low cost	Some

2. The decision tree

Need sub-millisecond latency for real-time agent context?

→ MemoryDB vector

Already running OpenSearch for hybrid BM25+vector search?

→ OpenSearch k-NN (FAISS HNSW)

Already running Aurora Postgres for OLTP, and dataset < 100M vectors?

→ Aurora pgvector (same DB, no new service)

Bedrock Knowledge Bases user wanting the simplest path?

→ OpenSearch Serverless (Bedrock KB's default)

Need graph-aware vector retrieval (GraphRAG)?

→ Neptune Analytics

MongoDB-API-compatible document store with vectors?

→ DocumentDB vector

Massive scale (billions of vectors), willing to trade latency for cost?

→ S3 Vectors (or OpenSearch Service with IVF)

3. Bedrock Knowledge Bases native backends

Bedrock KB natively connects to: - **OpenSearch Serverless** (default) - **Aurora pgvector** - **Pinecone** (third-party) - **Redis Enterprise** (third-party) - **MongoDB Atlas** (third-party) - **Neptune Analytics** (2024)

For Capital One: **OpenSearch Serverless** + **Aurora pgvector** are the two AWS-native paths Bedrock KB integrates with cleanly.

4. Cost shapes

- **Aurora pgvector** — already paying for Aurora; vectors add storage + IOPS.
- **RDS pgvector** — same as Aurora but smaller scale.
- **DocumentDB vector** — instance-based; vector index lives in cluster.
- **MemoryDB vector** — premium (\$/hr × instance count); justified for sub-ms latency only.
- **OpenSearch Service** — per-instance; sized by RAM (HNSW memory-heavy).
- **OpenSearch Serverless** — OCU floor of 2 (~\$700/mo for dev).
- **Neptune Analytics** — per-NCU; in-memory analytical workloads.

5. Multi-tenant patterns

For SaaS use cases with vector data per tenant:

- **Namespace** in OpenSearch (one index per tenant or shared index with `tenant_id` filter).
- **Schema-per-tenant** in Aurora pgvector.
- **Cluster-per-tenant** in DocumentDB/MemoryDB (heavyweight).

6. The “two-tier” pattern (Capital One likely)

OpenSearch Serverless — hybrid BM25 + vector for policy/regulatory RAG
↓ (slower, recall-focused)

MemoryDB vector — sub-millisecond agent context retrieval
↑ (fast, latency-focused)

Tier 1 (OpenSearch) returns top candidates from a large corpus; tier 2 (MemoryDB) caches hot tenant context for in-flight agent calls.

7. When to pick which (cheat sheet)

Workload	Pick
New RAG with Bedrock	OpenSearch Serverless (Bedrock default)
Existing Aurora OLTP + vectors	Aurora pgvector
Sub-millisecond agent context	MemoryDB vector
MongoDB-compatible + vector	DocumentDB vector
Graph + vector (GraphRAG)	Neptune Analytics
Massive cold vector store	S3 Vectors
Hybrid lexical + semantic search	OpenSearch k-NN with hybrid query

8. Pitfalls

- **Adding a new DB just for vectors** when your existing OLTP could handle it (pgvector).
- **MemoryDB for cold vectors** — way overpriced.
- **HNSW without enough RAM** — index spills, performance collapses.
- **Forgetting Bedrock KB ingestion costs** — embedding generation per chunk.

9. Capital One lens

Two-tier likely: - **OpenSearch Serverless** for policy/regulatory document RAG (hybrid search) — paired with Bedrock KB. - **MemoryDB vector** for Eno / Servicing Tool agent context (sub-millisecond requirement). - **Aurora pgvector** for workloads that already live in Aurora.

10. Sanity check

1. When do you pick MemoryDB vector over OpenSearch?
2. What are Bedrock Knowledge Bases' native vector backends?
3. What's the two-tier vector pattern, and why does Capital One likely use it?
4. Why is HNSW memory-heavy, and what's the failure mode?
5. When does Aurora pgvector beat a dedicated vector DB?

11. Cross-references

- **Topic 01 Module 22** — GraphRAG deep dive
- **Modules 17, 19, 21, 25, 26** — the individual database modules
- **Module 42** — Bedrock Knowledge Bases

Primary sources

- [Bedrock_Knowledge_Bases.html](#)
- [MemoryDB_Vector.html](#)
- [Aurora_pgvector.html](#)
- Research report: [06c_analytical_vector_graph.md](#)

Module 28 — Amazon MSK Deep

What this is: Apache Kafka managed by AWS — MSK Provisioned vs Serverless, KRaft mode, IAM auth, MSK Connect, MSK Replicator, Glue Schema Registry. The streaming backbone for event-driven architectures.

1. MSK Provisioned vs Serverless

	MSK Provisioned	MSK Serverless
Sizing	You choose broker instance types (kafka.m5., <i>kafka.m7g.</i>)	AWS sizes automatically
Pricing	Per broker-hour + storage + data	Per-partition-hour + per-message + storage
Cluster control	Full	Limited
Use case	Predictable throughput, custom Kafka config	Variable workloads, small teams
KRaft mode	Yes (replacing ZooKeeper)	Yes

MSK Serverless (GA 2022) abstracts broker management. You define topics, MSK handles cluster scaling. Great for getting started; less control.

2. KRaft vs ZooKeeper

KRaft (Kafka Raft, GA in Apache Kafka 3.3+) **eliminates ZooKeeper** as the metadata coordinator. MSK now supports KRaft natively. New clusters should default to KRaft.

ZooKeeper-managed clusters still exist and AWS supports them, but the future is KRaft.

3. Authentication

Three options:

	IAM	SASL/AWS_MSK_IAM	SASL/SCRAM	mTLS
Mechanism	AWS IAM with SigV4	Username/password in Secrets Manager		X.509 client certs
Easiest?	Yes for AWS workloads	Need creds store		Complex PKI
Cross-account	Yes via IAM trust	Manual		Manual
AWS-native pick				

IAM auth is the default pick on AWS — no static creds, integrates with IRSA / Pod Identity / Lambda execution roles. Capital One almost certainly uses this exclusively given their “no static creds” policy.

4. MSK Connect

Managed Kafka Connect — connectors as fully managed workers.

- **Source connectors** — Debezium (CDC), JDBC, S3, Kinesis.
- **Sink connectors** — S3, OpenSearch, Redshift, Snowflake, JDBC.
- Custom plugins uploaded as JARs.

Used when you want CDC from Aurora/RDS into Kafka, or Kafka topic to S3 archive without a custom consumer.

5. Glue Schema Registry

The AWS-managed schema registry for Avro, JSON Schema, Protobuf.

- Schemas have versions; compatibility rules (BACKWARD, FORWARD, FULL, NONE) enforce contract evolution.
- Integration with MSK producers (`com.amazonaws.services.schemaregistry.serializers.avro.AWSKafkaAvroSer` and Kinesis).
- **Serializer caches schemas client-side**, registers new schemas on first encounter.

6. MSK Replicator

Cross-region (and within-region) Kafka replication. Lower-overhead than MirrorMaker 2; AWS-managed; preserves consumer offsets for active-passive DR.

7. Producer/Consumer patterns

- **Idempotent producer** — `enable.idempotence=true`; safe retries without duplicates within a partition.

- **Exactly-once semantics (EOS)** via transactions (`initTransactions`, `beginTransaction`, `sendOffsetsToTransaction`, `commitTransaction`).
- **Consumer groups + offsets** — Kafka-managed offsets in `__consumer_offsets`. Partition assignment via strategies:
 - `RangeAssignor` — older default
 - `RoundRobinAssignor`
 - `CooperativeStickyAssignor` — **the modern pick**; cooperative rebalances avoid stop-the-world

8. MSK vs Self-managed Kafka vs Confluent Cloud on AWS

	MSK	Self-managed (EC2/EKS)	Confluent Cloud on AWS
Ops effort	Low	High (you manage brokers)	Lowest
Cost (at scale)	Mid	Lowest (with engineer cost)	Highest
Feature parity with OSS Kafka	Latest stable Apache versions	Latest, including pre-releases	Confluent extensions (KSQL, Schema Registry, Cluster Linking)
Best for	AWS-native shops with mid-scale Kafka	Cost-sensitive scale, complex Kafka use	Heavy Kafka use needing Confluent features

9. 2024–2026 changes

- **MSK Serverless GA** (2022, but feature-rich by 2024).
- **MSK Replicator GA** (2023).
- **KRaft on MSK GA** (2024).
- **IAM auth GA in all regions** (2022+, but now default).

10. Pitfalls

- **Partition rebalancing pain** — bad partition assignment strategies cause stop-the-world rebalances. Use `CooperativeStickyAssignor`.
- **Replication factor 1** — never. Default to 3 (one per AZ).
- **Acks=1 producer** — risk of data loss on broker failure. Use `acks=all` for durability.
- **Schema-less producers** — schema drift breaks downstream consumers. Use Glue Schema Registry.
- **MSK Serverless not appropriate** for high-fanout, ultra-low-latency workloads → use Provisioned.

11. Capital One lens

Capital One’s public tech blog confirms **Kinesis** is the primary serverless event bus. MSK appears where Kafka-native consumers (Debezium CDC, acquired-company apps) already existed. The pattern: Kinesis for new internal services; MSK where Kafka was already in the picture.

If you’re interviewing, ask: “What’s the split between Kinesis and MSK across LOBs at Capital One? When do you pick MSK over Kinesis for new event buses?”

12. Sanity check

1. MSK Provisioned vs Serverless — when does Provisioned make sense?
2. What does KRaft replace and why does it matter?
3. Why is IAM SASL/AWS_MSK_IAM auth the AWS-native default?
4. What is `CooperativeStickyAssignor` and why is it preferred?
5. When is Confluent Cloud worth the premium over MSK?

13. Cross-references

- **Module 29** — Kinesis family (the AWS-native alternative)
- **Module 15** — orchestration (EventBridge Pipes can consume MSK)
- **Module 22** — Redshift Streaming Ingestion from MSK
- **Module 38** — SageMaker model inference triggered by Kafka events

Primary sources

- [MSK_Developer_Guide.pdf](#)
- [MSK_Best_Practices.html](#)
- Research report: [07_streaming.md](#)

Module 29 — Kinesis Family + Flink

What this is: Kinesis Data Streams (KDS), Amazon Data Firehose (formerly Kinesis Firehose), Managed Service for Apache Flink (formerly Kinesis Data Analytics), EventBridge, EventBridge Pipes. The decision matrix vs MSK.

1. Kinesis Data Streams (KDS)

Shard-based stream — the “Kafka, but serverless and AWS-native.”

- **Shard** = unit of throughput: 1 MB/s in, 2 MB/s out (or with **Enhanced Fan-Out** 2 MB/s per consumer).
- **Capacity modes:** Provisioned (you size shards) and On-Demand (auto-scales).
- **Retention** — 24h default, configurable up to 365 days (long retention \$\$).
- **Pricing** — per shard hour + PUT payload units (25 KB each). On-Demand more expensive but auto-scales.

Producer (KPL, Kinesis Producer Library) vs **Consumer (KCL)** — high-level libraries with batching, retries, checkpointing.

Enhanced Fan-Out (EFO) — per-consumer dedicated 2 MB/s pipe. Solves the noisy-neighbor problem at the cost of \$0.015/hr per consumer-shard.

2. Amazon Data Firehose (formerly Kinesis Firehose)

Fully managed delivery. **Source** → **buffer** → **optional transform** → **destination**.

- **Destinations:** S3, Redshift, OpenSearch, HTTP endpoint, Splunk, MongoDB, Snowflake, Datadog, New Relic.
- **Buffer hints:** by size (1-128 MB) or time (60-900 sec) — whichever first.
- **Transformations via Lambda:** per-record transform inline. Heavy use for PII redaction, format normalization.
- **Format conversion:** JSON → Parquet/ORC inline (using a Glue Catalog schema).
- **Dynamic partitioning** — keys from record content drive S3 path partitioning.

The “fire and forget” pattern — Firehose handles retry, buffering, format conversion, partitioning.

Capital One’s pattern per their tech blog: Kinesis → Lambda → S3, with Firehose for the simple straight-to-S3 sink.

3. Managed Service for Apache Flink (MSF, formerly Kinesis Data Analytics for Apache Flink)

Managed Flink for stream processing — windowing, joins, sessions, ML-via-PyFlink.

- **Flink jobs** run as Java/Scala/Python applications.
- **Studio notebooks** — Zeppelin-based interactive Flink for exploration.
- **Snapshots and savepoints** — versioned state for upgrade safety.
- **Autoscaling** — pay-per-KPU (Kinesis Processing Unit).
- **Flink SQL** for declarative stream processing.

When you need stateful stream processing (windows, sessions, exactly-once stream-stream joins) — MSF.

4. EventBridge

Default event bus (every account has one) + custom event buses + partner event buses (SaaS integrations).

- **Rules** match event patterns (JSON), route to up to 5 targets.
- **Schedules** (GA 2022) — cron + one-time scheduler, replaces CloudWatch Events scheduling.
- **API Destinations** — call external HTTP endpoints as targets.

5. EventBridge Pipes

Source → optional Filter → optional Enrich → Target. A serverless point-to-point pattern.

- **Sources:** SQS, Kinesis, MSK, DynamoDB Streams, Self-managed Apache Kafka.
- **Filter:** pattern match on event content (no compute).
- **Enrich:** Lambda, Step Functions, API Gateway, API Destinations.
- **Targets:** 14+ AWS services including Lambda, Step Functions, SQS, SNS, ECS, SageMaker pipelines.

Replaces the “SQS → Lambda → filter → transform → call SageMaker endpoint” boilerplate with declarative config.

6. Decision framework: Kinesis vs MSK vs EventBridge vs SQS/SNS

Scenario	Pick
AWS-native event bus, no Kafka ecosystem need	KDS

Scenario	Pick
Existing Kafka producers/consumers (Debezium, acquired company apps)	MSK
Just deliver records to S3/Redshift/OpenSearch	Firehose
Stateful stream processing (windows, joins, sessions)	MSF (Flink)
Cross-service event-driven architecture (S3 events, CloudWatch alarms, custom events)	EventBridge
One source → filter/enrich → one target	EventBridge Pipes
Queue with single consumer / pub-sub fanout	SQS / SNS

7. Pricing comparison (rough)

For 100 MB/s sustained throughput:

- **KDS Provisioned** — $\sim 100 \text{ shards} \times \$0.015/\text{hr} = \$1,100/\text{mo}$ + PUT payload + data egress
- **KDS On-Demand** — $\sim \$0.04/\text{GB ingest} + \$0.04/\text{GB retrieval} = \sim \$8\text{k}/\text{mo}$ at 100 MB/s
- **MSK Provisioned** — 3 kafka.m7g.large brokers $\sim \$0.42/\text{hr}$ each = $\sim \$900/\text{mo}$ + storage + data
- **Firehose** — $\$0.029/\text{GB to S3} = \sim \$7.5\text{k}/\text{mo}$ at 100 MB/s
- **MSF** — per-KPU pricing; for a moderate Flink job $\sim \$0.11/\text{KPU-hr} \times 4 \text{ KPUs} = \$320/\text{mo}$

8. Capital One serverless streaming SDK pattern

Per [their tech blog](#), Capital One built an internal SDK abstracting Kinesis + Lambda source/processing/sink:

Producer Lambda → KDS → Processor Lambda → KDS or DDB → Sink Lambda → S3 / Firehose

- IAM-native auth (no static creds).
- Per-LOB shard isolation.
- 7-day retention covers replay window.
- Firehose to S3 closes the loop for cold storage.

This is the AWS-native pattern Capital One built into a reusable SDK. **Talking point:** *“I’d expect the serverless streaming SDK to default to Kinesis with IAM auth, Firehose for cold archive, and EventBridge Pipes for cross-account fan-in.”*

9. Pitfalls

- **Hot shard** — bad partition key → one shard saturated. Use uniform-distribution keys.
- **Firehose buffer tuning** — too small → many small S3 files; too large → high end-to-end latency.
- **EventBridge throttle limits** — per-account quota; for high-throughput direct invocation, consider Kinesis.
- **MSF state size** — large state → slow snapshots → blocked deployments.

10. Sanity check

1. What's an EFO consumer and when do you need it?
2. When would you pick MSF over a simpler Lambda + DynamoDB stateful pattern?
3. EventBridge Pipes vs Lambda glue — what's the boilerplate it replaces?
4. Walk through the Capital One Kinesis SDK pattern.
5. Firehose dynamic partitioning — what does it do?

11. Cross-references

- **Module 28** — MSK (the Kafka alternative)
- **Module 22** — Redshift Streaming Ingestion (consume from KDS/MSK)
- **Module 32** — Lambda (the de-facto Kinesis processor)
- **Module 15** — EventBridge Pipes as orchestration glue

Primary sources

- [Kinesis_Data_Streams_Best_Practices.html](#)
- [Firehose_Developer_Guide.html](#)
- [MSF_Apache_Flink.html](#)
- [Streaming_Data_Solutions_Whitepaper.pdf](#)
- [c1tech_serverless_streaming.html](#) — Capital One internal SDK pattern
- Research report: [07_streaming.md](#)

Module 30 — EC2 & Accelerators

What this is: EC2 instance families for ML workloads — G5/G6/P4/P5/P5e (NVIDIA), Trainium / Inferentia (AWS chips), EC2 Capacity Blocks, UltraClusters, EFA, Spot economics.

1. The taxonomy

Family	Use
M	General-purpose (CPU+RAM balanced)
C	Compute-optimized
R	Memory-optimized
X	Extra memory
I, D	Storage-optimized
G	Graphics + general GPU (G5/G6)
P	Training-grade GPU (P4d/P5/P5e/P5en)
Inf	Inference accelerators (Inferentia2)
Trn	Training accelerators (Trainium, Trainium2)

2. NVIDIA GPU instances

Instance	GPU	VRAM/GPU	Best for
G5	A10G (24 GB)	small inference, single-GPU training	
G6 / G6e	L4 / L40S (24/48 GB)	mid-range inference, fine-tuning	
P4d / P4de	A100 (40/80 GB)	distributed training	
P5	H100 (80 GB)	FM-scale training	
P5e	H200 (141 GB)	larger FM training, 2024+	

Instance	GPU	VRAM/GPU	Best for
P5en	H200 + EFAv3	tightly-coupled GPU clusters	

3. AWS-designed accelerators

- **Trainium (Trn1)** — first-gen training chip. Cheaper per FLOP than NVIDIA.
- **Trainium2 (Trn2)** — 2024 launch; closer to H100 perf at much lower cost. **UltraServer** form factor.
- **Inferentia (Inf1, Inf2)** — inference chips. Inf2 supports larger models.

Trade-off: lower price/perf, but PyTorch / TensorFlow integration via **AWS Neuron SDK** is the long pole. Not all model architectures are well-optimized.

4. EC2 Capacity Blocks for ML

Reserve GPU capacity windows — guaranteed availability of P4/P5/P5e for 1-182 days.

- Announced 2023; expanded for H100/H200 in 2024.
- Use case: planned multi-day FM training runs where on-demand might not have capacity.
- **Pay for the window** whether you use the GPUs or not — accurate forecasting matters.

5. UltraClusters and EFA

UltraCluster — thousands of interconnected GPU instances with **Elastic Fabric Adapter (EFA)** networking.

- **EFA** — OS-bypass network adapter; ~100 Gbps with sub-microsecond latency.
- Required for tightly-coupled distributed training (NCCL all-reduce).
- Not all instance types support EFA; the high-end P5/P5e/P5en do.

EFAv3 (2024) — expanded throughput, lower latency.

6. Nitro System

AWS's hypervisor and security boundary. Most modern instances are Nitro-based: - Hardware-virtualized networking and storage (no hypervisor overhead). - Security: customer can't access the hypervisor / management plane. - Underpins all the GPU instance types.

7. AMIs (Amazon Machine Images)

For ML workloads: - **DLAMI (Deep Learning AMI)** — pre-installed with CUDA, cuDNN, NCCL, PyTorch, TensorFlow. - **Bottlerocket** — minimal container-optimized OS; good for EKS GPU nodes. - **Custom AMI** — your own image with org-specific tooling.

8. Auto Scaling Groups (ASG)

Scale EC2 by launch template + ASG. For GPU workloads, ASG with **mixed instances policy** + **Spot** is common.

9. Spot economics for ML

Up to 90% off On-Demand. Spot interruption rates by instance family:

Family	Typical interruption rate
c5, c6i	low (~5%)
m5, m6i	low
p4d	medium (10-20% in busy regions)
p5, p5e	high — variable; subject to capacity demand

For GPU training: combine Spot + checkpointing every 10-30 minutes. SageMaker Managed Spot Training handles this automatically.

10. Placement Groups

Hint to EC2 to place instances physically close (or far):

- **Cluster** — same low-latency network segment. For HPC, NCCL all-reduce.
- **Spread** — different racks. For HA.
- **Partition** — groups of cluster + spread. For Hadoop/Cassandra.

11. Pricing examples (us-east-1, May 2026)

- p5.48xlarge (8× H100) — ~\$98/hr on-demand
- p5e.48xlarge (8× H200) — ~\$110/hr on-demand
- g5.xlarge (1× A10G) — ~\$1.00/hr on-demand
- trn2.48xlarge — ~\$25/hr on-demand (vs ~\$98/hr for p5.48xlarge equivalent)

Spot can drop these by 60-90% with availability tradeoffs.

12. Capital One lens

For training: - **P5/P5e** for FM-scale training jobs. - **EC2 Capacity Blocks** to reserve GPU windows for planned training runs. - **Trainium2** if cost-driven and model fits AWS Neuron SDK support.

For inference: - **G6/G6e** for medium-sized inference workloads (KServe pods on EKS). - **Inf2** where the workload fits Neuron SDK.

13. Sanity check

1. P5 vs P5e vs P5en — what's different?
2. When is Trainium2 the right call vs P5?
3. What does EFA enable for distributed training?
4. When does Spot make sense for ML, and what's the operational requirement?
5. EC2 Capacity Blocks — what problem do they solve?

14. Cross-references

- **Module 36** — SageMaker training (uses these instance types)
- **Module 40** — HyperPod (clusters of these)
- **Module 44** — self-hosted FM serving on EC2/EKS

Primary sources

- [EC2_Capacity_Blocks_ML.html](#)
- [Spot_Best_Practices.html](#)
- Research report: [08_compute_for_ml.md](#)

Module 31 — ECS + Fargate + AWS Batch

What this is: the container orchestration alternatives to EKS — ECS (AWS-native), Fargate (serverless containers), AWS Batch (managed batch with EC2 or Fargate or EKS backend).

1. ECS

Elastic Container Service — AWS-native container orchestrator. Simpler than EKS but less portable.

- **Task definition** — JSON spec of containers, CPU, memory, networking, IAM.
- **Service** — desired count of tasks, load balancer integration.
- **Cluster** — logical grouping; can run on EC2 (you manage) or Fargate (AWS manages).
- **Capacity providers** — EC2 ASG (you choose instance types) or Fargate.

2. Fargate

Serverless container runtime. You give it a task definition; AWS runs it. No EC2 management.

- Platform versions (1.3, 1.4) define feature set. Currently 1.4 is the default.
- **Task networking modes** — awsvpc is default; each task gets its own ENI.
- **Up to 200 GB ephemeral storage** (raised from 20 GB).
- Pricing per vCPU + memory + duration.
- **No GPU support on Fargate** — for GPU tasks, use ECS-on-EC2 or EKS.

3. ECS vs EKS

	ECS	EKS
Control plane	AWS-managed (free for ECS itself)	AWS-managed (\$0.10/hr per cluster)
Portability	AWS-only	Kubernetes (portable)

	ECS	EKS
Operator ecosystem	Limited	Huge (Helm, KServe, Karpenter, Istio, etc.)
Best for	Simple workloads, AWS-only commitment	Complex platforms, hybrid/multi-cloud, K8s-native ML tooling

Capital One pattern: EKS for ML model serving (per Lead MLE job posting confirming KServe + Kubernetes). ECS likely for simpler service-tier deployments where K8s overhead isn't justified.

4. AWS Batch

Managed batch job runner. **Capacity environment** can use: - EC2 (you can use Spot, Auto Scaling). - Fargate. - **EKS** (the EKS backend lets you run Batch on your existing EKS cluster — great for ML training and batch processing).

Multi-Node Parallel (MNP) jobs — for tightly-coupled distributed training (MPI / NCCL).

Array jobs — parameterized N-task fan-out.

Job queues — priority-ordered queues; fair-share scheduling across users.

5. ECS Anywhere

Run ECS tasks on **on-prem hardware** (or third-party cloud). AWS-managed control plane; data plane is your iron. Used for hybrid scenarios.

6. Pricing comparison (~)

- **ECS on EC2** — pay only for EC2 instances.
- **ECS on Fargate** — ~40-60% more expensive than equivalent EC2 (but no ops overhead).
- **Fargate Spot** — interruptible, ~70% cheaper than Fargate on-demand.
- **AWS Batch on EC2 Spot** — cheapest path for non-urgent batch.

Rule of thumb: Fargate makes sense when EC2 utilization < ~40%. Above that, EC2 wins.

7. Capital One lens

- **EKS + KServe** for ML model serving (the differentiator).
- **ECS / Fargate** for simpler service-tier deployments.
- **AWS Batch on EKS** for ML batch training jobs.
- **AWS Batch with EC2 Spot** for cost-sensitive non-urgent batch.

8. Pitfalls

- **Fargate without GPUs** for ML training that needs them — pick ECS-on-EC2 or EKS instead.
- **ECS service auto-scaling on CPU only** — usually need a custom metric (queue depth, latency).
- **AWS Batch job queue without retries** — silent failure.
- **Mixing ECS and EKS** in the same team often creates split expertise debt.

9. Sanity check

1. ECS vs EKS — when does ECS win?
2. What does Fargate not support that EC2 does (relevant to ML)?
3. AWS Batch with EKS backend — when does that matter?
4. What's a Multi-Node Parallel job?
5. ECS Anywhere — niche use case?

10. Cross-references

- **Module 33** — EKS foundations
- **Module 36** — SageMaker training (alternative to AWS Batch for ML)
- **Module 54** — KServe on EKS (Capital One's ML serving pattern)

Primary sources

- Research report: [08_compute_for_ml.md](#)

Module 32 — AWS Lambda Deep

What this is: Lambda execution model, cold-start mechanics, SnapStart, container images, layers, concurrency, event sources, observability, and the Capital One serverless-first pattern.

1. Execution model

Lambda runs on **Firecracker microVMs**. The lifecycle:

1. **Init phase** — microVM boot, runtime init, code load, module imports. *This is the cold-start cost.*
2. **Invocation phase** — handler executes.
3. **Frozen** — microVM held warm; subsequent invocations skip init.
4. **Reclaimed** — eventually killed (typically after several minutes idle).

Cold start = init phase. Typical: 100-500 ms for Python/Node; 500-3000 ms for Java/.NET.

2. SnapStart (the cold-start killer)

SnapStart snapshots a fully-initialized microVM and restores from snapshot on cold start, dramatically reducing init time.

Runtime	SnapStart status
Java	GA 2022, free
Python	GA Nov 2024 (paid: \$0.0000015625 per request restoration)
.NET 8+	GA Nov 2024 (paid)

Java with SnapStart: 80%+ cold-start reduction, free. The killer feature for Lambda-heavy shops.

3. Container images

Up to **10 GB container images** (vs 250 MB zip layers). Useful for: - Heavyweight ML inference workloads. - Custom runtimes with proprietary dependencies. - Workloads that need specific OS

libraries.

4. Layers and Extensions

Layers — shared zip artifacts (libraries, custom runtimes). Up to 5 per function.

Extensions — managed processes that run alongside your function. Common uses: - Telemetry (Data-dog, New Relic). - Secrets caching. - Parameter Store caching.

5. Concurrency

	Description
Account concurrency	Default 1,000 concurrent executions per region (raisable)
Reserved concurrency	Reserve a slice for one function (also caps it)
Provisioned concurrency	Pre-warmed instances; no cold start; pay per instance-hour

When to use provisioned concurrency: latency-critical workloads where cold start matters more than cost.

6. Destinations

For async invocations, route success/failure events to: - SQS, SNS, EventBridge, another Lambda.

Cleaner than the older “DLQ” pattern.

7. Event sources

Lambda has built-in integration with: - **API Gateway** (HTTP). - **Function URLs** (direct HTTPS without API Gateway). - **S3, EventBridge, CloudWatch Events**. - **SQS, SNS, MSK, Kinesis, DynamoDB Streams**. - **Application Load Balancer**.

8. Lambda + VPC

Initially (pre-2019) putting Lambda in VPC added ~10s cold-start latency. AWS rebuilt the ENI model as **Hyperplane ENIs** (Sep 2019): ENI is pre-created and shared across invocations, eliminating the per-cold-start ENI cost.

Lambda + VPC is now a non-issue for cold start.

9. NAT GW cost trap

Lambda in VPC reaching the internet typically goes through NAT GW. At scale, NAT GW data-processing charges dominate.

Fix: VPC endpoints for AWS services. The big ones — S3 (gateway, free), DynamoDB (gateway, free), SQS, SNS, Kinesis, MSK, KMS, Secrets Manager — all have Interface endpoints. The bill drops significantly.

10. Observability

- **Lambda Insights** (CloudWatch agent) — per-invocation memory, duration, init time.
- **X-Ray** for distributed tracing.
- **Structured logging** to CloudWatch Logs.
- **Lambda Telemetry API** (extensions consume it for third-party APM).

11. Capital One serverless-first patterns

Per their re:Invent 2024 talk *“Celebrating 10 years of pioneering serverless”* (Catherine McGarvey):

- **Lambda is the centerpiece** of the architecture.
- **CodeDeploy gradual traffic shifting** — 2%/min canary deploys on Lambda.
- **Internal Kinesis SDK pattern** — Producer Lambda → KDS → Processor Lambda → KDS or DDB → Sink Lambda → S3 / Firehose.
- **Step Functions for orchestration**, Lambda for individual steps.
- **80% latency reduction** on check-processing pipeline via Step Functions + Lambda.

Talking point: *“Coming from an Azure-heavy shop, I see the Lambda serverless-first culture as the operational backbone — Lambda + Step Functions + DynamoDB + Kinesis is the platform default and ML pipelines integrate into it rather than replacing it.”*

12. Pricing

- **Per request:** \$0.20 per 1M requests.
- **Per duration:** \$0.0000166667 per GB-second.
- **Provisioned concurrency:** \$0.0000041667 per GB-second (~25% of execution price for warm capacity).
- **SnapStart restore for Python/.NET:** \$0.0000015625 per request restoration.

Example: 1B invocations/month × 100ms duration × 256MB → ~\$575/month execution + \$200 request fee = ~\$775/month.

13. Pitfalls

- **NAT GW cost** from Lambda in VPC to public internet.

- **Reserved concurrency = 0** accidentally → all invocations throttled.
- **Provisioned concurrency on rarely-invoked functions** → wasted spend.
- **Long-running Lambda** (15 min max) — graduate to Fargate or Step Functions.
- **Cold start in latency-critical paths** → use SnapStart or provisioned concurrency.

14. Sanity check

1. What does SnapStart do, and which runtimes are supported in 2026?
2. When does provisioned concurrency pay off?
3. Why is Lambda + VPC no longer a cold-start problem?
4. What's the NAT GW trap, and what's the fix?
5. Walk through the Capital One Kinesis SDK pattern.

15. Cross-references

- **Module 15** — Step Functions (Lambda's natural orchestrator)
- **Module 18** — DynamoDB (Lambda + DDB Streams)
- **Module 29** — Kinesis (Lambda processor pattern)
- **Module 8** — VPC endpoints (Lambda NAT cost fix)

Primary sources

- [Lambda_Operator_Guide.html](#)
- Research report: [08_compute_for_ml.md](#)

Module 33 — EKS Foundations

What this is: Amazon EKS — managed Kubernetes — cluster anatomy, Managed Node Groups vs Karpenter, Fargate Profiles, VPC CNI, IRSA / Pod Identity, EKS Auto Mode. The foundation for Capital One’s KServe model serving (Module 54).

1. Cluster anatomy

EKS = AWS-managed Kubernetes control plane + your data plane.

- **Control plane** (AWS-managed): API server, etcd, controller manager, scheduler. **\$0.10/hr per cluster.**
- **Data plane** (your nodes): EC2 instances (via Managed Node Groups or self-managed) or Fargate (serverless pods).

2. Data plane options

Managed Node Groups (MNG)

AWS-managed EC2 instance groups. You define instance type, scaling bounds, taints, labels. AWS handles AMI updates, draining.

Karpenter (the modern choice)

Karpenter v1.x GA late 2024. Replaces Cluster Autoscaler with a smarter, faster node provisioner:

- Reads pending-pod constraints (resources, taints, topology).
- Spins up the right instance type **directly** — no ASG indirection.
- Faster scale-up (10-30 sec vs minutes).
- Supports Spot + diverse instance types natively.
- Native consolidation: terminates underutilized nodes proactively.

New clusters in 2026 should default to Karpenter over MNG.

Fargate Profiles

Serverless pods. You define a selector (namespace + labels); matching pods run on Fargate. No node management.

Limitations: no GPU, no `hostPath`, no `DaemonSets` (use Fargate sidecars instead).

3. EKS Auto Mode (GA Dec 2024)

A higher-level abstraction: - **Karpenter built-in** — no manual install. - **System add-ons managed** by AWS (kube-proxy, VPC CNI, CoreDNS, EBS CSI, etc.). - **Compute and storage** automatically provisioned.

The “managed Kubernetes that actually feels managed.” New 2026 clusters should consider Auto Mode unless you have specific reasons to manage the data plane yourself.

4. VPC CNI (pod networking)

The default networking plugin. **Each pod gets a VPC IP** from an ENI attached to its node.

- **Pros:** native VPC integration; security groups apply at pod level; no overlay overhead.
- **Cons:** IP exhaustion is the #1 EKS scaling pain. A `m5.large` only supports ~30 pods.

Solutions: - **Prefix delegation** — allocate /28 prefixes to ENIs instead of individual IPs. Massive pod-per-node increase. - **Secondary CIDR / custom networking** — give pods IPs from a different (e.g., 100.64.0.0/10) CIDR.

5. IAM Roles for Service Accounts (IRSA) vs Pod Identity

IRSA (GA 2019-09-03)

Maps K8s ServiceAccount → IAM role via OIDC federation.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ml-trainer
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/MLTrainerRole
```

Trust policy on the role uses the cluster’s OIDC provider, conditioned on `system:serviceaccount:<namespace>:<sa-name>`.

EKS Pod Identity (GA 2023-11-26)

Simpler alternative using an **EKS-managed agent** instead of OIDC + cluster trust policies.

- Easier cross-account.

- Fewer trust-policy edits at scale.
- **Default pick for new EKS clusters in 2026.**

6. GPU support

- **NVIDIA GPU Operator** — manages GPU drivers, CUDA toolkit, MIG (Multi-Instance GPU) on nodes.
- Karpenter NodePools for GPU instance types (P5, G6, Inf2, Trn2).
- **EKS Auto Mode** handles GPU drivers automatically.

7. Upgrades

EKS supports **Kubernetes versions for ~14 months** (4 versions). Extended support pricing (\$0.60/hr per cluster) for older versions after standard support ends.

Upgrade path: 1. Upgrade control plane. 2. Upgrade managed node groups / Karpenter NodePools. 3. Upgrade add-ons.

Pin your add-on versions in IaC; surprises hurt.

8. EKS Anywhere & EKS Distro

- **EKS Anywhere** — Kubernetes on your bare metal / vSphere with AWS support.
- **EKS Distro** — the open-source Kubernetes distribution AWS uses; you can run it anywhere.

9. Pricing

- **Control plane:** $\$0.10/\text{hr} \times 24 \times 30 = \sim \$72/\text{month}$ per cluster.
- **Extended support:** \$0.60/hr per cluster (after standard EOL).
- **Nodes:** standard EC2 + EBS pricing.
- **Fargate Profiles:** per-vCPU + per-memory + per-duration.

10. 2024-2026 changes

- **EKS Auto Mode** GA Dec 2024.
- **Karpenter v1.x** stable.
- **EKS Pod Identity** widely adopted.
- **Pod Identity Agent** replaces IRSA for new clusters.
- **EKS in 14-month version support window.**

11. Pitfalls

- **VPC CNI IP exhaustion** — node packs few pods unless prefix delegation enabled.
- **Cluster upgrade without testing add-ons** — version drift breaks things.

- **MNG instead of Karpenter** for new builds — slower scale-up, harder Spot.
- **GPU operator misconfig** — pods stuck Pending with cryptic errors.
- **Default 0.0.0.0/0 egress** on cluster nodes → NAT GW cost.

12. Capital One lens

Per their Sr Lead AI/ML / Lead MLE postings, **EKS is mandatory for ML model serving** (KServe specifically). Likely architecture:

- **EKS clusters per LOB or per platform team.**
- **Karpenter for GPU node provisioning** as inference traffic scales.
- **IRSA / Pod Identity** for model pod → S3 / ECR access without static creds.
- **KServe + Istio** as the serving stack (Module 54).

13. Sanity check

1. Karpenter vs Managed Node Groups — what's the modern default?
2. What is the VPC CNI IP exhaustion problem, and the fix?
3. IRSA vs Pod Identity — when is each preferred?
4. What does EKS Auto Mode automate?
5. What's the extended-support cost penalty for old K8s versions?

14. Cross-references

- **Module 2** — IAM (IRSA / Pod Identity)
- **Module 6** — VPC + CNI / secondary CIDR
- **Module 30** — EC2 + GPU instances (the node hardware)
- **Module 54** — KServe deep (the ML serving layer on EKS)

Primary sources

- [EKS_Best_Practices.html](#)
- [Karpenter_Best_Practices.html](#)
- Research report: [08_compute_for_ml.md](#)

Module 34 — SageMaker Platform Map

What this is: the SageMaker product landscape — Studio Classic vs new Studio, Domains, Spaces, Unified Studio, Code Editor, the 2018→2026 evolution.

1. The three eras of SageMaker

- **Notebook Instances era (2017-2022)** — single-user EC2-based Jupyter. Still exists, increasingly deprecated.
- **Studio Classic era (2019-2023)** — multi-user JupyterLab inside a Domain, with the older console UI.
- **New Studio era (2023+)** — redesigned UX combining JupyterLab + Code Editor (VS Code-based) + Canvas + RStudio in a single web app.

2. Studio Domains

A **Domain** is the multi-user SageMaker workspace boundary. Each Domain has:

- **VPC config** (the network it lives in).
- **KMS key** for at-rest encryption.
- **IAM execution role**.
- **Authentication mode:** IAM Identity Center or IAM.
- **N user profiles** — one per ML practitioner.

3. User profiles and Spaces

- **User profile** = a person in the Domain.
- **Space** = a workspace within the Domain. Two flavors:
 - **Private space** — single-user JupyterLab / Code Editor / RStudio.
 - **Shared space** — multiple users collaborate in the same JupyterLab.

Each Space runs on a specific instance type (e.g., `ml.t3.medium` for cheap dev, `ml.g5.2xlarge` for GPU prototyping).

4. The 2024 story: SageMaker Unified Studio

SageMaker Unified Studio (GA late 2024) integrates SageMaker with: - **DataZone** (governance + discovery) - **Glue** (ETL) - **EMR** (Spark) - **Athena** (ad-hoc SQL) - **MWAA** (Airflow orchestration) - **SageMaker AI** (training, inference, etc.)

The pitch: **one interface for the entire data+AI lifecycle**, with DataZone-mediated discovery and subscription. Replaces “log into Glue console for ETL, SageMaker for training, EMR for Spark.”

This is the strategic direction — new builds in 2026 should target Unified Studio.

5. Code Editor

VS Code-based IDE in Studio. Same Domain, KMS, IAM as JupyterLab.

For engineers who prefer VS Code over JupyterLab — same underlying compute.

6. SageMaker Catalog

DataZone-integrated catalog visible in Studio. Browse, request access to datasets, then materialize them in your Space.

7. RStudio in SageMaker

Posit-licensed RStudio Workbench bundled in Studio Domains. For R users.

8. Custom Studio images

You can ship custom Docker images as Studio kernel choices — `tensorflow-with-our-tooling`, `pytorch-2.5-with-our-libs`. Critical for regulated orgs that vendor specific library versions.

9. Evolution timeline

Year	Event
2017	SageMaker GA (Notebook Instances era)
2019	Studio Classic GA
2020	Pipelines, Feature Store, Clarify GA
2021	Training Compiler, Inference Recommender
2022	Studio Lab (free educational), JumpStart matures
2023	New Studio redesigned UX; HyperPod GA
2024	Unified Studio, Inference Components, MLflow integration, HyperPod EKS-based

10. 2024-2026 changes

- **Unified Studio** (the headline).
- **MLflow integration** — SageMaker hosts MLflow tracking server.
- **Inference Components** (Module 37) — new endpoint cost model.
- **HyperPod EKS-based GA** (Module 40).
- **Code Editor** matured.

11. Pitfalls

- **Idle Studio Apps** — running JupyterServer or KernelGateway apps cost money. Auto-shutdown lifecycle config is the fix.
- **Studio Domain has a single EFS** — large user-profile homes accumulate; budget for EFS storage.
- **Migration from Notebook Instances** is non-trivial — different IAM, different code paths.
- **Two Studio versions** (Classic vs new) cause confusion. Stick to one per Domain.

12. Capital One lens

Capital One almost certainly runs many Studio Domains, one per LOB or per platform team. Custom Studio images vendored with internal libraries. Unified Studio likely in adoption.

13. Sanity check

1. Studio Classic vs new Studio — what's the difference?
2. What's a Domain vs a Space?
3. What does Unified Studio integrate?
4. Why do regulated orgs care about custom Studio images?
5. What's the auto-shutdown pattern for Studio Apps?

14. Cross-references

- **Modules 35-41** — the rest of SageMaker
- **Module 12** — DataZone integration via Unified Studio
- **Module 53** — Capital One MLOps spine

Primary sources

- SageMaker Developer Guide (archived as PDF)
- Unified Studio docs (archived)
- Research report: [09_sagemaker.md](#)

Module 35 — SageMaker Data

What this is: Data Wrangler, Feature Store (online + offline), Processing Jobs, Ground Truth, Clarify (bias + explainability), Model Cards.

1. Data Wrangler

Visual data prep tool — 300+ built-in transformations: type conversions, joins, custom transforms (Pandas/Spark), text/datetime parsing, outlier detection.

- Recipes compile to **Processing Jobs** (PySpark/Pandas).
- Output to S3, Feature Store, or directly to Training Job.

2. Feature Store

The managed feature platform. **Two tiers:**

	Online Store	Offline Store
Backing	DynamoDB	S3 + Iceberg (since 2023)
Latency	single-digit ms	seconds-minutes
Use case	Inference-time feature lookup	Training/batch scoring

Features are organized into **Feature Groups** — like tables. Ingest via boto3 / SageMaker SDK; query via Athena (offline) or `get_record` (online).

Capital One use case: per-customer behavioral features ingested from Glue → Feature Store → SageMaker training (offline) + KServe inference (online via DynamoDB or MemoryDB tier).

3. Processing Jobs

Run preprocessing / postprocessing code on managed containers. Three backends:

- **Built-in containers** — Spark, Scikit-Learn, Hugging Face.
- **Custom containers** — BYOC for your toolchain.

- **Bring Your Own Algorithm** for legacy patterns.

Output to S3. Pricing: per-instance-hour.

4. Ground Truth & Ground Truth Plus

Ground Truth — managed data labeling. - Workforce options: Mechanical Turk, vendor-managed, private (your own workforce). - Active learning to reduce labeling cost.

Ground Truth Plus — fully-managed labeling service (AWS handles the workforce).

5. Clarify

Bias and explainability:

- **Bias detection** — pre-training (data) and post-training (model). Statistical metrics like Class Imbalance, Difference in Positive Proportions in Labels, Demographic Disparity, etc.
- **SHAP-based explainability** — global and per-prediction feature importance.

Capital One relevance: SR 11-7 model risk management requires documented bias and explainability evidence. Clarify outputs feed Model Cards.

6. Model Cards (the SR 11-7 anchor)

Standardized model documentation:

- Intended use, training data, evaluation metrics, ethical considerations, decisions.
- Versioned with the model in Model Registry.
- Exportable as PDF for regulator submissions.

The **regulator-facing artifact**. Model Cards + Clarify + rubicon-ml (Capital One's OSS) form the audit trail for SR 11-7 / Fed model risk reviews.

7. A2I (Augmented AI)

Human-in-the-loop workflows: route low-confidence predictions to humans for review. Integrates with Textract, Comprehend, or custom workflows.

8. 2024-2026 changes

- **Feature Store offline-store on Iceberg** (2023+).
- **Data Wrangler in Unified Studio**.
- **Clarify Foundation Model evaluation** (for LLMs).
- **Model Cards integrated with Bedrock** for foundation models.

9. Pitfalls

- **Feature Store online cost** — DynamoDB writes for high-throughput features can dominate.
- **Data Wrangler recipes that don't scale** — built-in transforms work on samples; full data may OOM.
- **Forgetting Clarify in regulated workflows** — no SR 11-7 evidence trail.

10. Capital One lens

- **Feature Store** likely paired with MemoryDB vector for sub-ms online inference.
- **Clarify + Model Cards + rubicon-ml** for SR 11-7 model governance.
- **Ground Truth Plus** for high-quality labeling (financial transactions, fraud signals).

11. Sanity check

1. Online vs offline Feature Store — what's the backing store and latency?
2. What's the SR 11-7 chain: Clarify + Model Cards + ?
3. Ground Truth vs Ground Truth Plus — what's the difference?
4. When would you use a Processing Job vs Glue?
5. What does Data Wrangler compile its recipes to?

12. Cross-references

- **Module 21** — MemoryDB (online feature serving)
- **Module 38** — Model Registry, MLflow
- **Module 52** — SR 11-7 in regulatory context
- **Module 53** — rubicon-ml in Capital One MLOps spine

Primary sources

- SageMaker Feature Store docs (archived)
- Research report: [09_sagemaker.md](#)

Module 36 — SageMaker Training

What this is: Training Jobs, distributed training (SMDDP, FSDP), Spot training, warm pools, Trainium training, Heterogeneous Clusters, MLflow integration.

1. Training Jobs — the core

A SageMaker Training Job:

1. Provisions instance(s) of your chosen type.
2. Pulls a Docker image (built-in or BYOC).
3. Mounts S3 input data (or FSx Lustre, or EFS).
4. Runs your training script.
5. Writes output (model artifacts) to S3.
6. Tears down.

Pricing: **per-second** instance-hours. Single-instance or multi-instance.

2. Distributed training libraries

SMDDP (SageMaker Distributed Data Parallel)

AWS-optimized DDP library. Drop-in replacement for PyTorch DDP, optimized for AWS network topology (EFA, NCCL tuning).

SageMaker Model Parallel (SMP)

Tensor + pipeline parallelism for models too large for one GPU. **SMP v2** integrates with PyTorch **FSDP** (Fully Sharded Data Parallel).

Native PyTorch FSDP / DeepSpeed

You can also use vanilla PyTorch FSDP or DeepSpeed in a Training Job; SageMaker is the wrapper.

3. Spot training (SageMaker Managed Spot Training)

Up to **90% off On-Demand**. SageMaker handles: - Spot interruption → checkpoint restore. - Job restart on new instances. - Max wait time + max run time settings.

Your code must implement **periodic checkpointing** (every 10-30 min for large training).

4. Warm pools

A pool of pre-provisioned instances. Subsequent training jobs skip provisioning (~5-10 min savings).

Use for: iterative experimentation where you launch many short jobs back-to-back.

5. Trainium training

Trn1, **Trn2** instances for cost-optimized training via AWS Neuron SDK.

- PyTorch and TensorFlow via Neuron compiler.
- Up to ~40% cost reduction vs equivalent NVIDIA.
- Trade-off: not all architectures supported equally well.

Trainium2 UltraServer (Dec 2024 GA) — 64-chip super-node, comparable to NVIDIA HGX H100.

6. Heterogeneous Clusters

Mix CPU and GPU instances in one job: - **Data loading workers** on CPU instances. - **Training workers** on GPU instances.

Decouples data pipeline scaling from model compute. Useful when data preprocessing is the bottleneck.

7. Training Compiler (deprecated)

SageMaker Training Compiler (XLA/Inductor-based) is being deprecated in 2025. Use native PyTorch 2.x + `torch.compile()` instead.

8. MLflow integration

SageMaker hosts MLflow Tracking Server (2024+). Use as the experiment tracker alongside Training Jobs:

```
import mlflow
mlflow.set_tracking_uri("arn:aws:sagemaker:...:mlflow-tracking-server/...")
with mlflow.start_run():
    # train...
    mlflow.log_metric("loss", loss)
    mlflow.log_model(model, "model")
```

9. Entrypoint patterns

For distributed training:

```
from sagemaker.pytorch import PyTorch

estimator = PyTorch(
    entry_point="train.py",
    role=role,
    instance_count=8,
    instance_type="ml.p5.48xlarge",
    distribution={
        "torch_distributed": {"enabled": True}, # uses torchrun
        # or "smdistributed": {"dataparallel": {"enabled": True}}
    },
    framework_version="2.4.0",
    py_version="py311",
    hyperparameters={...}
)
estimator.fit({"train": "s3://.../train/", "val": "s3://.../val/"})
```

10. 2024-2026 changes

- **SMP v2 + FSDP integration.**
- **Trainium2 UltraServer GA.**
- **Heterogeneous Clusters** matured.
- **MLflow integration** as alternative to SageMaker Experiments (deprecating).
- **Training Compiler deprecation.**

11. Pitfalls

- **No checkpointing on Spot** → training restart loses progress.
- **Data loader bottleneck** → GPUs underutilized; check `nvidia-smi` and add CPU workers.
- **Wrong distribution config** → distributed-data-parallel mistakenly runs single-GPU.
- **NCCL timeouts** on slow networks → tune `NCCL_TIMEOUT` higher.
- **Cross-AZ training without EFA** → 10x slower than intra-AZ.

12. Capital One lens

For FM training: - **HyperPod** (Module 40) for multi-day, multi-thousand-GPU jobs. - **Spot + Capacity Blocks** mix — Capacity Blocks for SLA-bound runs, Spot for resumable experiments. - **MLflow on SageMaker** for experiment tracking, paired with **rubicon-ml** for git-linked audit trails.

For non-FM training: - **SageMaker Training Jobs** with Spot. - **Trainium2** evaluated where compatible.

13. Sanity check

1. SMDDP vs FSDP — when does each matter?
2. What’s the workflow for Spot training with checkpointing?
3. What problem do Heterogeneous Clusters solve?
4. When is Trainium worth picking over NVIDIA?
5. Why is MLflow integration replacing SageMaker Experiments?

14. Cross-references

- **Module 30** — EC2 instance types (the underlying hardware)
- **Module 40** — HyperPod (the FM-scale training cluster)
- **Module 11** — FSx Lustre (training I/O storage)
- **Module 38** — MLOps (Pipelines orchestrating Training Jobs)

Primary sources

- SageMaker Developer Guide (archived)
- Research report: [09_sagemaker.md](#)

Module 37 — SageMaker Inference

What this is: the inference endpoint variants — Real-time, Serverless, Async, Batch Transform, Multi-Model Endpoints (MME), Multi-Container, Inference Components, Inference Recommender, Shadow testing.

1. The four inference modes

Mode	Latency	Cost model	Use
Real-time endpoint	ms	Per instance-hour 24/7	Production low-latency
Serverless inference	ms-s (cold start)	Per request + per duration	Bursty, low-volume
Async endpoint	seconds-minutes	Per instance-hour while processing	Large input/output, long-running
Batch Transform	minutes-hours	Per instance-hour for batch	Offline scoring of large datasets

2. Real-time endpoints

Standard pattern:

```
predictor = model.deploy(
    instance_type="ml.g5.xlarge",
    initial_instance_count=2,
    endpoint_name="card-fraud-model"
)
```

- Multi-AZ behind a load balancer.
- Auto-scaling on `InvocationsPerInstance` or custom CloudWatch metrics.
- Configurable concurrency.

Pricing trap: even at 0 RPS, you pay per instance-hour. **The biggest SageMaker cost gotcha.**

3. Serverless inference

Pay per request:

```
serverless_config = ServerlessInferenceConfig(memory_size_in_mb=2048, max_concurrency=20)
predictor = model.deploy(serverless_inference_config=serverless_config)
```

- Up to 6 GB memory.
- Cold-start latency on first request.
- Max concurrency configurable.

Use when: bursty workload, <1k requests/hr, latency budget allows cold start.

4. Async inference

For **long-running inputs/outputs** (e.g., document AI processing a 50-page PDF):

- Input from S3, output to S3.
- SNS notification on completion.
- Scales to 0 when no requests.
- Up to 1 GB input, 1 hour processing time.

5. Batch Transform

Offline scoring:

```
transformer = model.transformer(
    instance_type="ml.m5.xlarge",
    instance_count=10,
    output_path="s3://.../predictions/"
)
transformer.transform(data="s3://.../inputs/", content_type="text/csv", split_type="Line")
```

Cheaper than spinning up a real-time endpoint just to score a static dataset.

6. Multi-Model Endpoints (MME)

Host **thousands of models on one endpoint**. Models load on-demand from S3 into instance memory.

- **CPU MMEs** — many small models per instance.
- **GPU MMEs** — fewer, larger models.

Use when: many models with similar runtime, modest QPS per model. **Memory pressure** is the failure mode — too many concurrent unique models triggers thrashing.

7. Multi-Container Endpoints

Different containers + routing logic on one endpoint. Each request routed to one container.

Useful for: A/B testing model versions, ensemble where each model is a different framework.

8. Inference Components (the 2024 cost story)

Inference Components (GA 2024) decouple model from endpoint compute:

- One endpoint hosts a pool of compute.
- Multiple “components” (models) packed onto the pool.
- Pack ratio: small models share a GPU.

The big cost win: **fractional GPU allocation per model**. Where MME was about hot-loading from S3, Inference Components is about static co-location with proper resource accounting.

9. Inference Recommender

Load-tests your model on multiple instance types and produces a Pareto cost/latency report. Use before going to production to pick the right instance.

10. Shadow testing

Mirror production traffic to a new model variant without affecting production. Compare metrics (latency, error rate, prediction distribution) before flipping traffic.

11. Auto-scaling

Configure on: - **InvocationsPerInstance** (most common). - **GPU/CPU utilization**. - **Custom CloudWatch metric**.

Cold-start under scale-up: scaling-out adds new instances which need ~3-5 min to load model and warm up. Plan accordingly.

12. Latency tuning

- **Right-size instance** — Inference Recommender.
- **Model compression** — quantization (INT8, FP8), pruning.
- **Compile with TensorRT** for NVIDIA, or **AWS Neuron** for Inferentia.
- **Batch on the server** when SLA allows.
- **SageMaker LMI (Large Model Inference)** container for FM serving.

13. SageMaker LMI

Large Model Inference container — purpose-built for FM serving with: - **DJL Serving** (Deep Java Library) under the hood. - **vLLM**, **TensorRT-LLM**, **TGI** backends. - **Tensor parallel inference** across multiple GPUs.

The AWS-native path for serving large open-source FMs without rolling your own KServe.

14. 2024-2026 changes

- **Inference Components** GA (the cost story).
- **DJL/LMI v12** with vLLM 0.6+, TensorRT-LLM updates.
- **Shadow testing** matured.
- **Serverless inference** memory raised to 6 GB.

15. Pitfalls

- **Idle real-time endpoint** — biggest SageMaker cost gotcha. Use serverless or scheduled shut-down.
- **MME memory pressure** — too many concurrent models.
- **Cold-start on serverless** for latency-critical workloads — use provisioned concurrency or real-time.
- **Auto-scale lag** on traffic spike — pre-warm or use Inference Components.

16. Capital One lens

Capital One **self-hosts on EKS + KServe** for many models (Module 54). But SageMaker endpoints likely still used for: - Lower-volume models where SageMaker simplicity beats EKS overhead. - **Inference Components** for cost-efficient packing of similar models. - **Shadow testing** before promoting to KServe production.

17. Sanity check

1. Real-time vs Serverless vs Async vs Batch — when does each win?
2. What's the MME failure mode?
3. What does Inference Components solve that MME doesn't?
4. When would you reach for SageMaker LMI?
5. What's the idle-endpoint cost gotcha?

18. Cross-references

- **Module 54** — KServe (the self-hosted alternative)
- **Module 56** — cost discipline (the idle-endpoint trap)
- **Module 30** — instance type pricing
- **Module 38** — model registry → deploy pattern

Primary sources

- SageMaker Inference Best Practices (archived)

- Research report: [09_sagemaker.md](#)

Module 38 — SageMaker MLOps

What this is: SageMaker Pipelines, Projects, Model Registry, Model Monitor, Lineage Tracking. The orchestration + governance layer.

1. SageMaker Pipelines

DAG of ML steps. Two definition styles:

Python DSL

```
from sagemaker.workflow.pipeline import Pipeline
from sagemaker.workflow.steps import ProcessingStep, TrainingStep
from sagemaker.workflow.model_step import ModelStep

pipeline = Pipeline(name="card-fraud-pipeline", steps=[
    ProcessingStep("preprocess", ...),
    TrainingStep("train", ...),
    ModelStep("register", ...),
])
pipeline.upsert(role_arn=role)
pipeline.start()
```

@step decorator (2024)

```
from sagemaker.workflow.function_step import step

@step(instance_type="ml.m5.xlarge")
def preprocess(input_path: str) -> str:
    # ...
    return output_path

@step
def train(data_path: str) -> str:
    # ...
```

```
return model_artifact
```

The decorator pattern compiles to a Pipeline. Easier for Python-first teams.

2. Step types

- **ProcessingStep** — preprocessing/postprocessing.
- **TrainingStep** — model training.
- **TuningStep** — hyperparameter tuning.
- **ModelStep / RegisterModelStep** — register to Model Registry.
- **ConditionStep** — branching.
- **ClarifyCheckStep, QualityCheckStep** — bias / quality gates.
- **LambdaStep, CallbackStep** — escape hatches.

3. Projects (templated CI/CD)

SageMaker Projects = Service Catalog templates that provision: - A Git repo. - CodePipeline for CI/CD. - A SageMaker Pipeline. - Model Registry deployment hooks.

Use when: you want a starter MLOps template per use case (regression, classification, NLP, vision).

4. Model Registry

Versioned model package store:

- **Model Package Groups** — collections of versions for a single model.
- **Approval status:** PendingManualApproval, Approved, Rejected.
- **Cross-account sharing** — share a registry across the AWS org via RAM.
- **Deploy from registry** to endpoints (real-time, serverless, async, batch).

Capital One pattern: Model Registry as the **approval gate**. PR merges to main trigger training; trained model lands in Registry as Pending; manual or automated reviewer approves; downstream deployment triggers.

5. Model Monitor

Continuous monitoring of deployed models. Four monitor types:

	What it detects
Data Quality	Input data drift (schema, stats)
Model Quality	Prediction quality (requires ground truth)
Bias Drift	Bias metrics shifting over time
Feature Attribution Drift	SHAP attributions shifting (via Clarify)

Schedules: hourly cron-like. Output: CloudWatch metrics + S3 reports.

6. Lineage Tracking

Artifacts, contexts, associations — the audit trail.

- **Artifacts** — model files, data sets, container images.
- **Contexts** — projects, pipelines, environments.
- **Associations** — “this model was trained on this data with this pipeline.”

The data structure that backs SR 11-7 audit responses: “show me what data trained this production model and who approved it.” Pairs with **rubicon-ml** for git-SHA-linked detail.

7. Edge Manager (deprecated)

SageMaker Edge Manager is being deprecated in favor of IoT Greengrass. Move existing edge-ML workloads.

8. The MLOps lifecycle (full picture)

```
Code commit
↓
CodePipeline / GitHub Actions OIDC
↓
Build container, run unit tests
↓
SageMaker Pipeline (Processing → Training → Evaluation → Register)
↓
Model Registry (Pending)
↓
Manual or automated approval
↓
Deploy to staging endpoint
↓
Shadow test or Inference Recommender
↓
Deploy to prod (Inference Components, MME, or KServe)
↓
Model Monitor + Lineage Tracking ongoing
↓
Drift detected → trigger retraining
```

9. 2024-2026 changes

- **@step decorator** for Pipelines.
- **MLflow integration** complements Model Registry.

- **Pipelines + Step Functions interop** improved.
- **Edge Manager deprecated**.

10. Pitfalls

- **Step max input/output size** — pass S3 references, not large blobs.
- **Manual approval bottleneck** — gate it on a SLA, fall back to automated approval.
- **Model Monitor without ground truth** — can detect drift but not quality directly.
- **Pipeline failures without retry policy** — single transient failure kills the run.

11. Capital One lens

- **SageMaker Pipelines** for training DAG.
- **Step Functions** for outer-loop orchestration (Module 15).
- **Model Registry** as the audit gate.
- **rubicon-ml** for git-SHA-linked experiment tracking (SR 11-7).
- **Model Cards + Clarify** for regulator-facing documentation.

12. Sanity check

1. Pipeline Python DSL vs @step decorator — when does each fit?
2. What four monitor types does Model Monitor support?
3. How does Lineage Tracking serve SR 11-7 audit?
4. Why is the Model Registry approval gate critical for regulated finance?
5. Edge Manager — what's it deprecating to?

13. Cross-references

- **Module 15** — Step Functions as outer orchestrator
- **Module 35** — Model Cards (in Registry)
- **Module 52** — SR 11-7 + Lineage
- **Module 53** — rubicon-ml integration

Primary sources

- SageMaker MLOps Whitepaper (archived)
- Research report: [09_sagemaker.md](#)

Module 39 — SageMaker JumpStart, Canvas, Autopilot

What this is: the pre-trained / no-code / AutoML surfaces in SageMaker — JumpStart’s FM hub, Canvas’s visual AutoML, and Autopilot’s programmatic AutoML.

1. JumpStart

Foundation model hub: - **LLMs:** Llama 3, Mistral, Falcon, Mixtral, Code Llama. - **Image models:** Stable Diffusion XL, FLUX. - **Embedding models:** Cohere, BGE, Sentence-Transformers. - **Domain-specific:** Hugging Face fine-tunes.

For each model: **deploy with one click** to a SageMaker endpoint, or **fine-tune** with sample notebooks.

2. JumpStart vs Bedrock

	JumpStart	Bedrock
Hosting	Your endpoint (SageMaker)	AWS-managed API
Customization	Full (fine-tune, modify)	Limited (Continued Pre-Training + FT via Bedrock Custom Models)
Cost model	Per-instance-hour	Per-token
Best for	Self-hosted + tunable	Quick start, multi-model, managed

Rule: JumpStart when you want control over the deployment artifact and operations.

3. Canvas

No-code ML for business analysts: - Tabular AutoML. - Image classification / object detection (via JumpStart models). - Text (sentiment, classification, summarization). - Time-series forecasting. - **Generative AI** (chat with Bedrock-hosted models, document Q&A).

Pricing: per-session-hour. Designed for non-engineers.

4. Autopilot (programmatic AutoML)

Programmatic API for tabular + time-series AutoML:

```
from sagemaker.automl.automl import AutoML

automl = AutoML(
    role=role,
    target_attribute_name="fraud",
    sagemaker_session=session,
    max_candidates=50,
    max_runtime_per_training_job_in_seconds=3600,
)
automl.fit(inputs="s3://.../train/")
best_candidate = automl.best_candidate()
```

- Tries N (default 250) algorithms × hyperparams.
- Auto-feature-engineers.
- Outputs explainability via Clarify.
- Best candidate deployable to endpoint.

5. Decision matrix

Need	Pick
Foundation model deploy / fine-tune	JumpStart
No-code tabular / time-series ML	Canvas
Programmatic tabular AutoML	Autopilot
Multi-FM API with managed serving	Bedrock
Custom training script	Module 36 Training Jobs

6. Bedrock integration

Canvas can use Bedrock-hosted FMs for chat / document Q&A — the unified UX.

7. Pitfalls

- **JumpStart instance type** mismatch — many FMs require GPU; you'll get OOM on CPU defaults.
- **Canvas pricing surprise** — per-session-hour adds up for analysts.
- **Autopilot run time** — large datasets blow past default time limits.

8. Capital One lens

- **JumpStart** for self-hosting open-source FMs (Llama, Mistral) when Bedrock model selection doesn't fit.
- **Canvas** for business-analyst self-service (likely with Databolt-tokenized data).
- **Autopilot** for quick tabular baselines (fraud, credit, churn).

9. Sanity check

1. JumpStart vs Bedrock — when does each win?
2. What's Canvas, and who's the user?
3. Autopilot programmatic flow — what do you give it, what do you get back?
4. Why is Canvas pricing per-session-hour?
5. Can Canvas talk to Bedrock?

10. Cross-references

- **Module 34** — SageMaker platform (Canvas lives in Studio)
- **Module 42** — Bedrock (alternative to JumpStart for managed FM)
- **Module 44** — self-hosted FM (JumpStart is one path here)

Primary sources

- SageMaker JumpStart docs (archived)
- Research report: [09_sagemaker.md](#)

Module 40 — SageMaker HyperPod

What this is: purpose-built clusters for foundation-model training — Slurm-based vs EKS-based HyperPod, Recipes, Task Governance, FSx Lustre integration.

1. The problem HyperPod solves

FM training runs span **days to weeks** on **thousands of GPUs**. At that scale:

- Node failures are inevitable (~1-5% per day on H100).
- Restart-from-scratch wastes compute.
- Manual recovery is a full-time job.

HyperPod is AWS's purpose-built cluster for this scale: long-running, fault-tolerant, multi-thousand-GPU jobs.

2. Two flavors

Slurm-based HyperPod (original GA 2023)

Cluster orchestrated by **Slurm** — the HPC standard. Familiar to research teams.

EKS-based HyperPod (GA 2024)

Cluster orchestrated by **Kubernetes / EKS** — familiar to cloud-native teams. **Aligns with Capital One's KServe + EKS choice.**

3. Resilience features

- **Auto-restart on node failure** — replaces a failed node, restores from checkpoint, resumes.
- **Health checks** — proactive detection of degraded GPUs.
- **Lifecycle scripts** — customize node setup (install drivers, mount storage).
- **Cluster controller** monitors the whole cluster.

4. HyperPod Recipes

Curated training scripts for popular FM architectures: - Llama family fine-tuning. - Falcon fine-tuning. - Mistral. - Custom architectures (BYO).

Recipes are a **starting point** — you adapt to your data and config. Faster than building from scratch.

5. Task Governance (2024)

Workload management across multi-team clusters: - Quota allocation per team. - Priority preemption. - Fair-share scheduling.

6. FSx Lustre storage

HyperPod clusters use **FSx Lustre** for shared training data — terabits/s throughput. Data Repository Association (DRA) syncs to S3 (lazy-load on first access).

7. Instance group definitions

A HyperPod cluster is a set of **instance groups** with different roles: - **Compute group** — training instances (P5/P5e/P5en). - **Controller group** — Slurm controller (Slurm flavor only). - **Login group** — SSH access nodes (Slurm flavor).

8. Picking Slurm vs EKS-based HyperPod

	Slurm	EKS
Familiar to	HPC researchers	Cloud-native engineers
Job specification	sbatch / srun	K8s Job / PyTorchJob CRD
Operator ecosystem	Slurm modules	Helm + operators
Integration with KServe / Karpenter	Limited	Native
Capital One lens	Possible for research teams	Strong fit given existing EKS investment

9. 2024-2026 changes

- **EKS-based HyperPod** GA 2024 — the major addition.
- **Task Governance** GA.
- **Recipes** expanded.

10. Pitfalls

- **Underused cluster** — HyperPod costs a lot when idle. Plan capacity.
- **Wrong instance group sizing** — controller too small breaks Slurm flavor.
- **No checkpointing** in your training script — defeats HyperPod’s recovery.
- **FSx Lustre cold-start** — first epoch slow without preload.

11. Capital One lens

Given Capital One’s heavy EKS/KServe investment for inference, **EKS-based HyperPod is the natural fit for FM training**: - Same EKS skillset. - Karpenter integration possible for non-HyperPod ancillary workloads. - Aligns with Sr Lead MLE / Lead MLE JD’s “build Kubernetes clusters, PyTorch, TensorFlow on AWS” language.

12. Sanity check

1. What problem does HyperPod solve that ordinary Training Jobs don’t?
2. Slurm vs EKS HyperPod — what’s the audience for each?
3. What are HyperPod Recipes?
4. What’s Task Governance, and why does it matter for multi-team clusters?
5. Why does FSx Lustre matter here?

13. Cross-references

- **Module 11** — FSx Lustre
- **Module 33** — EKS foundations (the EKS-based HyperPod data plane)
- **Module 36** — SageMaker Training (the non-HyperPod alternative)
- **Module 44** — self-hosted FM (where you’d use HyperPod)

Primary sources

- SageMaker HyperPod docs (archived)
- Research report: [09_sagemaker.md](#)

Module 41 — SageMaker Networking & Security

What this is: SageMaker VPC mode, no-internet mode, KMS encryption, private endpoints, IAM patterns, multi-account ML.

1. VPC mode

By default, SageMaker training/inference runs in **AWS-managed VPC** outside your VPC. For regulated workloads, **VPC mode** runs in **your** VPC:

- Training job ENIs live in your subnets.
- Inference endpoint ENIs live in your subnets.
- Studio Domains can also be VPC-only.

2. No-internet egress mode

Studio Domain “VPC-only” + no NAT GW = no internet access for the SageMaker workload. All AWS API calls must go via VPC endpoints.

Required endpoints for typical workloads: - `s3` (Gateway) - `sagemaker.api` (Interface) - `sagemaker.runtime` (Interface) - `kms` (Interface) - `ecr.dkr` + `ecr.api` (Interface — for pulling container images) - `sts` (Interface) - `cloudwatch logs` (Interface) - `secretsmanager` (Interface — if used)

This is the regulated-finance default. Capital One’s SageMaker Studio Domains likely run in this mode.

3. KMS encryption

SageMaker uses KMS for: - **Studio EFS** — Domain-level EFS that holds user-profile homes. - **Training Job EBS** — local storage on training instances. - **Inference endpoint EBS**. - **S3 model artifacts** (when uploaded). - **CloudWatch Logs** (optional).

Each can use **customer-managed KMS keys (CMKs)** for compliance.

4. Private SageMaker API endpoints

The SageMaker control plane API itself (creating training jobs, endpoints, etc.) can be reached via **VPC Interface endpoint** (`com.amazonaws.<region>.sagemaker.api`). Required for “no internet” Studio.

5. IAM execution roles

Every SageMaker resource has an **execution role**: - Training job execution role: permissions to read input S3, write output S3, pull container. - Endpoint execution role: similar. - Studio Domain execution role: per-user-profile.

Best practice: scope each role narrowly — prefix-scoped S3 access, KMS decrypt on specific keys, no `s3:*`.

6. Cross-account ML patterns

Cross-account Model Registry: - Producer account registers a model. - Consumer accounts pull the model via cross-account model package share (RAM).

Cross-account training: - Pull training data from a different account’s S3 (resource policy + KMS grant). - Run in your account’s VPC.

Cross-account inference: - Endpoint in your account. - Invoked from another account via cross-account IAM.

7. Custom Studio images

Custom Docker images vendored to ECR can be used as Studio kernels: - Pre-installed libraries. - Org-mandated security tools. - Pinned framework versions.

The path for “no `pip install` from public PyPI in regulated workloads.”

8. Network restrictions for sensitive workloads

Beyond no-internet: - **No public ALB** in front of endpoints — use API Gateway private endpoints. - **PrivateLink to consumer accounts** for inference (vs cross-account TGW). - **VPC Flow Logs** capturing all SageMaker ENI traffic. - **Network Access Analyzer** scopes enforcing reachability invariants.

9. Internet-free Studio domains

When fully locked down: - `pip install` only from a **private PyPI mirror** (AWS CodeArtifact, JFrog). - Git only from a private repo via VPC endpoint or hosted internally. - No npm / Maven from public — all proxied.

10. Multi-account ML topology

Typical regulated-finance pattern: - **Training account** — runs Training Jobs, owns experiment data. - **Model Registry account** — central registry, approval gates. - **Serving accounts** (per LOB or per env) — deploy from Registry, run endpoints. - **Audit account** — Lineage + CloudTrail aggregation.

11. 2024-2026 changes

- **SageMaker private endpoints** matured.
- **Studio Domain CMK** broadly supported.
- **Cross-account Model Registry** sharing improved.
- **Custom Studio images** for Code Editor too.

12. Pitfalls

- **Missing one VPC endpoint** → SageMaker job hangs.
- **KMS key policy** not granting SageMaker service principal → cryptic encryption failures.
- **VPC mode without enough subnet IPs** → distributed training fails.
- **NAT GW left on** in “no-internet” Studio → no actual isolation.

13. Capital One lens

- **VPC-only, no-internet Studio Domains** with all AWS service access via PrivateLink.
- **Custom Studio images** vendored with Databolt clients, internal libraries.
- **Cross-account Model Registry** centralized; per-LOB serving accounts.
- **CMK per LOB** with 90-day rotation.

14. Sanity check

1. What does “VPC-only, no-internet” SageMaker mean and what endpoints are required?
2. What does the SageMaker execution role typically need access to?
3. Multi-account ML topology — name the four account roles.
4. Why use custom Studio images in regulated finance?
5. What’s the network architecture for serving SageMaker endpoints to other accounts?

15. Cross-references

- **Module 2** — IAM execution roles
- **Module 6, 8** — VPC + PrivateLink
- **Module 50** — KMS deep
- **Module 52** — compliance posture

Primary sources

- SageMaker VPC Security docs (archived)
- Research report: [09_sagemaker.md](#)

Module 42 — Amazon Bedrock

What this is: Bedrock model catalog (Anthropic Claude, Amazon Nova, Llama, Mistral, Cohere), Knowledge Bases, Agents, Guardrails, Flows, Prompt Management, Distillation, custom models, Provisioned Throughput, Application Inference Profiles.

1. The model catalog

Bedrock is AWS's managed FM service. Models available (2026):

- **Anthropic Claude** — Claude 4.x family (Opus 4.7, Sonnet 4.6, Haiku 4.5). The largest portfolio.
- **Amazon Nova** (GA Dec 2024) — Nova Lite, Nova Pro, Nova Premier, Nova Micro. AWS-native FM family.
- **Meta Llama** — 3.x and 4.x.
- **Mistral** — Mistral, Mixtral, Mistral Large.
- **Cohere** — Command, Embed, Rerank.
- **AI21, Stability, DeepSeek** — additional.

Model access requests — Bedrock requires explicit per-region, per-account, per-model opt-in. Plan this in account vending.

2. Knowledge Bases (managed RAG)

Bedrock Knowledge Bases is a managed RAG implementation:

1. Ingest documents (S3, web crawler, Confluence, SharePoint, etc.).
2. Chunking + embedding (you pick model, e.g., Cohere Embed or Titan Embed).
3. Index into a **vector store** — Bedrock supports:
 - **OpenSearch Serverless** (default)
 - **Aurora pgvector**
 - **Pinecone**
 - **Redis Enterprise**
 - **MongoDB Atlas**
 - **Neptune Analytics** (2024)
4. Query: Bedrock retrieves, augments prompt, calls FM, returns response.

Capital One use case likely: regulatory document RAG with OpenSearch Serverless backend, paired with Guardrails.

3. Agents

Multi-step reasoning with action groups + knowledge bases.

- **Action groups** = OpenAPI specs that the agent can invoke (Lambda-backed).
- **Multi-agent collaboration** GA Dec 2024.
- **Inline agents** — define an agent in code, no console resource.
- **Return-of-control** — the agent yields control back to the application instead of calling a tool directly.

4. Guardrails

Six content filters: 1. **Content filters** — hate, insults, sexual, violence, misconduct, prompt injection. 2. **Denied topics** — define forbidden subjects. 3. **Word filters** — block specific words/phrases. 4. **Sensitive information filters** — PII detection (block or redact). GA includes CC numbers, SSN, etc. 5. **Contextual Grounding Check** — verify response is grounded in retrieved context. 6. **Automated Reasoning** (2024) — formal verification of claims.

Capital One: NeMo Guardrails appears in their job postings. They likely use Bedrock Guardrails as the first line for Bedrock-hosted models and NeMo Guardrails for self-hosted.

5. Flows

Visual workflow builder for chaining Bedrock prompts, FMs, code (Lambda), and Knowledge Bases. The “low-code agent” surface.

6. Prompt Management

Versioned prompt registry — store prompts as managed resources, deploy versions, A/B test.

7. Model Distillation (GA Dec 2024)

Train a smaller, cheaper model from outputs of a larger one. Use: - Teacher: Claude Opus. - Student: smaller Claude Haiku or Llama. - Result: 60-80% cost reduction with 85-95% quality retention.

8. Custom Models

- **Continued Pre-Training (CPT)** — feed domain corpus to adapt base model.
- **Fine-Tuning (FT)** — labeled task data.
- Custom models served via Provisioned Throughput.

9. Provisioned Throughput

Committed capacity for production: - **Per-MU (Model Unit) pricing** — predictable. - Required for custom models, optional for base models. - Commit 1 month or 6 months for discounts. - **Break-even vs on-demand**: ~8-12M output tokens / MU / month.

10. Application Inference Profiles (2024)

Tag-based cost allocation for Bedrock usage: - Profile = tags + cross-region routing rules. - **Per-tag cost reports**. - **Cross-region inference** — automatically route to a region with capacity.

Capital One uses this for **per-app chargeback** (per the re:Invent 2024 “Control the cost of your generative AI services” talk).

11. Bedrock Data Automation (2024)

Newer service for **automated document/image/video understanding**: extract structured data from invoices, IDs, contracts. The “Textract on steroids” pitched for GenAI workflows.

12. 2024-2026 changes

- **Amazon Nova GA** Dec 2024.
- **Model Distillation GA** Dec 2024.
- **Agents multi-agent collaboration GA** Dec 2024.
- **Guardrails Automated Reasoning + Contextual Grounding** new.
- **Application Inference Profiles GA**.
- **Cross-region inference GA** Aug 2024.
- **Bedrock Data Automation GA**.

13. Pitfalls

- **Throttling** — Bedrock has per-region, per-model rate limits. Use Provisioned Throughput at scale or distribute via cross-region profiles.
- **Guardrails false positives** — content filters can over-block; tune thresholds.
- **Knowledge Bases ingestion drift** — re-ingest as source docs change.
- **Forgetting model access requests** in new accounts/regions.

14. Pricing examples (May 2026, us-east-1)

- **Claude Haiku 4.5**: \$0.25 / 1M input tokens, \$1.25 / 1M output
- **Claude Sonnet 4.6**: \$3 / 1M input, \$15 / 1M output
- **Claude Opus 4.7**: \$15 / 1M input, \$75 / 1M output
- **Nova Lite**: \$0.06 / 1M input, \$0.24 / 1M output

- **Provisioned Throughput:** per-MU pricing; commit-based.

15. Capital One lens

Brent Segner’s re:Invent 2024 talk “*Control the cost of your generative AI services*” signals: - **Application Inference Profiles for chargeback.** - **Pre-deployment cost gate** (estimate token cost before launching a new agent). - **Dedicated FinOps-for-AI team.** - **Guardrails first-class** in compliance posture.

Likely workloads: - **Eno** uses Bedrock with Anthropic Claude (LLM upgrade announcement 2024). - **Servicing Tool** uses both Bedrock and self-hosted (NVIDIA GTC 2025 EXS74442). - **Knowledge Bases** for compliance/regulatory document retrieval.

16. Sanity check

1. What are Bedrock’s six guardrail filters?
2. When does Provisioned Throughput pay off vs on-demand?
3. What does Model Distillation do, and what’s the typical cost reduction?
4. What’s the use case for Cross-Region Inference?
5. Walk through how Capital One likely uses Application Inference Profiles.

17. Cross-references

- **Module 27** — vector backends for Bedrock KB
- **Module 44** — self-hosted FM alternative to Bedrock
- **Module 52** — Databolt tokenization upstream of Bedrock prompts
- **Module 56** — Bedrock cost discipline

Primary sources

- Bedrock User Guide (archived as HTML in `downloads/aws_whitepapers/`)
- Bedrock Pricing page (archived)
- [c1tech_reinvent_2024.html](#) — Capital One re:Invent 2024 GenAI cost talk
- Research report: [10_genai_on_aws.md](#)

Module 43 — Amazon Q Family

What this is: Q Developer (rebranded from CodeWhisperer), Q Business (enterprise assistant), Q Apps, Q in QuickSight, Q in Connect. The Amazon-branded LLM productivity layer.

1. Q Developer

Rebranded from **CodeWhisperer** in April 2024. The developer assistant:

- **Code generation** in 15+ languages.
- **Agent mode:** `/dev` (feature implementation), `/transform` (Java upgrade), `/test` (test generation), `/doc` (documentation), `/review` (code review).
- **Security scanning** inline.
- **IDE plugins** for VS Code, JetBrains, Eclipse, Visual Studio, command line.
- **AWS Console embedded** assistant.

Pricing: - **Free tier** — limited. - **Pro tier** — \$19/user/month.

2. Q Business

Enterprise assistant connected to corporate data sources. Conceptually a managed RAG-backed enterprise chatbot.

Features: - **40+ connectors:** Confluence, SharePoint, ServiceNow, Salesforce, Jira, Slack, Teams, GitHub, S3, etc. - **Identity-aware retrieval** — uses each user’s permissions in source systems (not a flat index). - **Requires IAM Identity Center** (organizational SSO).

Pricing: - **\$3/user/month Lite** - **\$20/user/month Pro** (includes Q Apps)

3. Q Apps

No-code app builder within Q Business. Create scoped chatbots / assistants for specific business workflows (e.g., “summarize regulatory updates by jurisdiction”).

4. Q in QuickSight

BI assistant. Natural-language questions translate to SQL/visualizations against your data sources.

- “Show me Q1 revenue by region” → auto-generated chart.
- Embedded in QuickSight dashboards.

5. Q in Connect

Contact-center assistant. Real-time agent assistance with relevant articles, suggested responses, customer history summarization. Comparable to Capital One’s Generative AI Agent Servicing Tool (which appears to be a custom build, possibly leveraging Q in Connect plus custom).

6. Governance

- **Identity-aware** — Q Business retrieval respects source-system permissions per user.
- **Audit logs** to CloudTrail.
- **Tags and chargeback** via Application Inference Profiles (Module 42).
- **No model training on customer data** (explicit policy).

7. 2024-2026 changes

- **CodeWhisperer** → **Q Developer** rebrand April 2024.
- **Q Apps** GA 2024.
- **Q in Connect** GA 2024.
- **Q Business connectors** expanded throughout 2024-2025.

8. Pitfalls

- **Q Business connector freshness** — index refresh latency means users see stale data; tune crawler intervals.
- **Connector permission propagation** — if source-system perms aren’t synced, users see stale denials.
- **Per-user pricing math** — $\$20/\text{user} \times 50,000 \text{ employees} = \$12\text{M}/\text{year}$, a significant line item.

9. Capital One lens

- **Q Business** is the natural fit for enterprise knowledge access — likely paired with their tokenization layer (Databolt) for sensitive data.
- **Q Developer** likely standard issue for engineers (or competing with internal/Anthropic-Claude-direct developer assistants).
- **Q in Connect** may underpin parts of the Servicing Tool, though Capital One built a custom system.

10. Sanity check

1. What was CodeWhisperer renamed to, and when?
2. What does “identity-aware retrieval” mean in Q Business?
3. What is the IAM Identity Center prerequisite for Q Business?
4. How does Q in QuickSight translate natural language to charts?
5. Per-user pricing at 50k users — what’s the annual order of magnitude?

11. Cross-references

- **Module 42** — Bedrock (the FM backend Q uses)
- **Module 12** — DataZone (governance overlap)
- **Module 56** — Q cost analysis

Primary sources

- Q Business / Q Developer docs (archived in downloads/aws_whitepapers/)
- Research report: [10_genai_on_aws.md](#)

Module 44 — Self-Hosted FM Training & Serving

What this is: when to self-host on EC2/EKS vs use Bedrock. NVIDIA NIM, Triton Inference Server, EC2 Capacity Blocks, NVIDIA AI Enterprise, Nemo Guardrails. Capital One's self-hosted path.

1. When to self-host vs Bedrock

Reason	Self-host	Bedrock
Control over runtime / container		
Custom CUDA kernels, vendored libs		
FedRAMP / air-gapped / strict isolation		
Very high token volume (>\$1M/mo Bedrock)	breakeven check	
Latency below Bedrock SLA	(in-VPC)	
Operational simplicity		
Model selection breadth (Claude, Llama, etc.)		
No GPU procurement burden		

Capital One's split (inferred from Eno + Servicing Tool architectures): - **Bedrock for Eno** — natural-language interface with guardrails fitting compliance posture. - **Self-hosted EKS + Triton + NeMo Guardrails for Servicing Tool** — performance-critical, NVIDIA-stack alignment (per GTC 2025).

2. NVIDIA NIM (Inference Microservices)

NIM = optimized container images for serving specific FMs (Llama, Mistral, custom). GA on AWS Marketplace 2024.

- Wraps **TensorRT-LLM**, **vLLM**, **Triton** — picks the best for the model.
- Standardized REST API.
- Auto-tunes batch size, KV cache for the GPU.

3. NVIDIA Triton Inference Server

Generic inference server supporting multiple frameworks (PyTorch, TensorFlow, ONNX, TensorRT, OpenVINO, Python).

- **Dynamic batching** for throughput.
- **Model ensembles** (chain models).
- **Concurrent model execution** on GPU.

Triton on EC2 GPU + behind ALB is a common self-hosted pattern.

4. EC2 Capacity Blocks for ML

(See Module 30 for the EC2 side.)

For self-hosted training: - **Reserve P5/P5e windows for planned FM training.** - 1-182 day windows. - Pay for the window regardless of use.

5. NVIDIA AI Enterprise on AWS

Enterprise-supported NVIDIA software stack (GPU drivers, NeMo, Triton, RAPIDS, etc.). Capital One specifically mentions this stack at re:Invent 2024 and NVIDIA GTC 2025.

6. Nemo Guardrails

Open-source LLM safety framework from NVIDIA. Defines guardrails as YAML/Colang programs: - Topic restrictions. - Conversation flow control. - PII detection. - Fact-checking against retrieved context.

Appears in Capital One Sr Distinguished MLE job posting as a preferred qualification. Their self-hosted ML path uses NeMo Guardrails as the equivalent of Bedrock Guardrails.

7. Self-hosted vector DBs (preview)

For self-hosted RAG (covered in Module 27): - **OpenSearch** (Service or Serverless). - **Aurora pgvector** for tight integration with OLTP. - **MemoryDB vector** for sub-ms agent context.

8. SageMaker HyperPod (the AWS-managed alternative)

If self-hosting feels like too much ops burden but you need foundation-model-scale training, **SageMaker HyperPod** (Module 40) is the middle ground — AWS-managed Slurm or EKS clusters for FM training.

9. EKS + KServe pattern (the Capital One way)

(Full treatment in Module 54.)

The serving stack: - **EKS** with Karpenter for GPU node provisioning. - **KServe InferenceService** CRDs to define models. - **Triton or NIM** as the runtime per model. - **Istio** for traffic management (canary, blue/green). - **NeMo Guardrails** as the safety layer. - **CloudWatch + Prometheus/Grafana** observability.

10. Cost math: self-hosted vs Bedrock

Rough rule of thumb: - For < 1M tokens/day, Bedrock wins (no infra to manage). - For 1-10M tokens/day, comparable. - For > 10M tokens/day, self-hosted breaks even, depending on model. - For > 100M tokens/day at low latency, self-hosted typically wins.

Capital One scale likely puts them in the “self-host the latency-critical and high-volume workloads, Bedrock everything else” zone.

11. 2024-2026 changes

- **NIM GA on AWS Marketplace** 2024.
- **Triton** matured.
- **NVIDIA AI Enterprise on AWS** standard offering.
- **EC2 Capacity Blocks H100/H200** expansion.

12. Pitfalls

- **Self-hosting “because we want control”** without doing the cost math.
- **GPU provisioning blind spots** — Capacity Blocks unused = wasted spend.
- **Model serving without proper batching** — order-of-magnitude waste.
- **Forgetting NeMo Guardrails** — self-hosted models without safety = compliance risk.

13. Capital One lens

- **Eno** likely on Bedrock with Claude (managed).
- **Servicing Tool** likely self-hosted EKS + Triton + NeMo (per GTC 2025).
- **Custom adapters / fine-tunes** for fraud-specific models on self-hosted P5/P5e via Capacity Blocks.

Talking point: *“I’d expect the self-host vs Bedrock split at Capital One to be based on three things: latency budget (sub-100ms favors self-host in VPC), token volume (>10M/day economics), and compliance fit (custom adapters/RLHF favor self-host).”*

14. Sanity check

1. What does NIM wrap, and what’s the value?
2. When does self-hosting beat Bedrock economically?
3. What does NeMo Guardrails do, and why does it appear in Capital One job postings?

4. Walk through the EKS + KServe + Triton + NeMo stack.
5. What's SageMaker HyperPod's role in the self-host vs managed decision?

15. Cross-references

- **Module 30** — EC2 + Capacity Blocks (the hardware)
- **Module 33** — EKS foundations
- **Module 40** — SageMaker HyperPod (managed alternative)
- **Module 42** — Bedrock (the contrast)
- **Module 54** — KServe deep (the serving layer)

Primary sources

- NIM on AWS Marketplace docs (archived)
- NeMo Guardrails GitHub (archived)
- Research report: [10_genai_on_aws.md](#)

Module 45 — Databricks on AWS

Architecture & Control Plane

What this is: Databricks-on-AWS deltas from Azure Databricks — control plane regions, classic vs serverless compute, MWS account model.

1. The control plane / data plane split (recap from Topic 02)

- **Control plane** — Databricks-hosted: web UI, Jobs scheduler, REST API, metastore.
- **Data plane** — your cloud (AWS in this module): compute clusters, storage access.

2. AWS deltas from Azure

	AWS	Azure
Identity	IAM Identity Center / Okta / IAM Users	Azure AD / Entra
Compute auth	Instance profiles (IAM role attached to EC2)	Managed identities
Storage	S3	ADLS Gen2
DNS	Route 53	Azure DNS
Workspace deployment	MWS (Multi-Workspace Service) Account API	Azure portal / ARM
PrivateLink terms	“AWS PrivateLink”	“Private Endpoint”
Networking	Customer VPC (BYO)	Customer VNet

3. Control plane regions (AWS)

Databricks runs control plane in specific AWS regions (us-east-1, us-east-2, us-west-2, eu-central-1, etc.). Your **workspace is bound to one control plane region**; data plane can be co-region.

4. Classic vs Serverless compute

	Classic	Serverless
Data plane location	Your AWS account / VPC	Databricks' AWS account
Compute startup	3-5 min	Seconds
Instance management	You see EC2 instances	None visible
Networking	BYO VPC; PrivateLink available	NCC (Network Connectivity Config) for egress
Cost	EC2 + DBUs	All-in DBUs
Compliance scope	Easier (workloads in your account)	Trickier (data plane outside your account)

The trade-off: classic gives full control and stays in your AWS account; serverless is faster but the compute lives in Databricks' AWS account (not yours).

5. The MWS Account API

The **Multi-Workspace Service Account API** is how you programmatically deploy workspaces on AWS:

- **Account-level resources:** credentials (cross-account IAM role), networks (VPC + subnets + SG), storage (S3 bucket for root storage), encryption keys (KMS).
- **Workspaces** — created with references to above.
- **Logs** — log delivery to S3.

Terraform provider: databricks/databricks. Two-provider pattern: - One provider at **account level** (account_id, host = https://accounts.cloud.databricks.com). - One provider per **workspace** (host = https://<workspace-url>, token = ...).

6. Deployment architecture (typical Capital One pattern)

AWS Account: Databricks Networking

- └ VPC (BYO, /16)
 - └ Private subnets (for clusters)
 - └ PrivateLink endpoints

AWS Account: Databricks Workspace A (Card LOB)

- └ Cross-account IAM role allows Databricks to provision EC2
- └ S3 bucket (root storage)
- └ KMS key (encryption)

Databricks Account API

- └ Creates workspace using above resources

7. 2024-2026 changes

- **Serverless on AWS** matured (general availability for SQL, Jobs, DLT, Notebooks).
- **Mosaic AI** features (Vector Search, Model Serving) on AWS.
- **Self-assuming IAM role** mandate (Jan 2025) — Databricks now requires the cross-account role to be self-assuming for security.

8. Pitfalls

- **Two-provider Terraform** complexity — easy to misconfigure account vs workspace.
- **Cross-account role permission scope** — too broad lets Databricks do too much.
- **Confusing classic and serverless cost models** — DBU rates differ.
- **Workspace bound to one control plane region** — multi-region requires multi-workspace.

9. Capital One lens

Confirmed via job posting “Snowflake, Databricks on AWS, Data Ops, Data Lakes.” Likely setup: - **Multi-workspace per LOB** (Card, Auto, Retail). - **Classic compute in their AWS accounts** for sensitive workloads. - **Serverless for non-sensitive analytical workloads** where control plane access is acceptable.

10. Sanity check

1. What’s the workspace vs control plane region relationship?
2. Classic vs serverless data plane — where does each live?
3. What does the MWS Account API let you do?
4. Self-assuming IAM role — what changed in 2025?
5. Azure-to-AWS Databricks substitutions (identity, storage, compute auth) — name three.

11. Cross-references

- **Topic 02** — Azure Databricks deep (the contrast)
- **Module 46** — Unity Catalog on AWS
- **Module 47** — Databricks networking on AWS
- **Module 2** — IAM (instance profiles, self-assuming roles)

Primary sources

- [databricks_on_aws_overview.html](#)
- Research report: [11_databricks_on_aws.md](#)

Module 46 — Unity Catalog on AWS

What this is: Unity Catalog on AWS — S3 backing, instance profiles vs Azure service principals, storage credentials, external locations, Lake Formation interop.

1. Unity Catalog on AWS — what changes

Topic 02 covered UC fundamentals on Azure. On AWS:

- **Managed storage** uses **S3 buckets** (vs ADLS Gen2 on Azure).
- **Storage credentials** are backed by **IAM roles** (vs Azure service principals).
- **Self-assuming IAM role** required (Jan 2025) for the cross-account role.
- **External locations** point to S3 paths, governed by UC.

2. Storage credentials

A **Storage Credential** in UC = a managed credential UC uses to access cloud storage.

On AWS:

Storage Credential

- └ Type: AWS IAM Role
- └ Role ARN: arn:aws:iam::<account>:role/<UCRole>
- └ (External ID for trust)

The role's trust policy allows the Databricks service principal (in Databricks' AWS account) to assume it with the external ID condition.

3. External locations

A **bridge between UC and S3**:

External Location

- └ URL: s3://<bucket>/<prefix>
- └ Storage Credential: <previous>
- └ Read/write permissions granted in UC

UC enforces grants on external locations; the underlying S3 bucket can still have its own IAM but should be **locked down to deny direct S3 access** to UC-managed prefixes — otherwise users can bypass UC.

4. Self-assuming role mandate (Jan 2025)

Databricks now requires the UC cross-account role to be **self-assuming** — it can assume itself. This adds defense in depth: even if Databricks' service principal is compromised, the attacker can't trivially escalate.

5. Lake Formation interop

The big question: **what happens when both UC and Lake Formation govern the same S3 bucket?**

Two patterns:

Pattern A: UC primary, no LF

UC governs S3; Lake Formation doesn't see those tables. Athena/Redshift Spectrum access via UC's Iceberg REST endpoint (2024 feature).

Pattern B: LF primary, UC reads via Glue Catalog federation

Lake Formation owns the catalog; UC federates to it. Databricks queries against LF-managed tables go through LF's permission model.

AWS Glue Catalog federation to UC (2024 GA) makes Pattern B viable. Tables governed by LF appear in UC as federated objects.

Capital One implication: they likely have both running. The pattern likely: - **Lake Formation** as the AWS-native catalog for non-Databricks consumers (Athena, Redshift Spectrum, EMR). - **Unity Catalog** as the catalog for Databricks workloads. - **Iceberg REST + Glue Catalog federation** as the bridge.

6. Three-level namespace

UC uses a **catalog.schema.table** namespace (vs Hive's **database.table**). Same on AWS as Azure.

7. Governance differences from Azure

- **No Purview integration** — DataZone is the AWS analog (third-party).
- **Audit log delivery** to S3 (vs Azure storage).
- **Lineage UI** same; backed by UC's metastore.

8. Credential vending

For workloads outside Databricks that need S3 access governed by UC: - UC issues **temporary STS credentials** scoped to a specific path/permission. - Caller uses these creds to access S3 directly.

Pattern: a Glue job or Lambda calls UC API → gets temp creds → reads S3.

9. Pitfalls

- **Direct S3 access bypasses UC** — must remove direct IAM grants on UC-managed prefixes.
- **LF and UC double-governing the same tables** without a clear primary causes confusion.
- **Trust policy without External ID** is insecure.
- **Self-assuming role mandate** missed → workspace creation fails (post-Jan 2025).

10. Capital One lens

Likely catalog stack:

```
S3 raw zone
↓
Glue Catalog (metastore) – Lake Formation governs
↓
Glue Iceberg REST Catalog
↓
Unity Catalog federates → Databricks consumers
↓
DataZone for discovery / subscription (overlays both)
```

This isn't trivial — operating two governance frameworks consistently is the open question worth raising in an interview.

11. Sanity check

1. What's the AWS storage credential made of?
2. Why is the self-assuming role mandate a defense-in-depth measure?
3. How can UC and Lake Formation coexist on the same S3 data?
4. What does Glue Iceberg REST Catalog federation enable?
5. What's credential vending used for?

12. Cross-references

- **Topic 02 Module 9** — UC fundamentals on Azure (pre-req)
- **Module 12** — Lake Formation + Glue Catalog + Iceberg
- **Module 10** — S3 + Iceberg + S3 Tables

Primary sources

- [databricks_aws_uc.html](#)
- Research report: [11_databricks_on_aws.md](#)

Module 47 — Databricks Networking on AWS

What this is: AWS PrivateLink for Databricks, Secure Cluster Connectivity (SCC), BYO VPC, NAT GW requirements, classic vs serverless network paths.

1. AWS PrivateLink for Databricks

Databricks supports **two PrivateLink paths**:

- **Front-end PrivateLink** — users → Databricks UI/API via PrivateLink. No traffic on public internet.
- **Back-end PrivateLink** — cluster (your VPC) → Databricks control plane via PrivateLink.

For regulated finance, both are typically required.

2. Secure Cluster Connectivity (SCC)

SCC is Databricks' name for back-end PrivateLink. The cluster initiates an outbound connection to the control plane via PrivateLink — **no inbound from Databricks needed**.

This means: - Cluster nodes don't need public IPs. - Inbound SG can be locked down. - All control-plane communication encrypted and private.

3. BYO VPC (Customer-managed VPC)

Default Databricks behavior creates a VPC for you. **BYO VPC** lets you use your own — required for org-managed networking standards.

Subnet sizing constraints: - VPC: /16 to /25. - Subnets: /17 to /26. - **2 IPs per cluster node** (one for driver, one for executor — and possibly more for shared storage).

So a /24 subnet (251 IPs) supports ~125 concurrent cluster nodes.

4. NAT Gateway requirements

Classic compute clusters need **internet egress** for: - Pulling DBR (Databricks Runtime) images. - Accessing Maven/PyPI/CRAN for libraries. - Webhook callbacks.

Solution: NAT GW with restricted egress (allowlist Databricks-related domains via Network Firewall) **OR** VPC endpoints for AWS services + private mirror for Maven/PyPI.

Serverless compute uses **NCC (Network Connectivity Configuration)** instead — a Databricks-managed egress story.

5. Classic vs serverless data plane network paths

Classic

User → Databricks UI → API call →
Control plane creates cluster in your VPC →
Cluster (in your VPC) reads S3 via S3 endpoint →
Results back to control plane via SCC →
User sees results

Serverless

User → Databricks UI → API call →
Serverless compute in Databricks' VPC →
Cluster reads S3 via Databricks' connectivity (NCC) →
Results back to control plane →
User sees results

Serverless requires you to allow the Databricks VPC to reach your S3 — this is the trickier compliance conversation.

6. Network Connectivity Configuration (NCC)

NCC defines how serverless compute reaches **your** resources: - **S3** access: PrivateLink from Databricks serverless to your S3 endpoint. - **Network Firewall rules** to restrict egress.

This is the equivalent of “BYO VPC” for the serverless world.

7. Cross-account considerations

The cross-account IAM role that Databricks assumes to manage EC2 needs: - `ec2:RunInstances`, `ec2:TerminateInstances` on cluster instances. - `iam:PassRole` on the instance profile. - VPC + ENI permissions.

Capital One pattern: explicit `Condition` restricting role to specific subnets/AMIs/instance types.

8. Pitfalls

- **Subnet too small** → can't scale beyond N nodes.
- **No NAT GW or VPC endpoints** → classic clusters can't pull DBR.
- **Direct S3 IAM** alongside UC governance → users bypass UC.
- **Public IP on cluster nodes** with no SCC → security risk.
- **No PrivateLink on UI** → users hit public Databricks endpoints.

9. Capital One lens

Almost certainly: - **Front-end + back-end PrivateLink** for both UI and SCC. - **BYO VPC** in their network account, shared via RAM with workspace accounts. - **Private mirrors** for PyPI/Maven (CodeArtifact or internal Artifactory). - **AWS Network Firewall** in front of internet egress where needed.

10. Sanity check

1. Front-end vs back-end PrivateLink — what does each protect?
2. What is SCC, and what does it remove the need for?
3. Why might /24 subnet limit you to ~125 cluster nodes?
4. What does NCC let serverless compute reach?
5. What classic-compute needs to pull from the internet, and how do you restrict it?

11. Cross-references

- **Module 8** — multi-VPC + PrivateLink
- **Module 6** — VPC + endpoints
- **Topic 02 Module 20** — Azure Databricks networking (the Azure contrast)

Primary sources

- [databricks_aws_privatelink.html](#)
- [databricks_aws_byovpc.html](#)
- Research report: [11_databricks_on_aws.md](#)

Module 48 — EMR vs Databricks Decision

What this is: the decision framework for EMR Serverless / EMR on EKS / EMR on EC2 vs Databricks. Photon-on-AWS economics. Mosaic AI Vector Search on AWS.

1. The four contenders

	EMR Serverless	EMR on EKS	EMR on EC2	Databricks
Form factor	Pay-per-job, no infra	Spark on your EKS cluster	Persistent EC2 cluster	Managed platform
Best for	Bursty, one-shot jobs	EKS-native shops	Steady-state ETL	ML/data engineering with broad ecosystem
Cost shape	Per-vCPU-second	EKS pricing + Spark	EC2 + EMR markup	DBU + EC2 (classic) or DBU only (serverless)
Spark engine	Apache Spark	Apache Spark	Apache Spark	Photon (proprietary, ~2× faster)
ML stack	Self-built	Self-built	Self-built	Mosaic AI (Vector Search, MLflow, Model Serving)
Notebooks	Basic	Basic	Basic	Polished
Governance	Lake Formation	Lake Formation	Lake Formation	Unity Catalog (or LF interop)

2. When EMR wins

- **AWS-native commitment** — fewer vendor relationships.
- **One-shot batch jobs** — EMR Serverless is cheaper than spinning up a Databricks job cluster.

- **EKS-native shops** — EMR on EKS lives alongside your other workloads.
- **Iceberg-first** — simpler integration in some respects.

3. When Databricks wins

- **Photon performance** — when query time $\times$ hourly cost favors Photon's $\sim 2\times$ speedup.
- **Mosaic AI ecosystem** — Vector Search, AutoML, model serving, MLflow tightly integrated.
- **Cross-team analytical platform** — UC governance, notebooks, SQL Warehouses, Workflows.
- **Multi-language workloads** — Python + Scala + SQL + R in same notebook.

4. Photon on AWS economics

Photon is Databricks' C++ rewrite of Spark's vectorized engine. AWS Databricks (and Azure) charges a **Photon premium DBU rate** ($\sim 2\text{-}3\times$ the standard rate).

Break-even rule of thumb: If Photon makes a query $2\times$ faster, the premium roughly cancels out — total cost stays similar. The win comes when: - Query is **$>2\times$ faster** (often true for large scans, joins, aggregations). - You value **wall-clock time** (interactive analytics, SLA-bound jobs).

For ETL where time is flexible, vanilla Spark (no Photon) is usually cheaper.

5. Mosaic AI Vector Search on AWS

Databricks' managed vector search: - **Delta-Sync** — keeps a vector index synced to a Delta source table. - **Hybrid BM25 + ANN** search. - **Storage-optimized vs compute-optimized endpoints**.

Comparable to AWS-native OpenSearch + Aurora pgvector. The pick depends on whether your data lives in Databricks-managed Delta tables or in S3 with Glue Catalog.

6. Cost-per-DBU on AWS

DBU rates vary by SKU:

SKU	\$/DBU (us-east-1, May 2026)
Jobs Compute (classic)	$\sim \$0.10$
All-Purpose Compute (classic)	$\sim \$0.40$
DLT (Delta Live Tables)	$\sim \$0.20$
SQL Pro	$\sim \$0.55$
Serverless SQL	$\sim \$0.70$
Serverless Compute (Jobs)	$\sim \$0.65$

Plus EC2 cost in classic. Serverless folds compute into DBU rate.

Rule: Jobs Compute (classic) for cost-sensitive batch ETL; All-Purpose / Serverless for interactive.

7. Decision tree

Need Mosaic AI / MLflow / Vector Search?

→ Databricks

Want lowest-cost bursty one-shot Spark?

→ EMR Serverless

Already heavy EKS investment, want Spark co-located?

→ EMR on EKS

Need Photon-speed analytical workloads, SLA-bound?

→ Databricks with Photon

Steady-state large ETL, cost-sensitive?

→ EMR on EC2 (with Spot)

Standard ETL, no ML, AWS-native shop?

→ EMR Serverless

8. Pitfalls

- **Photon premium on workloads that don't benefit** — wastes money.
- **All-Purpose clusters left on overnight** — biggest Databricks bill surprise.
- **EMR Serverless cold start** — first job has init latency.
- **EMR on EKS without Karpenter** — slow scale-up.

9. Capital One lens

Confirmed both Databricks and EMR (and Snowflake) in their stack. Likely pattern: - **EMR (some flavor)** for AWS-native batch ETL where Databricks isn't justified. - **Databricks** for ML/data engineering with Mosaic AI features. - **Snowflake** for enterprise warehousing.

Cost discipline matters at their scale — Photon premium on the wrong workload is real money.

10. Sanity check

1. EMR Serverless vs Databricks Jobs Compute — when does each win?
2. What's the Photon break-even rule?
3. Mosaic AI Vector Search vs OpenSearch — when does each win?
4. What's the DBU cost spread across Jobs / All-Purpose / Serverless?
5. Why is "All-Purpose cluster left overnight" the biggest cost gotcha?

11. Cross-references

- **Module 26** — OpenSearch (vector alternative)
- **Module 14** — Glue Spark (yet another Spark option)
- **Module 23** — Snowflake (the warehouse player)

Primary sources

- [aws_emr_serverless.html](#)
- [aws_emr_on_eks.html](#)
- Databricks pricing page (live, time-sensitive)
- Research report: [11_databricks_on_aws.md](#)

Module 49 — Databricks-AWS Admin & Ops

What this is: MWS Account API, Terraform Databricks provider, workspace deployment patterns, DBU economics, audit log delivery, KMS key separation.

1. MWS Account API endpoints

The Multi-Workspace Service API lives at <https://accounts.cloud.databricks.com>.

Key resources: - **Credentials** — cross-account IAM role. - **Networks** — VPC + subnets + SG references. - **Storage** — S3 bucket for root storage. - **Customer-managed keys (CMKs)** — KMS for workspace, managed services, S3. - **Workspaces** — composed of above. - **Log delivery** — config for audit logs to S3.

2. Terraform two-provider pattern

```
# Account-level provider
provider "databricks" {
  alias      = "mws"
  host       = "https://accounts.cloud.databricks.com"
  account_id = "..."
  username   = "..."
  password   = "..."
}

# Workspace-level provider
provider "databricks" {
  alias = "workspace"
  host  = databricks_mws_workspaces.this.workspace_url
  token = "..."
}
```

Use the account provider to create the workspace; use the workspace provider to manage inside it (clusters, jobs, secrets, UC).

3. Workspace deployment patterns

Single-account-multi-workspace: - Pro: simpler IAM. - Con: blast radius shared.

Multi-account-multi-workspace (Capital One likely): - Pro: isolation per LOB or per env. - Con: more complex Terraform; cross-account RAM for shared VPCs.

4. Billing and DBU economics

DBU rates vary by SKU (Module 48). System tables expose usage data inside Databricks:

```
SELECT workspace_id, sku_name, sum(usage_quantity) AS dbus
FROM system.billing.usage
WHERE usage_date >= current_date - INTERVAL 30 DAY
GROUP BY workspace_id, sku_name
ORDER BY dbus DESC;
```

Pair with AWS Cost Explorer for EC2 + EBS + S3 + NAT GW costs.

5. Audit log delivery

Configure log delivery to S3: - **Workspace audit logs** — every UI/API action. - **Account-level audit logs** — account-API actions.

Format: JSON, partitioned by date and workspace_id.

Consumers: - **Datadog** / **Splunk** via Kinesis Firehose forwarding. - **CloudTrail Lake** for cross-correlation. - **Athena queries** for ad-hoc investigations.

6. KMS key separation

Best practice for regulated finance: - **Workspace key** — encrypts notebook content, job results, secrets. - **Managed services key** — encrypts control-plane-stored metadata. - **Storage key** — encrypts S3 root storage and DBFS.

Separating keys lets you revoke one without breaking the others.

7. Audit logs vs CloudTrail

	Databricks audit logs	CloudTrail
Captures	Databricks API actions	AWS API actions (including those Databricks makes)

	Databricks audit logs	CloudTrail
Latency	Minutes	~15 min
Retention	S3-lifecycle-controlled	CloudTrail Lake or S3
Best for	“What did user X do in Databricks?”	“What AWS resources did Databricks create?”

For full audit story, **collect both**.

8. Workspace deployment in CI

```
resource "databricks_mws_workspaces" "card_ml_prod" {
  provider          = databricks.mws
  account_id        = var.databricks_account_id
  workspace_name     = "card-ml-prod"
  aws_region        = "us-east-1"
  credentials_id     = databricks_mws_credentials.card_ml_prod.credentials_id
  storage_configuration_id = databricks_mws_storage_configurations.card_ml_prod.storage_configuration_id
  network_id        = databricks_mws_networks.card_ml_prod.network_id
  managed_services_customer_managed_key_id = databricks_mws_customer_managed_keys.managed_services.customer_managed_key_id
  storage_customer_managed_key_id = databricks_mws_customer_managed_keys.storage.customer_managed_key_id
  private_access_settings_id = databricks_mws_private_access_settings.card_ml_prod.private_access_settings_id
}
```

9. Operational runbooks

- **Workspace deprovisioning** — graceful (drain jobs first) vs hard.
- **CMK rotation** — managed by AWS; verify workspaces don’t break.
- **Region migration** — workspace bound to one control-plane region; multi-region = multi-workspace.

10. Pitfalls

- **Two-provider Terraform** complexity — easy to misconfigure.
- **CMK without correct grants** — workspace creation fails.
- **Missing private access settings** — workspace UI publicly accessible.
- **Direct S3 access to root bucket** — bypasses UC + Databricks audit trail.

11. Capital One lens

Probable setup: - **Multi-account, multi-workspace** (per LOB × per env). - **Terraform CI** for workspace lifecycle. - **Audit logs to S3 → Datadog**. - **Per-LOB CMKs** with 90-day rotation. - **Slingshot** (their commercial product) likely used internally for Databricks cost optimization too.

12. Sanity check

1. What are the four MWS account-level resources required for a workspace?
2. Why are two Terraform providers (account vs workspace) needed?
3. What's in Databricks audit logs that's not in CloudTrail, and vice versa?
4. Why separate CMKs for workspace, managed services, and storage?
5. What does “private access settings” enforce?

13. Cross-references

- **Module 45** — Databricks-AWS architecture
- **Module 47** — networking (private access)
- **Module 50** — KMS deep
- **Module 56** — cost discipline

Primary sources

- [databricks_mws_api.html](#)
- Terraform Databricks provider docs (live)
- Research report: [11_databricks_on_aws.md](#)

Module 50 — AWS Security Primitives

What this is: KMS, CloudHSM, Secrets Manager, Parameter Store, Macie, GuardDuty, Inspector, Security Hub, Config, IAM Access Analyzer, CloudTrail Lake.

1. KMS (Key Management Service)

The cryptographic foundation.

Key types: - **AWS Owned** — invisible; AWS manages, no charge. Used by default for some services. - **AWS Managed** — visible (`aws/s3`, `aws/rds`), AWS owns rotation. Free. - **Customer Managed Keys (CMKs)** — you create, control policy, rotate. **Required for PCI-DSS-scoped data.**

Key sym/asym: - **Symmetric** — most common; encrypt/decrypt. - **Asymmetric** — sign/verify (RSA, ECC, SM2), encrypt/decrypt. - **HMAC** — MAC keys for authentication.

Multi-region keys — replicate a key to other regions for cross-region encryption with the same key ID.

Envelope encryption — encrypt a Data Encryption Key (DEK) with KMS; encrypt actual data with DEK. KMS only ever sees the DEK. **This is how S3 SSE-KMS and Bucket Keys work.**

Key policies vs IAM policies — both must allow the action. **The biggest KMS pitfall:** thinking IAM alone is sufficient. Key policy must explicitly allow the principal (or `kms:ViaService`).

Rotation — automatic annual rotation for AWS-managed (free); customer-managed can be manual or automatic. **2024:** support for shorter rotation periods (down to 90 days).

2. CloudHSM

FIPS 140-2 Level 3 HSM, dedicated to you. Use when: - Regulatory requirement for **single-tenant** hardware. - Cryptographic operations not supported by KMS.

KMS Custom Key Store can use CloudHSM as backing store. Expensive (\$1+/hr per HSM); justify carefully.

3. Secrets Manager vs Parameter Store

	Secrets Manager	SSM Parameter Store
Use case	Database passwords, API keys with rotation	App config, non-rotating settings
Rotation	Built-in Lambda-based	Manual
Cost	\$0.40/secret/month + \$0.05/10k API calls	Free (Standard); \$0.05/advanced parameter/mo + API calls
KMS encryption	Yes	Yes
Best for	Rotated database creds	App config, feature flags

4. Macie

S3 sensitive-data discovery.

- **150+ identifiers** for PII patterns (SSN, credit card, names, financial account numbers).
- **Automated discovery** (2024) — scans new S3 data automatically.
- **Cost:** per-GB scanned.

Use case: alert if PII lands in unexpected buckets (a post-2019-breach signal).

5. GuardDuty

Threat detection across multiple data sources: - **CloudTrail events**. - **VPC Flow Logs**. - **DNS logs**. - **S3 access patterns**. - **EKS audit logs** (extended). - **RDS login events** (extended). - **Lambda execution patterns** (extended). - **Runtime Monitoring** (eBPF agent for EKS, ECS, EC2).

GuardDuty Extended Threat Detection (GA Dec 2024) — correlates findings across data sources with MITRE ATT&CK mapping.

The detection engine for credential exfiltration patterns (`UnauthorizedAccess:IAMUser/InstanceCredentialExfiltrati`

6. Inspector v2

Vulnerability scanning: - **EC2** — OS package CVEs. - **ECR** — container image CVEs. - **Lambda** — function package CVEs + code security.

Integrates with Security Hub.

7. Security Hub

Aggregated security findings across GuardDuty, Inspector, Macie, IAM Access Analyzer, Config, partner integrations.

- **Conformance packs:** AWS Foundational Security Best Practices, CIS, PCI-DSS, NIST 800-53, etc.
- **Custom insights** — query findings.
- **Auto-remediation** via EventBridge → SSM Automation / Lambda.

The “single pane of glass” for AWS security posture.

8. AWS Config

Compliance posture over time.

- **Config Rules** — managed (e.g., `s3-bucket-public-write-prohibited`) or custom (Lambda or Guard).
- **Conformance Packs** — bundles of rules + remediation.
- **Aggregator** — multi-account view.
- **Compliance change history** — point-in-time state of every resource.

9. IAM Access Analyzer

(See Module 2 for IAM detail.)

- **External access** — what’s reachable from outside the account/org.
- **Unused access** — dormant IAM users/roles/permissions.
- **Custom policy checks** — CI gate for new permissions.
- **Internal access (2024)** — within-org access surface.

10. CloudTrail and CloudTrail Lake

CloudTrail: - Captures all AWS API calls. - Multi-region trail recommended. - S3 + CloudWatch Logs destinations. - **Org trail** — captures across all org accounts.

CloudTrail Lake (managed event lake): - SQL queries against captured events. - 7-year retention. - Replaces the “CloudTrail to S3 to Athena” pattern.

11. 2024-2026 changes

- **GuardDuty Extended Threat Detection** GA Dec 2024.
- **Macie automated discovery** GA 2024.
- **Audit Manager** with new AI/ML, GenAI frameworks.
- **Inspector** code security (2024).
- **KMS rotation periods** down to 90 days.

12. Pitfalls

- **KMS key policy vs IAM policy** interaction — both must allow.

- **GuardDuty without all data sources enabled** → blind spots.
- **Secrets Manager rotation Lambda failures** silently — alert on rotation failures.
- **Config aggregator missing accounts** — incomplete compliance picture.

13. Capital One lens

Post-2019-breach posture, almost certainly: - **CMK per LOB** with 90-day rotation. - **GuardDuty Extended Threat Detection** enabled org-wide. - **Macie automated discovery** on every S3 bucket. - **Security Hub** with PCI-DSS + NIST conformance packs. - **CloudTrail Lake** for forensics. - **Cloud Custodian** (their OSS) for continuous policy enforcement on top of all this.

14. Sanity check

1. KMS key policy vs IAM policy — what's the trap?
2. Secrets Manager vs Parameter Store — when does each win?
3. What does GuardDuty Extended Threat Detection add over plain GuardDuty?
4. Macie's job — when would you use it?
5. Security Hub conformance packs — name three relevant for regulated finance.

15. Cross-references

- **Module 2** — IAM (the access principals)
- **Module 51** — Cloud Custodian (policy-as-code on top of these primitives)
- **Module 52** — compliance frameworks

Primary sources

- [KMS_Cryptographic_Details.pdf](#)
- [AWS_Audit_Manager_User_Guide.pdf](#)
- Research report: [12_security_compliance.md](#)

Module 51 — Governance & Policy-as-Code

What this is: Control Tower, Landing Zone Accelerator, Organizations + SCPs + RCPs, Cloud Custodian (Capital One’s open-source), cfn-guard, cdk-nag, AWS Audit Manager.

1. The governance landscape

Tool	What it does
Control Tower / LZA	Multi-account landing zone (Module 1)
AWS Organizations	Org tree, SCPs, RCPs
Cloud Custodian	YAML policy-as-code for runtime governance (the Capital One-built tool)
cfn-guard	CloudFormation policy-as-code (deploy-time)
cdk-nag	CDK construct guardrails (synth-time)
CloudFormation Hooks	Deploy-time policy enforcement
AWS Audit Manager	Regulatory framework attestation

2. SCPs and RCPs (recap)

(Detailed in Module 1.)

- **SCPs** bound principal permissions.
- **RCPs** (GA Nov 2024) bound resource access — close the cross-org access gap that SCPs couldn’t.

3. Cloud Custodian — the Capital One project

Cloud Custodian (github.com/cloud-custodian/cloud-custodian) is YAML-based policy-as-code for AWS, GCP, Azure.

- Originally built by Capital One.

- **CNCF Incubating** since Sep 2022.
- Apache 2.0.
- 2026: nearing ten-year anniversary.

c7n architecture

Policies are YAML:

policies:

- name: terminate-untagged-ec2
 - resource: ec2
 - filters:
 - "tag:CostCenter": absent
 - actions:
 - type: notify
 - to: [security@example.com]
 - transport:
 - type: sns
 - topic: arn:aws:sns:...:security-alerts
 - type: stop

Custodian discovers AWS resources via API, filters them, applies actions (tag, stop, terminate, notify).

Common patterns

- **Tag enforcement** — find untagged resources, notify-then-act.
- **Encryption enforcement** — find unencrypted EBS, RDS, S3; remediate.
- **Public access prevention** — find S3 buckets with public ACLs; lock down.
- **Idle resource shutdown** — find EC2/RDS not used in 7 days; stop.
- **Cost guardrails** — find oversized instances; recommend right-sizing.

Dry-run discipline

Always test policies in `--dryrun` mode first. Custodian's actions can be destructive (terminate, delete) — dry-run shows what would happen without doing it.

Mode: deployment patterns

- **Pull mode** — Lambda triggered on schedule (EventBridge).
- **Event mode** — Lambda triggered on CloudTrail event (real-time enforcement).
- **CLI mode** — run from a CI/CD pipeline.

Why Capital One built this

Custodian was Capital One's answer to **continuous, runtime, policy-as-code governance** — preventive measures (SCPs) plus detective + responsive measures (Custodian).

Reading recommendation: before any interview, spend 90 minutes reading the [Cloud Custodian docs](#) and three example policies. You will impress interviewers by referencing it by name and identifying which AWS APIs it wraps.

4. cfn-guard

CloudFormation policy-as-code. Written in Rust; declarative DSL:

```
let ec2_instances = Resources.*[ Type == 'AWS::EC2::Instance' ]

rule require_tags when %ec2_instances !empty {
  %ec2_instances.Properties.Tags[*].Key == /CostCenter/
}
```

Run in CI to block deployments that violate rules. Native CloudFormation Hooks integration.

5. cdk-nag

CDK construct that scans your CDK app at **synth time** for security issues:

```
import * as cdkNag from 'cdk-nag';
cdk.Aspects.of(app).add(new cdkNag.AwsSolutionsChecks());
```

Catches: unencrypted resources, IAM wildcards, missing logging, public exposure.

Capital One uses cdk-nag heavily (per re:Invent 2024 talk by Ishu Gupta).

6. CloudFormation Hooks

Pre-deployment policy enforcement — fires before a stack creates/updates/deletes resources. Can block based on rules.

The “infra-level gatekeeper” — works even if someone bypasses your CI/CD.

7. AWS Audit Manager

Automated compliance evidence collection.

Frameworks: - AWS Best Practices, CIS, NIST 800-53, NIST CSF. - PCI-DSS, SOC 2, HIPAA, GDPR.
- **New 2024:** GenAI / AI-ML frameworks.

Audit Manager continuously collects evidence from CloudTrail, Config, etc., maps to control statements, produces reports for auditors.

8. The four pillars (when to use what)

1. **Preventive** — SCPs, RCPs, IAM permission boundaries.

2. **Deploy-time** — cdk-nag (synth), cfn-guard (template), CloudFormation Hooks (deploy).
3. **Detective + responsive** — Cloud Custodian.
4. **Audit/attestation** — Audit Manager + CloudTrail Lake + Config.

9. Pitfalls

- **Custodian without dry-run** — accidental terminations.
- **cfn-guard rules too loose** — gives false confidence.
- **Audit Manager unmapped controls** — incomplete evidence trail.
- **SCPs/RCPs only**, no runtime detection → drift accumulates.

10. Capital One lens

Capital One’s governance stack (inferred from public talks and OSS):

```

SCPs/RCPs (preventive)
  ↓
cdk-nag (synth)
  ↓
cfn-guard / CloudFormation Hooks (deploy-time)
  ↓
Cloud Custodian (runtime detection + auto-remediation)
  ↓
Audit Manager + CloudTrail Lake (attestation)

```

This four-layer model is the regulated-finance posture.

11. Sanity check

1. SCPs vs RCPs — what gap does RCP close?
2. What is Cloud Custodian’s project status (license, foundation)?
3. cfn-guard vs cdk-nag — when does each fire?
4. Audit Manager — what does it produce?
5. Why is dry-run discipline critical with Custodian?

12. Cross-references

- **Module 1** — Organizations, Control Tower, LZA
- **Module 2** — IAM (the permission layer)
- **Module 50** — security primitives (the data sources Custodian reads)
- **Module 52** — compliance frameworks (what Audit Manager attests against)

Primary sources

- [cloud_custodian_readme.md](#)
- [cloud_custodian_aws_provider.html](#)
- [AWS_Audit_Manager_User_Guide.pdf](#)
- Research report: [12_security_compliance.md](#)

Module 52 — Financial-Services Compliance Lens

What this is: PCI-DSS v4 on AWS, SOC, FFIEC, OCC, **SR 11-7 model risk management**, GLBA, Databolt-style tokenization patterns, and the 2019 Capital One breach postmortem mapping each lesson to a specific AWS feature.

1. The regulatory stack

A US bank operating on AWS must satisfy:

	What it constrains
PCI-DSS v4.0	Cardholder data protection (active Mar 31 2024; mandatory Mar 31 2025)
SOC 1/2/3	Operational controls (AWS provides; you provide on top)
FFIEC	Federal financial institution exam framework + 2020 cloud computing statement
OCC	Bank supervision (the 2019 enforcement action against Capital One)
SR 11-7	Federal Reserve / OCC guidance on model risk management
GLBA Safeguards Rule	Customer financial info privacy (FTC's 2023 update)

2. PCI-DSS v4 on AWS

Shared responsibility: - **AWS** provides PCI-DSS-certified infrastructure (regions, services). - **You** configure encryption, access, logging, network segmentation.

Key v4 requirements: - **4.x** — Encryption at rest and in transit. - **8.x** — Access control (MFA, no shared credentials). - **10.x** — Logging and monitoring.

For a SageMaker workload: VPC isolation, KMS CMK for S3 + EFS, IAM Identity Center MFA, CloudTrail with file integrity validation.

3. SOC reports

In **AWS Artifact**: - **SOC 1** — financial controls. - **SOC 2** — security, availability, confidentiality, processing integrity, privacy. - **SOC 3** — public summary.

Banks consume these from AWS and produce their own to customers/regulators.

4. FFIEC Cloud Computing Statement

The FFIEC's joint statement (originally 2020, updated since) on cloud computing for financial institutions. Covers: - Governance (who owns the cloud relationship). - Cloud security management. - Change management. - Resilience and recovery. - Audit.

The “compliance baseline” examiners reference.

5. OCC supervision

The OCC supervises banks; Capital One was fined **\$80M in Aug 2020** in the post-2019-breach consent order. The order specified controls Capital One had to implement — many became broader industry best practice (IMDSv2, IAM Access Analyzer, etc.).

6. SR 11-7 — Model Risk Management

Federal Reserve / OCC supervisory guidance on model risk. Three pillars:

1. **Model development** — sound theory, robust validation against alternatives.
2. **Model validation** — independent review (not by model developers).
3. **Governance** — policies, roles, documented risk appetite.

Mapping to AWS services

SR 11-7 requirement	AWS feature
Documented model purpose, data, methodology	SageMaker Model Cards
Versioned model artifacts	Model Registry
Bias and fairness analysis	Clarify (bias detection, SHAP)
Experiment lineage (which data, code, hyperparams trained which model)	Lineage Tracking + rubicon-ml (Capital One OSS)
Production monitoring for drift	Model Monitor
Approval gates	Model Registry approval status
Audit trail	CloudTrail + CloudTrail Lake

This is **the** map for regulated-finance ML engineering. Capital One’s rubicon-ml is purpose-built for the experiment-lineage piece.

7. GLBA Safeguards Rule (FTC 2023 update)

Required controls for non-bank financial institutions and applicable safeguards for banks: - Risk assessment. - Information security program. - Designated qualified individual. - Access controls. - Encryption at rest and in transit. - MFA. - Secure development. - Service-provider oversight.

8. Databolt-style tokenization

Tokenization replaces sensitive data (credit card numbers, SSNs) with surrogate tokens. The mapping is held in a secure vault.

Vaulted — tokens map to vault entries (lookup required). **Vaultless** — tokens generated algorithmically (often FPE — Format-Preserving Encryption: same length, same characters as original).

FPE algorithms: FF1, FF3-1 (NIST-recommended).

Capital One Databolt is the vaultless / FPE solution they productized. Designed to: - Keep raw PCI data **out of analytic systems** entirely. - Allow downstream queries on tokenized data (joins, filters). - Preserve formats for compatibility. - Reduce PCI-DSS audit scope to the tokenization layer.

Pattern: Databolt sits **upstream of S3** — raw PAN never lands; only tokens persist.

9. The 2019 Capital One breach postmortem

The breach (detail in Module 2): 1. SSRF via misconfigured WAF. 2. IMDSv1 returned EC2 instance role creds. 3. Instance role had over-permissive S3 access. 4. ~106M records exfiltrated.

Each lesson → AWS feature

Lesson	Defense
SSRF + IMDSv1	IMDSv2 required (now default on new instance types)
Over-permissive instance role	Least privilege + permission boundaries
Bucket policy didn’t constrain to org principals	RCPs (GA Nov 2024) + aws:PrincipalOrgID
No detection of unusual S3 access patterns	GuardDuty Extended Threat Detection
Sensitive data discoverable in compromised bucket	Macie + Databolt tokenization upstream
Public S3 bucket	Block Public Access at org level via RCP

Every regulated-finance AWS architecture review now references the breach (whether named or not).

10. AWS Artifact

The portal where you download AWS' compliance reports — SOC, PCI AOC, ISO 27001 audit reports, FedRAMP packages, etc. Required reading for audit packages.

11. Capital One lens

Compliance posture inferred from public sources:

- **PCI-DSS scope reduction** via Databolt tokenization upstream.
- **SR 11-7 alignment** via Model Cards + Model Registry + Clarify + rubicon-ml.
- **GLBA Safeguards** mapped to standard AWS controls.
- **FFIEC examiner-ready** architecture artifacts (audit reports, IR runbooks).
- **OCC consent-order controls** integrated and exceeded.

12. Sanity check

1. What's the PCI-DSS v4 mandatory date?
2. What are the three SR 11-7 pillars, and what AWS feature maps to each?
3. Vaulted vs vaultless tokenization — what's FPE?
4. Name three controls that would have stopped the 2019 breach.
5. What's in AWS Artifact, and who uses it?

13. Cross-references

- **Module 2** — IAM, IMDSv2, breach chain
- **Module 35, 38** — SageMaker Model Cards, Model Registry, Clarify
- **Module 50, 51** — Security primitives + Cloud Custodian
- **Module 53** — Capital One MLOps spine (where rubicon-ml lives)
- **CAPITAL_ONE.md** — breach context

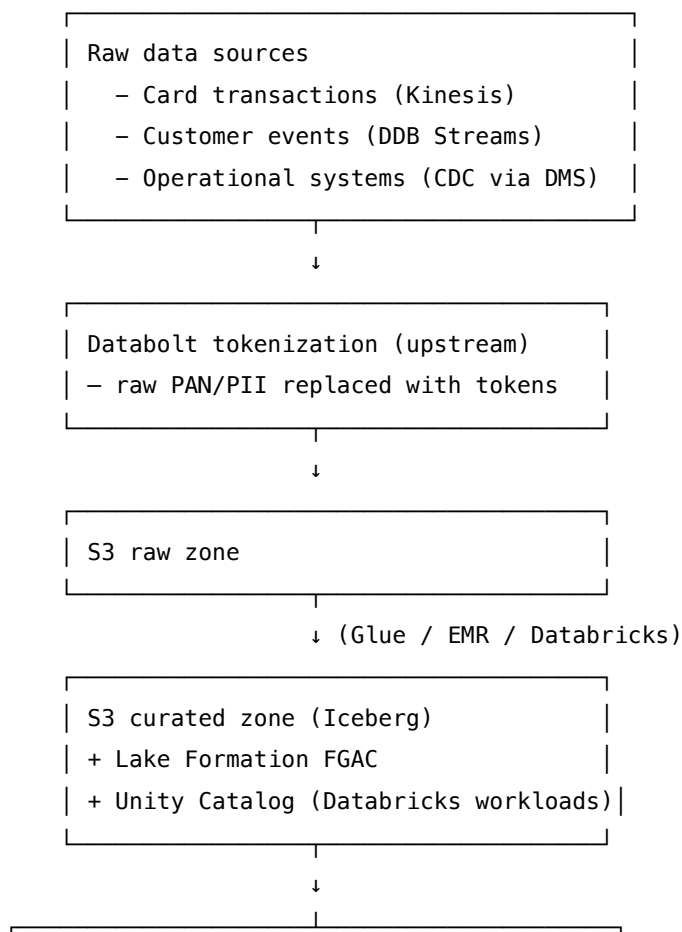
Primary sources

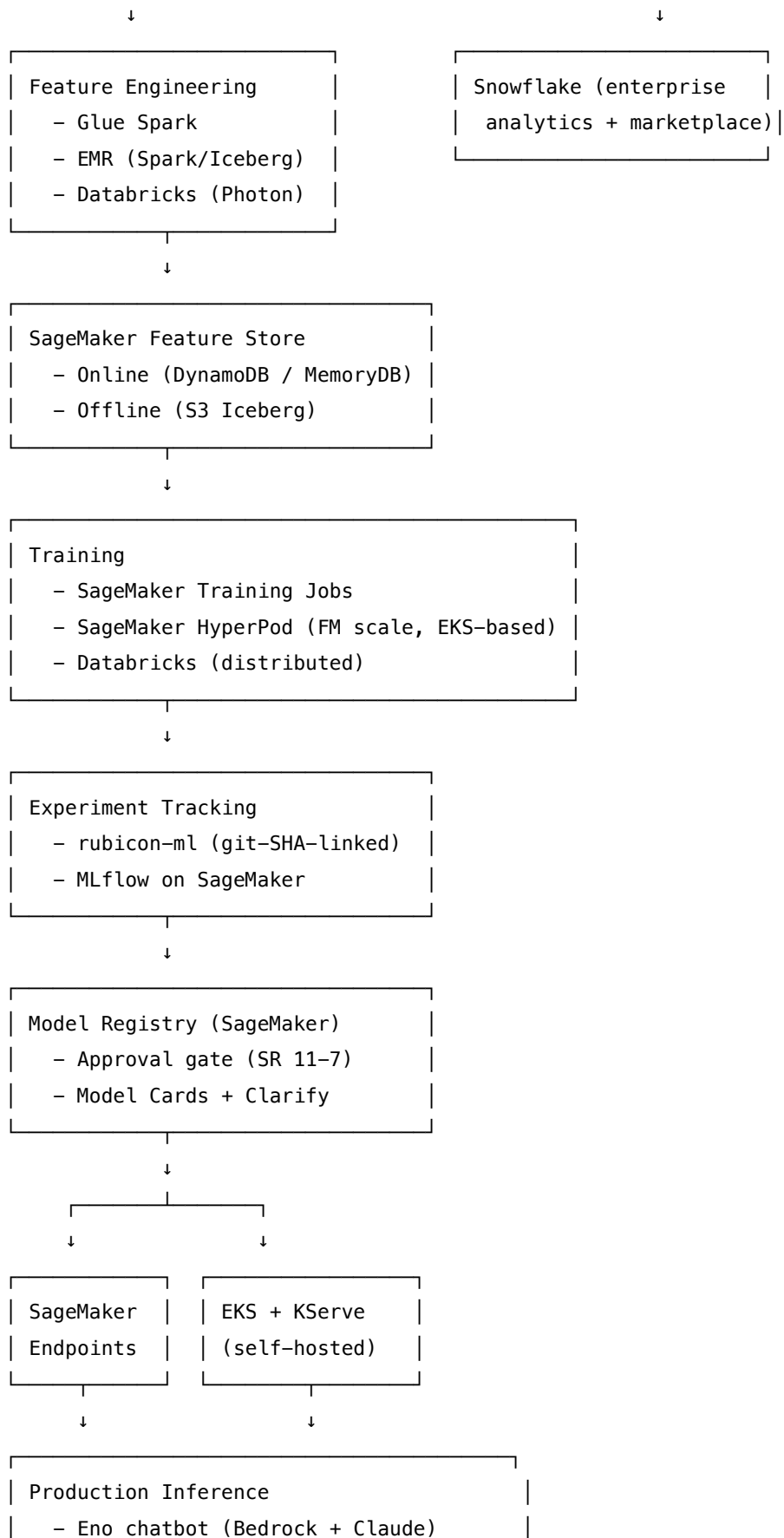
- [OCC_2020_Consent_Order_Capital_One.pdf](#)
- [FFIEC_Cloud_Computing_Statement.pdf](#)
- [SR_11-7_Model_Risk_Management.pdf](#) (2026 revision: SR 26-02)
- [PCI_DSS_v4_0_Quick_Reference.pdf](#)
- Research report: [12_security_compliance.md](#)

Module 53 — The Capital One MLOps Spine

What this is: the synthesized view of Capital One's MLOps platform — SageMaker + Step Functions + Glue + EMR + EKS/KServe + rubicon-ml. The integrated picture across previous modules.

1. The full diagram





	- Servicing Tool (EKS + Triton + NeMo)	
	- Fraud scoring (Neptune ML + SM)	

Orchestration overlay: EventBridge → Step Functions → above
 Observability overlay: CloudWatch + Datadog + Prom/Grafana
 Governance overlay: Lake Formation + UC + Model Cards + rubicon-ml
 Compliance overlay: Cloud Custodian + GuardDuty + Macie + Security Hub

2. The named platform team — IFX

Intelligent Foundations and Experiences (IFX) is Capital One’s internal ML platform team (per the Lead MLE job posting).

IFX likely owns: - The MLOps platform (above). - Internal SDKs (the serverless Kinesis SDK, model serving wrappers). - Shared infrastructure (EKS clusters, SageMaker domains). - Self-service model deployment paths.

3. The rubicon-ml choice

rubicon-ml (github.com/capitalone/rubicon-ml) is Capital One’s experiment-tracking tool.

The differentiator: **auto_git_enabled=True** binds every experiment to a specific git SHA. The lineage is automatic:

```
from rubicon_ml import Rubicon

rubicon = Rubicon(persistence="filesystem", root_dir="./rubicon", auto_git_enabled=True)
project = rubicon.create_project("card-fraud-v3")
exp = project.log_experiment(name="run-12")
exp.log_metric("auc", 0.92)
exp.log_parameter("learning_rate", 0.001)
# Each experiment automatically tags the git SHA of the code that ran it
```

This is the **SR 11-7 audit answer**: *“Show me which exact code (down to the commit) trained this production model.”*

4. Fraud-graph ML

Capital One’s tech blog publishes work on: - Graph ML for fraud detection. - University partnerships on global graph transformers, dynamic customer embeddings.

The architecture pattern: 1. Stream transactions to Neptune (writer). 2. Periodic Neptune ML GNN training. 3. Score new transactions via SageMaker inference using GNN-derived features.

5. The hybrid serving decision (SageMaker vs KServe)

The single most distinctive thing about Capital One’s MLOps:

Choose SageMaker endpoints when	Choose EKS + KServe when
Low-volume model	High-volume + latency-critical
Standard framework (PyTorch / TF / XGBoost)	Custom runtime (Triton + custom kernels)
Want zero ops on serving	Existing EKS skillset and operations
Don’t need fine-grained traffic routing	Need pod-level canary, blue/green
Acceptable to be on managed schedule	Need air-gapped, FedRAMP-style isolation

6. Talking points for interviews

Three crisp framings:

- *“Capital One’s MLOps spine is SageMaker + Step Functions + Glue + EMR + EKS/KServe with rubicon-ml binding every experiment to a git SHA — the SR 11-7 audit answer is built into the platform.”*
- *“The hybrid SageMaker-endpoint and KServe-on-EKS serving decision is what makes Capital One distinctive. SageMaker for managed simplicity, KServe for latency, custom runtime, and Kubernetes-native operations.”*
- *“Databolt sits upstream of S3 — raw PAN never lands in the lake, so PCI-DSS scope is bounded to the tokenization tier. This shapes every data pipeline downstream.”*

7. Cross-references

This module synthesizes: - **Modules 35, 38** — SageMaker MLOps - **Module 33, 54** — EKS + KServe - **Module 40** — HyperPod for FM training - **Module 52** — SR 11-7 + Databolt - **Module 25** — Neptune + fraud graphs - **Module 51** — Cloud Custodian governance - **CAPITAL_ONE.md** — the dossier

8. Sanity check

1. What does IFX stand for?
2. What does rubicon-ml’s `auto_git_enabled=True` enable, audit-wise?
3. Walk through the SageMaker vs KServe decision.
4. Where does Databolt sit in the data flow, and what does it bound?
5. How does fraud-graph ML use Neptune + SageMaker together?

Primary sources

- [rubicon_ml_readme.md](#)

- [c1tech_ai.html](#)
- [c1tech_reinvent_2024.html](#)
- [CAPITAL_ONE.md](#)
- Research report: [13_mlops_c1_patterns.md](#)

Module 54 — EKS for ML Serving — KServe Deep

What this is: KServe — the Kubernetes-native model serving project. CRDs (InferenceService, ServingRuntime), KServe ModelMesh, KEDA, Karpenter for GPU, Istio, IRSA/Pod Identity, NVIDIA GPU Operator. The differentiator that Capital One explicitly mentions in its Lead MLE job posting.

1. The KServe story

KServe (formerly KFServing, part of Kubeflow until it became independent) is the standard Kubernetes-native ML serving framework. Capital One’s Lead MLE posting is titled “**Lead Machine Learning Engineer (MLOps, KServe — building Kubernetes Clusters, PyTorch, TensorFlow on AWS)**” — explicit signal.

2. InferenceService CRD

The core abstraction:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: card-fraud
  namespace: ml-prod
spec:
  predictor:
    serviceAccountName: kserve-sa # IRSA / Pod Identity
    minReplicas: 2
    maxReplicas: 50
    model:
      modelFormat:
        name: pytorch
      storageUri: s3://co-card-prd-models/fraud/v3/
```

```

resources:
  requests:
    nvidia.com/gpu: 1
    memory: 16Gi
  limits:
    nvidia.com/gpu: 1
    memory: 16Gi
transformer:
  containers:
    - image: 123.dkr.ecr.us-east-1.amazonaws.com/transformer:v1
explainer:
  alibi:
    type: AnchorTabular
    storageUri: s3://co-card-prd-models/fraud/v3/explainer

```

Components: - **Predictor** — the model server. - **Transformer** — optional pre/post-processing. - **Explainer** — optional explainability sidecar (Alibi / SHAP).

3. Runtimes

KServe ships ServingRuntime definitions for: - **TensorFlow Serving** - **TorchServe** - **Triton Inference Server** (NVIDIA) - **ONNX Runtime** - **MLServer** - **vLLM** (LLM serving) - **Custom** — your own Docker image

4. ModelMesh — multi-model serving

For shops with many small models, **ModelMesh** packs multiple models into shared inference pods: - Models load on-demand from S3. - LRU eviction when memory full. - Routing transparent to clients.

Analogous to SageMaker MME but Kubernetes-native.

5. Deployment modes

- **Knative Serving** (the original) — serverless, scale-to-zero.
- **Raw Deployment** (GA, increasingly preferred at scale) — vanilla K8s Deployment + HPA + Service, avoiding Knative’s revision proliferation.

Capital One scale likely uses Raw Deployment — Knative’s revision history grows fast and adds operational complexity.

6. KEDA — event-driven autoscaling

Standard HPA scales on CPU/memory. **KEDA** scales on **external signals**: - SQS queue depth. - Kafka consumer lag. - Custom CloudWatch metrics. - Prometheus metrics.

For ML serving: scale on **incoming request rate from the messaging layer**, not just CPU.

7. Karpenter for GPU

(Module 33 introduces Karpenter.) For ML serving:

```
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: gpu-inference
spec:
  template:
    spec:
      requirements:
        - key: "node.kubernetes.io/instance-type"
          operator: In
          values: ["g5.xlarge", "g5.2xlarge", "g6e.xlarge", "g6e.2xlarge"]
        - key: "karpenter.sh/capacity-type"
          operator: In
          values: ["spot", "on-demand"]
      nodeClassRef:
        name: default
```

When a KServe pod is Pending due to no GPU node, Karpenter provisions one in 30 seconds. Idle GPU nodes terminate via consolidation.

8. Istio service mesh

For traffic management: - **VirtualService** — request routing rules. - **DestinationRule** — load balancing policy, circuit breaker. - **Canary deployments**: shift 5% of traffic to v2, monitor metrics, shift more. - **Blue/green**: maintain v1 + v2 pods simultaneously, flip traffic atomically.

KServe integrates with Istio for these patterns.

9. Gateway API (newer)

The Kubernetes Gateway API (more powerful than Ingress) is replacing some Istio patterns in 2025-26. Worth tracking.

10. IRSA / Pod Identity

(Module 33.) The KServe pod's ServiceAccount → IAM role → access to S3 model bucket, KMS decrypt, CloudWatch metrics, etc. **No static AWS keys** in the pod.

11. NVIDIA GPU Operator

Manages on GPU nodes: - **NVIDIA drivers**. - **CUDA toolkit**. - **Device plugin** (advertises `nvidia.com/gpu` to scheduler). - **MIG (Multi-Instance GPU)** — partition an A100/H100 into smaller logical GPUs.

EKS Auto Mode handles this automatically; otherwise you install it via Helm.

12. Observability

KServe + Istio + Prometheus + Grafana: - Per-model latency, throughput, error rate. - GPU utilization, memory, KV cache. - Request/response distributions.

13. Why Capital One picks KServe over SageMaker endpoints

Inferred reasons: - **Custom runtime control** — vendored Triton with specific TensorRT-LLM versions. - **Tight integration with their EKS investment** — same skillset and operations as the rest of their services. - **Multi-region portability** — easier to run identical KServe deployments across regions. - **Cost at high scale** — per-pod GPU sharing via ModelMesh and Karpenter consolidation can beat SageMaker endpoint pricing at very high volume. - **Pod-level traffic control** for canary / blue/green (Istio). - **Air-gapped, FedRAMP-style isolation** when needed.

14. KServe + Triton + NeMo Guardrails (the C1 Servicing Tool stack)

A plausible architecture for Capital One's Generative AI Agent Servicing Tool (presented at NVIDIA GTC 2025):

```
External request → Istio Gateway →  
  KServe InferenceService (transformer pod: input sanitization) →  
  Predictor pod (Triton + LLM model) →  
  NeMo Guardrails sidecar (safety filter) →  
  Response transformer →  
  Out
```

All running on EKS with Karpenter-provisioned GPU nodes.

15. 2024-2026 changes

- **Karpenter v1.x** stable (Module 33).
- **Istio Ambient Mode GA** — sidecar-less mesh.
- **Gateway API** replacing Ingress in many shops.
- **KServe Raw Deployment** mode dominant for scale.
- **EKS Pod Identity Agent** replacing IRSA for new clusters.

16. Pitfalls

- **KServe revision proliferation** (Knative mode) — clean up old revisions or use Raw Deployment.
- **OIDC trust-policy sub claim scoping** — too broad allows lateral movement across namespaces.
- **No PodDisruptionBudget** → cluster upgrades evict all model pods at once.
- **GPU operator misconfig** → pods stuck Pending with cryptic errors.
- **Karpenter without consolidation** → idle GPU nodes burn.

17. Sanity check

1. What's the InferenceService CRD anatomy (predictor, transformer, explainer)?
2. ModelMesh — what problem does it solve, and what's the SageMaker analog?
3. Knative vs Raw Deployment mode in KServe — when does each win?
4. What is KEDA, and why does it matter for ML serving?
5. Walk through the Karpenter + GPU node + KServe pod lifecycle.

18. Cross-references

- **Module 33** — EKS foundations (the underlying cluster)
- **Module 30** — EC2 GPU instances (the nodes)
- **Module 37** — SageMaker inference (the managed alternative)
- **Module 53** — Capital One MLOps spine (the integrated view)
- **Module 44** — self-hosted FM (the related concept)

Primary sources

- [kserve_main.html](#)
- [kserve_quickstart.html](#)
- [karpenter_main.html](#)
- [aws_ml_eks_best_practices.html](#)
- Research report: [13_mlops_c1_patterns.md](#)

Module 55 — CI/CD for ML on AWS

What this is: CodePipeline / CodeBuild / CodeArtifact, GitHub Actions with **OIDC federation to AWS IAM** (the modern pattern), SageMaker Pipelines vs Step Functions for ML CI/CD, model registry promotion strategies, IaC with CDK + Terraform.

1. The CI/CD chains

Stack	Use
CodePipeline + CodeBuild + CodeArtifact	AWS-native CI/CD
GitHub Actions + OIDC	Most common in 2026
GitLab CI + OIDC	Where GitLab is the source
Jenkins on EC2	Legacy; many shops migrating

GitHub Actions + OIDC pattern (the modern default)

Don't store AWS keys in GitHub. Instead, configure OIDC trust:

1. In AWS, create an **OIDC provider** for `https://token.actions.githubusercontent.com`.
2. Create an IAM role with trust policy allowing `sts:AssumeRoleWithWebIdentity` from that OIDC provider, conditioned on the GitHub sub claim.
3. In GitHub Actions, use the `aws-actions/configure-aws-credentials@v4` action with `role-to-assume`.

permissions:

```
id-token: write
contents: read
```

jobs:

deploy:

```
runs-on: ubuntu-latest
```

steps:

- uses: actions/checkout@v4
- uses: aws-actions/configure-aws-credentials@v4

```
with:
  role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
  aws-region: us-east-1
- run: terraform apply -auto-approve
```

The trust policy sub claim must be scoped tightly:

```
"Condition": {
  "StringEquals": {
    "token.actions.githubusercontent.com:sub": "repo:capitalone/card-ml:ref:refs/heads/main"
  }
}
```

A too-broad sub (repo:capitalone/card-ml:*) lets feature branches assume the same role — a real attack vector.

2. CodeCommit deprecation note

CodeCommit closed to new customers mid-2024. Existing customers retained; new shops use GitHub Enterprise, GitLab, or Bitbucket Cloud.

3. SageMaker Pipelines vs Step Functions for ML CI/CD

(Module 15.)

- **SageMaker Pipelines** — best for the ML-specific DAG (Processing → Training → Evaluation → Register).
- **Step Functions** — best for the outer orchestration (event → SM Pipeline → Model Registry → deploy gate → deploy).

Pattern: GitHub Actions kicks off Terraform/CDK; Terraform/CDK provisions resources; SageMaker Pipeline runs; Step Functions orchestrates the deployment after Model Registry approval.

4. Model registry promotion strategies

Three approaches:

Manual approval

PR merges to main → training → Model Registry as Pending → human approver flips to Approved → deployment.

Automated approval gates

Same flow, but a CI job runs evaluation checks (accuracy threshold, bias threshold, latency benchmark) and auto-approves if passed.

Progressive rollout

Approved model → deploy to staging → canary on prod (5%) → full prod after N hours of green metrics.

Capital One pattern: all three, depending on workload sensitivity. Manual for high-risk models (credit decisioning); automated + progressive for low-risk.

5. Packaging models

- **Model Registry** (SageMaker) — the metadata + S3 artifact reference.
- **Container image** in ECR — for KServe/Triton serving.
- **S3 artifact** — the raw weights for SageMaker endpoints.

Most regulated shops use **Model Registry as the gate**, regardless of where the artifact ends up.

6. IaC patterns — CDK + Terraform

A common pattern:

- **Terraform** for foundational/networking — VPC, TGW, IAM roles, KMS.
- **AWS CDK** for application-layer constructs — SageMaker domains, Lambda, Step Functions.
- **Separation rationale:** Terraform's state model and provider ecosystem suit networking; CDK's L2/L3 constructs are richer for app-level patterns.

cdk-nag + cfn-guard in CI

(Module 51.) Run cdk-nag at `cdk synth` time; run cfn-guard against the synthesized CloudFormation template. Both must pass before deploy.

7. Container security scanning

- **Inspector v2** scans ECR images for vulnerabilities.
- **Trivy / Snyk** in CI for early detection.

Common Capital One pattern: block deploys if container has CRITICAL or HIGH CVEs.

8. Secrets and config management

- **Secrets Manager** for rotated secrets (DB passwords, API keys).
- **Parameter Store** for non-rotated config.
- **IAM Identity Center** for human workforce access.
- **Doppler / HashiCorp Vault** for cross-platform secrets management.

9. Automated rollback

Patterns: - **CodeDeploy traffic shifting** (Lambda, ECS) — automatically rolls back if CloudWatch alarms trigger during deploy. - **Istio canary + Flagger** for K8s — auto-rollback on metric regression. - **SageMaker endpoint blue/green** — rollback variant on health-check failure.

10. 2024-2026 changes

- **GitHub Actions OIDC** widespread.
- **SageMaker Pipelines @step decorator** + MLflow integration.
- **CodeCatalyst** exists but adoption limited in regulated finance.
- **EKS Pod Identity** widely adopted (vs IRSA).
- **cdk-nag** matured.

11. Pitfalls

- **OIDC sub claim too broad** — high-privilege role assumable from any branch.
- **No IaC drift detection** — manual console changes diverge from code.
- **Model Registry approval bottleneck** — gate it on a SLA.
- **Container scanning at deploy** instead of build → late detection.

12. Capital One lens

Likely CI/CD: - **GitHub Actions + OIDC** to AWS. - **Terraform for VPC/IAM, CDK for app stacks**. - **cdk-nag + cfn-guard + Cloud Custodian** as the policy layers. - **SageMaker Pipelines + Step Functions** for ML orchestration. - **Model Registry + rubicon-ml + Model Cards** for the approval audit trail. - **Inspector** for container vulnerability scanning.

13. Sanity check

1. Why OIDC federation instead of GitHub-stored AWS keys?
2. What's the trust policy sub claim pitfall?
3. SageMaker Pipelines vs Step Functions for ML CI/CD — when does each fit?
4. Why use both CDK and Terraform in the same org?
5. What did CodeCommit do in 2024?

14. Cross-references

- **Module 38** — SageMaker MLOps (Pipelines + Registry)
- **Module 15** — orchestration (Step Functions)
- **Module 51** — cdk-nag + cfn-guard + Cloud Custodian
- **Module 2** — IAM trust policies

Primary sources

- Research report: [13_mlops_c1_patterns.md](#)

Module 56 — Observability & Cost for AI Workloads

What this is: CloudWatch, X-Ray, Prometheus / Grafana on EKS, Datadog patterns, AWS Cost Anomaly Detection, CUR 2.0, FOCUS 1.0, the **SageMaker cost gotchas hit list**, GenAI cost discipline, EKS / Lambda cost shapes.

1. CloudWatch — the AWS-native backbone

- **Metrics** — namespace per service, custom metrics via `PutMetricData`.
- **Alarms** — static, anomaly-detection-based, composite (combining multiple alarms).
- **Dashboards** — multi-service views.
- **Metric Streams** — push metrics to Kinesis Firehose for forwarding to Datadog, Splunk, etc.
- **CloudWatch Logs** — log groups with retention, **Logs Insights** (SQL-like queries), subscription filters.
- **Container Insights** — for ECS/EKS, per-cluster/per-pod metrics.
- **Application Signals** (GA Nov 2024) — APM-style auto-instrumentation for Lambda, ECS, EKS, EC2.

2. X-Ray

Distributed tracing. - **Service map** — visualizes service dependencies. - **Trace analytics** — query traces. - **Sampling rules** — control trace volume. - **OTel convergence** — X-Ray supports OTLP ingest (2024+).

3. AWS Distro for OpenTelemetry (ADOT)

AWS's OTel distribution. Instrument Lambda / EC2 / EKS / ECS workloads → send to CloudWatch + X-Ray + AMP + AMG.

4. AMP + AMG (Managed Prometheus + Managed Grafana)

- **Amazon Managed Service for Prometheus (AMP)** — scalable metrics backend with PromQL.
- **Amazon Managed Grafana (AMG)** — visualization on top of AMP, CloudWatch, X-Ray, OpenSearch, etc.

Decision rule: if you'd build it yourself with Prom + Grafana on K8s, use AMP + AMG instead unless you have specific reasons to self-host (cost at huge scale, customization).

5. Datadog and Splunk forwarding patterns

Most AWS-heavy shops still use Datadog or Splunk for unified observability. Forwarding patterns:

- **Datadog Lambda forwarder** — Lambda triggered on CloudWatch Logs subscription, forwards to Datadog.
- **Kinesis Firehose to Datadog** — Metric Streams → Firehose → Datadog HTTP endpoint.
- **Kinesis Firehose to Splunk** — same pattern; Splunk consumes HEC.

6. AWS Cost Anomaly Detection

ML-based anomaly detection on the bill. Cheap insurance: - Detects unexpected spikes per service / account / tag / cost category. - Free. - Alert via SNS / Slack / email.

Set this up day-one in every account.

7. Cost Explorer + Budgets + Budget Actions

(Module 4.) The standard cost-management toolkit.

Budget Actions — auto-stop EC2/RDS, auto-apply SCP — underused for non-prod environments.

8. CUR 2.0 + FOCUS 1.0

- **CUR 2.0** — Cost and Usage Report, parquet on S3, hourly.
- **FOCUS 1.0** — FinOps Open Cost & Usage Specification; vendor-neutral cost data format. AWS exports CUR in FOCUS format.

Capital One almost certainly pipes CUR 2.0 to Snowflake/Redshift for per-LOB chargeback dashboards.

9. Application Signals

(2024+.) APM-style auto-instrumentation: - Service-level objectives (SLOs). - Service map with golden signals. - Per-method latency/error rate.

The AWS-native APM offering, alternative to Datadog APM.

10. The SageMaker cost gotchas hit list

The line items that blow up bills for ML teams (the most-tested cost question in interviews):

Gotcha	Why expensive	Fix
Idle Studio domain	Always-on EFS + IDE compute	Lifecycle hooks to shutdown idle apps
Idle real-time endpoint	\$/hr per instance even at 0 RPS	Auto-scale to 0 (serverless inference) or scheduled shutdown
Idle Notebook Instance	\$/hr forever	Auto-shutdown lifecycle config (well-known script)
NAT GW for SageMaker in VPC	\$0.045/GB + \$0.045/hr per AZ	Use VPC endpoints
Unused EC2 Capacity Blocks	Pay-for-reserved-window regardless	Cancellation policy + accurate forecasting
Cross-AZ data transfer	\$0.01-\$0.02/GB	Same-AZ placement for training data + instances
S3 Standard for cold training data	1.5-3x cost vs IA	Lifecycle policies
PrivateLink endpoint hours	\$/hr $\times$ every region $\times$ every service $\times$ every AZ	Consolidate, share endpoints via RAM
EBS orphans	Volumes left after instance terminates	DLM policies + Custodian cleanup

11. GenAI cost discipline

Bedrock + Q cost levers:

- **Provisioned Throughput break-even:** roughly **8-12M output tokens / MU / month**. Below that, on-demand wins.
- **Application Inference Profiles** for per-app/per-tenant chargeback.
- **Guardrails cost per text unit** — non-trivial at high volume.
- **Knowledge Bases ingestion cost** — re-ingest on every doc change can add up.
- **OpenSearch Serverless OCU floor** — 2 OCU min ~\$700/mo.
- **Q Business per-user** — \$20/user $\times$ 50k employees = \$12M/year.
- **Custom model hosting** requires Provisioned Throughput (no on-demand).

12. EKS and Lambda cost shapes

EKS: - \$0.10/hr control plane $\times$ \$730 = \$73/mo per cluster. - Plus nodes (EC2 or Fargate). - Plus EBS for PVs. - Plus NAT GW + Interface endpoints. - **Extended support** for old K8s versions:

\$0.60/hr per cluster ($\times 6$ = additional ~\$430/mo). - **Karpenter consolidation** is the biggest cost lever — terminates underutilized nodes proactively.

Lambda: - Per request + per duration. - Provisioned concurrency adds fixed cost. - VPC endpoints avoid NAT GW cost.

13. Capital One pattern — cost-optimizing GenAI

Brent Segner’s re:Invent 2024 talk “*Control the cost of your generative AI services*” reveals:

- **Application Inference Profiles** for per-app cost attribution.
- **Tag-based chargeback.**
- **Dedicated FinOps-for-AI team.**
- **Pre-deployment cost gate** (estimate token cost before launching).

14. Slingshot insight (the Capital One product)

Slingshot is Capital One Software’s commercial Snowflake cost-optimization product. Concepts that almost certainly apply to their internal AWS cost governance:

- **Overprovisioning detection** (warehouses too big for workload).
- **Idle resource shutdown.**
- **Tag-based chargeback.**
- **Policy governance** (auto-apply rules).

These are the patterns you’d see on top of CloudWatch + Cost Anomaly Detection + Custodian internally.

15. Pitfalls

- **No baseline metrics** before optimizing → can’t tell if you improved.
- **Datadog cost** at AWS scale can exceed the AWS bill it monitors.
- **Forgetting NAT GW + VPC endpoint costs** in SageMaker / Lambda VPC workloads.
- **Provisioned Throughput** for sub-break-even Bedrock usage.

16. Sanity check

1. Name three SageMaker cost gotchas and the fix for each.
2. When does Bedrock Provisioned Throughput beat on-demand?
3. What’s CUR 2.0 + FOCUS 1.0 about?
4. Lambda in VPC — what’s the NAT GW trap, and what’s the fix?
5. What does Capital One’s Slingshot product tell you about their cost approach?

17. Cross-references

- **Module 4** — billing fundamentals
- **Module 8** — VPC endpoints (cost vs NAT)
- **Module 37** — SageMaker inference (idle endpoint gotcha)
- **Module 42** — Bedrock Provisioned Throughput
- **Module 51** — Cloud Custodian for cost policies

Primary sources

- [CloudWatch_User_Guide.pdf](#)
- [X-Ray_Developer_Guide.pdf](#)
- [AWS_Cost_Management_Whitepaper.pdf](#)
- Research report: [14_observability_cost.md](#)

Module 57 — AWS Cert Paths for Sr Lead AI/ML

What this is: the cert sequence (SAA → MLA → DEA → SCS → SAP → AIP), domain-to-module mapping, study plan, hands-on lab budget, exam-day strategy, recertification policy.

1. Recommended sequence

#	Cert	Code	Why now
1	Solutions Architect – Associate	SAA-C03	Recruiter-scannable baseline
2	Machine Learning Engineer – Associate	MLA-C01	Strongest AI/ML signal (replaces retired MLS-C01)
3	Data Engineer – Associate	DEA-C01	Glue/Redshift/Athena/Streaming credibility
4	Security – Specialty	SCS-C03	Regulated-finance differentiator (Dec 2025 refresh adds GenAI)
5	Solutions Architect – Professional	SAP-C02	EM / Architect track promotion lever
6	Generative AI Developer – Professional	AIP-C01	Capstone for AI Architect targeting RAG/Bedrock/agents

Skip: AIF-C01 (beneath your level), ANS-C01 (SAP covers enough networking).

2. Domain-to-module mapping

SAA-C03 (Solutions Architect Associate)

Domain	Weight	Modules
Design Secure Architectures	30%	1, 2, 6-9, 50, 51
Design Resilient Architectures	26%	3, 6, 8, 9, 16-18
Design High-Performing Architectures	24%	10-14, 22, 28-33
Design Cost-Optimized Architectures	20%	4, 56

MLA-C01 (ML Engineer Associate)

Domain	Weight	Modules
Data Preparation for ML	28%	11-15, 27, 35
ML Model Development	26%	36, 39
Deployment and Orchestration	22%	15, 37, 38
ML Solution Monitoring, Maintenance, Security	24%	2, 38, 41, 50, 56

DEA-C01 (Data Engineer Associate)

Domain	Weight	Modules
Data Ingestion and Transformation	34%	13-15, 28, 29
Data Store Management	26%	10-12, 16-27
Data Operations and Support	22%	4, 15, 24, 56
Data Security and Governance	18%	2, 12, 50-52

SCS-C03 (Security Specialty)

Domain	Weight	Modules
Threat Detection and Incident Response	14%	50, 52
Security Logging and Monitoring	18%	7, 50, 56
Infrastructure Security	20%	6-9, 50, 51
Identity and Access Management	16%	2, 50, 51
Data Protection	18%	10, 50, 52
Management and Security Governance	14%	1, 51, 52

Dec 2025 refresh: the SCS-C03 added an explicit “Implement protections and guardrails for generative AI applications (GenAI OWASP Top 10 for LLM Applications)” task statement under Domain 3.2. **This is brand new and most third-party prep courses haven’t caught up.** Supplement with AWS re:Inforce 2025 talks.

SAP-C02 (Solutions Architect Professional)

Domain	Weight	Modules
Design for Org Complexity	26%	1, 2, 8
Design for New Solutions	29%	All Part C-N modules apply
Continuous Improvement for Existing Solutions	25%	4, 56, 51
Accelerate Workload Migration & Modernization	20%	16, 17, 22

3. Study plan (46-week ladder)

Cert	Weeks	Hours/week
SAA-C03	4-6	10-12
MLA-C01	4-6	10-12
DEA-C01	3-5 (overlap with MLA)	10-12
SCS-C03	4-6	10-12
SAP-C02	8-10	12-15
AIP-C01	4-6 (after GA stabilizes)	10-12
Total	~9-12 months part-time	

4. Hands-on lab plan per cert

SAA-C03

Deploy a multi-tier app: - VPC + public/private subnets across 3 AZs. - ALB → ECS Fargate or Lambda → Aurora Postgres. - S3 for static assets + CloudFront. - IAM roles for each tier.

MLA-C01

End-to-end SageMaker pipeline: - Glue ETL → Feature Store ingest. - SageMaker Training Job (PyTorch or built-in). - SageMaker Pipeline orchestrating preprocessing → train → evaluate → register. - Real-time endpoint deployment + Model Monitor.

DEA-C01

Lake house: - Glue crawlers + ETL into S3 Iceberg. - Lake Formation FGAC. - Athena queries + Redshift Spectrum. - Kinesis Firehose for streaming ingest.

SCS-C03

Multi-account security: - Control Tower or LZA landing zone. - Cloud Custodian policies (encryption enforcement, tag enforcement). - KMS CMKs with rotation. - Security Hub conformance pack (PCI-DSS). - GuardDuty + Macie + Inspector enabled.

SAP-C02

Multi-region landing zone: - Transit Gateway hub-spoke. - DMS / MGN-style migration scenario. - Multi-region active-passive failover. - Network Firewall + egress filtering.

5. Hands-on lab budget

- **SageMaker Studio Lab** — free (no AWS account).
- **SageMaker Free Tier** — 2 months × 250 hours `ml.t3.medium`.
- **Realistic monthly spend after free tier:** \$75-150/month.
- Biggest line items to watch: NAT GW, idle SageMaker endpoints/Studio domains, idle EKS clusters, Capacity Blocks.

Discipline: AWS Budgets alert at \$100; terraform destroy after each lab session.

6. Mock-exam strategy

Two highest-signal resources: - **Tutorials Dojo** — practice exams. - **AWS Skill Builder Exam Prep** — official AWS-aligned questions.

Readiness gates: **78-82% on Tutorials Dojo** sustained across multiple practice tests indicates real-exam readiness.

Other useful: Stephane Maarek Udemy (well-paced), Adrian Cantrill (deep-dive video courses).

7. Exam-day strategy

- **Pearson VUE in-person testing center** > online proctoring (fewer issues, less stress).
- **Time per question budgets:**
 - SAA-C03 (130 min / 65 Q): ~2 min/question.
 - MLA-C01 (170 min / 85 Q): ~2 min/question.
 - SAP-C02 (180 min / 75 Q): **2.4 min/question** — this is the real bottleneck.
- **Flag and return** — flag any question you spend > 3 min on; return after first pass.
- **Elimination heuristics** — AWS exam answers typically eliminate two wrong answers easily (clearly out of scope, deprecated, wrong service). Pick between the remaining two.

8. Recertification

- **3-year validity** from date earned.
- **Auto-recert via higher-tier cert** — passing SAP-C02 automatically recerts SAA-C03 for another 3 years.
- **Skill Builder Recertification Assessment** — alternative to re-taking; shorter free assessment.

Stacking play: time DEA-C01 right after MLA-C01 so they both recert together when you take SAP-C02. Maximizes cert-window coverage with minimum exam re-takes.

9. Pitfalls

- **Skipping SAA-C03** — common mistake. It's the recruiter-scannable baseline.
- **Studying too much theory, not enough hands-on.**
- **Mock-exam % bias** — Tutorials Dojo tends to track AWS exam difficulty; if you're below 78%, you're not ready.
- **Going straight to Professional** without Associate foundation — possible but high-risk.

10. The Capital One signal

Job postings don't strictly require certs but list AWS Solutions Architect Pro + AWS ML Specialty as fit. With MLS-C01 retired, **the modern signal is SAA + MLA + DEA + SCS**, then SAP for AI Architect / EM tracks.

Cloud Custodian familiarity is a separate cultural plus that no cert covers.

11. Sanity check

1. What replaced MLS-C01?
2. What's new in SCS-C03's Dec 2025 refresh?
3. SAP-C02 bottleneck — what is it?
4. What's the auto-recert-via-higher-tier play?
5. What's the budget alarm threshold you'd set for self-paced labs?

12. Cross-references

- **CAPITAL_ONE.md** — section 7 lists Capital One hiring-bar cert priorities
- **FACTS.md** — exam costs, durations, retirement dates
- All modules in this topic — feed into one or more cert domains

Primary sources

- All exam guide PDFs in [downloads/certs/](#)
- AWS Skill Builder pages (archived)
- Research reports: [01_certification_landscape.md](#) and [15_cert_roadmap_synthesis.md](#)

Appendix A — FACTS.md

Atomic, citable facts. Single source of truth for dates, costs, GA, retirements, and exam metadata.

Topic 04 — FACTS.md

Atomic, citable facts. Single source of truth for numbers, dates, model names, GA/retirement events, and exam metadata.

Format: each fact is one line, with citation in square brackets and “last verified” date. If a fact is older than 90 days, re-verify before quoting.

Last updated: 2026-05-14 (seed)

Capital One on AWS

- Capital One completed exit from its last on-premises data center in **November 2020** — first US bank 100% all-in on public cloud. [Source: AWS case study + SiliconAngle re:Invent 2020 coverage] [Verified 2026-05-14]
- ~80% of applications were rebuilt cloud-native (microservices + REST) during the migration. [Source: AWS case study] [Verified 2026-05-14]
- Capital One runs both **Snowflake on AWS** and **Databricks on AWS** in production. Their commercial product line (Slingshot, Databolt) targets this combo. [Source: capitalone.com/software/, job postings] [Verified 2026-05-14]
- Capital One self-hosts model serving on **EKS with KServe** — confirmed in their public Lead MLE job posting titled “Lead Machine Learning Engineer (MLOps, KServe — building Kubernetes Clusters, PyTorch, TensorFlow on AWS)”. [Source: [capitalone.wd12.myworkdayjobs.com R239877](https://capitalone.wd12.myworkdayjobs.com/R239877)] [Verified 2026-05-14]
- **Eno** (Capital One’s AI assistant) launched 2017; received LLM upgrade in 2024; added Spanish in 2025; credited with ~50% call-center volume reduction. [Source: capitalone.com/tech/ai/] [Verified 2026-05-14]
- Capital One’s internal ML platform team is called **Intelligent Foundations and Experiences (IFX)**. [Source: Lead MLE job posting] [Verified 2026-05-14]
- Compensation bands (McLean, VA, 2026): Sr Lead MLE ~\$229k–\$262k; Sr Distinguished MLE ~\$315k–\$359k. [Source: capitalonecareers.com job postings] [Verified 2026-05-14]

Capital One open-source AWS tooling

- **Cloud Custodian** — YAML policy-as-code rules engine for AWS governance, originally from Capital One, now **CNCF Incubating** (Apache 2.0). Native Security Hub integration. [Source: [github.com/cloud-custodian](https://github.com/cloud-custodian/cloud-custodian), cnf.io] [Verified 2026-05-14]
- **rubicon-ml** — ML experiment tracking with git integration, designed for auditability and reproducibility in regulated environments. [Source: github.com/capitalone/rubicon-ml] [Verified 2026-05-14]
- **DataProfiler** — schema/stat/entity extraction for sensitive-data scanning; ~1.6k GitHub stars. [Source: [github.com/capitalone](https://github.com/capitalone/dataprofiler)] [Verified 2026-05-14]
- **datacompy** — Pandas/Polars/Spark/Snowpark dataframe diff library for pipeline validation. [Source: github.com/capitalone/datacompy] [Verified 2026-05-14]
- **locopy** — Redshift + Snowflake load/unload helper. [Source: github.com/capitalone/locopy] [Verified 2026-05-14]
- **Hygieia** — DevOps dashboard; first OSS release at OSCON 2015. [Source: github.com/Hygieia] [Verified 2026-05-14]

Capital One 2019 breach (compliance context)

- March/April 2019: misconfigured **WAF SSRF** vulnerability + over-permissive IAM allowed exfiltration of ~106 million customer records from S3. [Source: OCC consent order Aug 2020; SEC filings] [Verified 2026-05-14]
- \$80M OCC fine (Aug 2020); \$190M class-action settlement (Dec 2021). [Source: OCC and DOJ press releases] [Verified 2026-05-14]
- Aftermath shaped Capital One’s emphasis on **policy-as-code** (**Cloud Custodian**), **tokenization** (**Databolt**), and **IaC guardrails** (**cfn-guard**, **cdk-nag**, **CloudFormation hooks**). [Inference from re:Invent 2024 session content] [Verified 2026-05-14]

AWS Certification — active (as of 2026-05-14)

Cert	Code	Level	Cost USD	Duration
Cloud Practitioner	CLF-C02	Foundational	\$100	90 min / 65 Q
AI Practitioner	AIF-C01	Foundational	\$100	90 min / 65 Q
Solutions Architect – Associate	SAA-C03	Associate	\$150	130 min / 65 Q
Machine Learning Engineer – Associate	MLA-C01	Associate	\$150	170 min / 85 Q

Cert	Code	Level	Cost USD	Duration
Data Engineer – Associate Solutions Architect – Professional	DEA-C01	Associate	\$150	130 min / 65 Q
Generative AI Developer – Professional	AIP-C01	Professional	\$300	~205 min / 85 Q (GA early 2026)
Advanced Networking – Specialty	ANS-C01	Specialty	\$300	170 min / 65 Q
Security – Specialty	SCS-C03	Specialty	\$300	170 min / 65 Q

[Source: aws.amazon.com/certification/] [Verified 2026-05-14]

AWS Certification — retired

- **Machine Learning – Specialty (MLS-C01)** — last exam day **March 31, 2026**. Successor: MLA-C01 + AIF-C01 + AIP-C01. [Source: AWS training & certification blog] [Verified 2026-05-14]
- **Data Analytics – Specialty (DAS-C01)** — retired April 2024. Successor: DEA-C01. [Source: AWS training & certification blog] [Verified 2026-05-14]
- **Database – Specialty (DBS-C01)** — retired April 30, 2024. No direct replacement; content absorbed into DEA-C01 + SAP-C02. [Source: AWS training & certification blog] [Verified 2026-05-14]
- **Security – Specialty (SCS-C02)** — retired December 1, 2025. Successor: SCS-C03 (adds GenAI/ML security domain). [Source: AWS training & certification blog] [Verified 2026-05-14]
- All earned certifications remain valid for **3 years** from date earned regardless of retirement. [Source: AWS recertification policy] [Verified 2026-05-14]

Exam domain weightings (top three for this role)

SAA-C03 (Solutions Architect Associate): - Design Secure Architectures: 30% - Design Resilient Architectures: 26% - Design High-Performing Architectures: 24% - Design Cost-Optimized Architectures: 20%

MLA-C01 (ML Engineer Associate): - Data Preparation for ML: 28% - ML Model Development: 26% - ML Solution Monitoring, Maintenance, and Security: 24% - Deployment and Orchestration of ML Workflows: 22%

DEA-C01 (Data Engineer Associate): - Data Ingestion and Transformation: 34% - Data Store Management: 26% - Data Operations and Support: 22% - Data Security and Governance: 18%

[Source: official AWS exam guide PDFs] [Verified 2026-05-14]

Hands-on lab budget guidance

- SageMaker Studio Lab — **free**, no AWS account needed. [Source: aws.amazon.com/sagemaker/studio-lab/] [Verified 2026-05-14]
- SageMaker 2-month Free Tier: 250 hrs of `ml.t3.medium`. [Source: aws.amazon.com/sagemaker/pricing/] [Verified 2026-05-14]
- Realistic monthly self-paced lab spend (after free tier): **\$75–150/mo** covering SageMaker + Glue + Athena + Bedrock + occasional GPU. NAT Gateway and idle SageMaker endpoints/Studio domains are the three line items that blow up bills. [Source: estimation from AWS calculator + community reports] [Verified 2026-05-14]

(Module-specific FACTS to be added as modules are written)

— Aurora DSQL GA date, Limitless Database GA date — *to be verified at module 17 write-time* —
DynamoDB on-demand vs provisioned pricing per WCU/RCU — *to be verified at module 18 write-time*
— Glue worker types and pricing — *to be verified at module 14 write-time* — EKS pricing per cluster
(\$0.10/hr control plane) — *to be verified at module 33 write-time* — Bedrock model pricing per 1k
input/output tokens — *to be verified at module 42 write-time* — SageMaker Inference latency SLAs
and cold-start numbers — *to be verified at module 37 write-time*

Appendix — Sources & References

All material in this ebook was compiled from publicly-available primary sources. Every cited document is archived under `research_inputs/04_aws_for_ai_ml/downloads/` in the project repository so the corpus is self-contained against link rot.

This appendix lists every source consulted, grouped by category.

1. Capital One — primary sources

Capital One Tech blog and corporate sites

- **Capital One AWS Case Study** — <https://aws.amazon.com/solutions/case-studies/innovators/capital-one/>
- **Capital One Tech — re:Invent 2024 roundup** — <https://www.capitalone.com/tech/cloud/aws-reinvent-2024/>
- **Capital One Tech — AI page** — <https://www.capitalone.com/tech/ai/>
- **Capital One Tech — serverless streaming SDK pattern** — <https://www.capitalone.com/tech/cloud/serverless-streaming/>
- **Capital One Software — Databolt for AWS** — <https://www.capitalone.com/software/products/databolt/>
- **Capital One Software — Slingshot (Snowflake cost governance)** — <https://www.capitalone.com/software/slingshot/>

Capital One open-source projects (read the code to learn their idioms)

- **Cloud Custodian** (CNCF Incubating, originally from Capital One) — <https://github.com/cloud-custodian/cloud-custodian> — docs at <https://cloudcustodian.io/docs/>
- **rubicon-ml** (model experiment tracking with git lineage) — <https://github.com/capitalone/rubicon-ml>
- **DataProfiler** (sensitive-data scanning) — <https://github.com/capitalone/DataProfiler>
- **datacompy** (dataframe diff) — <https://github.com/capitalone/datacompy>
- **locopy** (Redshift + Snowflake load/unload) — <https://github.com/capitalone/locopy>
- **edgetest** (dependency-update testing) — <https://github.com/capitalone/edgetest>
- **Hygieia** (DevOps dashboard, OSCON 2015) — <https://github.com/Hygieia>

Capital One job postings (signal what the role expects)

- Sr Lead Machine Learning Engineer (GenAI / Python / AWS) — <https://jobs.physicstoday.org/job/41953304/>
- Lead Machine Learning Engineer (MLOps, KServe, Building Kubernetes Clusters on AWS) — https://capitalone.wd12.myworkdayjobs.com/Capital_One/job/McLean-VA/Lead-Machine-Learning-Engineer-MLOps-KServe-building-Kubernetes-Clusters-PyTorch-TensorFlow-on-AWS-_R239877-1
- Sr Distinguished Machine Learning Engineer (Personalization) — <https://www.capitalonecareers.com/job/mcldistinguished-machine-learning-engineer-remote-eligible/1732/93650794080>

Conference presentations referenced

- **NVIDIA GTC 2025, Session EXS74442** — “Generative AI Agent Servicing Tool” (Capital One)
 - **re:Invent 2024** — Capital One sessions on:
 - “Celebrating 10 years of pioneering serverless” (Catherine McGarvey)
 - “Reduce FM deployment costs and latency with SageMaker” (Grant Gillary)
 - “Control the cost of your generative AI services” (Brent Segner)
 - “Governance and security with infrastructure as code” (Ishu Gupta)
 - “Advancing state-of-the-art science and AI in financial services” (James Montgomery)
 - “AI-driven value” (Prem Natarajan + NVIDIA)
-

2. AWS official documentation (archived as PDFs/HTML)

AWS Well-Architected Framework

- **Well-Architected Framework** (the master) — <https://docs.aws.amazon.com/wellarchitected/latest/framework/framework.html>
- **Security Pillar** — <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/>
- **Machine Learning Lens** — <https://docs.aws.amazon.com/wellarchitected/latest/machine-learning-lens/>
- **Generative AI Lens** — <https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/>
- **Financial Services Industry Lens** — <https://docs.aws.amazon.com/wellarchitected/latest/financial-services-industry-lens/>

AWS Reference Architectures

- **AWS Security Reference Architecture (SRA)** — <https://docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/>
- **Organizing Your AWS Environment Using Multiple Accounts** — <https://docs.aws.amazon.com/whitepapers/latest/organizing-your-aws-environment/>
- **Tagging Best Practices** — <https://docs.aws.amazon.com/whitepapers/latest/tagging-best-practices/>

- **AWS Control Tower User Guide** — <https://docs.aws.amazon.com/controltower/latest/userguide/>
- **Landing Zone Accelerator on AWS** — <https://aws.amazon.com/solutions/implementations/landing-zone-accelerator-on-aws/>
- **AWS Resource Control Policies launch (Nov 2024)** — <https://aws.amazon.com/blogs/aws/introducing-resource-control-policies-rcps-a-new-authorization-policy/>

Networking

- **Amazon VPC Connectivity Options** — <https://docs.aws.amazon.com/whitepapers/latest/aws-vpc-connectivity-options/>
- **Hybrid Connectivity** — <https://docs.aws.amazon.com/whitepapers/latest/hybrid-connectivity/>
- **Building Scalable and Secure Multi-VPC Network Infrastructure** — <https://docs.aws.amazon.com/whitepapers/latest/building-scalable-secure-multi-vpc-network-infrastructure/>
- **Transit Gateway Best Practices** — <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-best-practices.html>
- **Route 53 Routing Policies** — <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html>

Storage & Lake

- **Amazon S3 User Guide** — <https://docs.aws.amazon.com/s3/>
- **AWS Lake Formation Developer Guide** — <https://docs.aws.amazon.com/lake-formation/>
- **AWS Glue Developer Guide** — <https://docs.aws.amazon.com/glue/>
- **AWS DataZone User Guide** — <https://docs.aws.amazon.com/datazone/>
- **Amazon EBS User Guide** — <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AmazonEBS.html>
- **Amazon EFS User Guide** — <https://docs.aws.amazon.com/efs/>
- **Amazon FSx for Lustre** — <https://docs.aws.amazon.com/fsx/latest/LustreGuide/>

Databases

- **Amazon RDS User Guide** — <https://docs.aws.amazon.com/rds/>
- **Aurora User Guide** — <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/>
- **Aurora DSQL** — <https://docs.aws.amazon.com/aurora-dsql/>
- **DynamoDB Developer Guide** — <https://docs.aws.amazon.com/dynamodb/>
- **DocumentDB Best Practices** — https://docs.aws.amazon.com/documentdb/latest/developerguide/best_practices.html
- **DocumentDB Vector Search** — <https://docs.aws.amazon.com/documentdb/latest/developerguide/vector-search.html>
- **Amazon Keyspaces Developer Guide** — <https://docs.aws.amazon.com/keyspaces/>
- **ElastiCache User Guide** — <https://docs.aws.amazon.com/elasticache/>
- **Valkey announcement and adoption** — <https://aws.amazon.com/blogs/database/amazon-elasticache-supports-valkey-7-2-with-a-major-price-reduction/>
- **MemoryDB User Guide** — <https://docs.aws.amazon.com/memorydb/>
- **MemoryDB Vector Search** — <https://docs.aws.amazon.com/memorydb/latest/devguide/vector-search.html>

- **Amazon Redshift Best Practices** — <https://docs.aws.amazon.com/redshift/latest/dg/best-practices.html>
- **Redshift Zero-ETL** — <https://docs.aws.amazon.com/redshift/latest/mgmt/zero-etl-using.html>
- **Snowflake on AWS** — <https://aws.amazon.com/snowflake/>
- **Amazon Athena Performance Tuning** — <https://docs.aws.amazon.com/athena/latest/ug/performance-tuning.html>
- **Amazon Neptune Best Practices** — <https://docs.aws.amazon.com/neptune/latest/userguide/best-practices.html>
- **Neptune Analytics** — <https://docs.aws.amazon.com/neptune-analytics/latest/userguide/>
- **Neptune ML** — <https://docs.aws.amazon.com/neptune/latest/userguide/machine-learning.html>
- **OpenSearch Vector Search** — <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/vector-search.html>
- **Aurora pgvector** — <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.html>

Streaming

- **Amazon MSK Developer Guide** — <https://docs.aws.amazon.com/msk/>
- **Amazon Kinesis Data Streams** — <https://docs.aws.amazon.com/kinesis/latest/dev/>
- **Amazon Data Firehose** — <https://docs.aws.amazon.com/firehose/>
- **Amazon Managed Service for Apache Flink** — <https://docs.aws.amazon.com/managed-flink/>
- **EventBridge Pipes** — <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes.html>
- **Streaming Data Solutions on AWS (whitepaper)** — <https://docs.aws.amazon.com/whitepapers/latest/streaming-data-solutions-amazon-kinesis/>

Compute

- **EC2 Capacity Blocks for ML** — <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-capacity-blocks.html>
- **AWS Lambda Operator's Guide** — <https://docs.aws.amazon.com/lambda/latest/operatorguide/>
- **Amazon EKS Best Practices Guide** — <https://docs.aws.amazon.com/eks/latest/best-practices/>
- **Karpenter** — <https://karpenter.sh/docs/>
- **EC2 Spot Best Practices** — <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-best-practices.html>

SageMaker (the biggest section)

- **SageMaker Developer Guide** — <https://docs.aws.amazon.com/sagemaker/>
- **SageMaker Best Practices** — <https://docs.aws.amazon.com/sagemaker/latest/dg/best-practices.html>
- **SageMaker Pipelines Developer Guide** — <https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html>
- **SageMaker Studio (new) Admin Guide** — <https://docs.aws.amazon.com/sagemaker/latest/dg/studio-updated.html>

- **SageMaker HyperPod** — <https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-hyperpod.html>
- **SageMaker Inference Best Practices** — <https://docs.aws.amazon.com/sagemaker/latest/dg/inference-recommender.html>
- **SageMaker JumpStart** — <https://docs.aws.amazon.com/sagemaker/latest/dg/studio-jumpstart.html>
- **SageMaker Unified Studio** — <https://docs.aws.amazon.com/sagemaker-unified-studio/>
- **SageMaker Feature Store** — <https://docs.aws.amazon.com/sagemaker/latest/dg/feature-store.html>
- **SageMaker Model Monitor** — <https://docs.aws.amazon.com/sagemaker/latest/dg/model-monitor.html>
- **SageMaker Clarify** — <https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-configure-processing-jobs.html>
- **SageMaker Distributed Training** — <https://docs.aws.amazon.com/sagemaker/latest/dg/distributed-training.html>
- **SageMaker Inference Components** — <https://aws.amazon.com/blogs/machine-learning/amazon-sagemaker-launches-inference-components/>

Generative AI

- **Amazon Bedrock User Guide** — <https://docs.aws.amazon.com/bedrock/>
- **Amazon Bedrock Knowledge Bases** — <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html>
- **Amazon Bedrock Agents** — <https://docs.aws.amazon.com/bedrock/latest/userguide/agents.html>
- **Amazon Bedrock Guardrails** — <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html>
- **Bedrock pricing** — <https://aws.amazon.com/bedrock/pricing/>
- **Amazon Nova** — <https://aws.amazon.com/nova/>
- **Amazon Q Developer** — <https://aws.amazon.com/q/developer/>
- **Amazon Q Business** — <https://aws.amazon.com/q/business/>
- **NVIDIA NIM on AWS Marketplace** — <https://aws.amazon.com/marketplace/seller-profile?id=c568fe05-e33b-411c-b0ab-047218431da9>
- **NVIDIA NeMo Guardrails** — <https://github.com/NVIDIA/NeMo-Guardrails>

Databricks on AWS

- **Databricks on AWS — High-level architecture** — <https://docs.databricks.com/aws/en/getting-started/high-level-architecture>
- **Databricks on AWS — Configure customer-managed VPC** — <https://docs.databricks.com/aws/en/security/managed-vpc>
- **Databricks on AWS — Classic PrivateLink** — <https://docs.databricks.com/aws/en/security/network/classic-private-link>
- **Databricks on AWS — Serverless compute plane networking** — <https://docs.databricks.com/aws/en/security/network-security/serverless-compute-plane-networking>
- **Databricks on AWS — Unity Catalog S3 external location** — <https://docs.databricks.com/aws/en/unity-catalog/cloud-storage/s3/>

- **AWS Glue Catalog federation to Databricks Unity Catalog** — <https://aws.amazon.com/blogs/big-data/access-databricks-unity-catalog-data-using-catalog-federation-in-the-aws-glue-data-catalog/>
- **Terraform Databricks Provider** — <https://registry.terraform.io/providers/databricks/databricks/latest/docs>
- **Mosaic AI Vector Search** — <https://docs.databricks.com/aws/en/vector-search/vector-search>
- **Databricks pricing** — <https://www.databricks.com/product/pricing>
- **Amazon EMR Serverless** — <https://docs.aws.amazon.com/emr/latest/EMR-Serverless-UserGuide/>
- **Amazon EMR on EKS** — <https://docs.aws.amazon.com/emr/latest/EMR-on-EKS-DevelopmentGuide/>

Security & Compliance

- **AWS KMS Cryptographic Details** — <https://docs.aws.amazon.com/kms/latest/ cryptographic-details/>
- **AWS Audit Manager User Guide** — <https://docs.aws.amazon.com/audit-manager/>
- **AWS GuardDuty** — <https://docs.aws.amazon.com/guardduty/>
- **AWS Macie** — <https://docs.aws.amazon.com/maciek/>
- **AWS Security Hub** — <https://docs.aws.amazon.com/securityhub/>
- **AWS Config** — <https://docs.aws.amazon.com/config/>
- **AWS Inspector** — <https://docs.aws.amazon.com/inspector/>
- **IAM Access Analyzer** — <https://docs.aws.amazon.com/IAM/latest/UserGuide/what-is-access-analyzer.html>
- **CloudTrail Lake** — <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-lake.html>

Observability & Cost

- **Amazon CloudWatch User Guide** — <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/>
- **AWS X-Ray Developer Guide** — <https://docs.aws.amazon.com/xray/>
- **AWS Cost Management** — <https://docs.aws.amazon.com/cost-management/>
- **AWS Cost and Usage Report (CUR 2.0)** — <https://docs.aws.amazon.com/cur/>
- **AWS Cost Anomaly Detection** — <https://docs.aws.amazon.com/cost-management/latest/userguide/manaad.html>
- **AWS Application Inference Profiles** — <https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles.html>
- **Application Signals (APM)** — <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudApplication-Signals.html>
- **Amazon Managed Prometheus (AMP)** — <https://docs.aws.amazon.com/prometheus/>
- **Amazon Managed Grafana (AMG)** — <https://docs.aws.amazon.com/grafana/>
- **AWS Distro for OpenTelemetry (ADOT)** — <https://aws-otel.github.io/>
- **FOCUS 1.0 (FinOps Open Cost Specification)** — <https://focus.finops.org/>

3. AWS Certifications (official exam guides)

All retrieved from <https://aws.amazon.com/certification/> portal as of 2026-05.

- **AWS Certified Cloud Practitioner (CLF-C02)** — <https://aws.amazon.com/certification/certified-cloud-practitioner/>
 - **AWS Certified AI Practitioner (AIF-C01)** — <https://aws.amazon.com/certification/certified-ai-practitioner/>
 - **AWS Certified Solutions Architect – Associate (SAA-C03)** — <https://aws.amazon.com/certification/certified-solutions-architect-associate/>
 - **AWS Certified Machine Learning Engineer – Associate (MLA-C01)** — <https://aws.amazon.com/certification/certified-machine-learning-engineer-associate/>
 - **AWS Certified Data Engineer – Associate (DEA-C01)** — <https://aws.amazon.com/certification/certified-data-engineer-associate/>
 - **AWS Certified Solutions Architect – Professional (SAP-C02)** — <https://aws.amazon.com/certification/certified-solutions-architect-professional/>
 - **AWS Certified Generative AI Developer – Professional (AIP-C01)** — <https://aws.amazon.com/certification/certified-generative-ai-developer-professional/>
 - **AWS Certified Advanced Networking – Specialty (ANS-C01)** — <https://aws.amazon.com/certification/certified-advanced-networking-specialty/>
 - **AWS Certified Security – Specialty (SCS-C03)** — <https://aws.amazon.com/certification/certified-security-specialty/>
 - **AWS Certification retirements and launches blog** — <https://aws.amazon.com/blogs/training-and-certification/aws-certification-retirements-and-launches/>
-

4. Regulatory and standards documents

- **OCC Consent Order against Capital One (Aug 2020)** — public document
 - **FFIEC Cloud Computing Statement (June 2020)** — Federal Financial Institutions Examination Council
 - **SR 11-7 — Federal Reserve / OCC Guidance on Model Risk Management** (with 2026 revision SR 26-02)
 - **PCI-DSS v4.0** (active Mar 2024, mandatory Mar 2025) — PCI Security Standards Council
 - **GLBA Safeguards Rule** (FTC, 2023 update) — <https://www.ftc.gov/legal-library/browse/rules/safeguards-rule>
 - **FedRAMP Documentation** — <https://www.fedramp.gov/>
 - **NIST 800-53** — <https://csrc.nist.gov/Projects/risk-management/sp800-53-controls>
-

5. Open-source projects referenced

- **Cloud Custodian (CNCF Incubating)** — <https://github.com/cloud-custodian/cloud-custodian>
 - **KServe** — <https://github.com/kserve/kserve>
 - **Karpenter** — <https://github.com/aws/karpenter-provider-aws>
 - **KEDA (Kubernetes Event-Driven Autoscaling)** — <https://keda.sh/>
 - **NVIDIA NeMo Guardrails** — <https://github.com/NVIDIA/NeMo-Guardrails>
 - **NVIDIA Triton Inference Server** — <https://github.com/triton-inference-server/server>
 - **Apache Iceberg** — <https://iceberg.apache.org/>
 - **Apache Hudi** — <https://hudi.apache.org/>
 - **Delta Lake** — <https://delta.io/>
 - **Apache Spark** — <https://spark.apache.org/>
 - **Apache Kafka** — <https://kafka.apache.org/>
 - **Valkey** (open-source Redis fork) — <https://valkey.io/>
 - **rubicon-ml** — <https://github.com/capitalone/rubicon-ml>
 - **Istio** — <https://istio.io/>
 - **OpenTelemetry** — <https://opentelemetry.io/>
 - **MLflow** — <https://mlflow.org/>
-

6. Internal research reports (in this repository)

The 14 deep-research reports under `research_inputs/04_aws_for_ai_ml/` that drove each Part:

#	Report	Covers Modules
00	Capital One AWS usage	CAPITAL_ONE.md
01	AWS Certification landscape (May 2026)	57
02	AWS Foundations	1–4
03	Networking deep dive	5–9
04	Storage & data lake	10–12
05	Glue & ETL orchestration	13–15
06a	Relational + DynamoDB	16–18
06b	NoSQL siblings (DocumentDB, Keyspaces, ElastiCache, MemoryDB)	19–21
06c	Analytical + Vector + Graph DBs	22–27
07	Streaming (MSK + Kinesis)	28–29
08	Compute for ML	30–33
09	SageMaker	34–41

#	Report	Covers Modules
10	Generative AI on AWS	42–44
11	Databricks on AWS	45–49
12	Security & Compliance	50–52
13	MLOps & Capital One patterns	53–55
14	Observability & cost	56
15	Cert roadmap synthesis	57

Each report contains a full citations section with archived primary-source URLs.

7. Books, papers, and additional references

- **Mark Brooker** — “Stable and Predictable Distributed Locks” / Aurora design papers (referenced for Aurora Limitless and DSQL background)
- **Pat Helland** — papers on consistency and distributed systems (Aurora DSQL background)
- **The Phoenix Project + The Unicorn Project** (Gene Kim) — DevOps culture context
- **AWS re:Invent 2024 session recordings** — Capital One session list (see Section 1)
- **HBR sponsored piece (2021)** — “From Data to Insights: The Capital One Journey” — <https://hbr.org/sponsored/2021/06/from-data-to-insights-the-capital-one-journey>

8. About archived documents

Every URL listed in this appendix has a corresponding file archived under `research_inputs/04_aws_for_ai_ml/downloads/` in the project repository:

```
research_inputs/04_aws_for_ai_ml/downloads/
├─ aws_whitepapers/      # Well-Architected pillars, IAM, S3, Lambda, EKS, SageMaker, Bedrock, etc.
├─ capital_one/          # AWS case study, Tech blog posts, Databolt product page
├─ certs/                # 7 AWS exam guide PDFs
├─ databricks_on_aws/    # Architecture, PrivateLink, UC, EMR docs
├─ oss_tooling/          # Cloud Custodian, rubicon-ml, KServe, Karpenter
├─ regulatory/           # OCC consent order, FFIEC, SR 11-7, PCI-DSS
└─ INDEX.md              # Manifest mapping each file to its source URL + retrieval date + citing modules
```

~162 files, all retrieved 2026-05-14 to 2026-05-15. Refer to `INDEX.md` for the canonical mapping.

9. Caveats

- **AWS feature velocity is high.** Re-verify any GA dates, pricing numbers, and exam-content domains at write-time. The `FACTS.md` companion file in this topic tracks “last verified” dates per fact.
- **Some primary-source PDFs have moved or been deprecated** since archival. Where the original URL returns 404 (e.g., the SR 11-7 PDF was superseded by SR 26-02 in 2026), the archived file in `downloads/` remains as the corpus’s authoritative copy of the version cited.
- **Capital One’s actual internal architecture is not public.** This ebook synthesizes from public talks, job postings, open-source projects, and reasonable inference from their public engineering blog. Specifics may differ from internal reality.
- **The 2019 Capital One breach** is referenced extensively as a teaching case study. The architectural lessons are well-documented in the OCC consent order and SEC filings. The treatment in this ebook is non-attribution where possible — the patterns are now industry-wide best practices.

End of ebook.